

**Entwicklung eines Softwaremoduls für kamerabasierte visuelle Prüfungen im
Industrienumfeld**

Masterarbeit

des Studienganges Praktische Informatik
an der Fernuniversität Hagen

von
Henning Brands

Abgabedatum: 30.06.2026

Bearbeitungszeitraum:	6 Monate
Matrikelnummer	3267679
Erstgutachter:	Dr. Daniela Keller
Zweitgutachter	Dr. Rainer Nitsche rainer.nitsche@festo.com

Sperrvermerk

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anderslautende Genehmigung des Unternehmens Festo vorliegt.

Erklärung zur Eigenleistung

Ich versichere hiermit, dass ich meine Masterarbeit mit dem Thema: „Entwicklung eines Softwaremoduls für kamerabasierte visuelle Prüfungen im Industrieumfeld“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Stuttgart, den 30.06.2026

Ort

Datum



Unterschrift

Abstract

This thesis addresses the development and implementation of a computer vision solution for use in automated testing systems. For this purpose, the software module *TEaMVision* is created, which integrates into an existing testing ecosystem based on the software *TestStand*.

At the industrial company Festo, manual visual inspections are to be automated through the use of camera-based systems, so-called *Vision Systems*. However, large-scale implementation of such systems has repeatedly failed due to high costs, significant development effort, and insufficient interfaces for implementing complex applications. *TEaMVision* closes these gaps by utilizing open-source software, providing reusable detection components, and enabling seamless integration as a software module.

To achieve this, established computer vision solutions and the existing testing ecosystem are first analyzed to understand their functionality and to identify desirable properties for *TEaMVision*. Key elements include reusable functional components and GUI-based configuration dialogs with live camera images.

The operating concept of *TEaMVision* defines several simple and reusable detection methods, which can be configured via *Live-Preview dialogs* and immediately display results within a live camera image. *TEaMVision* can be fully integrated into existing programming environments, allowing further processing of results and enabling the implementation of complex applications. During implementation, the core aspects of performance, stability, and extensibility ensure that the module can be used effectively in productive environments.

Subsequently, several real-world inspection applications in industrial manufacturing are successfully implemented using *TEaMVision*. Thanks to the *Live-Preview dialogs*, even users without extensive programming knowledge or expertise in computer vision can quickly and successfully implement various detection tasks. The performance of the solution exceeds expectations: *TEaMVision* enables applications that could not be realized even with expensive commercial vision systems.

In the field of computer vision, there is often an excessive focus on improving recognition of increasingly complex features. In doing so, the practical applicability of these mechanisms is frequently overlooked. This thesis demonstrates convincingly that even very simple and long-established methods can cover many practical use cases. The primary added value lies not in the detection itself, but in its thoughtful integration into existing systems. The *TEaMVision* components empower a broad range of users to independently develop their own applications, significantly increasing development capacity and productivity.

While this thesis focuses on vision systems, the underlying problem is a general one. Through well-designed software engineering involving modular components, open interfaces, and user-oriented operation, core functionality can be made accessible to a wide audience of non-experts. This approach has proven successful in this context and can be transferred to many other domains.

Zusammenfassung

Diese Arbeit beschäftigt sich mit der Entwicklung und Implementierung einer Computer-Vision Lösung für den Einsatz in automatisierten Prüfsystemen. Dafür wird das Softwaremodul *TEaMVision* erstellt, das sich in ein bestehendes Testing-Ökosystem auf Basis der Software *TestStand* einfügt.

Beim Industrieunternehmen Festo sollen manuelle Sichtprüfungen durch den Einsatz von kamerabasierten Systemen, so genannten *Vision-Systemen*, automatisiert werden. Die großflächige Implementierung von Vision-Systemen scheiterte aber immer wieder an hohen Kosten, großen Entwicklungsaufwänden und den mangelnden Schnittstellen für die Umsetzung komplexer Anwendungen. TEaMVision schließt diese Lücken durch den Einsatz freier Software, das Bereitstellen wiederverwendbarer Erkennungsbausteine und die Integrierbarkeit als Softwaremodul.

gut benannt

Aber ab jetzt wird es wieder schwulstig

Dafür werden zuerst etablierte Vision-Lösungen und das bestehende Testing-Ökosystem analysiert, um ihre Funktionsweise zu verstehen und wünschenswerte Eigenschaften für TEaMVision zu identifizieren. Wesentliche Bestandteile sind wiederverwendbare Funktionsbausteine und GUI-Konfigurationsdialoge mit Kameralivebild.

Das Bedienkonzept für TEaMVision definiert daraufhin mehrere einfache und wiederverwendbare Erkennungsmethoden, die mit Live-Preview Dialogen konfiguriert werden und sofort ein Ergebnis im Livebild der Kamera anzeigen. TEaMVision ist voll integrierbar in bestehende Programmierumgebungen, um dort Ergebnisse weiterverarbeiten und so auch komplexe Applikationen umsetzen zu können. Bei der Implementierung stellen die Kernaspekte Performance, Stabilität und Erweiterbarkeit sicher, dass das Modul effektiv und produktiv eingesetzt werden kann.

Anschließend werden mit TEaMVision erfolgreich mehrere reale Prüfapplikationen in der industriellen Fertigung umgesetzt. Durch die Live-Preview Dialoge können auch Anwender ohne große Programmierkenntnisse und Fachwissen zu Computer Vision schnell und erfolgreich verschiedene Erkennungen umsetzen. Die Leistungsfähigkeit der Lösung ist besser als erwartet, mit TEaMVision können erfolgreich Applikationen umgesetzt werden, die mit teuren kommerziellen Vision-Systemen nicht abbildbar waren.

Beim Thema Computer Vision wird meist ein zu großer Fokus auf die immer bessere Erkennung neuer, noch komplizierterer Merkmale gelegt. Dabei wird oft vernachlässigt, wie diese Mechanismen tatsächlich in der Realität eingesetzt werden können. Diese Arbeit zeigt eindrucksvoll, dass bereits sehr einfache und lange etablierte Methoden viele praktische Anwendungen abdecken können. Der große Mehrwert wird dabei nicht von der Erkennung generiert, sondern durch die durchdachte Integration in bestehende Systeme. Die TEaMVision-Bausteine befähigen einen großen Kreis von Anwendern, ihre Applikationen selbständig zu entwickeln. Dadurch werden Entwicklungskapazität und Produktivität massiv erhöht.

Das Thema dieser Arbeit sind Vision-Systeme. Die Problemstellung ist jedoch eine allgemeine. Durch geschicktes Software-Engineering mit modularen Komponenten, offenen Schnittstellen und Anwenderorientierter Bedienung wird eine Kernfunktionalität einem breiten Kreis von Nicht-Experten zugänglich gemacht. Dieses Vorgehen führte hier zum Erfolg und lässt sich auch auf viele andere Themengebiete übertragen.

Inhaltsverzeichnis

Sperrvermerk	I
Erklärung zur Eigenleistung	II
Abstract	III
Zusammenfassung	IV
Inhaltsverzeichnis	V
Abkürzungen	IX
Abbildungsverzeichnis	X
1 Einleitung	1
1.1 Umfeld	1
1.2 Prüfsysteme	2
1.3 Kamerabasierte Prüfungen	3
1.4 Standardisierte Testinfrastruktur	5
1.5 TestStand & TEaM-Framework	5
1.6 Motivation und Zielsetzung	6
1.7 Aufbau der Arbeit	7
2 Vision-Systeme	8
2.1 Aufbau von Vision-Systemen	8
2.1.1 Kameras	8
2.1.2 Software	9
2.1.3 Kamerasysteme	12
2.1.4 Applikationslösungen	14
2.1.5 Fazit	15
2.2 Vergleichbare Themen in der Wissenschaft	16
2.3 Methoden der Bildverarbeitung	17
2.3.1 Bildkanäle & Farben	18
2.3.2 Quantisierung	19
2.3.3 Kanten und Konturen	19
2.3.4 Vorverarbeitung	19
2.3.5 Klassifizierung	21
2.4 Zusammenfassung	22
3 Testing Ökosystem	23
3.1 TestStand	23
3.1.1 Sequenzen	24
3.1.2 Variablen	24
3.1.3 Step Types	25

3.2	TEaM-Framework	27
3.2.1	Funktionen.....	27
3.2.2	Parametrierung über Edit-Dialoge	28
3.2.3	DLL-Aufbau.....	31
3.2.4	Standardisierte Prüfberichte	31
3.2.5	Report Viewer & User Interface TEaMView.....	32
3.3	Fazit.....	32
4	Entwurf der Anwendung	34
4.1	Anforderungen	34
4.1.1	A1 – Erkennungen.....	34
4.1.2	A2 – Applikationen	35
4.1.3	A3 – Bedienung.....	35
4.1.4	A4 - Nichtfunktionale Anforderungen	36
4.2	Bedienkonzept.....	36
4.2.1	Live-Preview Dialoge	36
4.2.2	Kameraintegration	37
4.2.3	Bildhandhabung	38
4.2.4	Erkennungsmethoden	39
4.2.5	Image Message Box	41
4.2.6	Datenstrukturen	41
4.3	Beschreibung der Erkennungsmethoden.....	42
4.3.1	FindShapes	42
4.3.2	TemplateMatching	46
4.3.3	DmtxDetection	48
4.3.4	TextDetection	49
4.4	Softwarearchitektur	50
4.4.1	Integration in TestStand	50
4.4.2	Datentypen	51
4.4.3	Image Handles.....	51
4.4.4	Kameras.....	51
4.4.5	Erkennungsmethoden	52
4.4.6	Vorschau-Dialoge.....	54
4.4.7	ShowImageWindow	56
4.4.8	Gesamtarchitektur	57
4.5	Programmablauf.....	57
4.5.1	Kamera konfigurieren	58
4.5.2	Erkennung konfigurieren.....	59

4.5.3	Auswahl Bildbereich im Preview Dialog.....	60
4.5.4	Laufzeit: Testprogramm	61
4.6	Zusammenfassung	64
5	Verwendete Technologien.....	65
5.1	WPF.....	65
5.2	OpenCV	66
5.3	Kameratreiber	66
5.4	Dmtx-Erkennung	67
5.5	Tesseract-OCR	67
6	Implementierung	68
6.1	TEaMVision	68
6.1.1	Statische Funktionen	68
6.1.2	Interaktion mit TestStand Context	69
6.1.3	Laden von weiteren Abhängigkeiten.....	70
6.2	ImageHandle	70
6.3	Camera	71
6.3.1	FileCamera	72
6.3.2	BaumerCamera.....	73
6.3.3	Kameraparameter	74
6.4	FindShapes	75
6.4.1	Maskenbildung	75
6.4.2	Konturen in Maske zerschneiden	77
6.4.3	Erkennung in Maske.....	79
6.4.4	Dynamische Intensitätsgrenzwertermittlung.....	80
6.5	Template Matching	81
6.5.1	Vorverarbeitung	81
6.5.2	Template Matching	81
6.6	DmtxDetection	83
6.7	Preview-Dialoge.....	84
6.7.1	Definition GUI-Elemente	84
6.7.2	Dynamische GUI-Elemente	85
6.7.3	Bildanzeige.....	86
6.8	Zusammenfassung	89
7	Praxiseinsatz & Anwender-Feedback	90
7.1	Entwicklung Testapplikation für Fixdrosseln in der VTUX-Montage	90
7.1.1	TestStand Kernprogramm	90
7.1.2	Integration in Produktionslinie	93

7.1.3	Fazit	94
7.2	Überprüfung des Bauzustands von VTUX-Ventilinsel.....	95
7.2.1	Erkennung der Merkmale	95
7.2.2	Testapplikation	96
7.2.3	Erkennung weiterer Merkmale	97
7.2.4	Zukünftiges Projekt - DataMatrix Massenscanner	100
7.2.5	Benutzerfreundlichkeit und Anwenderfeedback	101
7.3	Integration TEaMVision in bestehendes Prüfsystem	102
7.4	Speicherfragmentierung	103
7.5	Erfüllung der Anforderungen	104
7.5.1	A1 – Erkennungen.....	104
7.5.2	A2 – Applikation	105
7.5.3	A3 – Bedienung.....	105
7.5.4	A4 - Nichtfunktionale Anforderungen	106
7.6	Fazit	106
8	Reflexion und Ausblick.....	107
8.1	Diskussion der Ergebnisse	107
8.2	Erweiterungspotenzial	107
8.3	Fortführung des Projekts	108
8.4	Fazit	108
9	Literaturverzeichnis.....	109

Abkürzungen

TEaM	Test Environment and Modules
DLL	Dynamic Link Library
API	Application Programming Interface
TDMS	Technical Data Management Streaming (Dateiformat)
TreX	Testreport eXtensible (Dateiformat)
WPF	Windows Presentation Foundation
OpenCV	Open Computer Vision
ROI	Region of Interest
TDI	TEaM Dialog
CAN	Controller Area Network
XAML	Extensible Application Markup Language
DMTX	DataMatrix
KI	Künstliche Intelligenz
HHB	Handhilfsbetätigung
OBB	OrientedBoundingBox

Abbildungsverzeichnis

Abbildung 1: Festo Ventilinsel aus der Baureihe VTUX [Festo SE & Co.KG, 2025]	1
Abbildung 2: Illustration des Aufbaus der Komponenten einer VTUX-Ventilinsel.....	2
Abbildung 3: Explosionszeichnung VTUX-Ventilinsel [Festo SE & Co.KG, 2025]	3
Abbildung 4: Erkennung der Druckluftanschlüsse bei einer Festo-Ventilinsel	4
Abbildung 5: Industriekameras vom Hersteller Baumer [Baumer GmbH, 2026]	8
Abbildung 6: Einbinden und Konfigurieren von Kamera in HDevelop [MVTec, 2026]	9
Abbildung 7: MVTec HDevelop Oberfläche [MVTec, 2026]	10
Abbildung 8: Keyence All-In-One Kamerasystem [Keyence, 2026].....	12
Abbildung 9: Keyence Entwicklungsumgebung für All-In-One Kamerasystem.....	12
Abbildung 10: a) Originalbild, b) Graustufenbild, c) RGB Rot, d) RGB Grün, e) RGB Blau, f) HSV Hue, g) HSV Saturation, h) HSV Value.....	18
Abbildung 11: Ergebnis Binarisierung Farbkanal eines HSV-Bildes	19
Abbildung 12: a) Originalbild, b) Dilation, c) Erosion [OpenCV, 2026]	20
Abbildung 13: a) Erosion gefolgt von Dilation, b) Dilation gefolgt von Erosion [OpenCV, 2026]	20
Abbildung 14: a) Bild mit Rauschen [He, 2008], b) Bild mit Unschärfe-Filter	20
Abbildung 15: Ergebnis Laplace-Filter angewandt auf Abbildung 14 a) und b).....	21
Abbildung 16: Screenshot der TestStand Programmierungsumgebung	23
Abbildung 17: Post Action Menü eines TestStand Steps.....	24
Abbildung 18: TestStand Variablen	25
Abbildung 19: Modulkonfiguration eines TestStand Action-Steps	26
Abbildung 20: TEaM Edit-Dialog für den Step Type ENGT_ReadParameter.....	28
Abbildung 21: Buttons zum Öffnen des Edit-Dialogs	28
Abbildung 22: Pre- und Post-Expression eines TEaM-Modul Schrittes.....	29
Abbildung 23: TDI Auszug für den Edit Dialog zum Step Type ENGT_ReadParameter.....	29
Abbildung 24: TestStand Substep-Menü eines Step Types	30
Abbildung 25: Funktionsaufruf teamdiag.dll für die Konfiguration eines Edit-Substeps	30
Abbildung 26: TEaMView Benutzeroberfläche mit TDMS-Bericht	32
Abbildung 27: Live Preview Button neben Edit Dialog Button	36
Abbildung 28: Moduldefinition Substep für Live-Preview Dialog	37
Abbildung 29: Edit-Dialog InitCamera	37
Abbildung 30: Live-Preiew Dialog InitCamera	38
Abbildung 31: Image Handle Step Types	38
Abbildung 32: Edit-Dialog Erkennungsmethode	39
Abbildung 33: Live-Preview Dialog FindShapes	40
Abbildung 34: ShowImageMessageBox Step Types	41
Abbildung 35: TestStand Datentypen für Kameraeinstellungen.....	41
Abbildung 36: TestStand Datentyp für Erkennungsergebnis.....	42
Abbildung 37: Struct Passing Einstellungen von TestStand Datentyp	42
Abbildung 38: FindShapes Live Preview mit Binärmasken Ansicht.....	43
Abbildung 39: Ausschnitt FindShapes Live Preview mit variabler Intensität	44
Abbildung 40: Ausschnitt FindShapes Live Preview mit aktivem Outline Modus	45
Abbildung 41: Ausschnitt FindShapes Preview Dialog mit Farbmodus für Maskenbildung ..	45
Abbildung 42: Live-Preview für Template Matching.....	46

Abbildung 43: Template Matching Live Preview mit Preprocessing Ansicht.....	47
Abbildung 44: Datenstrukturen für Einstellungen und Ergebnisse von TemplateMatching ...	47
Abbildung 45: Live Preview Dialog von DmtxDetection.....	48
Abbildung 46: Maskenansicht des Live-Preview Dialogs von DmtxDetection	48
Abbildung 47: Live Preview Dialog von TextDetection	49
Abbildung 48: Klasse TEaMVision	50
Abbildung 49: Klasse ImageHandle	51
Abbildung 50: Klassendiagramm Camera und ihre Implementierungen	52
Abbildung 51: Erkennungsklassen FindShapes, TemplateMatching, DmtxDetection, TextDetection	53
Abbildung 52: Klassendiagramm Vorschaudialoge.....	54
Abbildung 53: GUI-Element RoiSettingsControl	55
Abbildung 54: GUI-Element PreviewBaseControl.....	55
Abbildung 55: Klassendiagramm von ShowImageWindow	56
Abbildung 56: Klassendiagramm aller Hauptkomponenten von TEaMVision	57
Abbildung 57: Programmablauf Camera Live Preview	58
Abbildung 58: Programmablauf Live Preview einer Erkennung.....	59
Abbildung 59: Programmablauf Bildbereich in Vorschaubild auswählen.....	60
Abbildung 60: Programmablauf Init-Bereich Testprogramm	61
Abbildung 61: Programmablauf Main-Bereich Testprogramm	62
Abbildung 62: Programmablauf Cleanup-Bereich Testprogramm	63
Abbildung 63: Programmablauf in TestStand.....	63
Abbildung 64: XAML Beispielcode	65
Abbildung 65: Eventhandler in C#.....	66
Abbildung 66: Code zum Einsetzen von Tesseract-OCR	67
Abbildung 67: Codebeispiel statische Funktion von Klasse TEaMVision	68
Abbildung 68: Codebeispiel Auflösen von TestStand Pre-Expression in TEaMVision.....	69
Abbildung 69: DLL-Suchpfade ergänzen in TEaMVision	70
Abbildung 70: Implementierung des Multiton-Patterns bei ImageHandle	71
Abbildung 71: Methode CreateCamera der Klasse Camera	72
Abbildung 72: Implementierung der Kameravariante FileCamera.....	72
Abbildung 73: GetImageInternal von BaumerCamera	73
Abbildung 74: Teil der Parameterdefintion von BaumerCamera	74
Abbildung 75: Schematischer Ablauf der Methode Run von FindShapes.....	75
Abbildung 76: Bildung der Intensitätsmaske in FindShapes	75
Abbildung 77: Bildung der Farbmaske in FindShapes	76
Abbildung 78: Nachverarbeitung der Binärmaske in FindShapes	77
Abbildung 79: Methode SplitContours von FindShapes.....	77
Abbildung 80: Methode CreateDepthProfiles von FindShapes	78
Abbildung 81: Effekt von SplitContours auf die Binärmaske in FindShapes.....	79
Abbildung 82: Methode FindCircles von FindShapes	79
Abbildung 83: Ablauf der dynamischen Intensitätsgrenzwertermittlung in FindShapes	80
Abbildung 84: Vorverarbeitungsmethode von TemplateMatching	81
Abbildung 85: Methode MatchTemplate von TemplateMatching.....	82

Abbildung 86: Vergleich Template Matching Ergebnisse vor und nach Ermittlung lokaler Maxima.....	82
Abbildung 87: Vorverarbeitung für DataMatrix-Code Erkennung.....	83
Abbildung 88: XAML-Definition der PreviewBaseControl Oberfläche	84
Abbildung 89: XAML-Definition der GUI von FindShapesPreview	84
Abbildung 90: XAML-Definition der GUI von ShowImageWindow	85
Abbildung 91: Methode AddParameterControls von ConfigureCameraLivePreview.....	85
Abbildung 92: Initialisierung PreviewBaseControl mit leerem WriteableBitmap	86
Abbildung 93: Funktion ConvertMatData von PreviewBaseControl	87
Abbildung 94: Methode DrawImageInternal von PreviewBaseControl	87
Abbildung 95: Konstruktor von FindShapesPreview.....	88
Abbildung 96: Illustration Fixdrosseleinsätze für VTUX-Ventilinselfamilie.....	90
Abbildung 97: TestStand Programmablauf Fixdrosselerkennung	90
Abbildung 98: Ansicht Bedienerhilfe Fixdrosselmontage	91
Abbildung 99: Programmablauf zum Gruppieren von Shapes in Spalten	92
Abbildung 100: Programmablauf zur Entfernung von falsch positiven Ergebnissen	92
Abbildung 101: Fixdrosselerkennung in VTUX-Montagelinie	93
Abbildung 102: Ergebnisbild Vorderseite VTUX-Bauzustand	95
Abbildung 103: Ergebnisbild Oberseite VTUX-Bauzustand.....	95
Abbildung 104: Ergebnisbild VTUX-Soll-Bauzustand ohne Ventilinsel	96
Abbildung 105: SplitContours Problem	97
Abbildung 106: Erkennung Hutschienelemente.....	97
Abbildung 107: Erkennung Klemmbügel	98
Abbildung 108: Erkennung Leerplatten	98
Abbildung 109: Erkennung Flächenschalldämpfer	98
Abbildung 110: Erkennung Schilderträger.....	99
Abbildung 111: Erkennung Varianten Verkettungsplatten	99
Abbildung 112: Erkennung Platinenvarianten	99
Abbildung 113: Einscannen Dmtx-Codes mit Handscanner und Schablone	100
Abbildung 114: Erkennung aller DMTX-Codes auf einer Ventilinsel	100
Abbildung 115: Ergebnisbild PCFU-Prüfstand mit Klassifizierung der Anschlussvarianten	102
Abbildung 116: Ergebnisbild PCFU-Prüfstand mit DMTX-Erkennung.....	102
Abbildung 117: Prüfstand zum Test des Kontrollmoduls PCFU	103

1 Einleitung

In der Industrie und im produzierenden Gewerbe dienen Automatisierung und Digitalisierung zunehmend als Hebel, um steigendem Preisdruck und gleichzeitig zunehmenden Qualitätsanforderungen gerecht werden zu können. Essenziell für die Qualität der hergestellten Kundenprodukte ist dabei das ausführliche Testen in der Produktion. Beim Unternehmen Festo werden dafür schon lange automatisierte Prüfstände eingesetzt, die manuelle Sichtprüfung durch einen Menschen ist in vielen Fällen dennoch weiterhin ein notwendiger Schritt.

Diese Sichtprüfungen sollen nun durch den Einsatz von kamerabasierten Systemen, so genannten *Vision-Systemen*, automatisiert werden. Dadurch sollen Geschwindigkeit und Produktivität gesteigert, sowie Fehlerquote und Kundenreklamationen reduziert werden.

Diese Arbeit beschäftigt sich mit der Entwicklung und Implementierung einer Vision-Lösung für den Einsatz in automatisierten Prüfsystemen. Dafür soll ein Softwarebaustein erstellt werden, der sich in das bestehende Ökosystem für Testautomatisierung einfügt. Darüber hinaus soll die neue Lösung in bestehende Produktionslinien integriert und ihre Tauglichkeit anhand von Testergebnissen und Anwenderfeedback bewertet werden.

gute Einleitung

1.1 Umfeld

Das Unternehmen Festo ist einer der führenden Anbieter von Komponenten für die Fabrikautomatisierung. Festo ist einer der Begründer der pneumatischen Automatisierungstechnik, seit den 1950er Jahren entwickelt und produziert Festo Zylinder, Ventile, und weitere Komponenten. 1989 bringt Festo die weltweit erste Ventilinsel auf den Markt. Heute ist Festo Weltmarktführer im Bereich Pneumatik und Fabrikautomatisierungstechnik. [Festo SE & Co.KG, 2025]

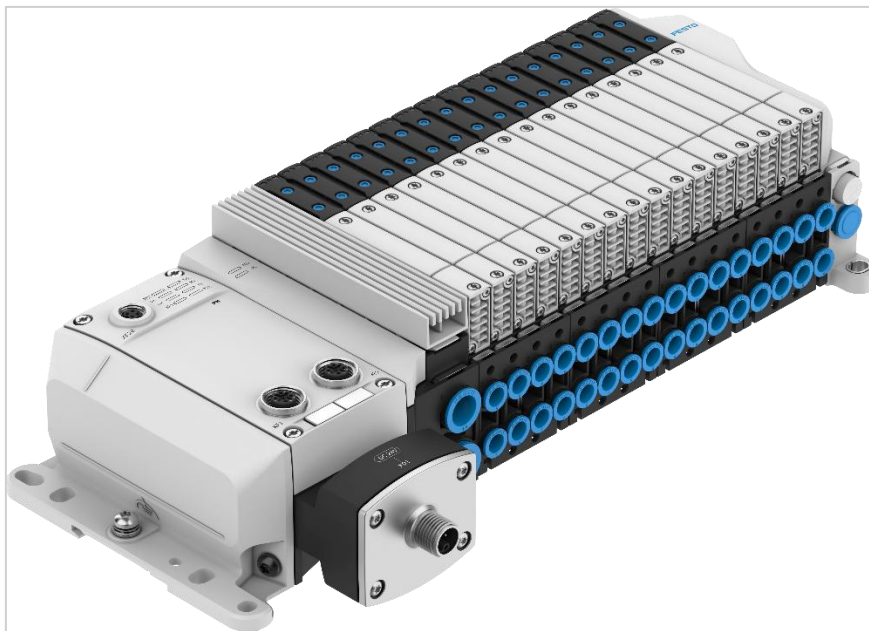


Abbildung 1: Festo Ventilinsel aus der Baureihe VTUX [Festo SE & Co.KG, 2025]

Abbildung 1 zeigt eine beispielhafte Festo Ventilinsel mit 16 Ventilen. In einer Ventilinsel sind mehrere elektromagnetisch angesteuerte Ventile zu einer Einheit verbunden. Die Ventile werden durch eine zentrale Verteilerplatte mit Druckluft versorgt und von einer zentralen Steuereinheit betätigt. Die Art, Anzahl und Anordnung der Ventile sind variabel. Neben Ventilen können auch andere Komponenten wie Vakuumsaugdüsen montiert werden. Vorteile von Ventilinseln sind unter anderem die kompakte Bauweise, der geringe Verdrahtungsaufwand und die zentrale Ansteuerung aller Ventile.

Abbildung 2 zeigt den modularen Aufbau einer Festo Ventilinsel. Sie besteht aus mehreren so genannten *Verkettungsplatten*, auf denen Ventile oder andere Komponenten montiert werden. Die Verkettungsplatten leiten elektrische Signale und Druckluft zu jedem Ventil und weiter zur nächsten Verkettungsplatte.

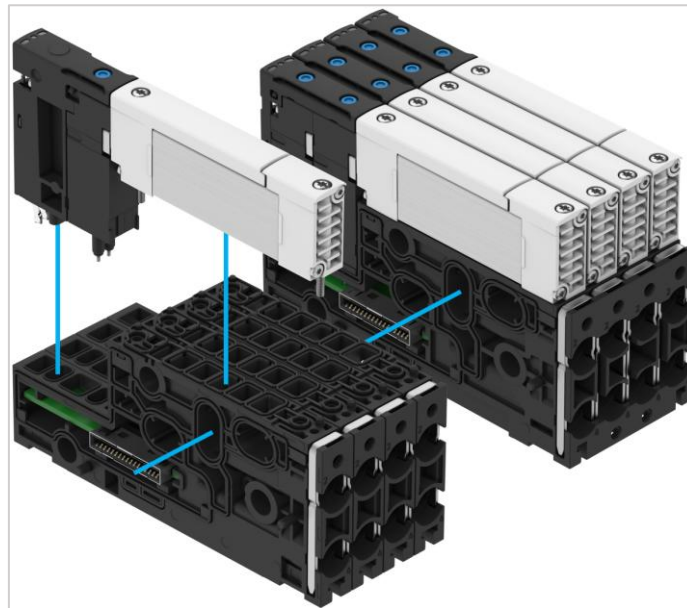


Abbildung 2: Illustration des Aufbaus der Komponenten einer VTUX-Ventilinsel

Der Aufbau der Ventilinsel kann vom Kunden beliebig konfiguriert werden. Auch Inseln mit über 100 Einzelmodulen sind möglich. Daraus und aus der Vielzahl an möglichen Komponenten, ergibt sich eine Anzahl von möglichen Kombinationen in der Größenordnung 10^{100} .

Neben der Pneumatik führt Festo auch ein Portfolio an Greifer- und Handlingsystemen, elektrischer Antriebs- und Steuerungstechnik, sowie Sensorik. Zusätzlich ist Festo mit dem Bereich Didactic Weltmarktführer im Feld der technischen Aus- und Weiterbildung. Heute erzielt Festo mit rund 20.600 Mitarbeitern einen Umsatz von 3,45 Mrd. € und bedient mit 36.000 Katalogprodukten über 300.000 Kunden in 176 Ländern. [Festo SE & Co.KG, 2025]

1.2 Prüfsysteme

Qualität und Zuverlässigkeit eines Produktes werden zunächst sichergestellt durch geeignete Prozesse und Methoden während der Entwicklung. Prüflinge aus jeder Entwicklungsiteration durchlaufen ausführliche Tests, dadurch kann aber nur die korrekte Funktion weniger Versuchsmuster bewertet werden.

In der industriellen Produktion werden oft mehrere hundert Exemplare täglich gefertigt. Jedes einzelne Exemplar durchläuft dabei einen Prüfprozess, um sicherzustellen, dass kein Materialfehler vorliegt und das Produkt korrekt montiert wurde. Dabei wird unterschieden zwischen *In-Line* (In der Linie) und *End-of-Line* (Am Ende der Linie) Tests. Kleinere Komponenten wie ein einzelnes Ventil laufen nach der Montage nur durch einen End-of-Line Prüfstand. Komplexere Produkte wie eine ganze Ventilinsel werden in mehreren Schritten gefertigt.

Am Ende verschiedener Schritte werden bestimmte Merkmale überprüft, denn manche Merkmale können später im fertig montierten Zustand nicht mehr geprüft werden. Zusätzlich sollen Fehler so früh wie möglich erkannt werden (*Fail-early* Prinzip), um sie auch frühestmöglich wieder zu beheben.

Zum Beispiel werden zwischen den Verkettungsplatten einer Ventilinsel teilweise Verschlussstopfen eingesetzt, um in einer Ventilinsel mehrere getrennte Druckbereiche zu realisieren. Abbildung 3 zeigt die Vielzahl von Einzelteilen, die bei der VTUX-Produktion mehrheitlich von Hand montiert werden. An Pos. 16 sind beispielhaft die Verschlussstopfen zum Abtrennen von Druckbereichen zu sehen.

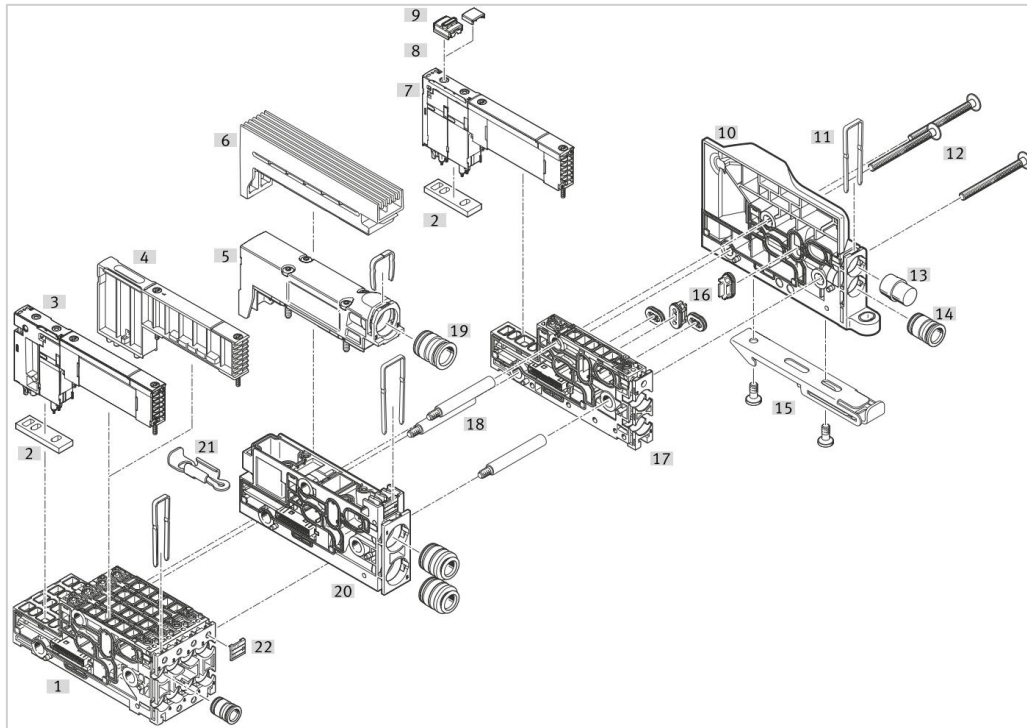


Abbildung 3: Explosionszeichnung VTUX-Ventilinsel [Festo SE & Co.KG, 2025]

In den Prüfsystemen wird verschiedenste hochentwickelte pneumatische und elektrische Steuerungs- und Messtechnik eingesetzt. Bei einer Ventilinsel wird jeder Ventilplatz mit Luftdruck beaufschlagt und jedes Ventil mehrmals angesteuert. Dabei werden Parameter wie Funktion, Schaltzeit und Dichtheit/Leckage ermittelt. Erst wenn die Insel alle vorgegebenen Grenzwerte einhält, wird sie für die Auslieferung freigegeben.

In einem detaillierten Prüfbericht werden Informationen zum Prüfling, sowie alle Testergebnisse mit Soll- und Istwerten festgehalten. Der Prüfbericht wird in einer zentralen Datenbank abgelegt und archiviert. Somit ist auch nach Jahren noch eine Rückführbarkeit möglich. Zusätzlich werden über diese Daten Statistiken und Berichte generiert, die in der Produktionssteuerung und in der Qualitätssicherung verwendet werden.

1.3 Kamerabasierte Prüfungen

Die pneumatischen und elektrischen Prüfungen erreichen im End-of-Line Test eine hohe Testabdeckung. Viele Merkmale können im In-Line Betrieb mit diesen Methoden allerdings nicht geprüft werden. Ein falsch eingesetzter oder fehlender Verschlussstopfen zwischen Ventilblöcken würde zum Beispiel in der fertig montierten Insel erkannt, weil an bestimmten Ventilen der falsche oder kein Druck anliegt. In der Linie ist allerdings keine pneumatische Prüfung möglich, eben weil die Insel noch nicht fertig montiert ist. Andere Merkmale, wie ein fehlender Schalldämpfer, werden auch im Endtest nicht automatisch erkannt. Dieser fällt höchstens dem Mitarbeiter aufgrund der erhöhten Geräuschentwicklung auf.

Somit sind verschiedene Schritte weiterhin abhängig von der manuellen Sichtprüfung durch den Mitarbeiter in der Montage. Das ist naturgemäß fehleranfällig. Besonders bei Ventilinseln,

die aus dutzenden gleich aussehenden Ventilplätzen bestehen, kommt der Mensch an seine Grenzen. Wenn bei der Montage die Ventilplätze von Hand abgezählt werden müssen, um einen Einsatz in die richtige Öffnung zu platzieren und dazu noch der in der Fertigung übliche Zeitdruck mit ins Spiel kommt, sind Fehler vorprogrammiert.

Industrielle kamerabasierte Erkennungssysteme sind auf dem Markt schon lange verfügbar und etabliert - Festo hat selbst eine Handvoll spezialisierter Kamerasysteme im Portfolio. In der Produktion bei Festo werden an verschiedenen Stellen schon länger Kamerasysteme eingesetzt, somit ist bereits einiges an Erfahrung und Know-how im Unternehmen vorhanden. Dennoch sind die Vision-Methoden bei Festo noch lange nicht so weit verbreitet und etabliert wie die pneumatische und elektrische Prüftechnik.

Bei den etablierten Vision-Systemen zeichnen sich durchgehend ähnliche Hindernisse ab, die einem breiteren Einsatz im Wege stehen. Die Erkennungsmechanismen funktionieren in der Regel sehr gut, allerdings handelt es sich fast immer um geschlossene Systeme, die sich nicht oder nur schwer in andere Infrastruktur integrieren lassen. Die Konfiguration der Erkennung erfolgt in der Regel mit einer proprietären Software des Herstellers. Hier können sehr detailliert eines oder mehrere Merkmale eingestellt werden. Anschließend wird die Erkennung per externem Trigger gestartet und ein gut/schlecht Ergebnis, teilweise zusammen mit einem proprietären Prüfbericht, erzeugt.

Komplexe Applikationen wie das Testen von Ventilinseln erfordern eine dynamische Konfiguration der Erkennung, da sich der Aufbau und die Zusammensetzung des Prüflings mit jedem Ablauf stark unterscheiden können. Aufgrund der Kombinatorik ist es somit auch nicht möglich, eine feste Erkennung pro Produktvariante im Voraus zu konfigurieren.

Ein weiteres Hindernis ist der Preis solcher Kamerasysteme, welcher sich in der Regel im Bereich von mehreren tausend bis mehreren zehntausend Euro bewegt. Dadurch ist der Einsatz nur in großen Systemen mit hoher Auslastung möglich. Für kleinere oder unregelmäßig eingesetzte Systeme ist die Investition meist nicht zu rechtfertigen.

Frei verfügbare Software-Bibliotheken zur Bildverarbeitung existieren, sind jedoch meist ausgerichtet auf eine Zielgruppe mit ausgeprägten Programmierkenntnissen. Die Erkennungen müssen über Grundmechanismen der Bildverarbeitung selbst geschrieben werden, dadurch ist die Einstiegshürde für die meisten potenziellen Anwender bei Festo zu hoch.



Abbildung 4: Erkennung der Druckluftanschlüsse bei einer Festo-Ventilinsel

Als Alternative zu den kommerziellen Vision-Systemen wurden in einem Vorprojekt Erkennungen auf Basis frei verfügbarer Softwarebibliotheken entwickelt. Das Prüfprogramm kann für eine Handvoll von Ventilinsel-Konfigurationen der Baureihe MPA erkennen, ob alle Druckluftanschlüsse und alle Ventile korrekt montiert wurden. Abbildung 4 zeigt ein Ergebnisbild dieses Prüfprogramms. Die Lösung existiert allerdings als eigenständiger, isolierter Prüfstand, der nicht in das weitere Ökosystem integriert ist.

1.4 Standardisierte Testinfrastruktur

Festo hat viele verschiedene Entwicklungsbereiche und Produktionsstandorte. Die Entwicklung ist unterteilt in verschiedene Geschäftsbereiche, wie die elektrische Automatisierungstechnik, Pneumatische Antriebe, Pneumatische Kontrolltechnik oder Prozessautomatisierung. In jedem Geschäftsbereich gibt es eigene Versuchsabteilungen, die sich mit der entwicklungsbegleitenden Prüfung ihrer eigenen Produkte beschäftigen. Diese Struktur zeichnet sich auch in den Produktionsstätten ab, wo Verantwortliche für einzelne Produktgruppen sich primär mit der Herstellung von Produkten aus einem oder nur wenigen der Geschäftsbereiche beschäftigen. Somit gibt es an verschiedenen Punkten im Unternehmen viele Verantwortliche für Prüfsysteme, die nicht immer mit den Verantwortlichen aus anderen Geschäftsbereichen kommunizieren oder zusammenarbeiten.

Deshalb gibt es parallel zu den Geschäftsbereichen einen Zentralbereich, in dem für die Produktentwicklung unternehmensweit relevante Funktionen gebündelt sind. Eine Abteilung in diesem Zentralbereich ist verantwortlich für die Entwicklung, Verwaltung und Bereitstellung von Versuchsinfrastruktur, die bereichsübergreifend im Unternehmen eingesetzt wird. Das umfasst Standards für Prozesse, Hardware und Software. So wird Mehrfachentwicklung vermieden und Wissen gebündelt. Ohne den Zentralbereich hätten die einzelnen Versuchsbereiche einen schlechten Überblick darüber, welche anderen Bereiche sich bereits mit einer Technologie auseinandergesetzt haben. Ein Softwaremodul, das im Zentralbereich zunächst für einen Anwendungsfall entwickelt wird, kann später auch für weitere Projekte in anderen Bereichen eingesetzt werden. Dabei achtet der Zentralbereich bereits bei der ersten Entwicklung darauf, dass das Modul generisch genug ausgelegt und kompatibel mit dem restlichen Ökosystem ist. Es wird gewährleistet, dass auch über die Dauer eines einzelnen Projektes hinaus ein Ansprechpartner vorhanden bleibt, der dediziert beim Einsatz der Technologie unterstützen kann. So können sich die Versuchsbereiche in Entwicklung und Produktion auf die Realisierung von Prüfapplikationen konzentrieren und müssen keine Ressourcen für Grundlagenarbeit investieren. Gleichzeitig kann durch die Wiederverwendung von Lösungen in späteren Projekten ein großer Mehrwert bei kleinem Aufwand geschaffen werden.

1.5 TestStand & TEaM-Framework

Die Entwickler von Prüfsystemen sind oft Ingenieure oder Techniker, keine Informatiker oder Softwarespezialisten. Deshalb wurde durch den Zentralbereich bei Festo die Software *TestStand*¹ (siehe Kapitel 3.1) als Programmierungsumgebung für Prüfsysteme eingeführt. TestStand ist eine blockbasierte Programmiersprache, in der durch die Aneinanderreihung und Parametrierung von vorkonfigurierten Programmbausteinen schnell mächtige Prüfprogramme erstellt werden können. TestStand bietet dafür Adapter zum Einbinden von Softwaremodulen, die in verschiedenen Sprachen, wie C, C# oder Python, erstellt wurden.

Zur Erweiterung von TestStand wurde vom Zentralbereich das *TEaM-Framework* (Test Environment and Modules) entwickelt. Das Framework bietet eine große Menge von Programmbausteinen für TestStand, die so genannten *TEaM-Module*, mit denen zum einen die breite Palette elektrischer und pneumatischer Standard-Prüftechnik eingebunden und genutzt werden kann. Dazu kommen Bausteine zur Datenanalyse, Erstellung standardisierter Prüfberichte und Anbindung an relevante IT-Systeme sowie Beispielpprogramme und eine ausführliche Dokumentation.

¹ NI TestStand ist eine Softwareumgebung für die Erstellung und Bereitstellung von Testsystemen. [NI, 2025]

1.6 Motivation und Zielsetzung

Etablierte kommerzielle Vision-Systeme sind zum einen teuer und gleichzeitig nur eingeschränkt für die Anwendungsfälle bei Festo geeignet. Neben dem Einsatz von fertig verfügbaren Vision-Systemen wurden bereits eigenentwickelte Applikationen auf Basis von Open-Source Software erprobt. Die selbst entwickelten Erkennungen haben sich für die zuverlässige Erkennung erster Merkmale als geeignet erwiesen. Deshalb soll die Entwicklung einer eigenen Vision-Lösung fortgesetzt werden.

An vielen Stellen tun sich immer wieder Anwendungsfälle für kleine, sowie große und komplexe Vision-Applikationen auf. Deshalb soll die Vision-Lösung für kleine Anwendungen mit sehr niedrigem Initialaufwand einsetzbar sein und gleichzeitig auch große Applikationen mit hoher Komplexität unterstützen.

Idealerweise sollen Anwender im ganzen Unternehmen befähigt werden, kamerabasierte Erkennungen ohne tiefes technisches Know-How und ohne große finanzielle Aufwände in ihre Prüfsysteme zu integrieren.

Ziel dieser Arbeit ist die Entwicklung eines Softwaremoduls für Vision-Applikationen, das als TEaM-Modul in das bestehende Ökosystem aus TestStand und TEaM-Framework integriert wird. Das Modul soll verschiedene Kameratypen einbinden und auf den Kamerabildern visuelle Erkennungen ausführen können.

Die Konfiguration und Bedienung der Funktionen soll einfach und intuitiv sein, damit auch Anwender ohne große Programmierkenntnisse mit dem Modul visuelle Prüfungen umsetzen können. Aufbau und Bedienung des Moduls sollen sich an den existierenden TEaM-Modulen orientieren.

Zunächst sollen Erkennungen für die Anwendungsfälle in der Ventilinselmontage umgesetzt werden. Die Lösung soll aber so generisch gestaltet sein, dass sie auch für andere Anwendungsfälle einsetzbar ist. Um auch komplexe Anwendungen zu ermöglichen, sollen die Erkennungsergebnisse in TestStand weiter verarbeitbar sein.

Der Anspruch ist nicht, im Rahmen dieser Arbeit die volle Leistungsfähigkeit und den Funktionsumfang eines kommerziellen Vision-Systems zu erreichen. Deshalb sollen zunächst Erkennungen umgesetzt werden, mit denen sich ein großer Mehrwert bei überschaubarem Aufwand generieren lässt. Wichtig ist, dass einfache Merkmale zuverlässig erkannt werden und die Erkennungen performant genug für den Einsatz in einem zeitsensitiven Produktionsumfeld sind. Die Softwarearchitektur des Moduls soll so aufgebaut sein, dass weitere Funktionen, Erkennungen und Kameratypen möglichst einfach ergänzt werden können.

Mit dem Modul soll eine Pilotapplikation in der Ventilinselmontage umgesetzt werden. Die Leistungsfähigkeit und Zuverlässigkeit der Erkennungen soll anhand von Testergebnissen bewertet werden. Die Benutzerfreundlichkeit soll anhand von Anwenderfeedback beurteilt und die Allgemeingültigkeit der Erkennungen mit der Betrachtung weiterer möglicher Einsatzzwecke beleuchtet werden.

1.7 Aufbau der Arbeit

In diesem Kapitel wurden das Umfeld und der Kontext dieser Arbeit eingeführt. Die Problemstellung, Motivation und Zielsetzung wurden initial abgesteckt.



Kapitel 2 betrachtet die verschiedenen Komponenten und Arten von Vision-Systemen, analysiert ihre Eigenschaften, Limitierungen und Vorteile. Anschließend wird Problemstellung dieser Arbeit im Kontext der aktuellen Wissenschaft beurteilt und zuletzt werden einige der Grundmechanismen der Bildverarbeitung vorgestellt, die im Rahmen der Arbeit zum Einsatz kommen.

Kapitel 3 stellt das bestehende Testing-Ökosystem aus TestStand und das TEaM-Framework vor. Dabei wird analysiert, welche Aspekte aus technischer und Anwender-Sicht relevant sind für Entwurf und Implementierung des neuen Softwaremoduls.

In Kapitel 4 werden die Anforderungen an das Softwaremodul detailliert formuliert. Anschließend werden ein Bedienkonzept, mehrere Erkennungsmethoden und die Softwarearchitektur spezifiziert.

Kapitel 5 führt die ausgewählten Technologien für die Umsetzung des Konzepts ein.

Kapitel 6 zeigt die Implementierung des Moduls. Dafür werden einige Schlüsselstellen und die Umsetzung der Erkennungsmethoden vorgestellt.

Kapitel 7 behandelt den Praxiseinsatz des Moduls. Es werden die Pilotapplikationen, die umgesetzten Erkennungen, die Ergebnisse des Piloteinsatzes und das Anwenderfeedback vorgestellt. Damit wird die Erfüllung der Anforderungen und somit die Zielerreichung des Projekts beurteilt.

Kapitel 8 reflektiert den Inhalt der Arbeit, diskutiert die Ergebnisse und gibt einen Ausblick auf mögliche Erweiterungen und die Fortführung des Projekts.

2 Vision-Systeme

Dieses Kapitel betrachtet zunächst die Komponenten, Funktionsweise und Eigenschaften von Vision-Systemen. Dabei erfolgt eine Beurteilung, inwieweit die verfügbaren und etablierten Komponenten für die Zielsetzung dieser Arbeit eingesetzt werden können, welche Limitierungen sie besitzen und welche Eigenschaften für die Entwicklung einer eigenen Lösung übernommen werden sollten. Dafür werden auch aktuelle verwandte Themen aus der Wissenschaft mitbezogen und es wird betrachtet, welche Lösungen hier für ähnliche Problemstellungen vorgeschlagen werden.

Als nächstes werden einige gängige Grundmechanismen der Bildverarbeitung eingeführt und daran erläutert, wie damit visuelle Erkennungen und Erkennungsmethoden umgesetzt werden können.

Eine Erkennung ist ein Programmablauf, der in einem Eingabebild ein spezifiziertes Merkmal detektieren kann. So ein Programmablauf kann als parametrierbare Erkennungsmethode verallgemeinert werden, die mit der gleichen Kombination von Grundmechanismen je nach Parametrierung auch unterschiedliche Merkmale erkennen kann.

2.1 Aufbau von Vision-Systemen

Vision-Systeme im Testing-Kontext erfassen ein oder mehrere Bilder von physischen Objekten, versuchen in diesen Bildern verschiedene zuvor spezifizierte Merkmale zu erkennen und treffen dann anhand dieser Erkennungsergebnisse eine Aussage über den Zustand des betrachteten Objektes. Solche Vision-Systeme können verschiedenste Ausprägungen von Größe, Form und Funktionsumfang aufweisen. Bereits eine Webcam in Verbindung mit einem einfachen Skript auf einem Laptop kann als Vision-System herhalten, Maximalausprägungen könnten zur Überwachung vollautomatisierter Fertigungslinien mit dutzenden Kameras eingesetzt werden.

2.1.1 Kameras

In jedem Fall ist die beschreibende Komponente eines Vision-Systems die Kamera. Für den Einsatz in professionellen Vision-Applikationen gibt es mit den spezialisierten Industriekameras eine eigene Produktkategorie, die von verschiedensten Herstellern bedient wird. Diese Kameras besitzen keinerlei eigene Auswertelogik, sie sind als modulare Komponenten für größere Systeme darauf ausgelegt, ein möglichst gutes Bild mit konsistenter Qualität zu liefern.

Abbildung 5 zeigt einige Kameramodelle des Herstellers Baumer, die Modelle anderer Hersteller haben in der Regel ähnliche Eigenschaften. Diese Industriekameras verfügen typischerweise über leistungsstarke integrierte Mechanismen zum Ausgleich von Belichtung, Farbe und Kontrast, um auch unter veränderbaren Bedingungen möglichst gleichbleibend gute Bilder zu liefern. Die Bildauflösung ist abhängig je nach Modell. Fokus sowie Vergrößerung werden über verschiedene physische Objektive kontrolliert. Die Kameras sind in der Lage, je nach Konfiguration Bilder in verschiedenen Farben, Kodierungen und Repräsentationsformaten zu liefern. Angebunden werden die Kameras üblicherweise per Ethernet-Schnittstelle, teilweise auch per USB. Für die Anbindung via Ethernet unterstützen die meisten Hersteller das standardisierte GigE-Vision Protokoll. Über dieses können Kameras in einem Netzwerk identifiziert, eine Verbindung aufgebaut, Bilder abgerufen und verschiedene Konfigurationsparameter gesetzt werden. [Association for Advanced Automation, 2026]



Abbildung 5: Industriekameras vom Hersteller Baumer
[Baumer GmbH, 2026]

Über die standardisierte API können die Kameras somit aus verschiedenen Programmierumgebungen heraus eingebunden und angesprochen werden. Zusätzlich bieten die verschiedenen Hersteller meist noch eigene Software-Suiten und Treiber mit höherem Funktionsumfang zur Konfiguration ihrer Kameras an. So liefert Baumer zum Beispiel eine Software mit grafischer Oberfläche, die alle Kameras im Netzwerk anzeigt, eine Verbindung per Doppelklick erlaubt und dann direkt ein Livebild anzeigt. In der GUI werden alle Konfigurationsmöglichkeiten der Kamera aufgelistet und bei Veränderung eines Parameters wird das Ergebnis direkt im Bild sichtbar. Auf diese Weise können optimale Einstellungen für eine Applikation ermittelt und anschließend aus dem eigenen Applikationsprogramm per API-Schnittstelle gesetzt werden. Die Kosten für gängige Industriekameras, inklusive Objektiv, liegen üblicherweise im mittleren dreistelligen bis niedrig vierstelligen Euro-Bereich.

2.1.2 Software

Nach der Akquise eines Kamerabildes werden darauf diverse Erkennungen ausgeführt. Die Software hierfür kann auf verschiedenen Zielsystemen laufen, von eingebetteten Systemen und Mikrocontrollern über gängige PCs bis hin zu Cloud-Containern finden alle Varianten in der Praxis Anwendung. Die eingesetzte Basissoftware zur Implementierung von Bildverarbeitung und Erkennungen unterscheidet sich zwischen den verschiedenen Zielsystemen nicht wesentlich. Viele der dominanten Softwarebibliotheken sind auf Embedded-, Windows- und Unix-Systemen lauffähig. Als repräsentative Beispiele für kommerzielle und frei verfügbare Softwarebibliotheken werden hier zwei der am weitest verbreiteten Vertreter vorgestellt.

Die Softwareumgebung *Halcon* vom Anbieter MVTec ist einer der Marktführer unter den kommerziellen Vision-Lösungen und wird von vielen Organisationen zur Umsetzung eigener Vision-Anwendungen, sowie zur Entwicklung vollumfänglicher Vision-Systeme für den Weiterverkauf verwendet. Auch Festo hat in der Vergangenheit Kamerasysteme mit integrierter Erkennungslogik auf Basis von MVTec Halcon entwickelt und verkauft.

Halcon besteht aus einer umfassenden Funktionsbibliothek, die in verschiedenen Programmiersprachen und auf unterschiedlichen Zielsystemen eingebunden werden kann, sowie einer eigenen Programmierumgebung *HDevelop*. Halcon umfasst einen Baukasten von Funktionen zum Einbinden verschiedener Kameras, unter anderem über das GigE-Vision Protokoll, diverse Grundfunktionen der Bildverarbeitung, sowie Funktionen zum Generieren von Ergebnisbildern.

Die Umgebung HDevelop erlaubt das interaktive Entwickeln von Erkennungen mit der Möglichkeit, die Erkennungsergebnisse direkt im Livebild der Kamera anzuzeigen.

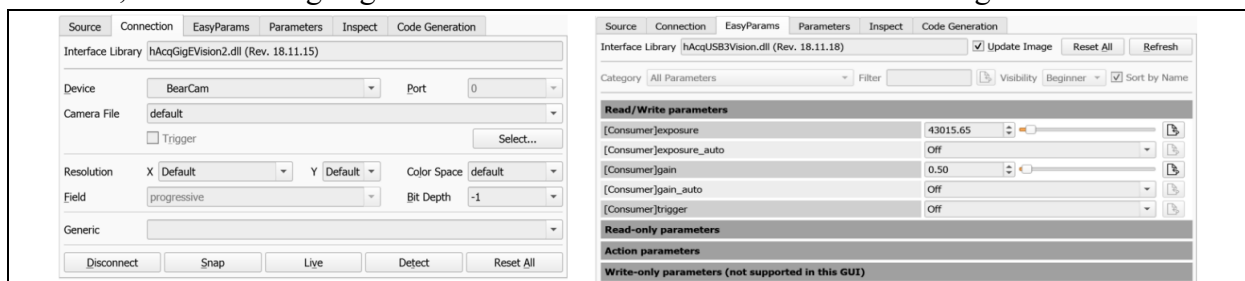


Abbildung 6: Einbinden und Konfigurieren von Kamera in HDevelop [MVTec, 2026]

Abbildung 6 zeigt das GUI-Menü in HDevelop, über das verschiedene Kameras eingebunden und konfiguriert werden können. Je nach Kameratyp werden hier automatisch alle unterstützten

Parameter der Kamera aufgelistet. Das Livebild der Kamera wird anschließend oben links in der Hauptoberfläche angezeigt, die in Abbildung 7 zu sehen ist.

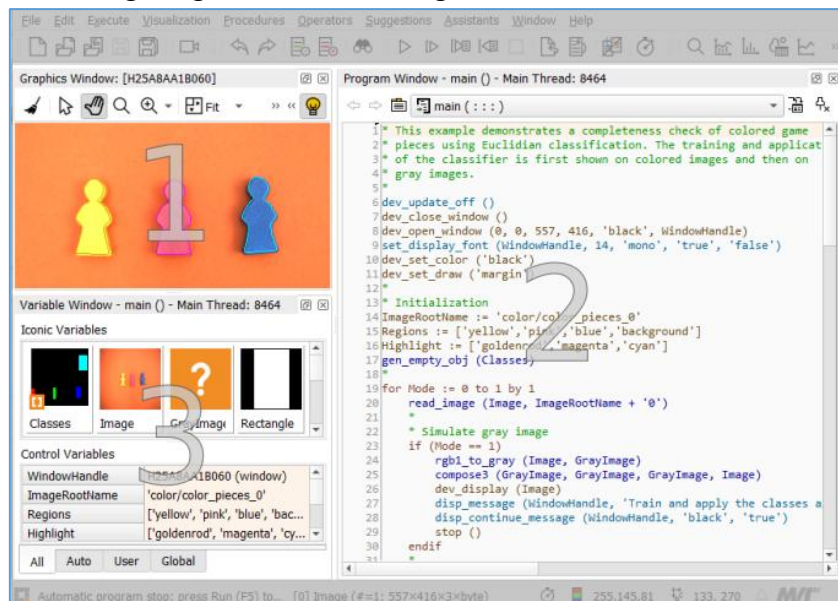


Abbildung 7: MVTec HDevelop Oberfläche [MVTec, 2026]

Im Bereich unten links sind verschiedene Variablen des Erkennungsprogramms als GUI-Elemente veränderbar. Die Erkennung wird im Hauptbereich in einer proprietären Skripting-Sprache definiert. Hier können die diversen Funktionen der Halcon-Bibliothek auf das Kamerabild angewandt werden. Dabei bietet Halcon keine fertigen Erkennungsbausteine, die mit einem einzelnen Aufruf ein gewünschtes Objekt erkennen und markieren können, die Erkennungslogik muss vom Anwender selbst aus einfacheren Bausteinen und Grundfunktionen der Bildverarbeitung erstellt werden. Halcon bietet allerdings fertig integrierte und leistungsstarke Methoden zur Erkennung und Dekodierung von Texten und Barcodes. Die fertigen Erkennungsabläufe können anschließend als Code für alle unterstützten Zielprogrammiersprachen und -systeme exportiert und somit dort in andere Applikationen integriert werden. [MVTec, 2026] Der Mehrwert von HDevelop liegt primär darin, dass durch die integrierte Umgebung mit Live-Vorschau und Parametrierung ein rapid Prototyping betrieben und damit Erkennungen deutlich schneller und effizienter angepasst und optimiert werden können, als würde man diese direkt auf dem Zielsystem entwickeln. Zusätzlich wird das Einbinden verschiedener Kameras durch das einheitliche Interface deutlich vereinfacht.

Mit Halcon lassen sich leistungsfähige Erkennungsmethoden schneller umsetzen als mit frei verfügbaren Bibliotheken. Dennoch sind dafür signifikanter Programmieraufwand und Expertenwissen zu Bildverarbeitung notwendig. Die Software ist somit nicht dafür geeignet, dass auch weniger technisch versierte Anwender damit selbstständig Vision-Applikationen umsetzen. Halcon könnte aber eine interessante technische Basis darstellen, um damit weitere Erkennungsmethoden umzusetzen.

Der größte Nachteil von Halcon für den Anwendungsfall in dieser Arbeit sind das Lizenzierungsmodell und die Preisgestaltung. Halcon ist ausgesprochen teuer, Einzellizenzen für Entwickler bewegen sich im mittleren vierstelligen Euro-Bereich. Für jedes einzelne System mit der Entwicklungsumgebung muss eine eigene Lizenz angeschafft werden und auch Zielsysteme benötigen eine Lizenz, für die ein USB-Stick mit Lizenzschlüssel installiert werden muss. Kos-

tengünstige Volumenlizenzen werden nicht angeboten. Dadurch würde die finanzielle Einstiegshürde für jeden potenziellen Anwender so hoch, dass die meisten, vor allem kleine Anwendungen nie angegangen würden. Somit scheidet Halcon als Basis für die in dieser Arbeit zu entwickelnden Lösung leider aus.

Die mit Abstand **am weitesten verbreitete frei verfügbare** Bibliothek für Vision-Applikationen ist *OpenCV* (Open Source Computer Vision Library). [Open Source Vision Foundation, 2025] Die Bibliothek existiert bereits seit mehreren Jahrzehnten und wird stetig weiterentwickelt.

OpenCV bündelt in mehreren Modulen alle wesentlichen Grundfunktionen der Bildverarbeitung, Datenstrukturen zur Handhabung von Bildern, Methoden zum Bearbeiten von und Zeichnen in Bildern und vieles mehr, was über den Rahmen dieser Arbeit hinausgeht. Die Bibliothek ist in C++ implementiert und über diverse Wrapper in vielen verschiedenen Programmiersprachen aufrufbar. Die Implementierung der verschiedenen Grundmechanismen ist leistungsstark und performant und es sind Distributionen für so gut wie alle denkbaren kommerziell verfügbaren Zielsysteme verfügbar. [OpenCV Team, 2026] Die Dokumentation von OpenCV ist sehr ausführlich, es werden viele offizielle Beispiele bereitgestellt und durch die weite Verbreitung der Bibliothek gibt es weitreichende Erfahrungen und Projekte, an denen Anwender sich orientieren können. Fast jede Pipeline für Computer Vision verwendet heutzutage an irgendeinem Punkt OpenCV. [Maddula, 2025]

Ähnlich wie Halcon stellt auch OpenCV nur Grundmechanismen und Basisfunktionen für die Verarbeitung von Bildern bereit. Erkennungen müssen im Quellcode eigenständig entwickelt werden. Im Gegensatz zu Halcon bringt OpenCV keine Funktionalität für die Anbindung von Kameras mit und besitzt auch keinerlei grafische Oberfläche, Konfigurationsdialoge, geschweige denn eine eigene Entwicklungsumgebung. OpenCV kann ein statisches Bild in einem Debugging-Dialog anzeigen, jegliche GUI-Funktionalität darüber hinaus muss vom Anwender selber implementiert werden. Dadurch ist die Entwicklungsarbeit mit OpenCV mühsamer und zeitaufwändiger als mit Halcon.

Dennoch lassen sich mit OpenCV und dem entsprechenden Entwicklungsaufwand leistungsstarke und stabile Erkennungen umsetzen. Der weitreichende Funktionsumfang, die ausführliche Dokumentation und nicht zuletzt auch die Erfahrungen aus einem Vorprojekt dieser Arbeit führen zu dem Schluss, dass OpenCV für die Lösung dieser Arbeit eingesetzt werden kann.

Neben OpenCV **als am weitesten verbreiteter freier Lösung** gibt es viele andere Bibliotheken, die sich oft auf kleinere, spezialisiertere Anwendungsfälle konzentrieren. OpenCV besitzt im Gegensatz zu Halcon keine Funktionalität zum Erkennen und Dekodieren von Barcodes und Texten. Dafür können verschiedene Freie Lösungen eingesetzt werden.

Die Open-Source Bibliothek *LibDmtx* erlaubt das Erkennen und Dekodieren von Barcodes im DataMatrix-Format. Die Bibliothek ist auf vielen Plattformen und über Wrapper in mehreren Programmiersprachen verfügbar. [Laughton, 2026]

Die Bibliothek *Tesseract-OCR* (Optical Character Recognition) stellt eine leistungsstarke freie Lösung zur automatisierten Texterkennung bereit und wird weltweit von einer Vielzahl von Anwendern in kommerziellen wie nichtkommerziellen Umfeldern eingesetzt. Die Bibliothek wurde anfangs von Hewlett Packard, später von Google entwickelt und wird seit 2019 von einer Open-Source Community gepflegt und weiterentwickelt. [Tesseract OCR, 2026]

Kommerzielle Softwarelösungen wie Halcon bieten einen großen und gut in einem Paket gebündelten Funktionsumfang, in Verbindung mit leistungsstarker GUI-Software, die eine effektivere und effizientere Entwicklung von Erkennungsapplikationen ermöglicht. Open-Source Software bietet keine vergleichbaren integrierten Entwicklungsumgebungen, aber der grundlegende Funktionsumfang zum Entwickeln von Erkennungen kann auch mit der Kombination mehrerer frei verfügbarer Lösungen abgedeckt werden.

2.1.3 Kamerasysteme

Verbreitet im Markt für Vision-Systeme sind auch integrierte Kamerasysteme, die Kamera und Erkennungssoftware in einer geschlossenen Einheit kombinieren. Die Erkennungslogik läuft hier üblicherweise direkt auf leistungsstarker Spezialhardware, die ins Kameragehäuse integriert ist. Einer der dominanten Anbieter in diesem Feld ist Keyence, Abbildung 8 zeigt solch ein *All-In-One* Kamerasystem. Diese Systeme vereinen eine leistungsstarke Kamera, teilweise integrierte, softwaregesteuerte Beleuchtung und Objektive mit dynamisch kontrollierbarer Vergrößerung, sowie eingebettete Prozessoren, die eine direkte Verbindung zum Kamerasensor haben und für die Ausführung von Bildverarbeitungsalgorithmen optimiert sind. Dadurch werden regelbasierte² Erkennungsabläufe schnell und effizient durchgeführt und es können teilweise sogar KI-basierte Methoden direkt auf der Kamerahardware ausgeführt werden.



Abbildung 8: Keyence All-In-One Kamerasystem [Keyence, 2026]

Die Konfiguration der Kamera und die Entwicklung von Erkennungsabläufen erfolgt in der Regel über eine proprietäre Software des Herstellers. Abbildung 9 zeigt die Oberfläche der Keyence-Software für das All-In-One Kamerasystem.

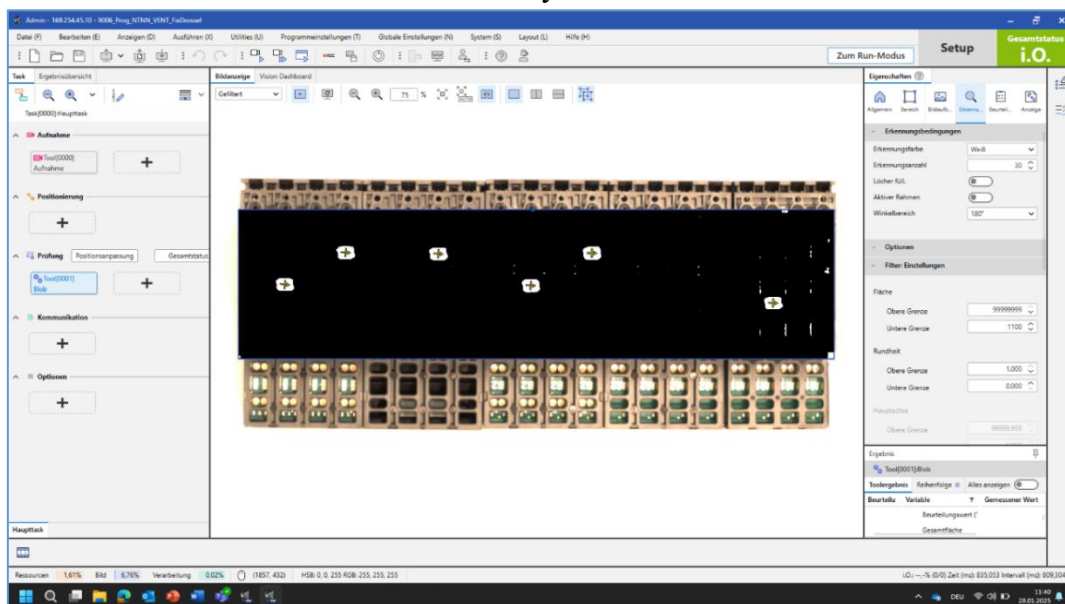


Abbildung 9: Keyence Entwicklungsumgebung für All-In-One Kamerasystem

Die Keyence-Software ist darauf ausgelegt, dass auch nicht-Experten damit Vision-Applikationen erstellen können. Die Konfiguration erfolgt vollständig über GUI-Elemente und die Er-

² Siehe Kapitel 2.3

kennungsabläufe werden durch die Verkettung von vordefinierten, grafisch dargestellten Bausteinen definiert. Der Hauptbereich in der Mitte zeigt ein Livebild der Kamera, in dem auch in Echtzeit die Ergebnisse der aktuellen Erkennung eingezeichnet werden. In dem Menü links werden Bildquellen, Bildausschnitte und Erkennungsbausteine ausgewählt und verkettet. Bausteine können über den Plus-Button hinzugefügt und aus einer Palette verfügbarer Funktionen ausgewählt werden. Die Menüs auf der rechten Seite zeigen alle verfügbaren Parameter für den aktuell ausgewählten Baustein an. Das in der Software erstellte Programm wird auf der Kamera hinterlegt und kann dann dort durch einen externen Trigger, etwa mit einem physischen Taster oder per Web-API Aufruf gestartet werden. Die Keyence Kamera besitzt einen integrierten Webserver, über den von einem anderen Rechner aus in einem Browserfenster eine Live-Ergebnisansicht der Kamera angezeigt und Testergebnisse abgerufen werden können.

Die Keyence-Software verfügt über fertige, direkt einsatzfähige Erkennungsmethoden, die mit minimaler Parametrierung bereits Merkmale erkennen und beurteilen können. Somit ist kein weiterer Programmieraufwand durch den Anwender vonnöten. Die vordefinierten Erkennungsbausteine umfassen diverse klassische regelbasierte Methoden, etwa um Objekte, Kanten und Defekte über Farbe, Helligkeit, geometrische Form oder Bildabgleich zu erkennen und zu beurteilen, sowie mehrere leistungsstarke KI-Basierte Erkennungsmethoden und Bausteine zum Dekodieren von Text und Barcodes.

Kamerasysteme von Keyence und auch von anderen Herstellern wurden bei Festo bereits an mehreren Stellen eingesetzt. Mit dem Baukasten von Keyence können die allermeisten Merkmale mit geringem Aufwand gut und zuverlässig erkannt werden. Die Erfahrung zeigt, dass durch die vordefinierten Erkennungsmethoden, die gut aufbereitete grafische Oberfläche und die direkte Rückmeldung im Livebild auch Anwender ohne tiefen technischen Hintergrund in die Lage versetzt werden, effektive Erkennungen umzusetzen.

Bei den gängigen Kamerasystemen zeigen sich im Praxiseinsatz schnell einige Limitierungen, die einem breiteren Einsatz im Weg stehen. Die Natur solch hochintegrierter Systeme erschwert die Einbindung in größere, dynamische Produktionsprozesse. Die Beurteilung der Erkennungsergebnisse erfolgt über zuvor definierte Regeln innerhalb des Kameraprogramms, das Kamerasystem gibt im Wesentlichen eine gut/schlecht Beurteilung nach außen zurück. Als Ergebnis wird meistens ein proprietärer, menschenlesbarer Prüfbericht erzeugt, der kaum maschinell weiterverarbeitet werden kann. Für die Prüfung verschiedener Produkte können mehrere Erkennungsprogramme in der Kamera hinterlegt werden, aber in einer Applikation mit hochvariablen konfigurierbaren Produkten kann die volle Kombinatorik so kaum abgebildet werden. Die Keyence-Software erlaubt eine limitierte Parametrierung eines Erkennungsprogramms von außen über die Web-Schnittstelle, aber in der Praxis hat sich gezeigt, dass auch dieser Ansatz bei der Prüfung von Ventilinseln an seine Grenzen stößt. Wünschenswert wäre, dass das Kamerasystem alle erkannten Merkmale als weiter verarbeitbare Datenstruktur bereitstellt, die anschließend im aufrufenden Testprogramm analysiert wird. Solch eine Schnittstelle wird aber leider nicht bereitgestellt.

Die gängigen Kamerasysteme sind ausgesprochen leistungsstark und gut geeignet für Anwendungsfälle mit geringer Prüflingsvarianz und hoher Stückzahl, bei denen durch die visuelle Prüfung ein großer, direkt messbarer Mehrwert erzeugt werden kann. Für viele weitere Anwendungsfälle ist der Kostenaspekt ein großes Hindernis. Leistungsstarke Kamerasysteme bewegen

sich meist im mittleren fünfstelligen Euro-Bereich pro Einheit, teilweise fallen noch zusätzliche Lizenzgebühren für die Entwicklungsumgebung und jährliche Wartungskosten an. Somit erfordert bereits die Evaluation eines potenziellen neuen Anwendungsfalls eine signifikante finanzielle Investition, wodurch kommerzielle Kamerasysteme als Option für die meisten Anwendungsfälle leider ausscheiden.

2.1.4 Applikationslösungen

Leistungsstarke Vision-Systeme werden oft speziell für einen dedizierten Anwendungsfall aufgebaut und entwickelt. Dafür erfasst meist ein spezialisierter Anbieter oder Dienstleister die Kundenanforderungen und Parameter, konstruiert einen passenden Hardwareaufbau und entwickelt die Erkennungssoftware für die Applikation. Dabei kommen nicht selten auch die zuvor beschriebenen Industriekameras und Softwarebibliotheken zum Einsatz. Für viele Anwendungsfälle gibt es auch fertige Baukastenlösungen, die vom Anbieter nur für die spezielle Applikation angepasst wird. Typische Einsatzfelder solcher dedizierten Applikationslösungen sind als Teil von Hochautomatisierungsanlagen, sowie Stationen mit sehr großem Durchsatz und hoher Kritikalität. Ein gängiger Einsatzort ist zum Beispiel in der vollautomatisierten Elektronikfertigung, wo nach der automatisierten Bestückung bei jeder Platine das Vorhandensein und die korrekte Position aller elektronischen Bauteile überprüft wird. Solche Systeme werden auch bei Festo eingesetzt.

Diese dedizierten Applikationslösungen sind mit eine der teuersten Varianten von Vision-Systemen, die Kosten für eine einzelne Station liegen je nach Art und Größe oft im sechsstelligen Euro-Bereich. Die Systeme sind in der Regel komplett geschlossen und nur über zuvor fest definierte limitierte Schnittstellen mit anderen Systemen verbunden. Die Erweiterung zur Abdeckung zusätzlicher Merkmale erfordert üblicherweise Entwicklungsarbeit durch den Anbieter und somit weitere Kosten, eine Replikation zur Umsetzung weiterer Anwendungsfälle ist in der Regel nicht möglich.

In einem Vorprojekt unter dem Arbeitstitel *VISUS* wurde solch eine Applikationslösung intern für die Überprüfung des Bauzustands von Ventilinseln entwickelt. Dabei wurden zwei Baumer Industriekameras, OpenCV, Tesseract und LibDmtx eingesetzt, um die Anzahl und Größe der Druckluftanschlüsse, verschiedene Beschriftungen und die DataMatrix Codes auf dem Prüfling zu erkennen. Das Prüfprogramm vergleicht Anzahl, Typ und Position aller Merkmale mit einer Soll-Definition und zeigt Abweichungen in einem Ergebnisbild an. Der Prüfstand wurde speziell für einen Kunden mit besonderen Qualitätsanforderungen aufgebaut, um Reklamationen zu vermeiden. Das Prüfprogramm kann mehrere verschiedene Produktkonfigurationen anhand des Identifikationscodes des Prüflings unterscheiden und so die richtige Definition für den aktuellen Prüfling anwenden. Mit dem Prüfstand konnte die Anzahl von Kundenreklamationen für dieses Produkt um über 90% reduziert werden.

Das Prüfprogramm wurde in Python implementiert und läuft als Web-Applikation in einem Cloud-Container. Die Anwenderoberfläche wird am Prüfstand in einem Browserfenster angezeigt und die Anwendung verbindet sich über das interne Netzwerk mit den Kameras. Die Erkennungen funktionieren mit einer hinreichenden Zuverlässigkeit, es ist jedoch immer wieder eine manuelle Nachkontrolle notwendig. Die Applikation ist empfindlich gegenüber veränderbaren Lichtbedingungen, wodurch Merkmale sporadisch nicht korrekt erkannt werden. Diese falsch-negativen Ergebnisse können in der Regel aber bereits mit einem zweiten Durchlauf behoben werden. Die Performance der Anwendung ist bisher noch nicht zufriedenstellend, die

Beurteilung eines Prüflings dauert meist länger als 30 Sekunden. Der Prüfstand existiert als eigenständige Insellösung und ist nicht in das bestehende Testing-Ökosystem integriert. Dadurch ist keine Anbindung an die restliche Infrastruktur gegeben und es werden keine standardisierten Prüfberichte erzeugt.

Die Entwicklung des Systems war dank dem Einsatz von freier Software sehr kostengünstig, es wurde aber ein signifikanter Zeit- und Arbeitsaufwand benötigt. Die Infrastruktur und Anwendungsumgebung für das Programm mussten von null an aufgesetzt werden und auch für die Kameraanbindung und Erkennungsschritte gab es keine Basis. Feintuning der Parametrierung und Weiterentwicklung der Erkennungen gestalten sich ebenfalls als mühsam, denn jede Änderung muss durch den Softwareentwickler direkt im Quellcode durchgeführt werden. Nach Vornehmen einer Änderung muss das Programm gestartet, die Auswirkung beurteilt und ggf. die nächste Veränderung gemacht werden. Dieser Ablauf macht die Entwicklung langwierig und abhängig von einem spezifischen Softwareentwickler.

2.1.5 Fazit

Die Erfahrungen und Erfolge aus dem VISUS Projekt sind der Auslöser für die Motivation, noch mehr Prüfapplikationen mit selbst entwickelter Erkennungssoftware umzusetzen. Die hohen Kosten von kommerziellen Vision-Systemen sind bisher das größte Hindernis für den breiten Einsatz, eine interne Lösung könnte breit verteilt und auch für kleinere Anwendungsfälle eingesetzt werden.

Bei der Eigenentwicklung von Vision-Applikationen sind Personalressourcen der größte Engpass. Deshalb wird eine Lösung benötigt, die auch Anwendern ohne großes Expertenwissen ermöglicht, Erkennungen und Applikationen zu erstellen. Die betrachteten kommerziellen Lösungen bringen mehr fertig einsetzbare Funktionalität mit, können den Anforderungen aber nicht vollständig gerecht werden. Zusätzlich ist es durch Lizenzierungsmodelle meist nicht möglich, diese intern bereitzustellen, ohne dass direkt beim Anwender hohe Kosten für jede neue Applikation anfallen.

Die benötigte Erkennungsfunktionalität kann mit freien Bibliotheken abgebildet werden, auf dieser Basis kann ein Softwaremodul entwickelt werden, das Vision-Funktionen einem breiten Anwenderkreis bereitstellt. Von den kommerziellen Vision-Systemen können hier einige wesentliche Eigenschaften übernommen werden. Gängige Industriekameras müssen möglichst einfach anbindbar und konfigurierbar sein. Es müssen fertig einsetzbare Erkennungsmethoden angeboten werden, die nur minimale Parametrierung benötigen. Als besonders große Effizienzsteigerung für die Entwicklung gezeigt hat sich die Konfiguration mit dedizierten GUI-Menüs und mit Ergebnisanzeige direkt im Livebild der Kamera.

Um auf bestehende Infrastruktur und die Programmierkenntnisse potenzieller Anwender zurückzugreifen sollte das Modul vollständig in das bereits bestehende Testing-Ökosystem integrierbar sein. Durch die Integration in eine bestehende Programmierumgebung wird auch gewährleistet, dass beliebig komplexe Anwendungsfälle in Hinsicht auf die Prüflingsvariabilität und die Beurteilung abgedeckt werden können.

2.2 Vergleichbare Themen in der Wissenschaft

Das Feld der Vision-Systeme wird in der aktuellen Wissenschaft weitreichend behandelt. Jedoch liegt der Fokus hier zum großen Teil auf der Erforschung neuer Algorithmen und Methoden für die Lösung einzelner, spezifischer Anwendungsfälle. Besonders der Einsatz von Machine Learning und neuronalen Netzen für spezifische Problemstellung werden vielzählig thematisiert. Die Aufgabenstellung dieser Arbeit beschäftigt sich weniger damit, neue Ansätze für die Erkennung von Merkmalen zu erarbeiten, sondern damit, wie etablierte Mechanismen wiederverwendbar verallgemeinert, einem breiten Anwenderkreise ohne Expertenwissen verfügbar gemacht und in existierende Programmierumgebungen integriert werden können. Verschiedene Arbeiten beschäftigen sich zumindest mit Teilaspekten dieser Problemstellung.

Zhu et al. (2022) kommen zu dem Schluss, dass die Entwicklung von Vision-Systemen für den Produktionseinsatz oft ineffizient abläuft. Applikationen werden meist für einen speziellen Anwendungsfall entwickelt und setzen nicht auf wiederverwendbaren Basiskomponenten auf. Dadurch müssen Kameraanbindung, Ergebnisaufbereitung und Programmabläufe jedes Mal neu entwickelt werden. Als Lösung spezifizierten sie ein Framework, das die Anbindung von Kameras und weiterer Hardware, den Transport von Daten, die Ausführung von Analyseaufgaben und die Ergebnisaufbereitung übernimmt. Damit stellen sie eine Web-Anwendung für mehrere Nutzer und Applikationen bereit. Die Anwender können sich so auf die Ausarbeitung von Erkennungen konzentrieren und müssen nicht alle Programmkomponenten selbst aufsetzen. Die Erkennungen werden von den Anwendern mit OpenCV implementiert und anschließend an das Framework übergeben. [Zhu, Zhang, Luan, & Hu, 2022]

Der Ansatz unterstützt die These dieser Arbeit, dass durch die Bereitstellung von wiederverwendbaren Grundfunktionen die Entwicklung von Vision-Applikationen deutlich beschleunigt und effizienter gemacht werden kann. Es wird jedoch weiterhin vorausgesetzt, dass Anwender ihre Erkennungen direkt mit OpenCV Methoden für einen spezifischen Anwendungsfall programmieren. Wiederverwendbare Erkennungsmethoden und die Integration in größere Ökosysteme werden hier nicht thematisiert.

Intisar et al. (2021) setzten zur Auswahl eines Ziels für ihren kameragesteuerten Roboter-Greifarm auf eine per GUI-Dialog konfigurierbare Erkennungsmethode. Der Anwender die gewünschte Farbe, Form und Dimension eines Zielobjekts definieren und erhält in einem Kameralebendbild direkte Rückmeldung, welche Objekte im Bild als Ziel erkannt werden. Die Autoren stellten fest, dass durch die interaktive Konfiguration auch Anwender ohne Wissen über die Implementierung leicht diverse verschiedene Objekte als Ziel definieren konnten. [Intisar, Khan, Islam, & Masud, 2021]

Hier zeigt sich erneut, dass Erkennungen über Parametrierung gut für verschiedene Anwendungsfälle verallgemeinert und durch interaktive Konfiguration und Rückmeldung auch nicht-technischen Anwendern verfügbar gemacht werden können.

Martin Rudorfer (2026) thematisiert, dass im Industrieumfeld zunehmend auch Vision-Applikationen für verschiedene, variable Anwendungsfälle mit kleineren Stückzahlen benötigt werden. Um eine große Anzahl unterschiedlicher Erkennungs-Applikationen abzubilden, entwickelte er ein Framework für eine Web-Service Applikation, die sich mit mehreren Kameras verbinden und damit unterschiedliche Erkennungen ausführen kann. Dazu Entwickelte er drei

generalisierte Erkennungsmethoden, die für unterschiedliche Anwendungsfälle wiederverwendet werden können. Anwender des Frameworks können die vordefinierten Methoden konfigurieren und mit Trainingsdaten ihres Anwendungsfalles trainieren und diese anschließend mit den Bildern der verbundenen Kameras ausführen. Als Ergebnis des Projektes hält er fest, dass die Service-basierte Lösung effizientere Entwicklung von Erkennungsapplikationen ermöglicht und besser konfigurierbar, sowie verfügbarer ist als konventionelle Vision-Systeme. Als Nachteile zeigten sich die schlechtere Performance und Latenz der Webbasierten Lösung. [Rudorfer, 2026]

In diesem Projekt wird wieder der Vorteil von wiederverwendbaren und generischen Erkennungsmethoden thematisiert, die durch verschiedene Anwender effizient für die eigenen Anwendungsfälle eingesetzt werden können. Zusätzlich wird durch das Framework die Logik zum Anbinden von Kameras und der Ausführung von Erkennungsabläufen wiederverwendbar gemacht. Aber auch hier wird nicht thematisiert, inwiefern die Lösung in andere Programmierumgebungen integriert werden kann.

In der Wissenschaft wird mehrfach anerkannt, dass Frameworks für die effektivere Entwicklung von Vision-Systemen auf Basis freier Software benötigt werden. Aber keiner der bisher bekannten Lösungsansätze vereint die Kombination aus wiederverwendbaren Erkennungsmethoden, Konfiguration über GUI-Dialoge mit Live-Ergebnisbildern und der Integrierbarkeit in andere Programmierumgebungen mit maschinell verarbeitbaren Ergebnisdaten.

2.3 Methoden der Bildverarbeitung

Bei der Beschreibung von Vision-Systemen wurden mehrfach die Grundmechanismen der Bildverarbeitung erwähnt, diese werden hier nun einmal kurz vorgestellt. Dafür werden beispielhaft ein paar der Mechanismen betrachtet, die auch im weiteren Verlauf dieses Projektes zum Einsatz kommen.

Bei den Methoden der Bildverarbeitung zu Erkennungszwecken muss grundsätzlich zwischen zwei Themengebieten unterschieden werden: Den klassischen, regelbasierten Methoden und den KI³- und Machine-Learning basierten Mechanismen. Die klassischen Methoden nutzen explizit definierte Algorithmen und Regeln, um Bilder zu verarbeiten und aus ihnen Informationen zu ziehen. Die KI-Basierten Methoden nutzen neuronale Netze, die mit größeren Mengen an Beispieldaten für einen spezifischen Anwendungsfall trainiert werden. KI-Mechanismen sind üblicherweise besser geeignet für komplexere Anwendungen wie Anomalieerkennungen, Oberflächeninspektion und Objektklassifizierung. Sie benötigen aber meist sehr leistungsstarke dedizierte Hardware, müssen für Anwendungsfälle trainiert werden, sind meist weniger performant als klassische Methoden und produzieren nicht immer deterministische Ergebnisse. Klassische Methoden erfordern meist einen höheren initialen Entwicklungsaufwand, sind aber auf einfacherer Hardware lauffähig und generell besser geeignet für Anwendungsfälle mit kontrollierbaren Umgebungsbedingungen und einfacheren Merkmalen. [Pekat Vision, 2026]

Diese Arbeit beschäftigt sich zunächst nur mit klassischen, regelbasierten Methoden, es spricht allerdings nichts gegen die Ergänzung von KI-basierten Erkennungsmethoden in der Zukunft.

³ Künstliche Intelligenz

Zur Identifikation von Objekten in einem Bild können verschiedene Ansätze genutzt werden. Zwei verbreitete und gut verständliche Methoden sind die Blob-Analyse und das Template Matching. Bei der Blob-Analyse werden zusammenhängende Bildbereiche identifiziert, die sich durch Helligkeit oder Farbe vom Hintergrund absetzen, beim Template Matching wird eine Vorlage mit dem Bild verglichen und dabei nach Übereinstimmungen auf Pixelebene gesucht. Damit diese Methoden gut funktionieren, müssen die Eingabebilder zunächst so vereinfacht werden, dass sie nur noch die für die Erkennung relevanten Details beinhalten. Dafür werden auf jedem Bild mehrere anwendungsspezifische Vorverarbeitungsschritte ausgeführt.

2.3.1 Bildkanäle & Farben

Ein Bild ist nichts anderes als eine 2d-Matrix von Pixeln mit Helligkeits- oder Farbwerten. In einem Schwarzweißbild besitzt jedes Pixel einen Intensitätswert, der die Helligkeit des Bildpunktes angibt. In einem Farbbild besitzt jedes Pixel Intensitätswerte in mehreren Kanälen, die zusammen den resultierenden Farbwert bestimmen. In einem RGB-Bild besitzt jedes Pixel drei Intensitätswerte für den Rot- Grün- und Blau-Kanal.

Für die Weiterverarbeitung sind einkanalige Bilder meist am einfachsten handzuhaben. Ist für den Anwendungsfall nur die Helligkeit eines zu erkennenden Merkmals interessant, kann die Farbkomponente verworfen und daraus ein Graustufenbild generiert werden. Die Farbkanäle eines Bildes können auch extrahiert und als individuelles Graustufenbild betrachtet werden, etwa um farbige Bildbereiche zu ermitteln.

Der individuelle Rot- oder Grünanteil eines Pixels ist dafür nicht unbedingt die am besten geeignete Eigenschaft. Zur Abbildung von Farbbildern gibt es noch weitere Repräsentationsformen, das HSV-Format zum Beispiel definiert die drei Kanäle Hue (Farbton), Saturation (Farbsättigung) und Value (Helligkeit). Hier wird der Farbton in einem einzelnen Kanal definiert, wodurch sich markante Farben oft besser unterscheiden lassen als im RGB-Format. Abbildung 10 zeigt die Repräsentation eines Bildes einmal als Farb- und Graustufenbild, dann die drei RGB-Kanäle und zuletzt die drei HSV-Kanäle.

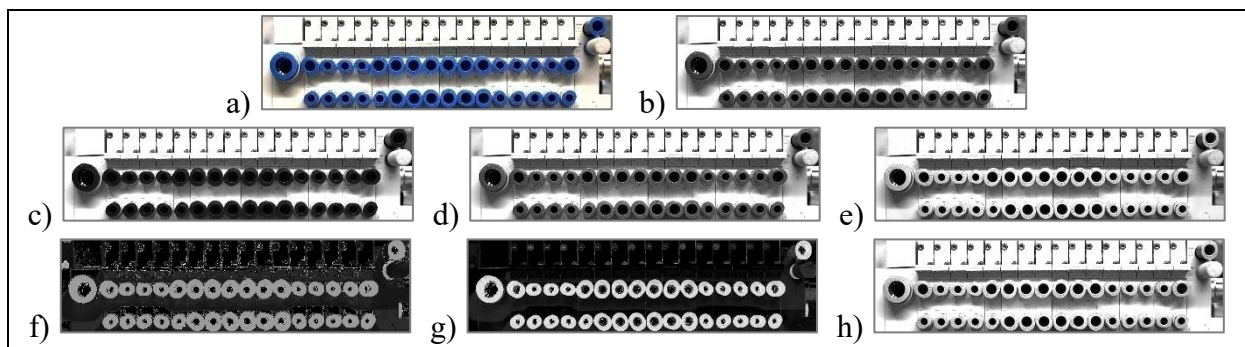


Abbildung 10: a) Originalbild, b) Graustufenbild, c) RGB Rot, d) RGB Grün, e) RGB Blau, f) HSV Hue, g) HSV Saturation, h) HSV Value

Im Beispiel ist gut der Unterschied zwischen der RGB- und der HSV-Repräsentation zu erkennen. In den Rot- und Grün-Kanälen der RGB-Repräsentation sind die blauen Bildbereiche dunkel und kaum von schwarzen Bildbereichen zu unterscheiden. Im Blau-Kanal verlaufen die farbigen Bereiche wiederum mit dem weißen Hintergrund. In der HSV-Repräsentation sind die farbigen Bereiche deutlich vom Hintergrund abgehoben und können so effektiv aus einem einzelnen Kanalbild extrahiert werden.

2.3.2 Quantisierung

Zur Extraktion von Merkmalen mit einer bestimmten Farbe oder Helligkeit müssen die jeweiligen Bildbereiche herausgestellt werden. Dazu kann die Quantisierung eingesetzt werden, wobei der Wertebereich der Pixel eines Bildes auf einige wenige diskrete Werte reduziert wird. So kann ein Farbbild etwa in Bereiche mit einer von fünf unterschiedlichen Farben unterteilt werden.

Die Maximalausprägung der Quantisierung ist die Binarisierung, hier wird ein Bild anhand eines Grenzwertes auf genau zwei Werte reduziert. Alle Pixel mit einem Wert über dem Grenzwert erhalten den Wert 1, alle darunter den Wert 0.

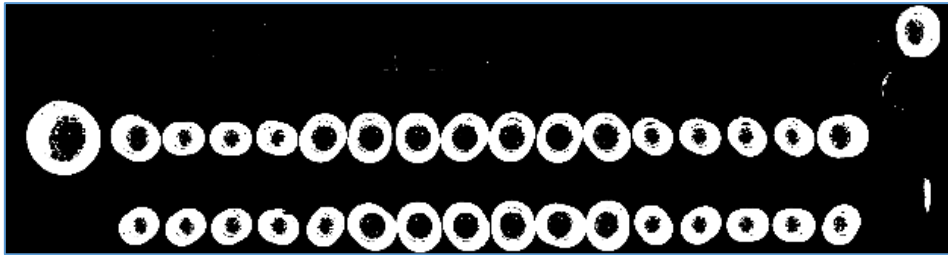


Abbildung 11: Ergebnis Binarisierung Farbkanal eines HSV-Bildes

So zeigt zum Beispiel Abbildung 11 das Ergebnis der Binarisierung des Farbkanals aus dem HSV-Bild in Abbildung 10. Hier ist gut zu sehen, wie nur noch die farbigen Objekte verbleiben. Durch Binarisierung mit einem gut gewählten Grenzwert können somit für viele Anwendungsfälle bereits die relevanten Bildbereiche ermittelt werden.

2.3.3 Kanten und Konturen

Zur Extraktion von Objekten in einem Bild werden üblicherweise Kanten und Konturen betrachtet. Eine Kante beschreibt den Übergang zwischen zwei Bildbereichen mit einer deutlichen Intensitätsveränderung, was den Beginn oder das Ende eines Objektes indizieren kann. Eine Kontur ist eine geschlossene Aneinanderreihung von Punkten, die einen Bildbereich einfasst. In dem Beispiel aus Abbildung 11 kann jeder der weißen Bildbereiche als Kontur betrachtet werden, die Übergänge zwischen Kontur und Hintergrund sind die Kanten des Objektes. Konturen werden ermittelt, indem eine Kante verfolgt wird, bis diese ohne Unterbrechung wieder ihren Ursprung erreicht. Kanten und Konturen sind besser erkennbar, je deutlicher sie sich vom Hintergrund absetzen. Deshalb sind sie in quantisierten Bildern am klarsten zu ermitteln. Eine extrahierte Kontur kann dann als gefundenes Objekt weiterbehandelt werden.

2.3.4 Vorverarbeitung

Nach der Quantisierung sind die verschiedenen Bildbereiche meist noch nicht sauber voneinander abgegrenzt. Es gibt Aus- und Einwüchse an den Kanten von Konturen sowie Löcher in Konturen und Artefakte in den nicht relevanten Bildbereichen.

Die zuverlässige Erkennung von Kanten und Konturen ist davon abhängig, dass diese klar abgegrenzt vom Hintergrund sind und konsistent zusammenhängen. Verschiedene Vorverarbeitungsmethoden können auf dem Originalbild oder auf dem quantisierten Bild ausgeführt werden, um Kanten und Konturen klarer herauszustellen und so die Erkennung und Klassifizierung von Merkmalen zu unterstützen.

In binarisierten Bildern werden dafür verschiedene morphologische Operationen eingesetzt. Bei der Dilation werden weiße Bildbereiche sowohl nach außen als auch nach innen vergrößert, was Löcher im Inneren und Einwüchse an den Kanten einer Kontur schließt oder reduziert.

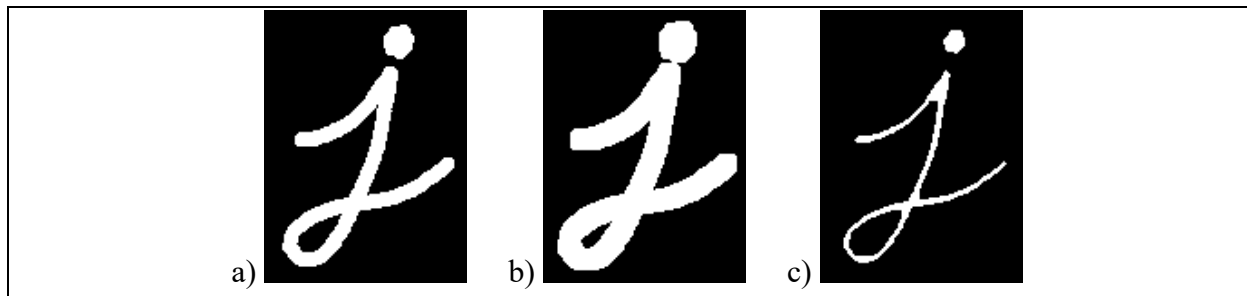


Abbildung 12: a) Originalbild, b) Dilation, c) Erosion [OpenCV, 2026]

Die Erosion hat den entgegengesetzten Effekt, hier werden Konturen von außen und von innen geschrumpft. Dadurch werden Auswüchse an den Rändern von Konturen verkleinert und besonders kleine Konturen, was meistens ungewollte Artefakte sind, werden dadurch gänzlich entfernt.

Jede Operation für sich verändert die Größe und Dicke von Konturen. Deshalb muss auf eine Operation üblicherweise eine entsprechende Gegenoperation folgen. Abbildung 13 zeigt Beispiele für Erosion gefolgt von Dilation, sowie den umgekehrten Fall. Durch die Kombination der beiden Operationen werden Artefakte entfernt, Löcher geschlossen und die gesuchten Objekte bleiben in ihrer Größe, Form und Dicke unverändert.

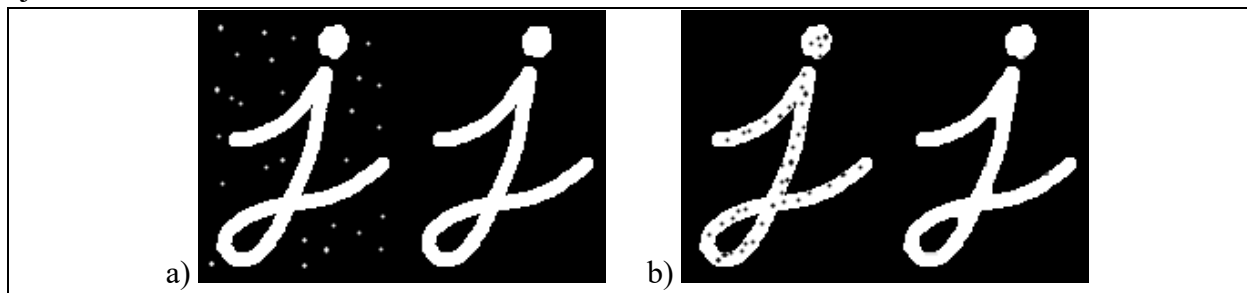


Abbildung 13: a) Erosion gefolgt von Dilation, b) Dilation gefolgt von Erosion [OpenCV, 2026]

Durch die geschickte und anwendungsbezogene Kombination von morphologischen Operationen kann die Qualität eines quantisierten Bildes deutlich verbessert werden, um anschließend relevante Konturen besser extrahieren und klassifizieren zu können.

Als Vorbereitung für die Quantisierung oder ein Template Matching können auch auf dem Originalbild verschiedene Vorverarbeitungsschritte ausgeführt werden. Zur Reduzierung von Rauschen und Artefakten in einem Bild kann ein einfacher Unschärfefilter bereits eine deutliche Verbesserung liefern. Dadurch werden kleine Punkte mit großen Intensitätssprüngen geglättet, sodass diese eine Kantenerkennung nicht mehr maßgeblich beeinflussen können. Das kann allerdings auch die Kanten von gesuchten Konturen verwaschen und abschwächen, deshalb muss dieser Filter mit Bedacht angewandt werden.

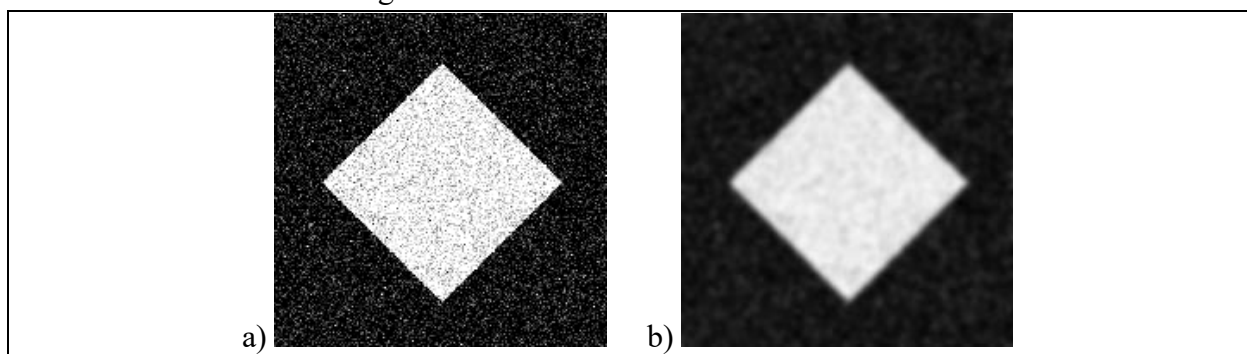


Abbildung 14: a) Bild mit Rauschen [He, 2008], b) Bild mit Unschärfe-Filter

Die Vorverarbeitung kann auch genutzt werden, um nur die Kanten in einem Bild herauszustellen. Der Laplace-Filter zum Beispiel betrachtet die Ableitung des Intensitätsverlaufes eines Bildes, wodurch Kanten als lokale Maxima und Minima sichtbar werden. Das Ergebnisbild zeigt dann die deutlichen Kanten des Originalbildes als Werte mit hoher Intensität.

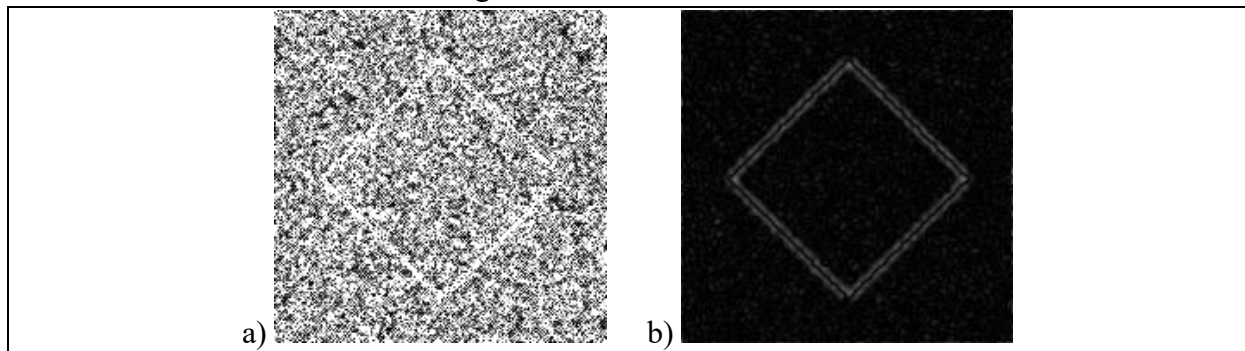


Abbildung 15: Ergebnis Laplace-Filter angewandt auf Abbildung 14 a) und b)

Abbildung 15 zeigt das Ergebnis eines Laplace-Filters, der auf die Bilder aus Abbildung 14 angewandt wird. Hier wird der Effekt des Unschärfefilters besonders deutlich. Durch die starken Artefakte im unbehandelten Bild lässt sich durch das Herausstellen der Kanten keine nützliche Information gewinnen, bei dem behandelten Bild sind die Kanten des Objektes deutlich zu erkennen. Eine saubere und anwendungsbezogene Vorverarbeitung ist meist der Schlüssel, um eine zuverlässige und effektive Erkennung zu ermöglichen.

2.3.5 Klassifizierung

Nachdem das Ursprungsbild in mehreren Schritten vorbehandelt und aufbereitet wurde, können in diesem relevante Bildbereiche extrahiert und klassifiziert werden.

Bei der Blob-Analyse werden zusammenhängende Konturen extrahiert und anschließend nach Parametern wie Größe und geometrischer Form gefiltert. Die Positionen der Konturen mit Parametern innerhalb eines festgelegten Bereiches bestimmen dann die Bildbereiche, in denen sich ein gesuchtes Objekt befindet. So können zum Beispiel blaue Kreise oder schwarze Quadrate identifiziert werden.

Beim Template Matching werden nicht die Form und Dimensionen einer Kontur betrachtet, sondern es wird ein Vorlagenbild (das *Template*) mit dem Kamerabild verglichen. Das Template ist typischerweise deutlich kleiner als das Kamerabild und es wird nach allen Positionen im Bild gesucht, an denen die Übereinstimmung oberhalb eines Grenzwertes liegt. Dafür werden Template und Kamerabild Pixel für Pixel verglichen, die Gesamtübereinstimmung ergibt sich aus den Übereinstimmungen an allen Pixelpositionen. Das Template wird dabei in jeder Pixelposition über das Bild gelegt und verglichen, sodass für jede einzelne Position ein Übereinstimmungsgrad mit dem Template ermittelt wird. Die Positionen mit den höchsten Übereinstimmungen bestimmen so die Bildbereiche, in denen sich ein gesuchtes Objekt befindet.

Das Template Matching ist gut geeignet, um möglichst exakte Übereinstimmungen zu finden. Größere Abweichungen, wie etwa ein Größenbereich zwischen 100 und 300 Pixeln sind damit nicht abbildbar. Zusätzlich ist die Methode sehr anfällig für Rauschen und kleinere Änderungen, hier ist eine gute Vorverarbeitung wichtig.

Die Blob-Analyse ist gut geeignet, um einfache Merkmale zu erkennen. Sie ist deutlich stabiler und weniger anfällig für rauschen und kleine Abweichungen. Sie kann auch gut größere Wertebereiche bei Größe und Form abdecken.

2.4 Zusammenfassung

Die Algorithmen und Mechanismen der Bildverarbeitung sind allgemein bekannt und weitreichend etabliert. Sowohl hochentwickelte kommerzielle Systeme als auch frei verfügbare Lösungen setzen intern auf den gleichen Grundfunktionen auf. Die große Herausforderung liegt darin, diese Mechanismen für Anwender verfügbar zu machen.

Kommerzielle Systeme vereinfachen die Entwicklung von Vision-Applikationen durch einheitliche Kameraanbindung, integrierte Entwicklungsumgebungen mit Live-Ergebnisbildern und teilweise vorgefertigten, wiederverwendbaren Erkennungsbausteinen.

Die Kosten und Lizenzmodelle der kommerziellen Systeme stehen einem breiten Einsatz im Weg, sie sind nur für große Anwendungen mit hohen Stückzahlen und messbarem Mehrwert zu rechtfertigen. Bei vielen komplexen Anwendungen stoßen kommerzielle Systeme jedoch an ihre Grenzen, da diese durch ihren geschlossenen Aufbau und fehlende Schnittstellen schlecht in bestehende Softwaresysteme integrierbar sind.

Somit ist ein eigenentwickeltes, wiederverwendbares Softwaremodul auf Basis frei verfügbarer Bibliotheken sowohl für kleine als auch für große Applikationen eine gleichermaßen interessante Lösung. Diese Lösung muss in bestehende Softwareumgebungen integrierbar sein, Standardkameras einbinden und einfach konfigurierbare, wiederverwendbare Erkennungsbausteine bereitstellen. Die Erkennungsmethoden sollten, wie die kommerziellen Lösungen, die Ergebnisse direkt im Livebild der Kamera anzeigen, um die Entwicklung zu beschleunigen und die Funktionen einem möglichst breiten Anwenderkreis zugänglich zu machen. Für die Entwicklung von Applikationen mit der Lösung sollte kein Fachwissen zur Bildverarbeitung notwendig sein.

Die volle Leistungsfähigkeit und den gesamten Funktionsumfang von dedizierten Systemen zu erreichen ist kein realistisches Ziel. Die Erfahrungen aus dem VISUS-Projekt haben jedoch gezeigt, dass auch mit ein paar einfachen Funktionen mächtige und effektive Applikationen aufgebaut werden können. Durch die Verallgemeinerung der dortigen Ansätze könnte bereits eine Vielzahl von Anwendungsfällen abgedeckt werden.

3 Testing Ökosystem

Das Vision-Modul soll so in das bestehende Testing Ökosystem integriert werden, sodass die Anwender der etablierten Umgebung sich auch mit den neuen Ergänzungen bestmöglich zu-rechtfinden. Dafür gilt es zu verstehen, wie das aktuelle Ökosystem funktioniert und welche Grundprinzipien und Mechanismen darin umgesetzt sind.

Das Ökosystem nutzt die Software *TestStand* von National Instruments⁴ als zentrale Programmierungsumgebung für Testapplikationen. Der Funktionsumfang von TestStand wurde von Festo durch eigens entwickelte Softwarekomponenten erweitert – dem so genannten *TEaM-Framework*. In diesem Kapitel werden zunächst die Oberfläche, der Aufbau und die wichtigsten Funktionen von TestStand eingeführt. Anschließend werden die Festo-eigenen Erweiterungen durch das TEaM-Framework beschrieben und die wichtigen Aspekte für dieses Projekt herausgearbeitet.

3.1 TestStand

TestStand ist eine Programmierungsumgebung mit dem Hauptfokus auf der Erstellung von Prüfprogrammen. TestStand verwendet einen blockbasierten Programmieransatz: Programme werden nicht in Zeilen von Quellcode in einem Texteditor erstellt, sondern als eine Aneinanderreihung von Testschritten, den so genannten *Steps*. Die Ambition dahinter ist, dass vor **Allem** auch nicht-Informatiker leicht mächtige Programme erstellen und verstehen können.

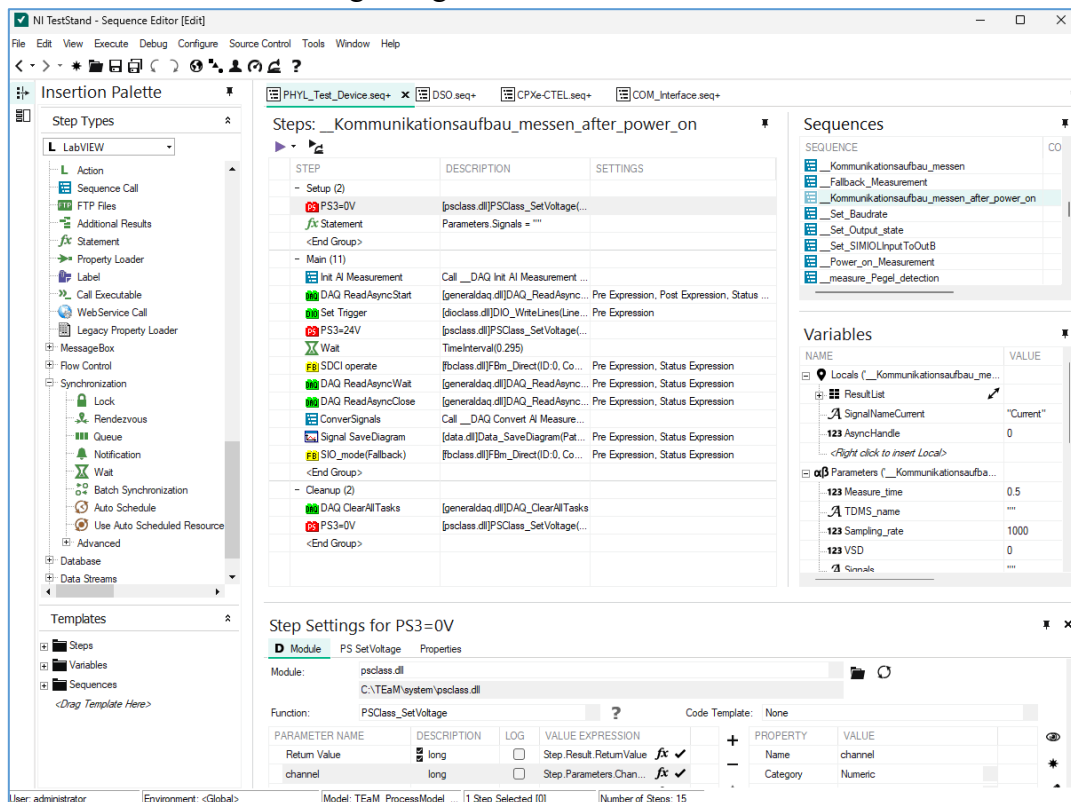


Abbildung 16: Screenshot der TestStand Programmierungsumgebung

Abbildung 16 zeigt die Programmierungsumgebung von TestStand, den Sequence Editor. Im mittleren Bereich befindet sich das aktuelle Testprogramm, bestehend aus einer Reihe von Steps. Auf die weiteren hier zu sehenden Bestandteile der Sprache und der Umgebung soll im Folgenden eingegangen werden.

⁴ National Instruments (NI) ist ein führender Anbieter von softwaregestützten Systemen für automatisierte Tests und Messungen. [NI, 2026]

3.1.1 Sequenzen

Die Aneinanderreihung von Steps erfolgt immer in den so genannten *Sequenzen*. Eine Test-Stand Datei, das so genannte *Sequence File*, besteht aus einer oder mehreren Sequenzen. Eine Sequenz kann wiederum mit einem *Sequence Call Step* aufgerufen werden, das Aufrufen von Sequenzen aus anderen Sequence Files ist ebenfalls möglich. Die aktuell geöffneten Sequence Files sind oben über dem Hauptbereich als Tabs aufgelistet, die Sequenzen im aktuellen Sequence File oben rechts in dem entsprechenden Abschnitt.

Jeder TestStand Schritt hat nach seiner Ausführung einen Ergebnisstatus. Der Statuswert *Passed*, respektive *Failed*, gibt das Bestehen oder Nichtbestehen des Schrittes an, und dieser Status wird auch auf das Ergebnis der Sequenz übertragen. So führt ein fehlgeschlagener Testschritt dazu, dass die gesamte Testsequenz den Status *failed* erhält. Bei einem Untersequenzaufruf wird das Sequenzergebnis wiederum übertragen auf den aufrufenden Sequence Call Schritt, und somit auch auf die aufrufende Sequenz, bis hin zur obersten Sequenz. So wird das *gut* oder *schlecht* Ergebnis für ein Testprogramm generiert.

Sequenzen sind unterteilt in die drei Bereiche Setup, Main und Cleanup. Dadurch kann eine Sequenz logisch unterteilt werden in Schritte zur Vorbereitung und Initialisierung, Schritte zur Durchführung der Hauptlogik und Schritte für das Aufräumen und Deinitialisieren von Ressourcen. Oft ist es nicht sinnvoll, eine Testsequenz fortzusetzen, wenn bestimmte Schritte fehlschlagen. Wenn zum Beispiel der Verbindungsaufbau zu einem Gerät scheitert, können weitere Operationen auf dem Gerät nicht ausgeführt werden.

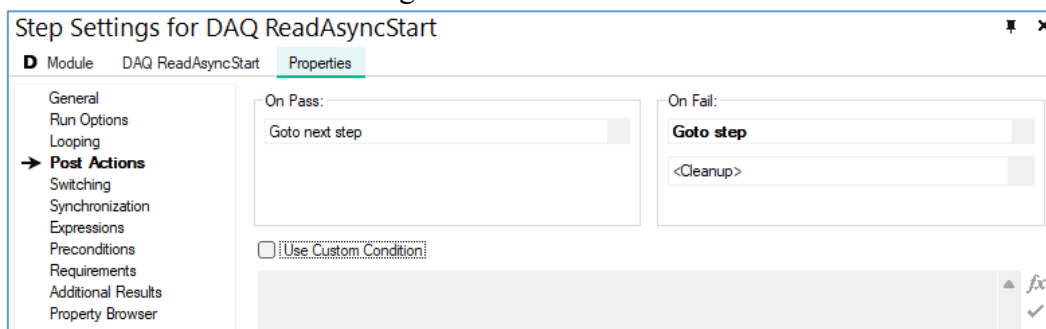


Abbildung 17: Post Action Menü eines TestStand Steps

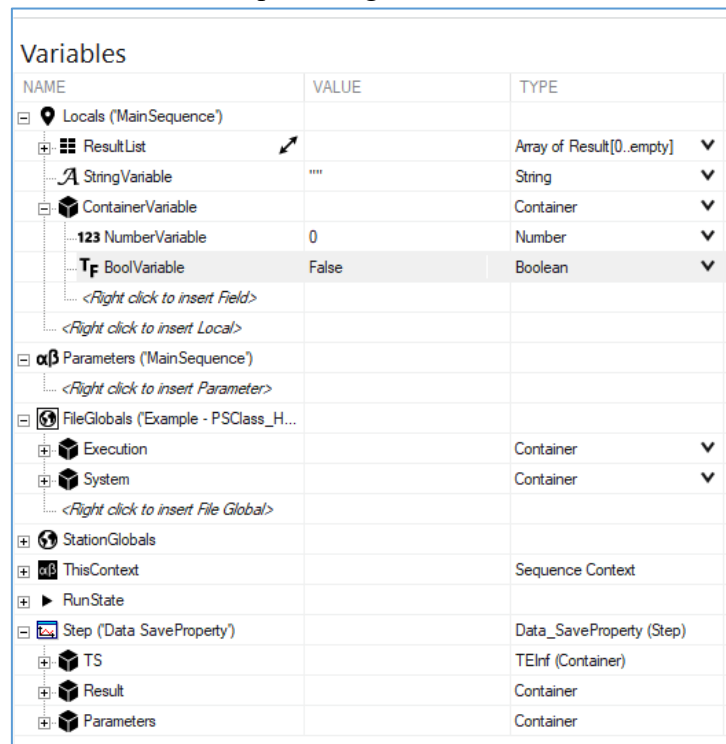
Abbildung 17 zeigt das Eigenschaftsmenü eines TestStand Steps. So kann über die *Post Action* eines Schrittes konfiguriert werden, dass die Ausführung nach einem Fehlschlagen direkt zum Cleanup-Bereich oder einem anderen definierten Schritt springen soll. Auf diese Weise lässt sich eine nachvollziehbare Ausführungskontrolle realisieren, ohne die Übersichtlichkeit der Testsequenz zu verschlechtern.

3.1.2 Variablen

Neben den Steps gibt es Variablen, die sich in einem von mehreren Kontexten befinden. Die Kontexte mit ihren Variablen sind auf der rechten Seite unterhalb der Sequenzen aufgeführt. So gibt es einen lokalen Kontext für die aktuelle Sequenz, einen Kontext *File Globals*, der global für das gesamte Sequence File existiert, oder den Kontext *Station Globals*, der von allen Sequence Files auf der aktuellen Maschine geteilt wird. Der Kontext *Parameters* gilt für eine Sequenz und enthält deren Übergabe- und Rückgabeparameter. Der Kontext *Step* existiert für einen einzelnen Schritt und enthält Informationen wie den Status oder die aufgerufene Modulfunktion. Die Kontexte *RunState* und *ThisContext* erlauben während der Ausführung Zugriff auf verschiedenste Eigenschaften der TestStand Laufzeit. Sie erlauben während der Laufzeit

Zugriff auf die Objekte zur aktuell ausgeführten Sequenz und Sequenzdatei, den zuvor ausgeführten Schritt, den aktuellen Aufrufstapel oder die globale Liste aller geladenen Codemodule. Auch die aktive Steuerung der Ausführung ist möglich, indem der Verweis auf den nächsten auszuführenden Schritt geändert wird.

ThisContext kann auch als Referenz an Modulfunktionen übergeben werden. So kann in externem Code die TestStand Umgebung ausgelesen und gesteuert werden. Eine Modulfunktion kann über den ThisContext zum Beispiel auf die Eigenschaften des aktuellen Schritts oder die lokalen Variablen der aufrufenden Sequenz zugreifen.



NAME	VALUE	TYPE
Locals ('MainSequence')		
ResultList		Array of Result[0..empty]
StringVariable	""	String
ContainerVariable		Container
123 NumberVariable	0	Number
TF BoolVariable	False	Boolean
<Right click to insert Field>		
<Right click to insert Local>		
Parameters ('MainSequence')		
<Right click to insert Parameter>		
FileGlobals ('Example - PSClass_H...')		
Execution		Container
System		Container
<Right click to insert File Global>		
StationGlobals		
ThisContext		Sequence Context
RunState		
Step ('Data SaveProperty')		Data_SaveProperty (Step)
TS		TEInf (Container)
Result		Container
Parameters		Container

Abbildung 18: TestStand Variablen

Abbildung 18 zeigt einen beispielhaften Ausschnitt der Variablen. TestStand unterstützt gängige Datentypen wie String, Boolean und Number, Enumerationen und Objektreferenzen werden ebenfalls unterstützt. Der Number-Typ hat standardmäßig die Repräsentation einer 64-Bit Gleitkommazahl, andere Repräsentationen wie 8-, 16-, 32- oder 64-Bit Integer können allerdings ebenfalls konfiguriert werden. Der Typ *Container* erlaubt das Erstellen einer Gruppierung von Variablen. Ein Container kann beliebige Variablen enthalten, inklusive weiterer Container. So können hierarchische Datenstrukturen abgebildet werden. Von allen Datentypen, inklusive Containern, können auch ein- und mehrdimensionales Array angelegt werden.

3.1.3 Step Types

Die in TestStand verfügbaren Funktionen werden als Typen von Schritten, den so genannten *Step Types*, in einer Auswahlliste auf der linken Seite, der *Insertion Palette*, zur Verfügung gestellt. Von Haus aus bietet TestStand hauptsächlich Schritte zur Ablaufkontrolle an. Die eigentliche Funktionalität eines Testprogramms ergibt sich aus den Schritten, mit denen externe Module aufgerufen werden können. Neben Schritten wie *Webservice Call* und *Call Executable* ist der *Action Step* das wichtigste Werkzeug in TestStand zur Umsetzung von Prüfprogrammen. Der Action Step erlaubt das Aufrufen von externen Codemodulen (etwa eine .dll- oder eine .py-

Datei), die in verschiedenen Sprachen und Technologien, wie C/C++, Python, LabView⁵ oder C#/.Net⁶ erstellt sind. Eine der größten Stärken von TestStand liegt darin, dass über Adapter die verschiedenen Technologien sehr gut und ohne großen Aufwand eingebunden werden können. Nach der Auswahl eines externen Codemoduls listet der Action Step alle Funktionen, Klassen und Methoden in diesem auf. Nach Auswahl einer Funktion werden alle definierten Parameter aufgelistet. Die TestStand Adapter sind in der Lage, zwischen allen gängigen Datentypen der jeweiligen Zieltechnologie und den Datentypen von TestStand zu konvertieren. Für jeden Parameter des Funktionsaufrufes kann eine Variable aus einem der TestStand Variablenkontexte angegeben werden.

Abbildung 19 zeigt den Aufruf der C-Funktion *PSClass_SetVoltage* in der Datei *psclass.dll*. Der Funktionsaufruf hat drei Eingabeparameter und einen Rückgabewert. Der TestStand C-Adapter legt für den *msg* Parameter automatisch einen 256 Byte großen Buffer an, auf den die C-Funktion zugreifen kann. Der Inhalt des Buffers wird nach dem Funktionsaufruf automatisch in einen String konvertiert und in die Variable *Result.ReportText* des Steps geschrieben.

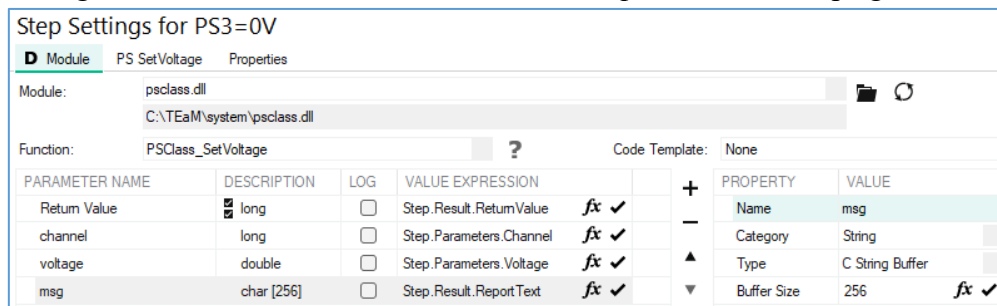


Abbildung 19: Modulkonfiguration eines TestStand Action-Steps

TestStand erlaubt das Erstellen von eigenen Step Types auf Basis der mitgelieferten Schritte. Die neuen Step Types können in einer Bibliotheksdatei gespeichert und so an andere Benutzer verteilt werden. Die Step Types sind dann in der Insertion Palette neben den standardmäßig vorhandenen Schritten auswählbar. So können gekapselte Codemodul-Aufrufe sehr einfach für alle Anwender in einer Organisation verfügbar gemacht werden.

⁵ NI LabView ist eine verbreitete grafische Programmierumgebung für die Entwicklung von Testsystemen und Prüfständen. [NI, 2025]

⁶ .Net ist eine Plattform zum Erstellen von Software aus dem Hause Microsoft. .Net unterstützt mehrere Programmiersprachen wie C#, F# und Visual Basic. Der Quellcode wird vom Compiler zu einem .Net Bytecode kompiliert, der in einer .Net Laufzeit ausgeführt wird. [Microsoft, 2025] TestStand integriert die .Net Modulschnittstelle. So werden in C# geschriebene Module eingebunden.

3.2 TEaM-Framework

Das TEaM-Framework ist die unternehmensweit standardisierte Funktionsbibliothek zur Erweiterung von TestStand bei Festo. Es besteht aus diversen DLLs (Dynamic Link Libraries)⁷ und Gerätetreibern, sowie einer großen Reihe von Step Types zur Integration dieser in TestStand. Das TEaM-Framework umfasst unter anderem diverse Kommunikations- und Feldbusprotokolle wie CAN⁸, IO-Link⁹ oder Profibus¹⁰. Ergänzend wird verschiedene Hardware unterstützt, wie zum Beispiel elektronische Netzteile, Digitalmultimeter und elektrische Lasten. Weiterhin gibt es ein Modul zum Erstellen von Reportfiles und detaillierter Datenanalysen, sowie ein Modul für die Anbindung an das unternehmensinterne SAP-System.

Ein TEaM-Modul für eine Technologie, zum Beispiel ein Netzteil, unterstützt diverse Varianten von unterschiedlichen Herstellern, mit verschiedenen Eigenschaften und Funktionen. Eine Hardware-Abstraktionsschicht erlaubt, dass alle unterstützten Geräte für den Anwender transparent mit den gleichen TEaM-Modulfunktionen genutzt werden können.

3.2.1 Funktionen

Die TEaM-Module sind nach einem einheitlichen Benutzungsparadigma aufgebaut. Zunächst muss eine Ressource, wie etwa eine Hardwarekomponente oder die Verbindung über ein Kommunikationsprotokoll, über einen *Init*- oder *Connect*-Schritt initialisiert werden. Diesem Schritt werden Parameter übergeben, etwa eine Zieladresse, ein zu nutzender Kommunikations-Port oder eine Gerätevariante. Der *Init*-Schritt gibt dann in der Regel einen Handle zurück, über den weitere Operationen auf der Ressource ausgeführt werden können. Vereinzelt wird der zu verwendende Handle auch vom Anwender vorgegeben, etwa der Name eines Dialogfensters.

Der Handle wird meist in Form eines Integers, teilweise auch als Objektreferenz zurückgegeben. Die *Init*-Funktion des Moduls erstellt intern Datenstrukturen mit allen wichtigen Informationen zur Interaktion mit der initialisierten Ressource und hält eine Liste aller vergebenen Handles mit Verknüpfung zu der jeweiligen Ressource. Auf diese Weise muss in TestStand vom Anwender keine komplexe Datenstruktur gehandhabt werden, sondern nur eine einfache Variable.

Neben dem *Init* besteht ein Modul hauptsächlich aus einfachen und gleichzeitig mächtigen Funktionen, wie etwa „Schreibe Parameter“, „Zeichne Spannungsverlauf auf“ oder „Starte Gerät neu“. Neben dem zuvor angelegten Handle benötigen die Funktionen je nach ihrer Natur mehr oder weniger zusätzliche Parametrierung. Abgesehen von der zuvor initialisierten Ressource sind die Funktionen in den meisten Fällen atomar und zustandslos. Der Nutzen von TEaM-Modulfunktionen besteht darin, ganze Anwendungsfälle oder Workflows in einzelne, möglichst einfach verständliche Aufrufe zu kapseln.

Der letzte Schritt in einem Programmablauf besteht darin, alle genutzten Ressourcen wieder ordnungsgemäß zu schließen. Dafür besitzen die TEaM-Module als Gegenstück zum *Init*-Schritt einen *Close*- oder *Disconnect*-Schritt. Dieser nimmt den zuvor angelegten Handle als

⁷ Dynamic Link Libraries (DLLs) sind Bibliotheken, die Code und Daten enthalten und mit denen Programme in separate Komponenten (Module) aufgegliedert werden können. DLLs können von mehreren Programmen geteilt werden. [Microsoft, 2026]

⁸ CAN (Controller Area Network) ist ein serielles Kommunikationsprotokoll für verteilte Echtzeitsysteme, unter anderem in der Automobilindustrie. [International Organization for Standardization, 2025]

⁹ IO-Link ist ein IEC 61131 Safety-konformes Punkt-zu-Punkt Protokoll zur Kommunikation mit Sensoren und Aktuatoren. [IO-Link Community, 2025]

¹⁰ Profibus ist ein weit verbreitetes Safety-konformes Feldbusprotokoll für Fertigungs- und Prozessautomatisierung. [PROFIBUS Nutzerorganisation, 2025]

Parameter, schließt offene Verbindungen und gibt alle reservierten Ressourcen ordnungsgemäß frei.

Externe DLLs bleiben in TestStand auch über eine Programmausführung hinaus geladen. Eine DLL wird beim ersten Aufruf einer ihrer Funktionen oder beim ersten Analysieren der DLL durch den Adapter von TestStand geladen und bleibt dann im Speicher, bis die DLL manuell entladen oder TestStand gänzlich geschlossen wird. Das bedeutet, dass auch angelegte Handles über mehrere Ausführungen hinweg bestehen bleiben. Dies kann leicht zu unerwarteten Seiteneffekten oder Konflikten führen, wenn versucht wird dieselbe Ressource mehrmals zu initialisieren. Deshalb ist es wichtig, dass die Init- und vor Allem die Close-Schritte bei jedem ProgrammDurchlauf ordnungsgemäß aufgerufen werden.

3.2.2 Parametrierung über Edit-Dialoge

Die Parametrierung einer DLL-Funktion in TestStand geschieht über die Felder in der Maske des jeweiligen Adapters. Die DLL-Funktionen besitzen in der Regel mehr Parameter als für den User direkt relevant sind, und die üblichen Benennungen von Parametern sind nicht jedem Anwender zumutbar, oder zumindest nicht immer intuitiv. Deshalb wurde für das TEaM-Framework der Mechanismus der *Edit-Dialoge* entwickelt, um die technischen Details zu abstrahieren und die Parametrierung für den Anwender so angenehm und verständlich wie möglich zu gestalten.

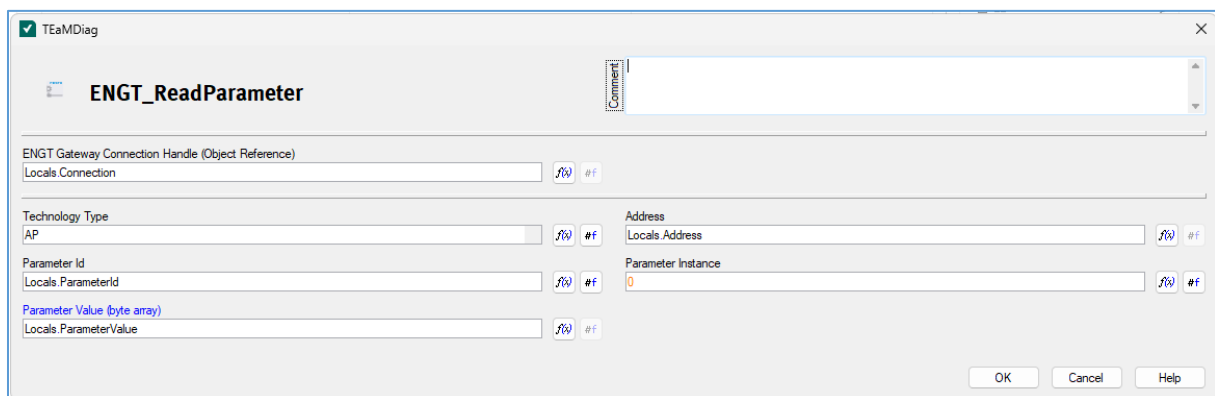


Abbildung 20: TEaM Edit-Dialog für den Step Type ENGT_ReadParameter

Abbildung 20 zeigt einen Edit Dialog für die Funktion *ReadParameter* für das Modul zum Festo-internen ENGT Kommunikationsprotokoll. Der Edit-Dialog ist ein Dialogfenster, in dem alle für den Anwender relevanten Parameter aufgelistet sind. Die Parameter sind mit sprechenden Titeln versehen, die besser verständlich sind als die Parameternamen der DLL-Funktion. Zusätzlich öffnet der „Help“-Button in jedem Dialog das Handbuch des Moduls und navigiert direkt zur Beschreibung der jeweiligen Funktion. Eingabeparameter sind schwarz und Ausgabe/Ergebnisparameter blau gekennzeichnet. Der Edit-Dialog wird über einen entsprechenden Button in den Eigenschaften und im Kontext-Menü des Steps, oder über die Tastenkombination **STRG+E** geöffnet.

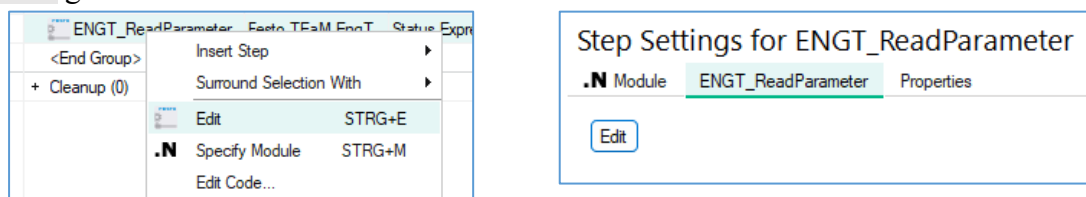


Abbildung 21: Buttons zum Öffnen des Edit-Dialogs

Die Parameterfelder im Edit-Dialog sind verknüpft mit Variablen, die direkt als Eigenschaften des Step Types im Parameter-Container angelegt sind. Der eigentliche DLL-Aufruf des Step Types ist so konfiguriert, dass für jeden Übergabeparameter der Funktion eine Variable des Steps übergeben wird. Als Parameterwert im Edit-Dialog kann entweder ein Literalwert oder ein Ausdruck eingegeben werden. Ein Literalwert wird direkt in die jeweilige Variable des Steps geschrieben. Für Ausdrücke wird der *Pre-/Post-Expression* Mechanismus von TestStand genutzt, der es erlaubt Variablenzuweisungen vor und nach der Ausführung eines Schrittes vorzunehmen. Ausdrücke für Eingabeparameter werden in der Pre- und Ausgabeparameter in der Post-Expression gesammelt. Ein Beispiel für diese Parametrierung ist in Abbildung 22 anhand des Step Types *ENGT_ReadParameter* zu sehen.

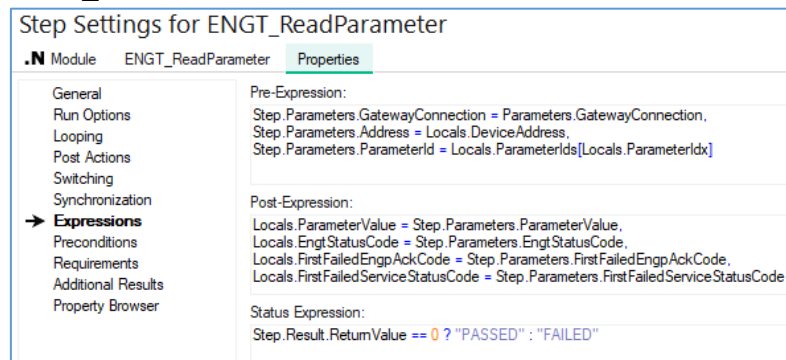


Abbildung 22: Pre- und Post-Expression eines TEaM-Modul Schrittes

Die Pre- und Post-Expression werden beim Ausführen des Steps vor, beziehungsweise nach Aufruf der DLL-Funktion evaluiert. So werden die während der Laufzeit aktuellen Werte der als Eingabeparameter konfigurierten Programmvariablen an den Schritt übergeben und die Ergebnisse des Schritts anschließend wieder an das Programm zurückgegeben.

Für die Definition der Edit-Dialoge wurde das Beschreibungsformat *TDI* (TEaM-Dialog) entwickelt. Die TDI-Definitionen werden in Form von Textdateien zusammen mit den TEaM-DLLs ausgeliefert. In einer TDI-Datei sind üblicherweise alle Dialoge zu einem TEaM-Modul zusammengefasst. Das folgende Beispiel zeigt die TDI-Definition zum zuvor gezeigten Edit-Dialog.

```
Dialog [ENGT_ReadParameter] {ENGT_ReadParameter}
  Property [ENGT Gateway Connection Handle (Object Reference)] {Step.Parameters.GatewayConnection}
  Separator
  Property [Technology Type] {Step.Parameters.TechnologyType}
    Choices [None] {0}
    Choices [AP] {1}
    Choices [Socket] {2}
    Choices [Pn] {3}
    Choices [Ec] {4}
    Choices [FreeRtosSockHandle] {5}
    Choices [BsdSockHandle] {6}
  Property [Address] {Step.Parameters.Address}
  Property [Parameter Id] {Step.Parameters.ParameterId}
  Property [Parameter Instance] {Step.Parameters.ParameterInstance}
  Property [Parameter Value (byte array)] {Step.Parameters.ParameterValue}
  RESULTEXPR
```

Abbildung 23: TDI Auszug für den Edit Dialog zum Step Type ENGT_ReadParameter

Ein Edit-Dialog wird über das Schlüsselwort *Dialog* eingeleitet. Der Dialog besitzt einen Titel, der in der Oberfläche angezeigt wird, sowie einen Bezeichner, über den die Zuweisung zum Step Type erfolgt. In einem Dialog werden über den Bezeichner *Property* die einzelnen Parameterfelder definiert. Hier werden wieder zuerst der Anzeigename und danach die zugehörige Eigenschaft des Steps angegeben. Im Anzeigenamen werden oft Hinweise zur Verwendung,

etwa zum erwarteten Datentyp, gegeben. Für einen Property-Eintrag kann auch eine Aufzählung von erlaubten oder Beispielwerten angegeben werden, wie hier für die unterstützten internen Technologietypen des Protokolls. Hier kann ebenfalls ein Anzeigename zusätzlich zum tatsächlichen Wert angegeben werden, um dem Anwender etwa die Bedeutung eines technischen Schlüssels zu erklären. Standardmäßig sind alle Properties Eingabefelder. Über das Schlüsselwort *ResultExpr* wird ein Feld als Ergebnisfeld deklariert. Durch diese Zuweisung wird das Feld als solches in der Oberfläche angezeigt, sowie der im Dialog übergebene Ausdruck in die Post-Expression des Steps übertragen. Die Felder innerhalb eines Dialoges können mit dem Schlüsselwort *Separator* noch einmal in Gruppen unterteilt werden, etwa um Pflichtfelder von optionalen Parametern abzugrenzen.

Die Integration des Edit-Dialogs in den TestStand Schritt erfolgt über den *Substep*-Mechanismus von TestStand. Substeps sind zusätzliche konfigurierbare Funktionen, die vor oder nach der Haupt-Modulfunktion ausgeführt werden. Abbildung 24 zeigt das Menü zum Anlegen von Substeps eines Step Types. Hier ist es auch möglich, Substeps vom Typ „Edit“ zu definieren. Diese werden dann als Button zur Programmierzeit zur Verfügung gestellt. Als Substep-Funktion ist ein Aufruf auf die Funktion *EditFunctionStep* der bei Festo entwickelten *teamdiag.dll* definiert. Dieser Aufruf nimmt unter Anderem das TDI-File, den Dialognamen, sowie das Handbuch-PDF und das entsprechende Kapitel als Parameter. Die Funktion interpretiert dann den Eintrag im TDI-File und zeigt den Dialog an.

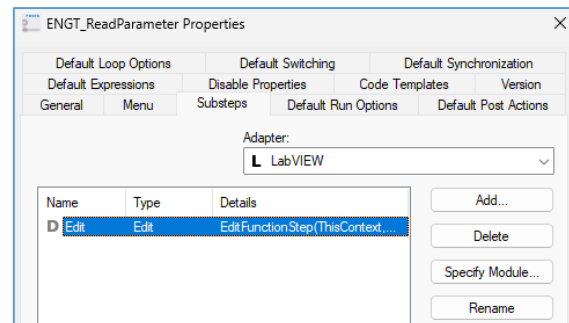


Abbildung 24: TestStand Substep-Menü eines Step Types

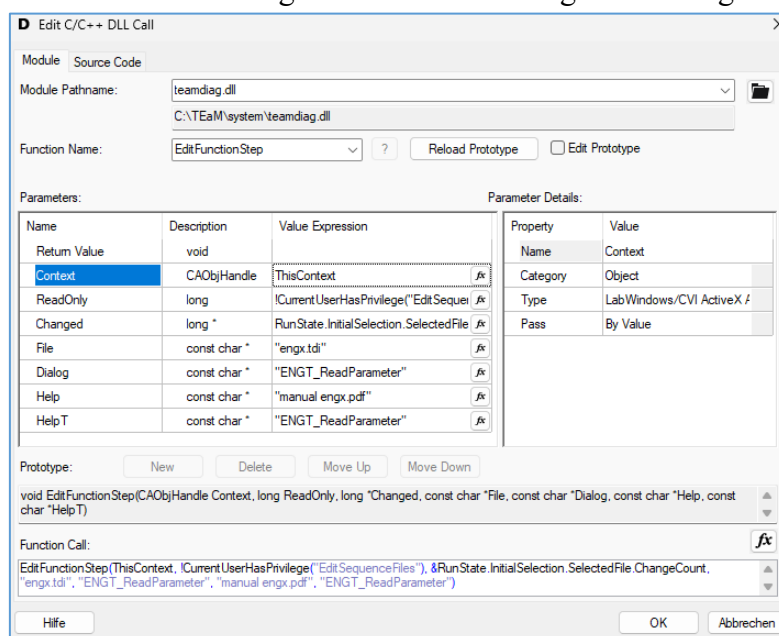


Abbildung 25: Funktionsaufruf teamdiag.dll für die Konfiguration eines Edit-Substeps

Die Konfiguration des Funktionsaufrufes ist in Abbildung 25 zu sehen. Beim Öffnen wird der aktuelle Inhalt der Pre- und Post-Expression des Schrittes analysiert und in die entsprechenden Dialogfelder eingetragen. Beim Schließen des Dialoges werden die Einträge von Pre- und Post-Expression mit dem Inhalt der Dialogfelder aktualisiert.

3.2.3 DLL-Aufbau

Alle TEaM-Module basieren auf bei Festo entwickelten DLLs. Diese sind zum Großteil in C und C# geschrieben. Die Funktionen aller TEaM-DLLs sind nach einem einheitlichen Muster aufgebaut. Jede Funktion besitzt einen Integer-Rückgabewert und der letzte Übergabeparameter ist ein String-Parameter mit dem Namen *msg*, *message*, o.Ä. Die Funktionen betreiben ein internes Fehlerhandling und geben Fehler über diese beiden Parameter an TestStand zurück. Der Integer Rückgabewert gibt 0 bei Erfolg und im Fehlerfall einen Return Code ungleich 0 zurück. Für jedes Modul sind die Return Codes in einer Liste im Handbuch dokumentiert. Der *msg* Parameter gibt im Fehlerfall eine zusätzliche Fehlerbeschreibung zurück.

Auch die C#-basierten DLL-Funktionen folgen diesem Muster. In C# üblich wäre, im Fehlerfall eine Exception zu werfen. Diese sind in TestStand allerdings nicht gut behandelbar, Exceptions führen in TestStand während der Ausführung zu einem Runtime Error, woraufhin ein Programmabbruch folgt. Dabei werden zwar noch alle Cleanup Blöcke ausgeführt, es gibt aber keine Einstellung, um einen Runtime Error als einen normalen Fehler handzuhaben. Das gilt auch für *NullReferenceExceptions*. Der Aufruf einer Methode auf einer uninitialisierten Objektreferenz führt zu einem Runtime Error, nach dem die Ausführung nicht wiederhergestellt werden kann.

Deshalb sind alle Funktionen der C#-basierten TEaM-Module statisch und nehmen Handle oder Objektreferenz als Parameter. Der Aufruf der Methode auf dem Objekt erfolgt dann intern, Exceptions werden in der DLL gehandhabt.

3.2.4 Standardisierte Prüfberichte

Neben den Prüfprogrammen selbst sind auch die standardisierten Prüfberichte von zentraler Bedeutung. Hierfür verwendet Festo das Dateiformat *TDMS*¹¹. TDMS ist ein binäres Format, deshalb eignet es sich im Gegensatz zu textbasierten Formaten wie JSON oder XML sehr gut dazu, auch große Datenmengen zu speichern.

Prüfberichte bei Festo enthalten Informationen zum Prüfling wie Produkttypcode und Seriennummer, Informationen wie Prüfsystembezeichnung und Testprogrammname und -version, sowie Informationen zur Ausführung wie Datum, Uhrzeit und Dauer. Darauf folgen die Ergebnisse aller durchgeführten Prüfungen mit Bezeichnung, Soll- und Grenzwert, Istwert und Ergebnis. Darüber hinaus werden nicht nur einzelne Kennwerte, sondern ganze Messwertkurven, so genannte Signale, gespeichert. Eine TDMS-Datei kann problemlos Millionen von Datenpunkten handhaben.

Das TDMS-Format von NI ist lediglich eine technische Spezifikation zum Speichern beliebiger Werte und Signale. Festo hat intern einen Standard für TDMS-Berichte erstellt, der den Aufbau der Datei und die Namen und Formate vieler Metadaten und Kennwerte vorgibt.

Als Erweiterung für den TDMS-Bericht wurde das Dateiformat TreX (Testreport eXtensible) definiert. Dabei handelt es sich um einen so genannten Report Container, der ein oder mehrere TDMS-Dateien, sowie weitere für den Bericht relevante Dateien, wie Log-Dateien und Bilder enthalten kann. Technisch handelt es sich dabei um ein Zip-Archiv mit geänderter Dateieindung.

Interessant, für mich als Nutzer....

¹¹ NI TDMS (Technical Data Management Streaming) ist ein binäres Dateiformat zum Speichern und Dokumentieren von Messdaten und Prüfergebnissen. TDMS unterstützt Streaming, wodurch hochfrequente Messdaten effizient direkt auf einem Datenträger gespeichert werden können. [NI, 2025]

3.2.5 Report Viewer & User Interface TEaMView

Da TDMS ein binäres Format ist, kann es nicht einfach mit einem Texteditor gelesen werden. Dafür wurde bei Festo die Software *TEaMView* entwickelt. Sie dient als Report Viewer für TDMS und TreX. Abbildung 26 zeigt einen TDMS/TreX Bericht in TEaMView mit mehreren Testergebnissen und einem Diagramm.

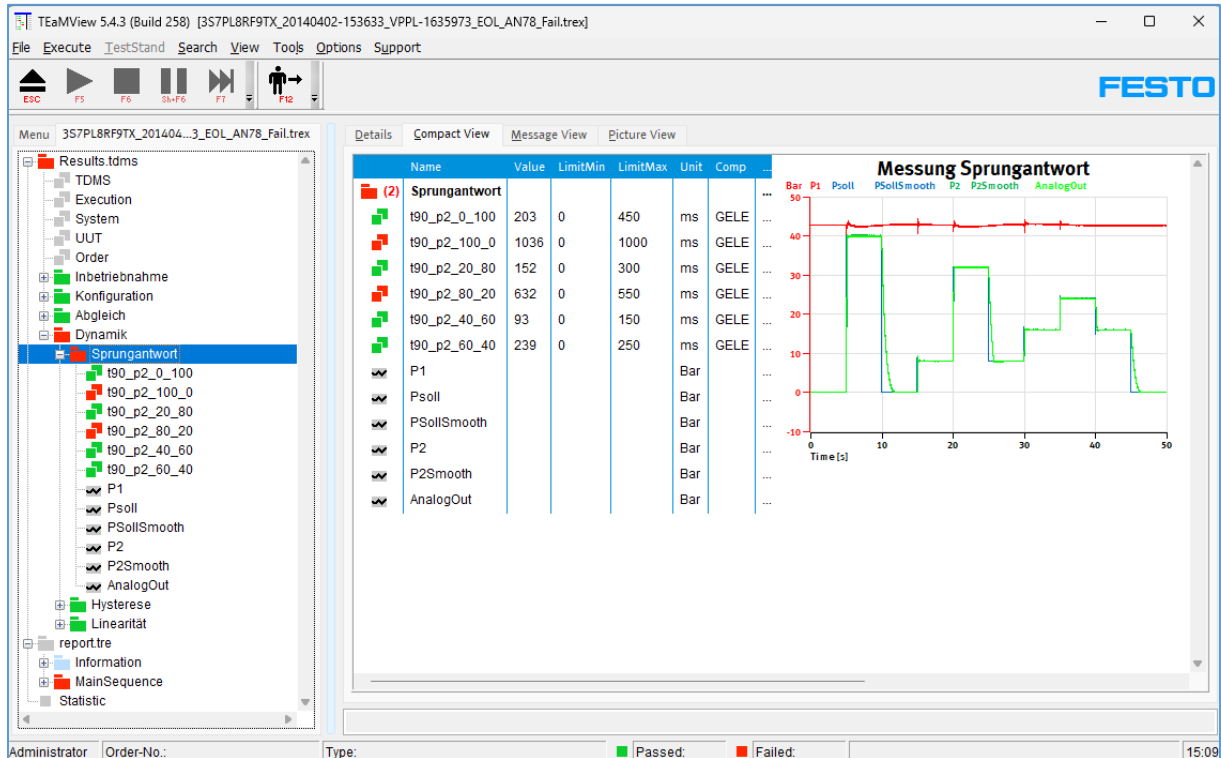


Abbildung 26: TEaMView Benutzeroberfläche mit TDMS-Bericht

TEaMView zeigt links in einem Baum den Inhalt des Berichts an. Im mittleren Bereich wird der Detailinhalt des aktuell ausgewählten Knoten angezeigt. Das Diagramm auf der rechten Seite ist kein statisches Bild, es ist ein Diagramm basierend auf realen Signaldaten und kann in der Anwendung durch Vergrößern/Verkleinern und weitere Einstellungen manipuliert werden. TEaMView ist kein reines Anzeigeprogramm für Prüfberichte. Es ist gleichzeitig auch ein User Interface für TestStand, mit dem Sequenzdateien ausgeführt werden können. Die Oberfläche ist bewusst einfach gestaltet und erlaubt lediglich das Ausführen von Testprogrammen, nicht jedoch das Bearbeiten. Das erlaubt die einfache Bedienung durch reine Anwender. Die Testergebnisse und die aufgezeichneten Diagramme können live während der Ausführung in der TEaMView Oberfläche angezeigt werden. Somit bekommt der Anwender eine unverzügliche Rückmeldung zu den Ergebnissen des aktuellen Testprogramms.

3.3 Fazit

TestStand ist eine blockbasierte Programmierungsumgebung, in der Testschritte aneinandergereiht und mithilfe vordefinierter grafischer Dialoge konfiguriert werden. Durch die mächtigen Schnittstellen können externe Codemodule sehr leicht eingebunden und gut in die Entwicklungsoberfläche integriert werden. Durch die Sammlung vordefinierter Funktionsbausteine, auch auf Basis externer Module, können Anwender sehr schnell mächtige Programme erstellen. Mit dem TEaM-Framework werden bereits viele Funktionsbausteine zur Nutzung diverser Technologien angeboten. Durch Mechanismen wird die Edit-Dialoge erweitert das TEaM-

Framework bereits die grafische Oberfläche von TestStand und bietet für die TEaM-Bausteine noch einmal vereinfachte und besser aufbereitete Konfigurationsmenüs an.

Die eigenentwickelte Vision-Lösung kann als weiteres Modul in das TEaM-Framework integriert werden. Dafür wird die Funktionalität in einer C oder C#-DLL implementiert und als Step Types mit Edit-Dialogen in TestStand eingebunden. Kameras können als Ressource wie ein Netzteil oder Feldbus-Master behandelt und über Handles angesprochen werden. Erkennungsmethoden können als eigene Step Types angeboten werden, die einen Kamerahandle als Eingabe nehmen und Erkennungsergebnisse als Array an TestStand zurückgeben.

Kamerabilder können zusammen mit dem TDMS-Bericht im TreX-Reportcontainer abgelegt werden. Livebilder könnten während der Programmlaufzeit auf der Oberfläche angezeigt werden. Zur Konfiguration der Erkennungsmethoden wäre ein Kameralivebild mit Ergebnisanzeige bereits zur Programmierzeit wünschenswert. Dafür könnte, wie auch für die Edit-Dialoge, der Substep-Mechanismus genutzt werden, um Dialoge mit Livebildern bereits zur Programmierzeit anzuzeigen.

TestStand und das TEaM-Framework bringen als einfache, grafische Programmierumgebung bereits viele der Eigenschaften mit, die zuvor als Vorteile von kommerziellen Vision-Systemen gegenüber freien Lösungen identifiziert wurden. Durch die Integration von Vision-Funktionalität auf Basis freier Software in TestStand und die Ergänzung benutzerfreundlicher Konfigurationsdialoge könnte eine Lösung geschaffen werden, die ernsthaft mit dem Funktionsumfang kommerzieller Lösungen konkurriert.

4 Entwurf der Anwendung

Nachdem nun die Eigenschaften von Vision-Systemen, die grundlegenden Mechanismen der Bilderkennung und die Zielumgebung für den Einsatz bekannt sind, kann in diesem Kapitel die Spezifikation für das Vision-Modul festgelegt werden.

Dafür werden zunächst die Anforderungen an das Modul gesammelt. Anschließend werden ein Bedienkonzept und mehrere Erkennungsmethoden entworfen, mit denen die Anforderungen umgesetzt werden können. Als nächstes werden die Softwarearchitektur und die zentralen Softwarekomponenten für die Umsetzung des Moduls modelliert. Zuletzt wird die Interaktion der Komponenten untereinander und mit dem Ökosystem anhand von mehreren Programmabläufen beschrieben.

4.1 Anforderungen

Die Anforderungen an das Vision-Modul sind gruppiert in vier Kategorien: Gruppe A1 beschreibt die Anforderungen zum visuellen Erkennen verschiedener Merkmale, in Gruppe A2 werden die Anforderungen für den Aufbau von Vision-Applikationen gesammelt. Gruppe 3 beinhaltet die Anforderungen an die Bedienbarkeit und Integration ins TEaM-Framework, Gruppe 4 sammelt nichtfunktionale Anforderungen zu Performance und Erweiterbarkeit.

4.1.1 A1 – Erkennungen

Die Kernfunktionalität des Vision-Moduls liegt im Wesentlichen in der Erkennung von technischen Merkmalen verschiedener Festo Produkte.

A1.1 – Komponenten: Mit dem Modul sollen mehrere gleiche oder verschiedene Komponenten eines Produktes erkannt werden, zum Beispiel Druckluftanschlüsse, Befestigungsklammern oder Abdeckkappen. Einzelne Merkmale können vielfach vorkommen und in ihrer Größe variieren. Für jedes Merkmal müssen Vorhandensein, Anzahl, Reihenfolge, Position und Typ unterscheidbar sein.

Deshalb müssen Erkennungsmethoden ein Ergebnis für jedes erkannte Merkmal zurückgeben und pro Ergebnis Position, Größe und ggf. Inhalt angeben. Die Präzision von Größe und Position muss eine Genauigkeit von plus minus 10% aufweisen.

Unter Annahme guter Bildqualität und angemessener Belichtungsverhältnisse sollen für 90% der getesteten Prüflinge alle Merkmale korrekt erkannt werden. Dafür können mehrere Bilder kurz hintereinander aufgenommen werden, wenn mit 5 oder weniger Bildern alle Merkmale korrekt erkannt werden, ist die Bedingung erfüllt.

A1.2 – DataMatrix-Codes: Mit dem Modul sollen DataMatrix-Codes erkannt und dekodiert werden. Es sollen mehrere Codes in einem Bild ermittelt werden können und für jeden Code ein Ergebnis zurückgegeben werden. Jedes Ergebnis soll Position, Größe und dekodierten Inhalt des Codes enthalten. Wenn aufgrund von schlechter Bildqualität der Code nicht vollständig oder nur fehlerhaft dekodierbar ist, soll kein Ergebnis anstatt eines fehlerhaften zurückgegeben werden.

A1.3 – Texte: Mit dem Modul sollen zusammenhängende Textblöcke erkannt und ausgelesen werden können. Für jeden zusammenhängenden Textblock im Bild soll ein Ergebnis mit Position, Größe und Inhalt zurückgegeben werden. Es sollen alle Texte erkannt und fehlerfrei ausgelesen werden, die auch durch einen Menschen korrekt und ohne Zweifel lesbar wären.

4.1.2 A2 – Applikationen

Mit dem Vision-Modul sollen in TestStand Prüfstände und Testapplikationen für den produktiven Einsatz entwickelt werden. Dafür muss das Modul vollständig in TestStand integrierbar sein.

A2.1 - Interface: Das Vision-Modul muss als Codemodul alle benötigten Funktionen so bereitstellen, dass sie in TestStand einbindbar und aufrufbar sind. Die Parametrierung von Erkennungsmethoden und allen weiteren Funktionen muss vollständig und direkt aus TestStand heraus möglich sein. Die Ergebnisse von Erkennungsmethoden müssen in einer Form zurückgegeben werden, die im TestStand Programm weiterverarbeitet werden können.

A2.2 - Kameras: Zum Aufnehmen von Bildern soll das Modul verschiedene Typen von Kameras ansteuern und konfigurieren können. Alle Kamerateypen sollen so weit wie möglich gleich konfigurierbar und benutzbar sein. Variantenspezifische Parametrierung muss in einer geeigneten, verallgemeinerbaren Form abgebildet werden.

Im ersten Schritt sollen die Baumer Industriekameras integriert werden. Zusätzlich soll eine virtuelle Kamera implementiert werden, mit der im Offline-Betrieb eine Kamera simuliert werden kann. Schnittstellen und Architektur sollen so aufgebaut sein, dass weitere Kamerateypen später ergänzt werden können.

A2.3 - Bildhandhabung: Kamerabilder sollen in einem einheitlichen Format bereitgestellt und weiter genutzt werden können. Als Bedienerückmeldung müssen während der Laufzeit Bilder auf der Oberfläche angezeigt werden können. In den Bildern sollen aus TestStand heraus Ergebnisse und sonstige Markierungen eingezeichnet werden können. Bilder sollen als Datei speicherbar sein.

4.1.3 A3 – Bedienung

Die Bedienbarkeit des Vision-Moduls soll so optimiert sein, dass Prüfsystem-Entwickler damit möglichst effizient Applikationen umsetzen können. Dazu sollen die etablierten Bedienparadigmen aus TestStand und dem TEaM-Framework befolgt werden.

A3.1 – Step Types: Alle Funktionen des Vision-Moduls sollen als Step Types in TestStand integriert werden. Jeder Step Type soll eine einfache, atomare und gut verständliche Funktion sein. Jede Funktion verfügt über Edit-Dialoge, in denen alle wichtigen Parameter gesetzt werden.

A3.2 - Handles: Kameras, Bilder und eventuelle weitere Komponenten sollen, wie im TEaM-Framework üblich, als Ressource behandelt werden. Eine Ressource wird initialisiert, über einen Handle können Aktionen auf der Ressource ausgeführt werden und zum Ende der Programmausführung wird die Ressource wieder geschlossen und freigegeben.

A3.3 – Livebilder: Erkennungsmethoden und Kameraeinstellungen sollen mithilfe von Livebildern der Kamera konfiguriert werden können, um schneller zu einer effektiven Parametrierung zu gelangen.

4.1.4 A4 - Nichtfunktionale Anforderungen

Die zuverlässige Erkennung von Merkmalen ist die wichtigste Anforderung an das Vision-Modul, für den effektiven Einsatz im Produktionsumfeld sind aber noch weitere nichtfunktionale Anforderungen entscheidend.

A4.1 - Performance: Die Testprogramme müssen schnell genug sein, um eine praktikable Durchlaufzeit zu gewährleisten. 5 bis 10 Merkmale pro Prüfling sollen in maximal 15 bis 20 Sekunden erkannt und beurteilt werden können. Zur Maximierung der Anwenderakzeptanz sollten idealerweise noch deutlich geringere Durchlaufzeiten erreicht werden.

A4.2: Langzeitstabilität: Testprogramme werden nicht nur einmal ausgeführt, sondern laufen in der Praxis kontinuierlich für mehrere Stunden, Tage oder sogar länger. Auch bei langer Laufzeit müssen die Funktionen des Vision-Moduls stabil bleiben. Der Speicherverbrauch soll konstant bleiben, Mechanismen zum Initialisieren und Freigeben von Ressourcen müssen sauber funktionieren.

A4.3: Erweiterbarkeit: Die Entwicklung des Vision-Moduls wird auch nach Abschluss dieser Arbeit noch weiter fortgeführt werden, um bestehende Funktionen anzupassen, neue Funktionen zu ergänzen und Fehler zu beheben. Deshalb muss die Vision-Bibliothek so konzipiert werden, dass weitere Funktionen mit überschaubarem Aufwand ergänzt werden können und Aktualisierungen möglichst einfach auf bestehende Systeme übertragbar sind.

4.2 Bedienkonzept

In diesem Abschnitt wird nun ein Bedienkonzept zum Umsetzen der zuvor gesammelten Anforderungen erarbeitet. Das Vision-Modul wird als eine Sammlung von Step Types in die Bibliothek des TEaM-Frameworks integriert. Jede Funktionalität ist somit als Step Type in der Insertion Palette verfügbar und wird von dort aus in das TestStand Programm eingefügt. Die Parametrierung der Schritte erfolgt zunächst über gängige TEaM Edit Dialoge, zusätzlich erhalten einige Schritte einen neuartigen *Live-Preview Dialog*. Die Gesamtheit des Vision-Moduls für das TEaM Framework, bestehend aus den Step Types und der darunterliegenden Softwarebibliothek, erhält an dieser Stelle den Namen *TEaMVision*.

4.2.1 Live-Preview Dialoge

Die Live-Preview Dialoge sind eine gänzlich neue Funktion, die es bisher in keinem anderen TEaM-Modul gibt. Ein großes Hindernis bei der Entwicklung von Vision-Lösungen ist oft das Feintuning der Parametrierung. Wenn das Ergebnis von Erkennungsmethoden nur während der Laufzeit des Programms sichtbar ist, muss für jede Änderung das Programm gestartet, das Ergebnis kontrolliert und das Programm wieder beendet werden. Dieses Vorgehen ist sehr zeitaufwändig und kräftezehrend für den Applikationsentwickler.

Deshalb ermöglicht der Live-Preview Mechanismus in TEaMVision, die Einstellungen einer Erkennungsmethode in einem GUI-Dialog anzupassen und in Echtzeit ein Ergebnis basierend auf einem Live-Kamerabild zu erhalten – Und das nicht im laufenden Programm, sondern bereits zur Programmierzeit.

Jede Erkennungsmethode erhält einen eigens definierten GUI-Dialog mit einem Livebild der Kamera und Kontrollelementen für alle Einstellungen der jeweiligen Methode.

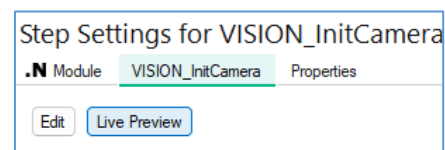


Abbildung 27: Live Preview Button neben Edit Dialog Button

Die Erkennung wird zyklisch auf jedem neuen Kamerabild ausgeführt und die gefundenen Ergebnisse werden auf dem Kamerabild als Annotation angezeigt. So kann der Applikationsentwickler Einstellungen anpassen und erhält sofort Rückmeldung über die Auswirkung der Änderung.

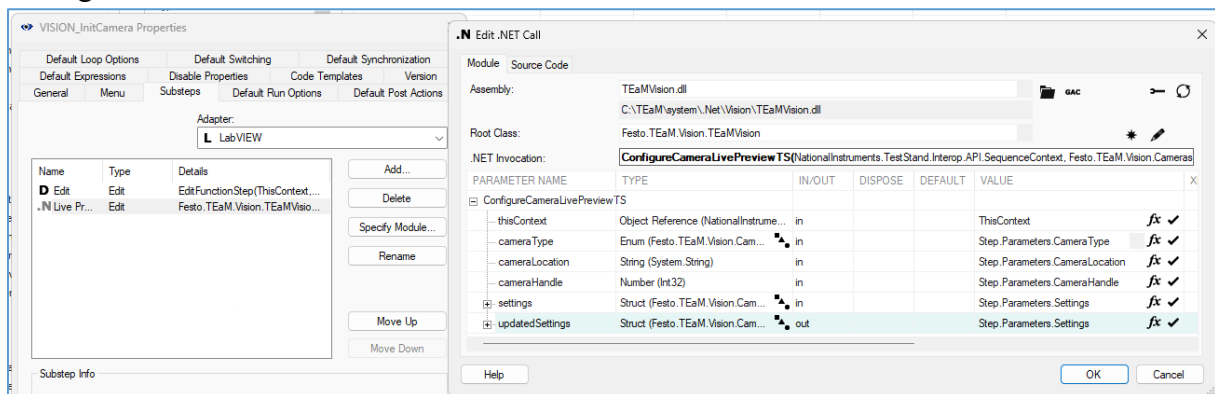


Abbildung 28: Moduldefinition Substep für Live-Preview Dialog

Der Live-Preview Dialog wird über einen zweiten Button neben dem Button für den gängigen Edit-Dialog aufgerufen. Abbildung 28 zeigt den Funktionsaufruf hinter dem Live-Preview Substep. Die Einstellungen für den Dialog sind als Parametervariable in dem TestStand Schritt hinterlegt. Beim Öffnen des Dialogs werden die Einstellungen an die DLL-Funktion für den Live-Preview Dialog übergeben, beim Schließen des Dialogs werden die aktualisierten Einstellungen wieder zurückgegeben und im TestStand Schritt gespeichert.

4.2.2 Kameraintegration

Für die Integration von Kameras wird der Step Type *InitCamera* eingeführt. Mit diesem wird eine Kamera angelegt, die für weitere Operationen genutzt werden kann. Die Initialisierung erfolgt über einen TEaM Edit-Dialog. Im Dropdown *Camera Type* wird einer von mehreren unterstützten Kamerateypen ausgewählt, in dem Feld *Camera Locator* wird ein String mit typspezifischen Informationen zur Initialisierung übergeben.

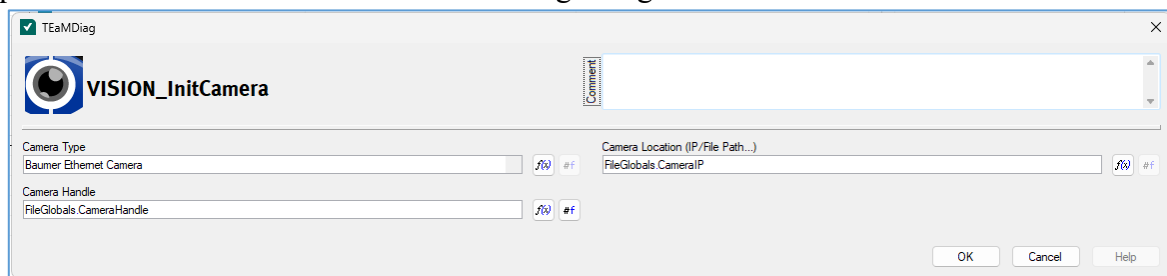


Abbildung 29: Edit-Dialog InitCamera

Für den ersten Schritt werden Ethernet-basierte Industriekameras vom Hersteller Baumer, sowie die so genannte *File Camera* unterstützt. Zusätzliche Kamerateypen können durch Erweiterung des Dropdown-Menüs ergänzt werden. Die *Baumer Ethernet Camera* nimmt zur Initialisierung die IP-Adresse der Kamera, die *File Camera* wird zur Offline-Simulation eingesetzt und nimmt den Dateipfad zu einem Bild, das bei jedem Aufruf zurückgegeben wird. Nach der Initialisierung werden alle Kameras gleich angesprochen.

InitCamera erhält ebenfalls einen Live-Preview Dialog, der in Abbildung 30 zu sehen ist. Im Hauptbereich in der Mitte wird das Livebild angezeigt, auf der rechten Seite befinden sich die Kontrollelemente für die verschiedenen Einstellungen. Das Textfeld am unteren linken Rand zeigt für jedes Bild die zum Auslesen benötigte Zeit, sowie Fehlermeldungen, etwa im Falle

einer Netzwerkunterbrechung. Die Elemente unten rechts zeigen die Pixelposition und die Farbe des aktuellen Pixels unter dem Mauscursor im Bild an. Der Button daneben erlaubt das Speichern des aktuellen Bildes auf der Festplatte, die Buttons *Ok* und *Abbrechen* schließen den Dialog. Bei *Ok* werden die Einstellungen gespeichert, bei *Abbrechen* werden sie verworfen.



Abbildung 30: Live-Preview Dialog InitCamera

Für jede Kamera kann eine *Region of Interest* (ROI), also ein vordefinierter Bildausschnitt definiert werden. Später abgerufene Kamerabilder enthalten dann nur den hier definierten Ausschnitt des Kamerabildes. Die ROI wird im Live Preview Dialog direkt im Bild ausgewählt. Dafür wird nach Klick auf *Select* ein rechteckiger Bereich durch Linksklicken und Halten, Maus ziehen und Linke Maustaste wieder loslassen definiert. Das Kamerabild kann über die Schaltfläche *Rotation* in 90 Grad Schritten gedreht werden, was sich ebenfalls auf die später abgerufenen Kamerabilder widerspiegelt. Die Felder oben rechts in *Parameters* unterscheiden sich je nach Kameratyp. So können weitere Funktionen abgebildet werden, die nicht jeder Kameratyp unterstützt. Die Baumer Kamera hier erlaubt zum Beispiel das Konfigurieren von Parametern zu Helligkeit und Belichtungsdauer.

Im Edit-Dialog wird ein Kamerahandle in Form eines Integer Werts festgelegt. Dieser kann direkt im Dialog definiert oder aus einer TestStand Variable übernommen werden. Üblicherweise werden Handles beim Aufruf einer Init-Methode automatisch generiert. Da der Kamerahandle und die initialisierte Kamera dahinter jedoch bereits zur Programmierzeit von verschiedenen weiteren Live-Preview Dialogen benötigt werden, ist eine automatische Vergabe hier nicht möglich.

4.2.3 Bildhandhabung

Für die Handhabung von Bildern wird das Konstrukt *Image Handle* eingeführt. Hinter einem Image Handle verbirgt sich ein Bild im Speicher, das von anderen Funktionen genutzt und auf dem weitere Methoden ausgeführt werden können. Abbildung 31 zeigt die Step Types zur Nutzung von Image Handles.

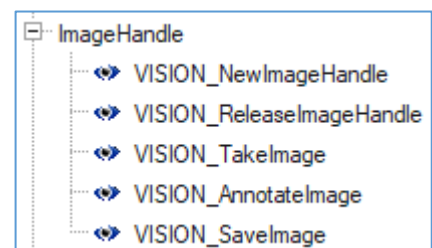


Abbildung 31: Image Handle Step Types

- ***NewImageHandle*** legt einen neuen Image Handle im Speicher an. Die Methode gibt einen Integer zurück, der in einer TestStand variable gespeichert und an weitere Methoden übergeben werden kann.
- ***ReleaseImageHandle*** gibt den Image Handle zum Ende des Programmablaufs wieder frei, wobei auch der reservierte Speicher des Bildes freigegeben wird
- ***TakeImage*** nimmt einen Camera Handle und einen Image Handle als Eingabeparameter. Die Methode nimmt über die gegebene Kamera ein Bild auf und speichert es im gegebenen Image Handle ab, so werden Bilder für weitere Operationen verfügbar gemacht.
- ***AnnotateImage*** erlaubt das Hinzufügen von Annotierungen in einem Image Handle. Annotierungen sind zum Beispiel ein farbiger Kreis oder ein Rechteck, die an einer definierten Stelle im Bild eingezeichnet werden. Somit kann der Applikationsentwickler nach Interpretation der Erkennungsergebnisse beliebige Markierungen im Kamerabild einfügen, um diese zu dokumentieren oder dem Anwender anzuzeigen.
- ***SaveImage*** kann das Bild hinter einem Image Handle auf der Festplatte abgespeichert werden. Die Methode nimmt einen Image Handle und einen Dateipfad als Parameter.

4.2.4 Erkennungsmethoden

Eine Erkennungsmethode ist ein atomarer Aufruf, der ein Bild und eine Reihe methodenspezifischer Parameter übergeben bekommt. Die Methode sucht in dem Bild nach Ergebnissen, die den gegebenen Parametern entsprechen. Als Rückgabe liefert sie eine Liste dieser gefundenen Ergebnisse, jedes Ergebnis beinhaltet methodenspezifisch festgelegte Eigenschaften wie Größe und Position. Dieser Abschnitt beschreibt zunächst die grundlegenden Eigenschaften aller Erkennungsmethoden, die einzelnen Methoden werden später im Detail vorgestellt.

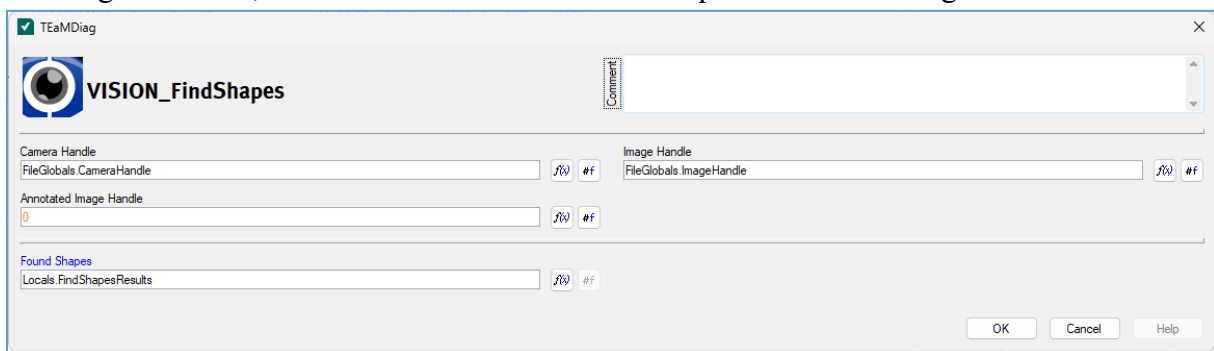


Abbildung 32: Edit-Dialog Erkennungsmethode

Jede Methode ist ein eigener Step Type, der einen Edit-Dialog und einen Live-Preview Dialog besitzt. Abbildung 32 zeigt beispielhaft den Edit-Dialog der Erkennungsmethode *FindShapes*, der Aufbau ist auf alle weiteren Erkennungsmethoden übertragbar.

Hier werden ein Kamerahandle und ein Image Handle übergeben. Der Kamerahandle wird zur Programmierzeit für die Anzeige und Verarbeitung von Livebildern im Live-Preview Dialog verwendet. Für die Durchführung der Erkennung zur Laufzeit wird ein Image Handle verwendet, der zuvor über einen TakeImage Schritt beschrieben werden muss. So können zur Laufzeit mehrere Erkennungen auf demselben Kamerabild ausgeführt werden. Das Aufnehmen eines neuen Bildes für jeden Schritt würde bei vielen Erkennungen die Durchlaufzeit deutlich beeinträchtigen.

Die Parametrierung der Erkennung erfolgt im Live-Preview Dialog. Dafür ist eine Datenstruktur mit Einstellungen in der Variablenstruktur des TestStand Steps hinterlegt, die beim Aufruf des Live-Previews übergeben, dort bearbeitet und beim Schließen wieder zurückgegeben und im Step gespeichert wird. Abbildung 33 zeigt so einen Live-Preview Dialog. Der Hauptbereich zeigt das Livebild der Kamera. Am oberen Rand kann der Vorschaumodus für das Livebild geändert werden, im Beispiel hier sind drei Vorschau Modi verfügbar. Zum einen das rohe, unverarbeitete Kamerabild, dann eine Vorschau für ein vorverarbeitetes Bild (dazu später mehr) und schließlich das Ergebnisbild, in dem alle gefundenen Ergebnisse als Annotationen eingezeichnet sind.

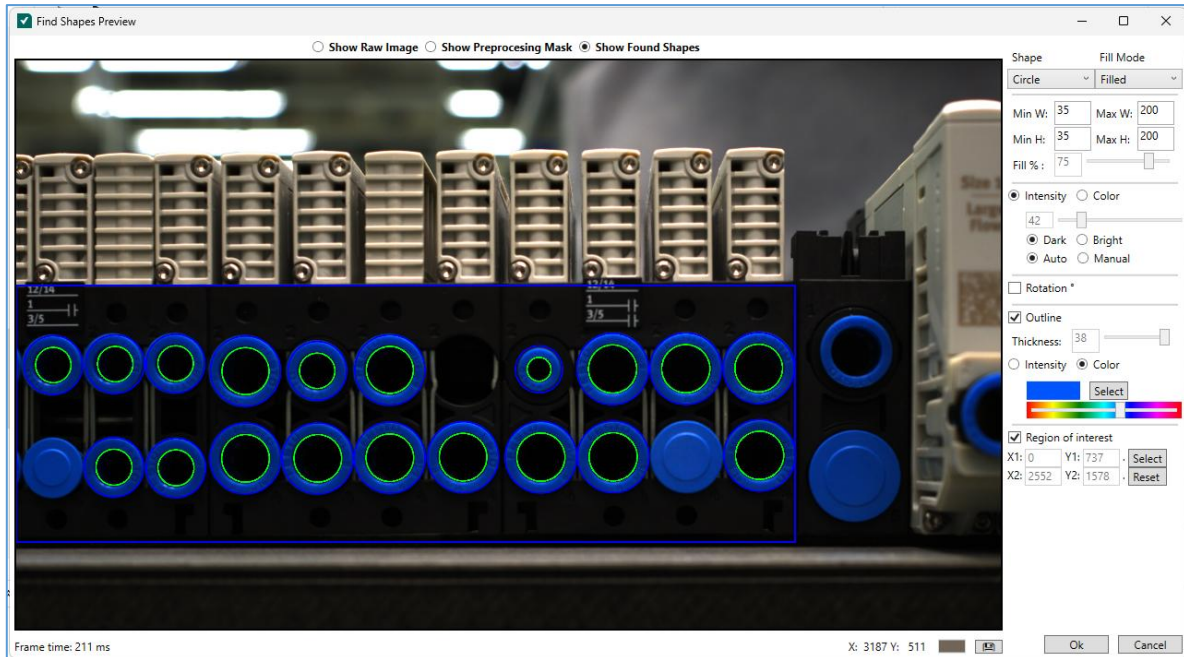


Abbildung 33: Live-Preview Dialog FindShapes

Die Art und Anzahl der verfügbaren Vorschau-Modi sind abhängig von der jeweiligen Erkennungsmethode, aber jede Erkennung besitzt eine Roh- und eine Ergebnisansicht. Unten links wird in einem Textfeld die Durchlaufzeit des letzten Erkennungszyklus angezeigt oder im unerwarteten Fehlerfall eine entsprechende Fehlermeldung. Unten rechts sind wieder die Koordinaten und die Farbe des Pixels unter dem Cursor im Bildbereich zu sehen, sowie der Button zum Abspeichern des aktuell angezeigten Bildes und die Buttons zum Schließen des Dialogs. Im Bereich an der rechten Seite sind die Einstellungen für die Erkennungsmethode aufgelistet. Die Einstellungen unterscheiden sich je nach Erkennungsmethode. Änderungen an einem Parameter werden direkt auf die Erkennungsmethode angewandt und produzieren somit beim nächsten Durchlauf direkt ein Ergebnis.

So wie die Kamera besitzt auch jede Erkennungsmethode neben den methodenspezifischen Einstellungen eine Region of Interest. Im Beispielbild ist die aktive ROI als blaues Rechteck im Ergebnisbild eingezeichnet. Die ROI definiert den Teilbereich des Bildes, in dem die Erkennung ausgeführt werden soll. Durch die Begrenzung des Bildbereiches wird zum einen die Performance der Erkennung verbessert und zusätzlich sinkt die Wahrscheinlichkeit für falsch positive Ergebnisse in den Randbereichen des Bildes, was die Stabilität der Erkennung erhöht. Auch hier wird die Region of Interest über den *Select* Button und anschließend mit Linksklick halten, Maus ziehen und Maustaste wieder loslassen festgelegt.

Bei Aufrufen eines Erkennungsschritts zur Programmlaufzeit wird die Erkennungsmethode auf einem gegebenen Bild mit den konfigurierten Einstellungen ausgeführt. Der Schritt gibt die gefundenen Ergebnisse in Form eines Array of Container zurück, das in der im Edit-Dialog definierten Zielvariable für das Ergebnis abgelegt wird. Die Ergebnisse können dann vom Applikationsentwickler beliebig weiterverarbeitet werden.

4.2.5 Image Message Box

Zum Anzeigen von Bildern auf der Benutzeroberfläche zur Laufzeit wird die *Image Message Box* definiert. Das ist ein Dialogfenster ähnlich dem Live Preview Dialog. Es zeigt das Bild an, das in einem gegebenen Image Handle hinterlegt ist. Abbildung 34 zeigt die Step Types zum Arbeiten mit der Image Message Box.

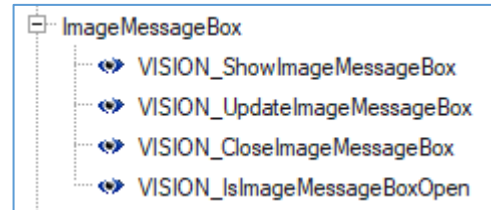


Abbildung 34: ShowImageMessageBox Step Types

- **ShowImageMessageBox** öffnet einen neuen Dialog. Eingabeparameter sind ein Image Handle und der Name des Dialogs. Der Dialogname ist gleichzeitig der Identifizierer und der Handle für das Dialogfenster.
- **UpdateImageMessageBox** aktualisiert das im Dialog angezeigte Bild. Änderungen am Bild im Image Handle werden nicht automatisch auf den Dialog übertragen, das passiert nur bei Aufruf dieser Methode.
- **CloseImageMessageBox** schließt den Dialog. Die Update- & Close-Methode nehmen beide zur Identifizierung den Dialognamen als Parameter.

Die Image Message Box wird vom Applikationsentwickler genutzt, um dem Anwender zur Laufzeit Rückmeldung oder Anweisungen zu geben. Der Applikationsentwickler hat in Verbindung mit AnnotateImage die volle Freiheit, welche Erkennungsergebnisse oder daraus gewonnene Erkenntnisse dem Anwender angezeigt werden.

4.2.6 Datenstrukturen

Verschiedene Bausteine wie die Kamera und die Erkennungsmethoden benötigen diverse Parameter für ihre Einstellungen. Gleichmaßen geben die Erkennungsmethoden als Ergebnis komplexe Datenstrukturen zurück, die vom Anwender weiterverarbeitet werden sollen. Für die Einstellungsparameter und die Erkennungsergebnisse werden in TestStand eigene Datentypen angelegt. Diese Datentypen sind TestStand Container mit festgelegten Feldern, vergleichbar mit Strukturtypen in anderen Programmiersprachen. Abbildung 35 zeigt den Datentyp für die Einstellungen von Camera. Er enthält die Einstellungen für die Region of Interest, die Bildrotation und ein Array für die weiteren kameraspezifischen Parameter. Die Parameter sind wieder als eigener Datentyp definiert, ein *CameraParameter* hat einen Namen, einen String-kodierten Wert, ein Feld für den Datentyp des dekodierten Wertes und zulässige Min- und Max-Werte.

Vision_CameraSettings			Container, Type ...
Parameters			Array of Visio...
UseRoi	False		Boolean
RoiX1	0		Number
RoiY1	0		Number
RoiX2	0		Number
RoiY2	0		Number
RotateDeg	0		Number
<Right click to insert Field>			
Vision_CameraParameter			Container, Type ...
Name	""		String
_Value	""		String
Type	""		String
MinValue	""		String
MaxValue	""		String
<Right click to insert Field>			

Abbildung 35: TestStand Datentypen für Kameraeinstellungen

Abbildung 36 zeigt den Datentyp *Shape*. Er wird als Erkennungsergebnis von der Methode *FindShapes* zurückgegeben. Er beschreibt ein Objekt mit einer spezifischen Form und Größe an einer bestimmten Position. Zusätzlich dient der gleiche Datentyp auch als Eingabe an *AnnotateImage*, deshalb enthält er zusätzlich Felder für die Farbe, in der das Objekt im Bild eingezeichnet werden soll. Für die Form des Objekts wird ein eigener Enumerationstyp *Shapes* definiert, der die unterstützten Formen enthält.

Field	Value	Type
X	123	Number
Y	123	Number
Width	123	Number
Height	123	Number
ShapeType	Circle (0)	Vision_Shapes (Enumeration)
ColorR	123	Number
ColorG	123	Number
ColorB	123	Number

Abbildung 36: TestStand Datentyp für Erkennungsergebnis

Solche Datentypen werden für alle Erkennungsmethoden erstellt, dadurch lassen sich die Einstellungen in einem einzigen Feld im TestStand Step abbilden und können als einzelne Variable an die Funktionen im entsprechenden Codemodul übergeben werden. Durch die vordefinierten Datentypen der Erkennungsergebnisse ist dem Anwender direkt klar, welche Felder er von der Methode zurückbekommt. Abbildung 36 zeigt eine lokale Variable *DetectionResults* in einer TestStand Sequenz, in die bereits ein Array-Element vom Typ *Shape* eingefügt wurde. Durch den vordefinierten Typ sieht der Anwender klar die verfügbaren Felder und bekommt diese zum Beispiel auch bei der Auto-Vervollständigung vorgeschlagen.

Ein weiterer Vorteil der vordefinierten Container-Datentypen ist, dass diese sehr komfortabel zwischen TestStand und verschiedenen Codemodulen ausgetauscht werden können. Abbildung 37 zeigt die Einstellungen für das TestStand *Struct Passing* für den Datentyp *CameraSettings*.

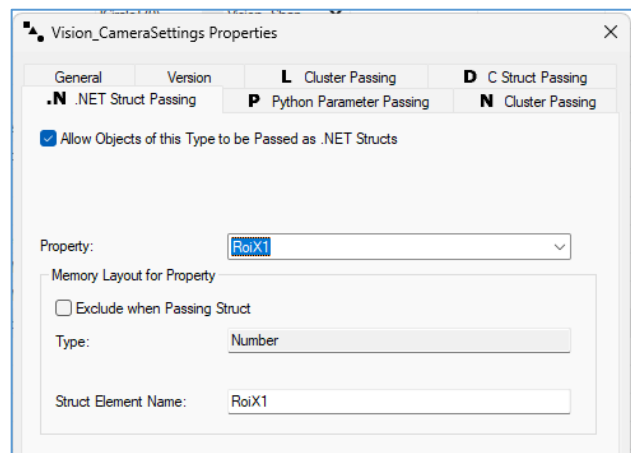


Abbildung 37: Struct Passing Einstellungen von TestStand Datentyp

Ein TestStand Container wird dabei direkt zu einer Instanz eines Strukturtypen übersetzt, für jedes Feld des TestStand Typs wird der Name der Eigenschaft im Zieldatentyp festgelegt. Zusätzlich kann der Mechanismus auf die gleiche Weise auch Enumerationstypen übersetzen. Mit dem Struct Passing können auch komplexe Datenstrukturen sehr komfortabel zwischen TestStand und externen Codemodulen verschiedener Technologien, hier .Net, übergeben werden.

4.3 Beschreibung der Erkennungsmethoden

Nachdem die Erkennungsmethode zuvor im Bedienkonzept als eine der Kernkomponenten von TEaMVision allgemein eingeführt wurde, werden hier die verschiedenen Erkennungsmethoden, ihre Parameter und ihre Preview Dialoge nun im Detail beschrieben.

4.3.1 FindShapes

Viele der zu erkennenden Produktkomponenten heben sich gut vom Hintergrund ab, entweder durch ihre Farbe oder einen Helligkeitsunterschied. Diese Objekte haben in der Regel eine runde oder annähernd rechteckige Form und gleichbleibende, bekannte Dimensionen. Für diese Szenarien bietet sich eine Blob-Analyse an.

Im Rahmen des VISUS-Vorprojekts wurde die Erkennung von Druckluftanschlüssen bereits erfolgreich über einen Hell-Dunkel Kontrast realisiert. Dabei wurde das Bild anhand eines Helligkeitsgrenzwertes binarisiert und in diesem anschließend nach geschlossenen und gefüllten kreisförmigen Konturen gesucht. Dieser Ansatz kann auch zur Erkennung weiterer einfacher Objekte verallgemeinert werden.

FindShapes generiert für ein Bild eine Binärmaske anhand vorgegebener Parameter + Grenzwerte und sucht in dieser Maske nach Konturen. Die gefundenen Konturen werden anschließend nach weiteren Kriterien gefiltert. Für jede verbleibende Kontur wird ein Ergebnis erzeugt und zurückgegeben.

Abbildung 38 zeigt den Live-Preview Dialog von FindShapes. Im Vorschaubild ist die Maskenansicht zu sehen. Dabei wird die intern generierte Binärmaske als Overlay über das Originalbild gelegt, um anzuzeigen welche Bildbereiche wie binarisiert wurden. Rote Pixel repräsentieren hier den Wert 1 in der Binärmaske, die restlichen Pixel werden nicht eingefärbt. Es ist gut zu sehen, dass die Innenräume aller Druckluftanschlüsse durch die Binarisierung erfasst werden und dass sie durch die blaue Fläche vom Hintergrund abgetrennt werden. Somit können alle Anschlüsse als Kontur erkannt werden.

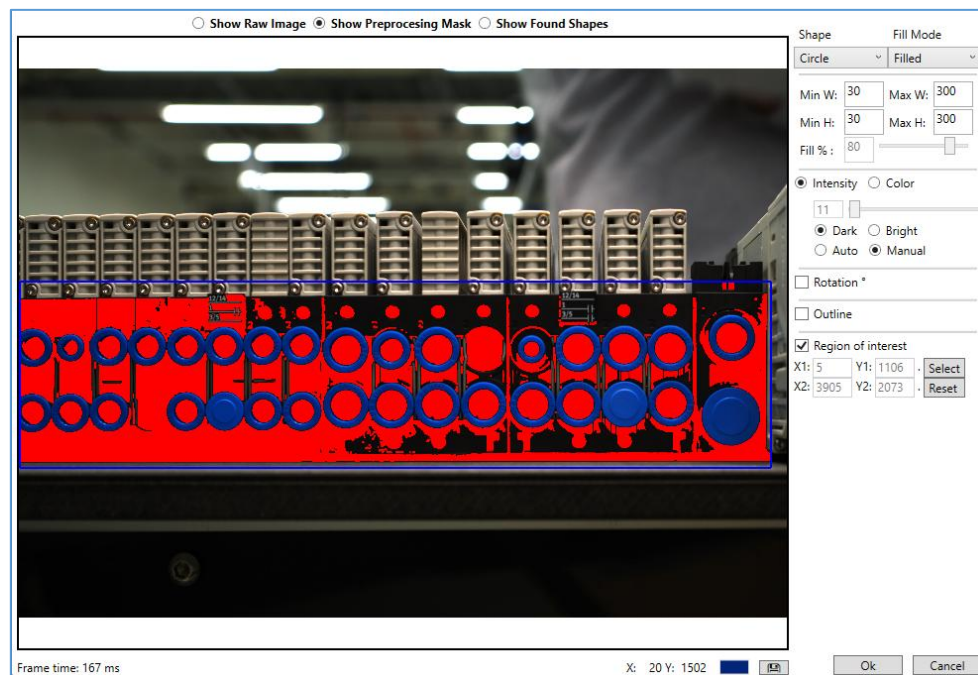


Abbildung 38: FindShapes Live Preview mit Binärmasken Ansicht

Für die Filterung von Konturen sind oben rechts mehrere Einstellungen vorhanden.

- **Shape:** In dem Dropdown-Menü kann die gewünschte Form der Kontur gewählt werden. Zunächst sind hier Kreise und Rechtecke auswählbar, weitere Formen können bei Bedarf in Zukunft ergänzt werden. Die Form einer Kontur wird unter anderem über das Verhältnis aus Fläche und Umfang klassifiziert.
- **Fill Mode:** In dem Dropdown kann zwischen *Filled* und *Contour* gewählt werden. Der Filled-Modus sucht nach komplett gefüllten Objekten, während der Contour-Modus nur nach Konturen mit der korrekten Form sucht, die im Inneren nicht gefüllt sind.
- **Min / Max W / H:** Die vier Filter geben Minimal- und Maximalwerte für Breite und Höhe in Pixeln vor. Breite und Höhe beziehen sich auf das Rechteck, das die Kontur einschließt.

- **Fill %:** Über einen Schieberegler wird der minimale Füllgrad einer Kontur vorgegeben. Der Füllgrad ergibt sich aus dem Verhältnis der Fläche der gesuchten Form und der Anzahl von Pixeln in der gefundenen Kontur.
- **Thickness:** Der Schieberegler gibt die Dicke einer gesuchten Kontur vor. Der Regler ist nur im Contour-Modus aktiv und sichtbar. (Siehe Abbildung 41)

Mit den weiteren Einstellungen wird die Bildung der Binärmaske parametrisiert.

- **ROI:** Die Maske in Abbildung 38 wird nur im mittleren Bereich des Vorschaubildes gebildet. Die Region of Interest gibt den von einem blauen Rechteck umschlossenen Arbeitsbereich der Erkennung vor. Die ROI wird über das Menü unten rechts konfiguriert. Da die Maske nur innerhalb der ROI gebildet wird, werden auch nur in diesem Bereich Erkennungsergebnisse gefunden.
- **Intensity:** Die Bildung der Maske geschieht über einen Intensitätsgrenzwert. Der Schieberegler gibt den Grenzwert vor, über/unter dem die Helligkeit eines Pixels als 1 oder 0 interpretiert wird.
- **Dark / Bright:** Für die Binarisierung gibt es einen Hell- und einen Dunkelmodus. Im Hellmodus wird jedes Pixel oberhalb des Grenzwertes, im Dunkelmodus jedes Pixel darunter als 1 interpretiert. Somit lassen sich entweder helle Objekte auf dunklem oder dunkle auf hellem Hintergrund erkennen.
- **Auto / Manual:** Die Wahl des korrekten Intensitätsgrenzwertes ist stark abhängig von den Belichtungsverhältnissen im Kamerabild. Deshalb bietet FindShapes einen Automatikmodus für die Ermittlung des Grenzwertes. Im Automatikmodus wird die Erkennung intern mehrmals ausgeführt, für verschiedene Intensitäten bis hin zum vorgegebenen Grenzwert. Ausgewählt wird dann der Wert, für den die Erkennung die meisten Ergebnisse findet.

An dem folgenden Beispiel wird ersichtlich, warum in verschiedenen Fällen eine weitere Filterung notwendig ist. In Abbildung 39 werden neben den schwarzen Kreisen innerhalb der Druckluftanschlüsse auch viele weitere schwarze Kreise gefunden. Die Auswahl des Grenzwertes mit den meisten Ergebnissen favorisiert auch die Generierung von falsch-positiven Ergebnissen, wodurch eigentlich gesuchte Objekte sogar wieder verworfen werden können.

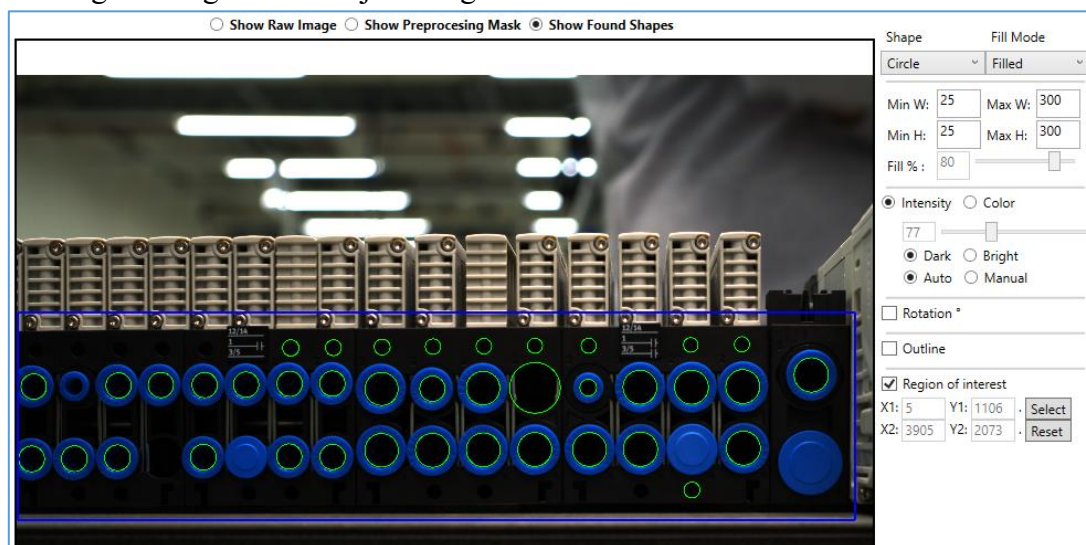


Abbildung 39: Ausschnitt FindShapes Live Preview mit variabler Intensität

Die **Outline** Funktion ist ein weiterer Filter, mit dem für jede gefundene Kontur eine weitere Umrandung gesucht wird. In Abbildung 40 wird für jedes gefundene Ergebnis überprüft, ob dieses von einem farbigen Rahmen mit vorgegebener Dicke umgeben ist.

Der Farbton der Outline kann über einen Schieberegler oder direkt im Bild ausgewählt werden. Nach Klick auf *Select* kann der Anwender ein Pixel im Bild anklicken und der Farbton wird direkt daraus übernommen. Alternativ kann wie bei der Maskenbildung auch eine Intensität für die Outline angegeben werden, so kann etwa nach einem dunklen Objekt gesucht werden, das in einem Mindestbereich von hellen Pixeln umgeben ist. In Abbildung 40 ist gut zu erkennen, dass die falsch-positiven Ergebnisse aus Abbildung 39 erfolgreich entfernt wurden und jetzt alle Druckluftanschlüsse gefunden werden.



Abbildung 40: Ausschnitt FindShapes Live Preview mit aktivem Outline Modus

So wie die Outline eines Objektes nach Farbe gefiltert werden kann, ist es auch möglich, direkt nach farbigen Objekten zu suchen.

Im **Color** Modus wird die Binärmaske nicht nach der Helligkeit, sondern für einen bestimmten Farbton gebildet. Abbildung 41 zeigt den Farbmodus für die Maskenbildung.

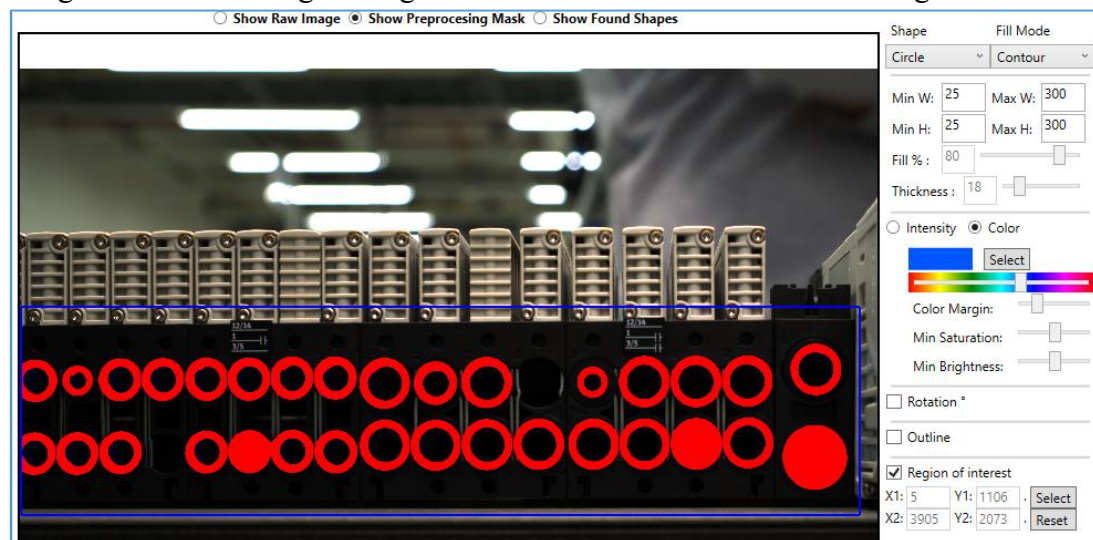


Abbildung 41: Ausschnitt FindShapes Preview Dialog mit Farbmodus für Maskenbildung

Der Farbton kann wieder entweder per Schieberegler oder über den *Select* Button direkt im Bild ausgewählt werden. Über zwei weitere Schieberegler kann die minimale Farbsättigung und die minimale Helligkeit jedes Pixels festgelegt werden.

Der Grundmechanismus, der bereits im VISUS-Projekt vielversprechende Ergebnisse lieferte, wird mit der FindShapes Methode einem breiten Anwenderfeld zugänglich gemacht. Durch die beliebige Kombinierbarkeit der Parameter und Filteroptionen kann der Mechanismus voraussichtlich für eine Vielzahl von Anwendungsfällen eingesetzt werden.

4.3.2 TemplateMatching

Nicht alle Merkmale können durch eine einfache Blob-Analyse abgedeckt werden. Eine pikto-graphische Kennzeichnung neben einem Anschluss oder ein Symbol auf einem Schwarz-Weiß Display kann kaum als einfache geometrische Form beschrieben werden und es ist sicherlich keine Unterscheidung zweier ähnlicher Symbole möglich.

Die **TemplateMatching** Erkennungsmethode nimmt einen Bildausschnitt (das Template) als Vorlage und sucht in einem gegebenen Bild nach Übereinstimmungen.

Abbildung 42 zeigt den Live Preview Dialog von TemplateMatching. Im Livebild der Ergebnisansicht werden alle Übereinstimmungen mit dem Template eingezeichnet. Das aktive Template wird rechts oben angezeigt. Ausgewählt wird das Template mit den Buttons darunter.

- **Select** erlaubt das Auswählen eines Templates direkt im Livebild, ähnlich wie bei der ROI.
- **Save** öffnet einen Dialog zum Abspeichern des Templates als Bilddatei.
- **Load** öffnet einen Dialog zum Laden einer Bilddatei und setzt diese als aktives Template.



Abbildung 42: Live-Preview für Template Matching

Für jedes Ergebnis wird der Übereinstimmungsgrad mit dem Template angegeben. Der Schieberegler **Match %** gibt den minimalen Übereinstimmungsgrad für die Ergebnissuche an. Alle Ergebnisse mit einer niedrigeren Übereinstimmung werden verworfen.

Im Beispiel hier wird nach Schildern mit einer pneumatischen Funktionsbeschreibung des jeweiligen Ventils gesucht. Erwartungsgemäß werden beide Schilder erkannt, die beiden Abbildungen unterscheiden sich aber so weit, dass das Schild rechts eine Übereinstimmung von 96% erreicht und das Schild links lediglich 82%. Somit können über den Übereinstimmungsgrad ähnliche Merkmale abgegrenzt und falsch-positive Ergebnisse ausgefiltert werden.

Darunter befindet sich der Schieberegler **Rotation**. Das Template Matching ist potenziell sehr empfindlich für die Orientierung des Motives, bereits wenige Grad Drehung können die Übereinstimmung und somit die Erkennungsrate deutlich verschlechtern. Deshalb kann hier ein maximaler Rotationswert R angegeben werden. Das Template wird dann in mehreren Schritten von -R bis R rotiert und damit mehrfach die Erkennung ausgeführt. Die Ergebnisse aller Rotationsschritte werden kombiniert und bei Überschneidung das mit der höchsten Übereinstim-

mung beibehalten. Die Rotation und wiederholte Ausführung der Erkennung verlangsamen natürlich die Gesamtdurchlaufzeit, deshalb ist der Wert standardmäßig niedrig gesetzt und sollte nur mit Bedacht höher gewählt werden.

Template Matching reagiert empfindlich auf Helligkeitsunterschiede, Farbabweichungen und sonstige Artefakte. Wirklich interessant ist nicht die Übereinstimmung jedes einzelnen Pixels, sondern die Übereinstimmung von klar erkennbaren Strukturen im Bild. Deshalb werden in der Vorverarbeitung Rauschen und Artefakte minimiert und Kanten, sowie sonstige markante Strukturen herausgestellt.

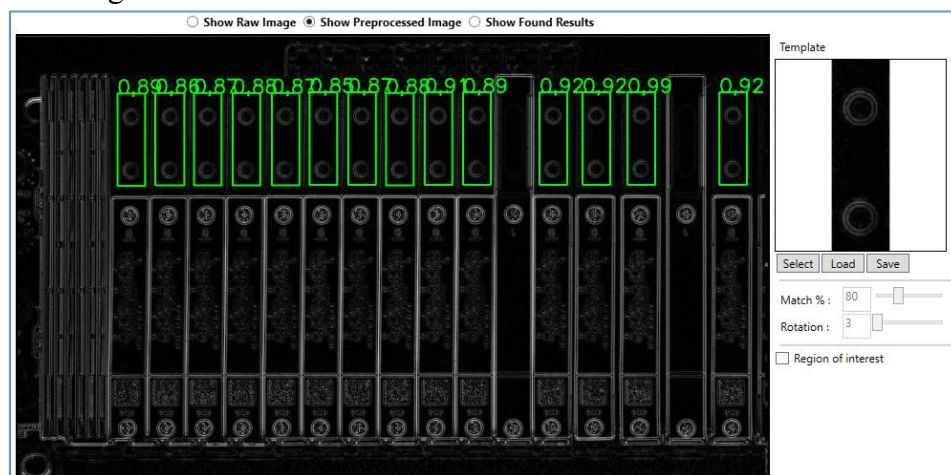


Abbildung 43: Template Matching Live Preview mit Preprocessing Ansicht

In Abbildung 43 sind das vorverarbeitete Bild und Template zu sehen. Durch eine adäquate Vorverarbeitung werden bei dem Pixel-für-Pixel Vergleich nur noch die formgebenden Merkmale technischer Komponenten verglichen.

Beim Klick auf Ok wird der Dialog geschlossen und die Einstellungen werden im TestStand Step gespeichert. Das Template Bild wird nicht direkt in einer TestStand Variable abgelegt. Es wird als Datei auf der Festplatte gespeichert und der Dateipfad wird als Parameter in der Settings-Datenstruktur hinterlegt.

Abbildung 44 zeigt die Einstellungs- und Ergebnis-Datenstrukturen der TemplateMatching Methode in TestStand. Die Einstellungen enthalten den Pfad zur Template-Datei, die Parameter für die Region of Interest und die beiden Werte für minimalen Übereinstimmungsgrad und Rotationsbereich. Jedes Ergebnis enthält neben Größe und Position die Rotation und den tatsächlichen Übereinstimmungsgrad.

Settings	
123 MinMatchingScore	75
123 RotationMargin	5
TemplateFile	"C:\App\Work\St...
UseRoi	False
123 RoiX1	0
123 RoiX2	0
123 RoiY1	0
123 RoiY2	0
FoundResults	
[0]	
123 X	131
123 Y	418
123 Rotation	1
123 Score	86
123 Width	65
123 Height	98

Abbildung 44: Datenstrukturen für Einstellungen und Ergebnisse von TemplateMatching

Der Anwender kann im TestStand Programm mehrere TemplateMatching Schritte kombinieren, um verschiedene ähnliche Varianten eines Merkmals zu unterscheiden. Mit dem Übereinstimmungsgrad kann für jede Position im Bild das Ergebnis der Erkennung mit der höchsten Übereinstimmung gewählt werden. Das ermöglicht die Unterscheidung und somit die Klassifizierung als eine bestimmte Variante des Merkmals.

4.3.3 DmtxDetection

Praktisch jedes Produkt hat eine eindeutige Identifikationsnummer, den *Product Key*. Dieser wird neben Klartext auch als DataMatrix-Code (DMTX) auf dem Produkt abgebildet. Komplexe Produkte bestehen oft aus mehreren Einzelkomponenten, die wiederum eigene Product Keys in Form von Dmtx-Codes besitzen. An Prüfständen werden die Dmtx-Codes üblicherweise per Handscanner eingelesen. Für die Testautomatisierung mit Vision-Funktionen liegt es also nahe, DataMatrix Codes und andere Barcodes erkennen und dekodieren zu können.

Die Erkennungsmethode **DmtxDetection** erkennt und dekodiert alle DataMatrix-Codes in einem gegebenen Bild. Abbildung 45 zeigt den Live-Preview Dialog der Methode. Die Erkennung wird eingegrenzt über die minimale und maximale Größe eines DMTX-Codes. Der dekodierte Inhalt eines Codes wird in der Ergebnisansicht neben jedem Ergebnis angezeigt.



Abbildung 45: Live Preview Dialog von DmtxDetection

Bildbereiche mit einem potenziellen Dmtx-Code werden in einem Vorverarbeitungsschritt ermittelt. Dabei werden zusammenhängende quadratische Konturen gesucht, die den Größenparametern entsprechen. Die Bereiche werden anschließend einzeln dekodiert und als Ergebnis beibehalten, sollten sie tatsächlich einen Code enthalten. Abbildung 46 zeigt Teil eines solchen Maskenbildes mit einigen identifizierten Code-Bereichen.

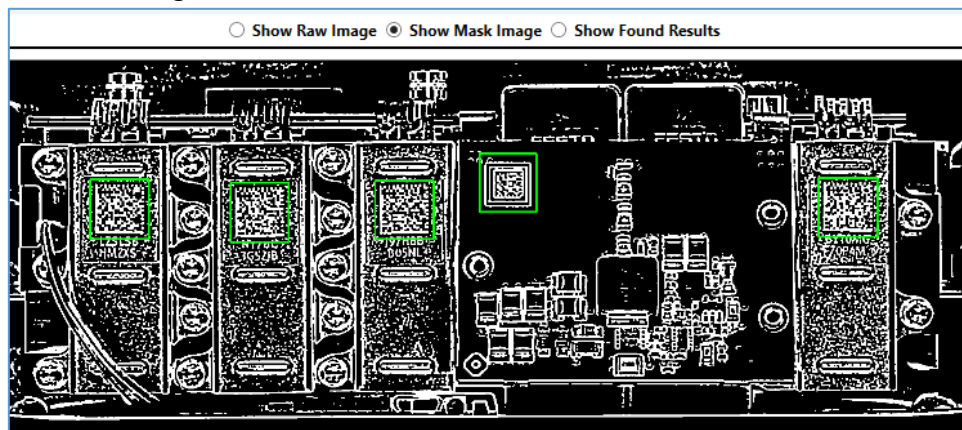


Abbildung 46: Maskenansicht des Live-Preview Dialogs von DmtxDetection

DmtxDetection liefert ein Ergebnis je gefundenem dekodierbaren Dmtx-Code. Ein Ergebnis enthält die Position und Größe, sowie den dekodierten Inhalt des Codes als String. Das Scannen von DMTX-Codes ist auch abseits von dedizierten Vision-Systemen ein Grundbestandteil jedes automatisierten Prüfstandes, der immer einen zusätzlichen Handgriff vom Bediener erfordert. Somit lassen sich mit diesem Baustein potenziell auch Anwendungsfälle optimieren, die weit über den Kernfokus dieser Arbeit hinausgehen.

4.3.4 TextDetection

Oft müssen auch Beschriftungen auf Prüflingen verifiziert werden. Die Erkennungsmethode **TextDetection** erkennt zusammenhängende Klartexte im Bild, liest diese ein und generiert ein Ergebnis pro Textblock.

Abbildung 47 zeigt den Live Preview Dialog von TextDetection. Die ausgelesenen Texte der vier erkannten Beschriftungen in der ROI werden neben dem jeweiligen Erkennungsergebnis dargestellt. Die Ergebnisse können nach minimaler und maximaler Breite des Textblocks gefiltert werden.



Abbildung 47: Live Preview Dialog von TextDetection

Die an TestStand zurückgegebenen Erkennungsergebnisse enthalten Position, Größe und Textinhalt jedes Textblocks. Für den ersten Schritt genügt der hier beschriebene Funktionsumfang, um die relevanten Anwendungsfälle umzusetzen. Weitere Filtermöglichkeiten können in der Zukunft noch ergänzt werden.

4.4 Softwarearchitektur

Dieser Abschnitt beschreibt die Softwarearchitektur, mit der das Bedienkonzept und die definierten Anforderungen umgesetzt werden können. Dafür werden die wesentlichen Softwarekomponenten, ihre öffentlichen Schnittstellen und ihre Beziehungen untereinander skizziert.

4.4.1 Integration in TestStand

Als zentrale Verbindungsschicht zwischen TestStand und dem Funktionsmodul wird die Klasse *TEaMVision* definiert. Sie beinhaltet alle statischen Funktionen, die in TestStand aufgerufen werden. Eine Funktion in *TEaMVision* bildet immer eine spezifische *StepType* Funktionalität ab. Durch die klare Schnittstelle wird die Einbindung in TestStand vereinfacht und die Programmlogik ist vollständig im Funktionsmodulcode gebündelt. Die statischen Funktionen benötigen keine Objektinstanzen, somit sind in TestStand keine Konstruktor- oder Factory-Aufrufe notwendig und es werden Null-Reference Fehler in TestStand verhindert.

Die *TEaMVision* Funktionen validieren alle Eingabeparameter, wodurch in den tieferen Schichten das Potenzial für Fehler durch unerwartete Eingaben reduziert wird. Zusätzlich fangen die statischen Funktionen alle Exceptions ab, damit diese nicht zu Laufzeitfehlern in TestStand führen.

Die Funktionen haben, wie im TEaM-Ökosystem üblich, ein einheitliches Interface mit Integer Rückgabetypp für Fehlercodes und einem String out-Parameter *msg* für Fehlerbeschreibungen. Im Fall einer Exception werden ein Fehlercode ungleich 0 und eine entsprechende Fehlermeldung zurückgegeben.

TEaMVision
+NewImageHandle(out imgHandle, out msg): int +ReleaseImageHandle(imgHandle, out msg): int +AnnotateImage(imgHandle, annotations, out msg): int +CopyImage(srcImgHandle, destImgHandle, out msg): int +SaveImage(imgHandle, filePath, out msg): int +InitCamera(cameraType, cameraLocator, cameraHandle, settings, out msg): int +ConfigureCameraLivePreview(thisContext, cameraType, cameraLocator, cameraHandle, settings, out updatedSettings): int +CameraTakeImage(cameraHandle, imageHandle, out msg): int +FindShapes(imageHandle, settings, annotatedImageHandle, out foundResults, out msg): int +FindShapesLivePreview(thisContext, cameraHandle, settings, out updatedSettings) +TemplateMatching(imageHandle, settings, annotatedImageHandle, out foundResults, out msg): int +TemplateMatchingLivePreview(thisContext, cameraHandle, settings, out updatedSettings) +DmtxDetection(imageHandle, settings, annotatedImageHandle, out foundResults, out msg): int +DmtxDetectionLivePreview(thisContext, cameraHandle, settings, out updatedSettings) +TextDetection(imageHandle, settings, annotatedImageHandle, out foundResults, out msg): int +TextDetectionLivePreview(thisContext, cameraHandle, settings, out updatedSettings) +ShowImageWindow(windowName, imgHandle, out msg): int +UpdateImageWindow(windowName, textMessage, out msg): int +IsImageWindowOpen(windowName, out windowIsOpen, out msg): int +GetImageWindowDialogResult(windowName, out dialogResult, out msg): int +CloseImageWindow(windowName, out msg): int

Abbildung 48: Klasse *TEaMVision*

Ausgenommen von dieser Regel sind die Funktionen für die Live Preview Dialoge. Diese werden nicht zur Laufzeit aufgerufen, deshalb ist eine automatisierte Handhabung der Fehler nicht notwendig und es ist auch kein Mechanismus dafür definiert. Ein unerwarteter Fehler wird einfach als Dialog in der Oberfläche angezeigt. Dementsprechend gibt es für jeden Step Type mit Live-Preview Dialog zwei Funktionen: Eine Funktion für den Aufruf zur Laufzeit und eine Funktion für den Live-Preview Dialog zur Programmierzeit.

4.4.2 Datentypen

Diverse Datenstrukturen werden für die Speicherung und Übergabe von Konfigurationen und Ergebnissen eingesetzt. Dabei handelt es sich um reine Strukturtypen ohne eigene Funktionslogik, die nur zur Sammlung mehrerer Felder mit primitiven Datentypen dienen. Zusätzlich werden an verschiedenen Stellen Enumerationstypen eingesetzt. Die Strukturen und Enumerationstypen werden an der Schnittstelle zwischen TestStand und TEaMVision geparkt und in entsprechende TestStand Datentypen übersetzt.

Intern wird zur Handhabung und Verarbeitung von Bildern die Bibliothek OpenCV¹² eingesetzt. Diese definiert für Bilder den Datentyp *Mat*¹³, der hier intern zur Speicherung von Bildinstanzen eingesetzt wird.

4.4.3 Image Handles

Zur Bereitstellung und Verwaltung von Bildhandles wird die Klasse *ImageHandle* definiert. Eine *ImageHandle* Instanz beinhaltet eine OpenCV *Mat*, in der das eigentliche Bild abgelegt ist.

- *GetMat* erlaubt das Abrufen der Bildinstanz.
- *SetMat* erlaubt das Beschreiben der Bildinstanz.
- *Handle* beinhaltet den Integer-Handle, der in TestStand zur Referenzierung einer Instanz verwendet wird.

ImageHandle
-ImageHandles: Dictionary<int, ImageHandle>
-Mat: Mat
+Handle: int {readOnly}
-ImageHandle()
+CreateImageHandle(): ImageHandle
+GetHandle(handle: int): ImageHandle
+ReleaseImageHandle(handle: int)
+GetMat(): Mat
+SetMat(mat: Mat)

Abbildung 49: Klasse ImageHandle

Die vergebenen Handles innerhalb einer Laufzeit sollen immer einzigartig sein und bleiben, dafür wird das Multiton¹⁴ Entwurfsmuster angewandt. Der Konstruktor von *ImageHandle* ist privat, Instanzen können nur über statische Funktionen der Klasse angelegt werden.

- *CreateImageHandle* erstellt eine neue Instanz und vergibt einen neuen, einzigartigen Integer Handle. Die Instanzen werden in einem statischen Dictionary abgelegt, wobei der Handle der Schlüssel ist.
- *GetHandle* ruft eine existierende Instanz über den Integer Handle ab.
- *ReleaseImageHandle* gibt eine Instanz wieder frei, wobei auch die Bildinstanz und der dadurch reservierte Speicher freigegeben wird.

4.4.4 Kameras

Für die Implementierung der verschiedenen Kameravarianten wird die abstrakte Basisklasse *Camera* definiert. Wie auch bei *ImageHandle* hat jede *Camera* einen eindeutigen Integer Handle. Auch hier wird das Multiton Entwurfsmuster eingesetzt, um die Eindeutigkeit von Kamerahandles zu gewährleisten. Der Integer Handle wird hier aber von außen vorgegeben, um Livebilder zur Programmierzeit zu ermöglichen. Der Enumerationstyp *CameraTypes* gibt vor, welche konkrete Implementierung von *Camera* instanziiert wird. Somit wird hier zusätzlich ein

¹² Siehe Kapitel 5

¹³ *Mat* steht als Abkürzung für Matrix und ist ein Datentyp, der in der Bibliothek OpenCV für Bilder und andere ein- oder mehrdimensionale Datenfelder genutzt wird.

¹⁴ Das Multiton Muster ist eine Erweiterung des Singleton Musters und erlaubt das kontrollierte Erstellen einer limitierten Anzahl von Instanzen, die alle mit einem einzigartigen Schlüssel versehen sind. Neue Instanzen werden erst bei Bedarf erstellt. Der Konstruktor der Klasse ist Privat, Instanzen werden über statische Methoden der Klasse erstellt. [Kurek, 2026]

Factory-Mechanismus eingesetzt, um je nach gegebenem Typ die entsprechende Kameravariante anzulegen.

- *CreateCamera* legt eine neue Kamerainstanz an. Neben der Eindeutigkeit des Bildhandles wird auch sichergestellt, dass es immer nur eine aktive Instanz je Paarung aus *CameraType* und *SourceLocator* gibt, um mögliche Ressourcenkonflikte zu vermeiden.
- *Settings* enthält die Einstellungen, die über den Live Preview Dialog konfiguriert und im TestStand Step gespeichert werden. Die Definition des Typs *CameraSettings* wird von allen Kameratypen geteilt.
- *GetImage* ruft ein aktuelles Kamerabild ab, wendet ROI und Rotation an und legt es im übergebenen ImageHandle ab.
- *GetImageInternal* wird intern von *GetImage* aufgerufen. Die konkreten Implementierungen von Camera realisieren die abstrakte Methode, um technologiespezifisch ein Bild der jeweiligen Kamera zur Verfügung zu stellen.

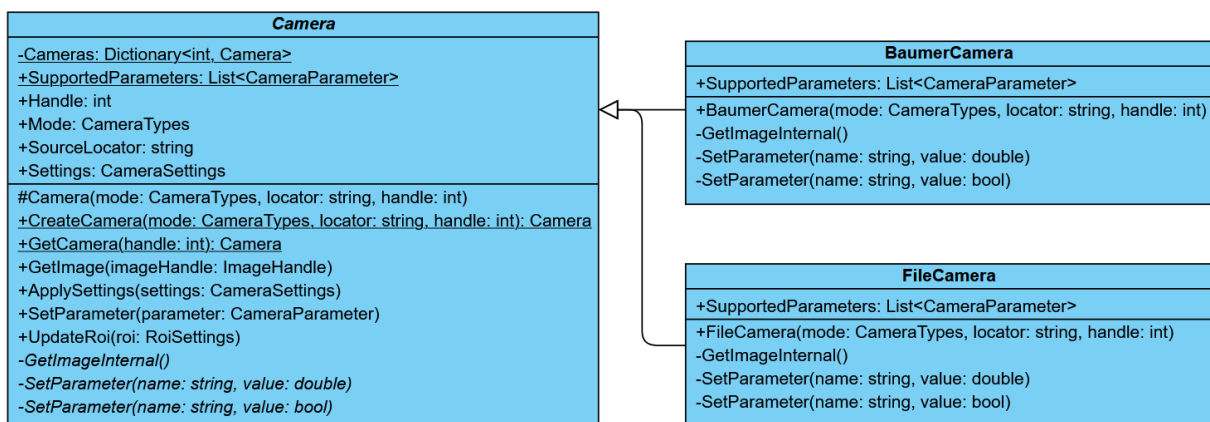


Abbildung 50: Klassendiagramm Camera und ihre Implementierungen

Zusätzlich zu den Einstellungen können verschiedene Kameratypen unterschiedliche spezifische Parameter unterstützen. Dazu enthält *CameraSettings* ein Array vom Typ *CameraParameter*. Ein *CameraParameter* hat einen Namen und einen Wert, beide sind als String gespeichert. Der Stringwert kann je nach Parametertyp als Boolean oder Double-Wert interpretiert werden.

- *ApplySettings* wendet bei Aufruf alle Einstellungen und auch alle Parameter auf die Camera an. Die Methode wird zur Programmierzeit bei Öffnen des Live Preview Dialogs aufgerufen, sowie zur Laufzeit bei Aufruf von *InitCamera*.
- *SupportedParameters* ist die Liste der Parameter, die eine Kameraimplementierung unterstützt.
- *SetParameter* vergleicht den gegebenen Parameter mit den unterstützten Parametern der Instanz, konvertiert den Stringwert in den Zieltyp und überprüft den erlaubten Wertebereich. Fehler wie undefinierte Parameternamen oder falsch kodierte Werte werden hier abgefangen. Anschließend wird der geparsete Parameter an die entsprechende *SetParameter*-Methode der Kameravariante übergeben.

4.4.5 Erkennungsmethoden

Als nächstes werden die Klassen *FindShapes*, *TemplateMatching*, *DmtxDetection* und *TextDetection* für die vier Erkennungsmethoden definiert. Die Erkennungsklassen haben keinen Handle und auch sonst keine weitere Beschränkung der Instanziierung. Eine Instanz ist für den

exklusiven Einsatz in einer einzelnen Funktion oder sonstigem Kontext ausgelegt und kann somit beliebig angelegt und wieder freigegeben werden.

- *Settings* ist eine Datenstruktur mit spezifischen Einstellungen für die jeweilige Erkennungsmethode. Beim Konstruktoraufruf werden die Einstellungen und ein ImageHandle mit dem zu analysierenden Bild übergeben.
- *Run* liest das aktuelle Bild aus dem ImageHandle aus, führt darauf die Erkennungsmethode mit den gegebenen Einstellungen aus und generiert Erkennungsergebnisse.
- *FoundResults* enthält nach dem Abschluss von Run alle gefundenen Ergebnisse.

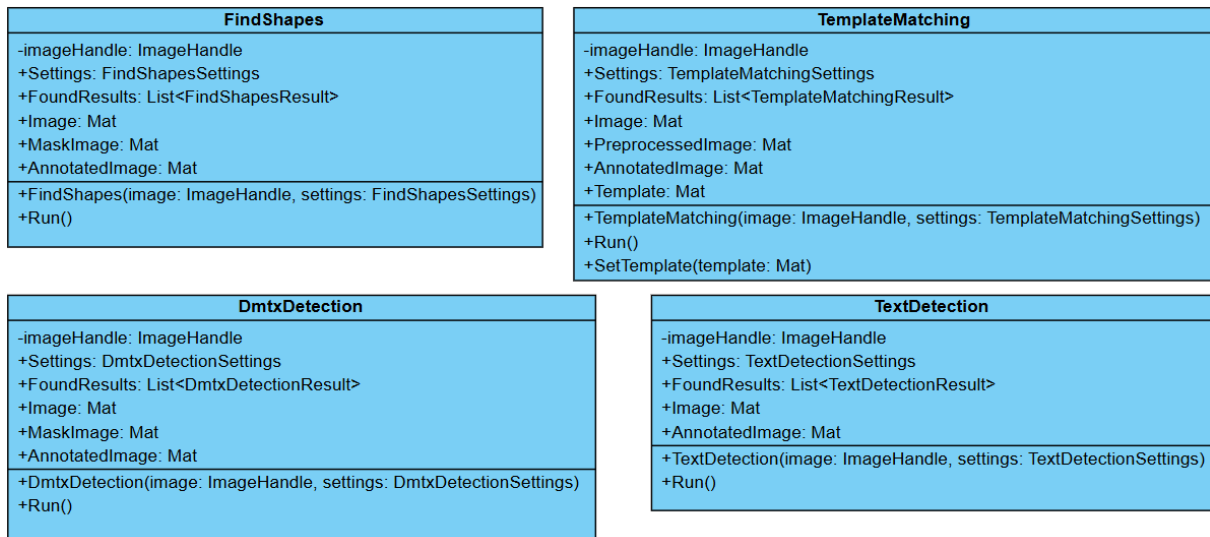


Abbildung 51: Erkennungsklassen FindShapes, TemplateMatching, DmtxDetection, TextDetection

Neben den Erkennungsergebnissen werden in mehreren Eigenschaften vom Typ Mat mehrere Bilder bereitgestellt. Die Art und Anzahl der bereitgestellten Bilder ist abhängig von der jeweiligen Erkennungsklasse.

- *Image* wird von allen Klassen bereitgestellt und enthält das unbearbeitete Originalbild.
- *AnnotatedImage* wird ebenfalls von allen Klassen bereitgestellt und enthält das Originalbild, mit Annotierungen aller Ergebnisse wie sie in FoundResults hinterlegt sind.
- *MaskImage* wird von FindShapes und DmtxDetection bereitgestellt. Es enthält das Binärmaskenbild, auf dessen Basis die Erkennung ausgeführt wird.
- *PreprocessedImage* von TemplateMatching enthält das vorverarbeitete Bild.
- *Template* wird von TemplateMatching bereitgestellt und enthält das aktuell aktive Template Bild. Ein neues Template kann über die Methode *SetTemplate* gesetzt werden.

Die vier Erkennungsklassen sind in ihrem Aufbau so ähnlich, dass auf den ersten Blick eine Abstraktion über eine gemeinsame Basisklasse sinnvoll erscheint. Jedoch werden die Erkennungen in der Praxis nie so eingesetzt, dass sie gegeneinander austauschbar wären. Eine bestimmte Erkennung wird für einen spezifischen Anwendungsfall eingesetzt, es gibt separate StepTypes für die verschiedenen Erkennungen und auch die Live Preview Dialoge werden in jedem Fall spezifisch auf die Erkennungsmethode zugeschnitten. Zusätzlich unterscheiden sich die Datentypen der Settings- und Result-Felder. Das wäre durch generische Typisierung abbildbar, würde aber die Anwendung wieder verkomplizieren. Eine Abstraktion der Erkennungsklassen bietet aktuell keinen realen Mehrwert.

4.4.6 Vorschau-Dialoge

Ein Live-Preview Dialog zeigt in Echtzeit ein aktuelles Kamerabild an, führt darauf eine Erkennungsmethode aus und zeichnet die Erkennungsergebnisse im Livebild der Kamera ein. Dafür nimmt der Konstruktor eines Preview-Dialogs eine Instanz der Erkennungsklasse, eine Camera Instanz und eine ImageHandle Instanz als Parameter. Der Preview-Dialog liest zyklisch Bilder über die gegebene Camera ein, übergibt diese mittels des ImageHandles an die gegebene Erkennung, führt die Erkennung aus und zeigt die Erkennungsergebnisse auf der Oberfläche an. Als Beispiel wird hier der Aufbau des *FindShapesPreview* Live-Dialogs gezeigt. Der Aufbau ist bis auf den Typ der Erkennungsmethode vollständig auf die anderen Erkennungsdialoge übertragbar.

Der Preview-Dialog hat GUI-Elemente zum Modifizieren jeder Einstellung für die jeweilige Erkennung. Die Elemente sind verknüpft mit den Feldern in der Settings-Eigenschaft der Erkennungsklasse. Für die Ergebnisanzeige werden die von der Erkennungsmethode bereitgestellten Mat-Eigenschaften verwendet. Der Preview-Dialog liest das Image, das AnnotatedImage und ggf. weitere Bilder aus und zeigt das ausgewählte Bild an. So werden bei jedem Durchlauf der Erkennung die aktualisierten Einstellungen aus der Oberfläche angewandt und das Erkennungsergebnis für diese Einstellungen direkt angezeigt.

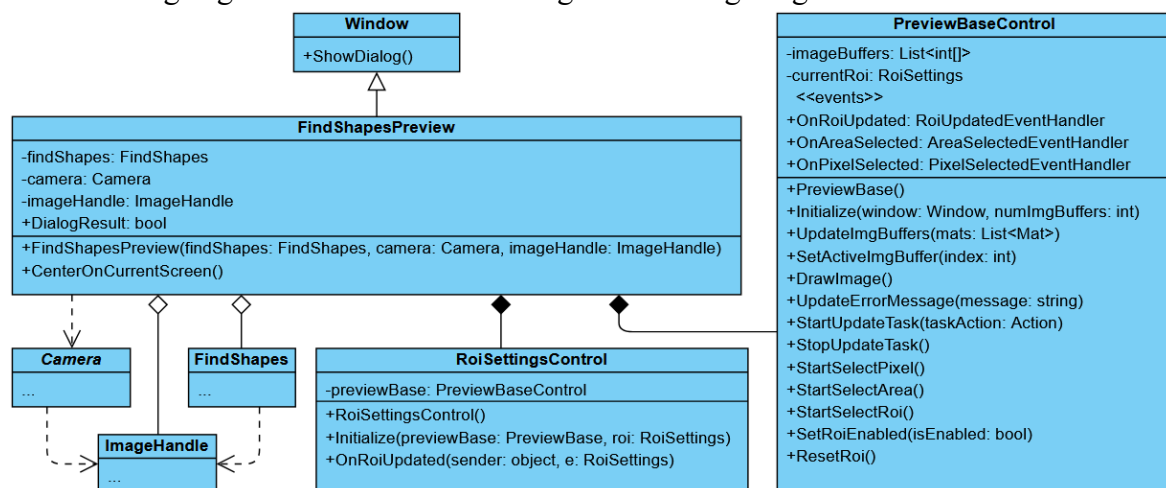


Abbildung 52: Klassendiagramm Vorschauldialoge

Die verschiedenen Live-Preview Dialoge sind abgeleitet von der *Window*-Klasse des Microsoft WPF-Frameworks¹⁵, der Standard GUI-Bibliothek für C# unter Windows. Nach Aufrufen des Konstruktors wird über die Methode *ShowDialog* von *Window* das Fenster geöffnet und die Ausführung im aufrufenden Thread blockiert, bis das Fenster wieder geschlossen wird. Nach Schließen des Preview-Dialogs kann der aufrufende Kontext über das Erkennungs-Objekt auf die aktualisierten Einstellungen und die letzten Erkennungsergebnisse zugreifen.

Wie auch bei den Erkennungsklassen ähneln sich die Preview-Dialoge für Erkennungen so sehr, dass eine Abstraktion naheliegen würde. Ebenso wird ersichtlich, dass eine Abstraktion wieder keinen realen Vorteil bietet. Anstatt die externe Schnittstelle über eine Basisklasse zu abstrahieren, wird intern mehrfach verwendete Logik ausgelagert in die zwei Basiskomponenten *PreviewBaseControl* und *RoiSettingsControl*.

¹⁵ Siehe Kapitel 5

Das Einstellungsmenü für die ROI ist in `RoiSettingsControl` als separates Kontrollelement ausgelagert, denn nicht jeder Dialog mit Bildanzeige unterstützt zwangsweise eine konfigurierbare ROI. Zusätzlich soll das `RoiSettingsControl` sich in die weiteren Einstellungen des jeweiligen Vorschaudialogs einreihen und muss somit als eigenes Element definiert werden, um es frei platzieren zu können.

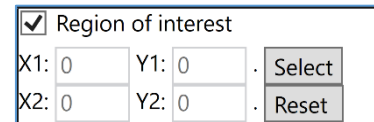


Abbildung 53: GUI-Element `RoiSettingsControl`

`PreviewBaseControl` beinhaltet alle GUI-Komponenten und die Logik zum Anzeigen, Verarbeiten und zyklischen Aktualisieren der Vorschaubilder, sowie alle weiteren Funktionen, die verallgemeinert werden können. Abbildung 54 zeigt das `PreviewBaseControl`. Neben dem Bildanzeigeelement und der Textmeldung am unteren Rand beinhaltet es auch die Anzeige der Pixelposition des Cursors, die Farbe des Pixels an dieser Position, den Button zum Speichern des aktuell angezeigten Bildes und sogar die *Ok* und *Cancel* Buttons zum Schließen des Dialogs. Der Preview-Dialog beinhaltet somit ausschließlich die Kontrollelemente zum Bearbeiten der erkenntungsspezifischen Einstellungen, sowie die Auswahl des angezeigten Bildes.

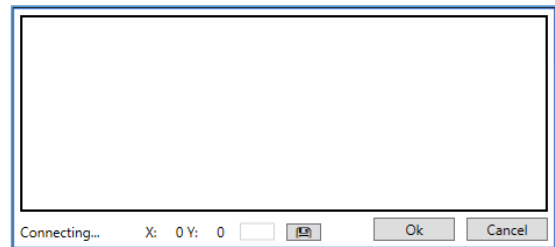


Abbildung 54: GUI-Element `PreviewBaseControl`

- *imageBuffers* beinhaltet ein oder mehrere Speicherpuffer, die direkt ohne Konvertierung in das Bildanzeigeelement geschrieben werden können. Die Anzahl der Puffer ist abhängig von der Anzahl der verschiedenen Bilder, die der jeweilige Preview-Dialog anzeigen kann.
- *Initialize* wird im Konstruktor des Preview-Dialogs aufgerufen. Dort werden eine Referenz zum übergeordneten Fenster und die Anzahl der Puffer übergeben.
- *UpdateImgBuffers* wird vom Preview-Dialog aufgerufen, um die als Mat vorliegenden Bilder zu konvertieren und somit die Anzeigepuffer zu aktualisieren.
- *SetActiveImgBuffer* setzt den aktiven Bildpuffer für die Bildanzeige.
- *DrawImage* kopiert den Inhalt des aktiven Puffers in das Bildk Kontrollelement und aktualisiert somit das angezeigte Bild.
- *UpdateErrorMessage* kann aufgerufen werden, um die Textmeldung am unteren Bildschirmrand zu aktualisieren.

Der Preview-Dialog ruft zur Anzeige von Livebildern also zyklisch *UpdateImgBuffers*, *DrawImage* und ggf. *UpdateErrorMessage* auf. *SetActiveImageBuffer* wird nur bei Bedarf aufgerufen, um das aktuell angezeigte Bild umzuschalten. Die Logik zur zyklischen Ausführung dieses Ablaufes als Hintergrundaufgabe ist ebenfalls in `PreviewBaseControl` enthalten.

- *StartUpdateTask* bekommt vom Preview-Dialog eine einfache Funktion übergeben. Anschließend legt `PreviewBaseControl` einen Hintergrundtask an, in dem die übergebene Funktion in Schleife aufgerufen wird. Dabei findet ein Fehlerhandling statt, das jegliche Exceptions abfängt. Beim Schließen des Dialogs wird sichergestellt, dass der Task korrekt beendet wird.

Für erkenntungsspezifische Interaktionen der Preview-Dialoge mit dem angezeigten Bild bietet `PreviewBaseControl` spezielle Schnittstellen an, etwa zum Abfragen der Farbe an einer Pixelposition, zur Auswahl des Templates oder zum Setzen der Region of Interest.

Die drei Methoden *StartSelectPixel*, *StartSelectRoi* und *StartSelectArea* aktivieren in *PreviewBaseControl* den Auswahlmodus für ein Pixel oder einen Bildbereich. Die Methoden sind asynchron, um den GUI-Thread nicht zu blockieren. Nach erfolgter Auswahl wird das jeweilige der drei Events *OnPixelSelected*, *OnRoiUpdated* und *OnAreaSelected* ausgelöst. Der Aufrufer muss sich zuvor mit einer Callback-Methode auf das jeweilige Event registrieren und bekommt dann das ausgewählte Pixel bzw. den Bereich zur Weiterverarbeitung übergeben.

4.4.7 ShowImageWindow

Das *ShowImageWindow* dient zum Anzeigen von Bildern auf der Benutzeroberfläche zur Laufzeit. Der Dialog erbt wie auch die Preview-Dialoge der Erkennungen von *Window* und nutzt das *PreviewBaseControl* zum Anzeigen des Bildes.

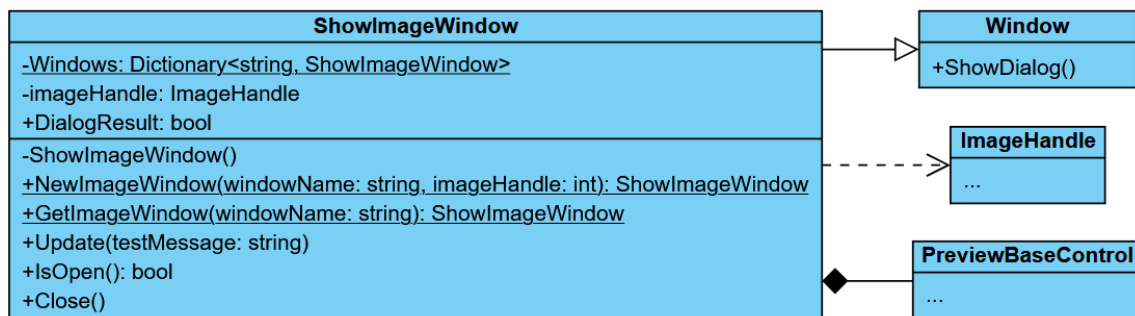


Abbildung 55: Klassendiagramm von ShowImageWindow

Zum Verwalten der *ShowImageWindow* Varianten wird wieder das Multiton Entwurfsmuster eingesetzt. Anstatt eines Integers wird hier ein String als Schlüssel verwendet, der gleichzeitig der Anzeigename des Dialogfensters ist.

- *NewImageWindow* erstellt eine neue Instanz vom *ShowImageWindow* und nimmt einen *ImageHandle* als Parameter. Das Bild wird geladen und via *PreviewBaseControl* angezeigt.
- *GetImageWindow* gibt für den gegebenen Dialognamen die bereits existierende Instanz zurück.
- *Update* wird auf einer Instanz aufgerufen, um das angezeigte Bild zu aktualisieren. Das *ShowImageWindow* nutzt keinen automatischen Aktualisierungs-Task, nur beim Aufruf von *Update* wird ein neues Bild aus dem *ImageHandle* geladen und angezeigt.
- *IsOpen* wird in der Anwendung für die Abfrage genutzt, ob das Dialogfenster geöffnet ist oder bereits vom Anwender geschlossen wurde.
- *DialogResult* gibt die Information zurück, ob der Anwender das Fenster über den *Ok* oder den *Cancel* Button geschlossen hat.
- *Close* erlaubt das Schließen des Dialogs aus dem Programm heraus.

4.4.8 Gesamtarchitektur

Abbildung 56 zeigt die Gesamtarchitektur der wichtigsten Komponenten des TEaMVision-Moduls. Die Klassen *LivePreviewDialog* und *Detection* stehen hier zur besseren Darstellbarkeit als Vertreter für alle Live-Preview Dialoge und alle Erkennungsklassen.

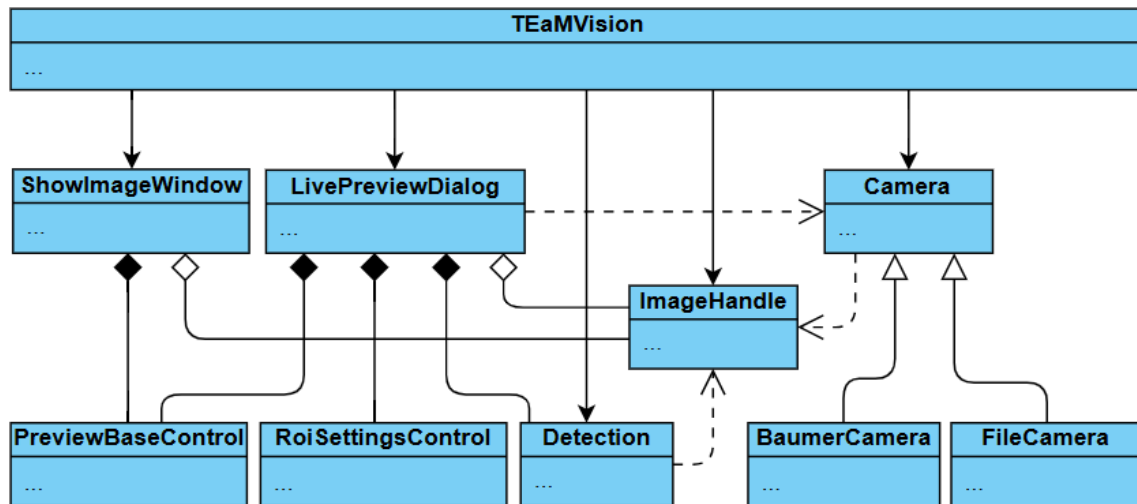


Abbildung 56: Klassendiagramm aller Hauptkomponenten von TEaMVision

Gut ersichtlich wird hier, wie *ImageHandle* als zentrale Komponente zum Austausch von Bild-daten von nahezu allen anderen Komponenten verwendet wird.

Die Wiederverwendbarkeit von *PreviewBaseControl* ist hier zumindest anhand der Nutzung durch *ShowImageWindow* und *LivePreviewDialog* zu sehen. Deutlicher wird dies, wenn man bedenkt, dass *LivePreviewDialog* für mehrere Einzelklassen steht. Die ausgelagerten Komponenten *PreviewBaseControl* und *RoiSettingsControl* sind somit die Basis für einige umfangreiche Dialoge.

Die Kameravarianten sind durch die abstrakte Basisklasse sauber vom Rest der Applikation getrennt, die anderen Komponenten sehen nur die Basisklasse und interagieren somit gleich mit allen Varianten.

Die statische Klasse *TEaMVision* zeigt sich hier noch einmal als zentraler Einstiegspunkt in das Modul. Sie bündelt die Funktionen der Einzelmodule und stellt sie in gesammelter Form für externe Aufrufer zur Verfügung.

4.5 Programmablauf

Im vorigen Abschnitt wurden die wesentlichen Softwarekomponenten für die Umsetzung von TEaMVision definiert und ihre Funktion skizziert. Um klarer zu zeigen, wie die einzelnen Komponenten in Verbindung miteinander eine funktionsfähige Lösung bilden, soll in diesem Abschnitt die Interaktion der Komponenten untereinander anhand einiger Beispielprogrammabläufe genauer beleuchtet werden.

4.5.1 Kamera konfigurieren

Das erste Programmablaufbeispiel zeigt die Funktionsweise des Live-Preview Dialogs für die Konfiguration einer Kamera.

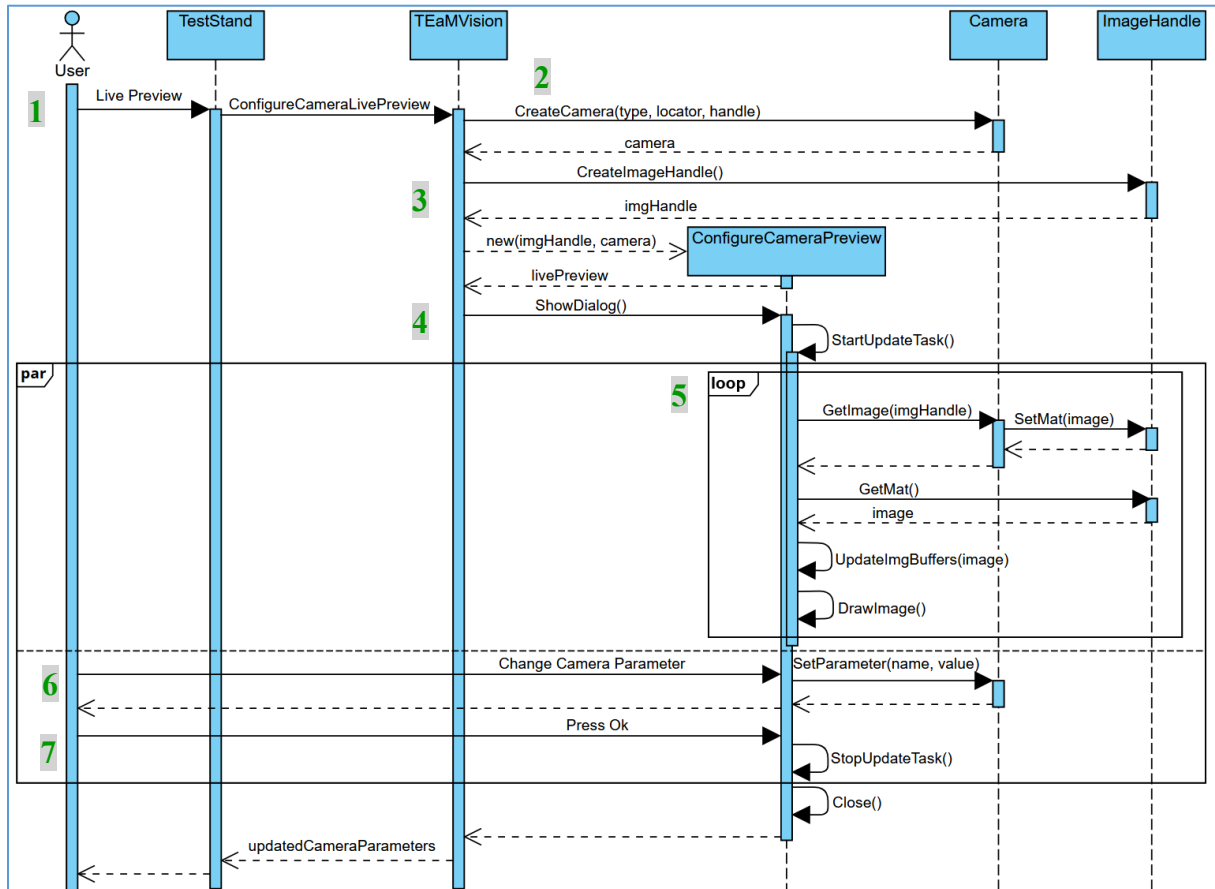


Abbildung 57: Programmablauf Camera Live Preview

1. Der Benutzer öffnet den Dialog über den Button *Live Preview* in TestStand. Der Button ruft *ConfigureCameraLivePreview* von TEaMVision auf. Dabei werden Kamertyp, Kamera-locator und der gewählte Kamerahandle aus den Step Parametern übergeben.
2. TEaMVision ruft mit diesen Parametern *CreateCamera* auf. Diese gibt entweder eine bereits existierende Kamera zurück oder erstellt eine neue Instanz.
3. Es wird ein neuer ImageHandle angelegt. Camera und ImageHandle werden an den Konstruktor von *ConfigureCameraPreview* übergeben, um einen neuen Dialog zu erstellen.
4. *ShowDialog* öffnet den Live-Preview Dialog. Intern wird über *StartUpdateTask* die Hintergrundaufgabe zur kontinuierlichen Aktualisierung des Livebildes gestartet.
5. In Dauerschleife wird über *GetImage* ein aktuelles Kamerabild im ImageHandle abgespeichert, über *UpdateImageBuffers* konvertiert und mit *DrawImage* angezeigt.
6. Parallel kann der Anwender über die Oberfläche Einstellungen und Parameter der Kamera bearbeiten. Ändern eines Parameters, z.B. ruft *SetParameter* auf. Die Änderung wird in der Camera Instanz gespeichert und im nächsten Aktualisierungszyklus automatisch angewandt.
7. Schließt der Anwender den Dialog über den Button *Ok*, wird *StopUpdateTask* ausgeführt. Daraufhin wird die Schleife im Update Task nach dem aktuellen Zyklus verlassen. Nach Beenden des Tasks schließt der Dialog final, die aktualisierten Einstellungen werden an TestStand zurückgegeben und dort im Step gespeichert.

4.5.2 Erkennung konfigurieren

Der folgende Programmablauf zeigt die Konfiguration einer Erkennung über den Live Update Dialog. Als Beispiel wird hier die Erkennung *FindShapes* herangenommen, der Ablauf ist auf die anderen Erkennungen übertragbar.

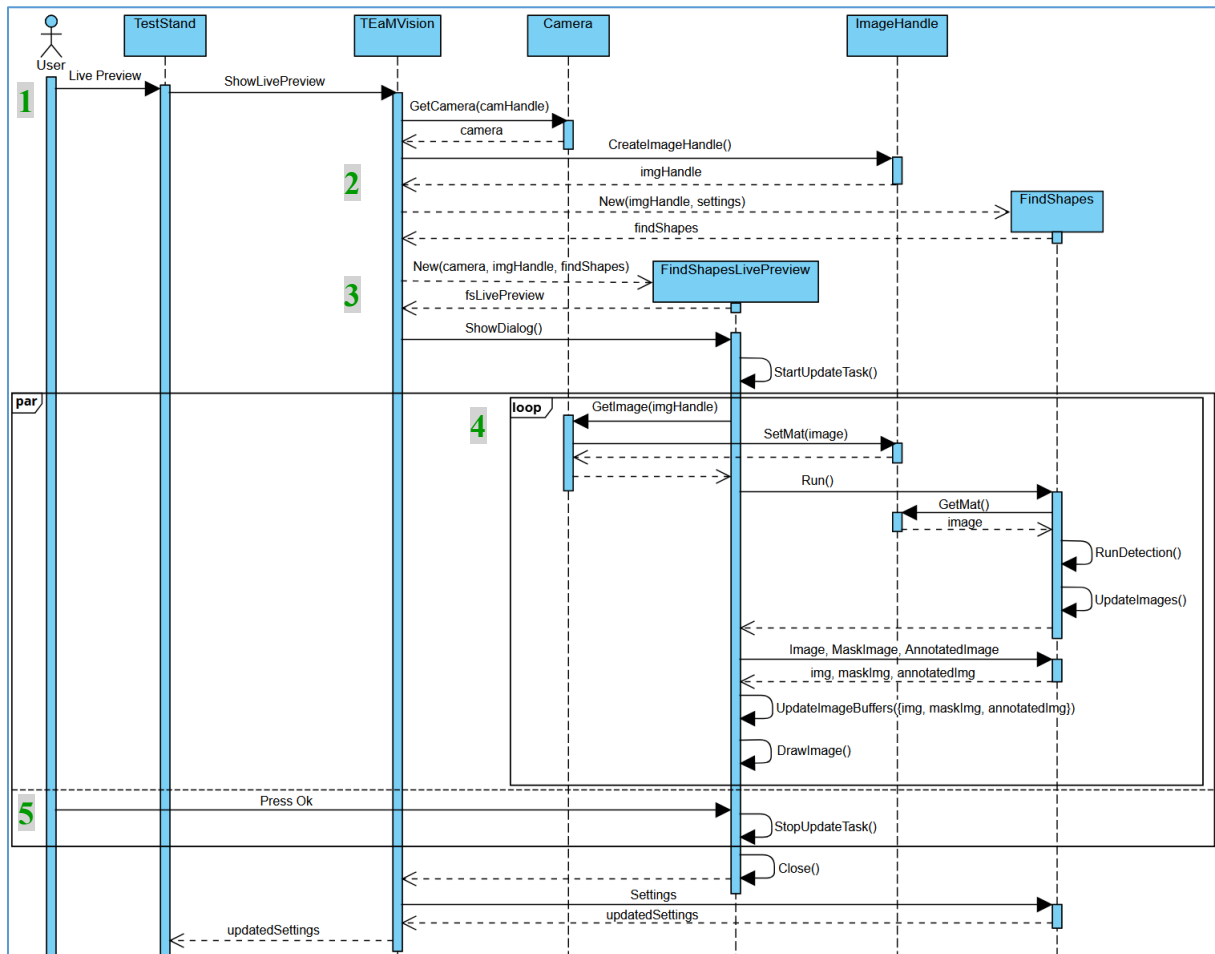


Abbildung 58: Programmablauf Live Preview einer Erkennung

1. Beim Aufruf von *ShowLivePreview* werden der Kamerahandle und die Erkennungseinstellungen aus den Step-Parametern übergeben. Über *GetCamera* wird die bereits initialisierte Kamerainstanz zu dem Handle geholt.
2. Es wird ein neuer ImageHandle angelegt. Mit dem ImageHandle und den Einstellungen aus TestStand wird eine neue Instanz der Erkennungsklasse erstellt, hier FindShapes.
3. Mit den Instanzen von Camera, ImageHandle und FindShapes wird der LivePreview-Dialog erzeugt, hier FindShapesLivePreview. *ShowDialog* öffnet das Fenster und startet den Update Task für die Aktualisierung der Bildvorschau.
4. In jeder Iteration der Dauerschleife wird ein Kamerabild aufgenommen und im ImageHandle abgelegt. *Run* ruft das Bild aus dem ImageHandle ab und führt darauf die Erkennungsmethode aus. *UpdateImageBuffers* konvertiert die Ergebnisbilder der Erkennung für die Bildanzeige. *DrawImage* aktualisiert die Vorschauanzeige mit dem neuen Inhalt des aktiven Bildpuffers.
5. Der Anwender schließt den Dialog über den *Ok* Button. Der Update Task wird beendet und der Dialog geschlossen. Die aktualisierten Erkennungseinstellungen aus der Instanz der Erkennung werden an TestStand zurückgegeben.

4.5.3 Auswahl Bildbereich im Preview Dialog

Bei den vorigen Sequenzdiagrammen wurden PreviewBaseControl und RoiSettingsControl zur besseren Darstellbarkeit verborgen und ihre Eigenschaften als Teil des Preview-Dialogs angenommen. Der Ablauf hier zeigt die Interaktionen zwischen Dialog und den Kontrollelementen bei der Eventbasierten Auswahl von Bildbereichen. Der erste Teil des Ablaufes zeigt die Auswahl eines Templates, im zweiten Teil wird eine ROI ausgewählt.

1. Der *Select Template* Button ruft *StartSelectArea* von PreviewBaseControl auf. Das aktiviert im Kontrollelement den Modus zur Auswahl eines Bildbereiches.
2. Der Anwender wählt durch Klicken und Ziehen einen Bereich im Anzeigebild aus. Das Lösen der Maustaste löst das Ereignis *OnAreaSelected* von PreviewBaseControl aus.
3. Das Event ruft die Callback-Methode *AreaSelected* vom Dialog auf und übergibt eine Datenstruktur mit dem Bildbereich. Der Dialog gibt den Bildbereich über *SetTemplate* weiter an die Erkennungsklasse. Das neue Template ist konfiguriert.

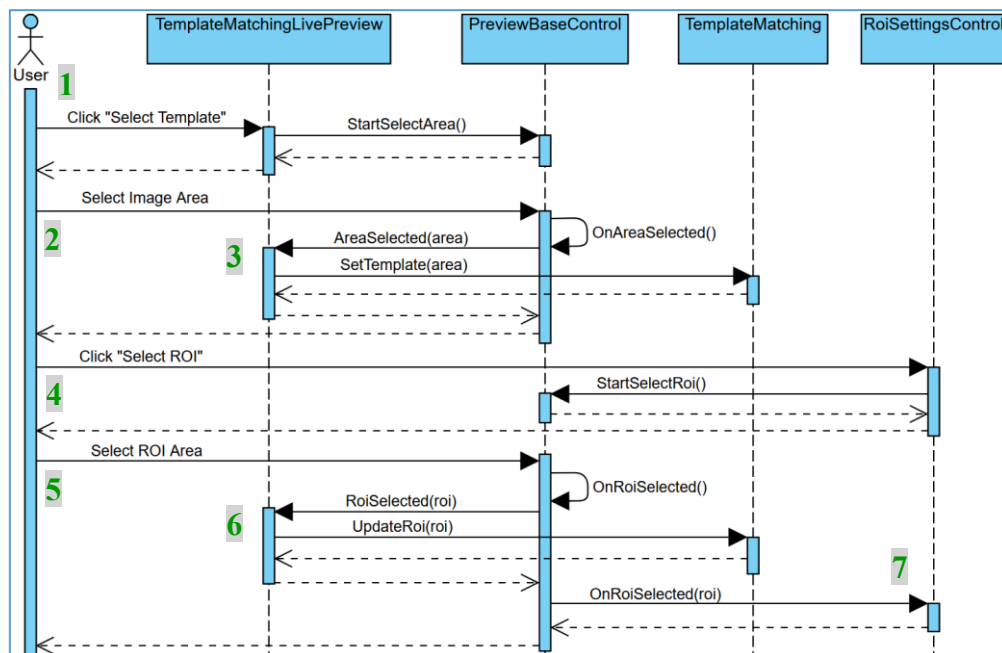


Abbildung 59: Programmablauf Bildbereich in Vorschaubild auswählen

4. Die Auswahl der ROI verläuft ähnlich, der *Select ROI* Button gehört aber zu RoiSettingsControl. Das hat eine Referenz auf PreviewBaseControl und ruft dort *StartSelectRoi* auf, um den ROI-Auswahlmodus zu aktivieren.
5. Die Auswahl des Bildbereiches durch den Anwender löst das Ereignis *OnRoiSelected* in PreviewBaseControl aus. Daraufhin werden zwei Callback Methoden aufgerufen.
6. Der Preview-Dialog erhält die ausgewählte ROI als Parameter der Callback Methode und gibt diese weiter an die Erkennungsmethode.
7. Die zweite Callback Methode gehört zu RoiSettingsControl, hier werden mit der ROI die Anzeigefelder für die ROI-Koordinaten aktualisiert.

So können mit dem Event-Mechanismus nach Auswahl eines Bildbereiches beliebig viele Empfänger über den neuen Bildbereich informiert werden.

4.5.4 Laufzeit: Testprogramm

Der Programmablauf in diesem Abschnitt zeigt das Zusammenspiel der Komponenten während der Ausführung eines beispielhaften Testprogramms in TestStand. Der Ablauf initialisiert eine Kamera, nimmt ein Bild auf, führt auf dem Bild eine Erkennungsmethode aus und zeigt anschließend Ergebnisse in einem Bild auf der Benutzeroberfläche an.

Der Ablauf ist zur Darstellbarkeit in drei Teile aufgeteilt. Der erste Teil in Abbildung 60 zeigt die Aktionen im Setup-Bereich der TestStand-Sequenz.

1. Über *InitCamera* wird eine neue Kamerainstanz initialisiert. Dafür werden die Einstellungen und Parameter aus dem TestStand Step übergeben und anschließend intern in der Kamera gesetzt.
2. Als nächstes wird über *CreateImageHandle* ein neuer ImageHandle angelegt. Der Integer-Handle wird an das TestStand Programm zurückgegeben und dort zur weiteren Verwendung in einer Variable abgelegt.
3. Über *ShowImageWindow* wird ein neues Dialogfenster zur Anzeige des Ergebnisbildes geöffnet. Der übergebene Bildhandle enthält noch keine Bilddaten, deshalb zeigt der Dialog zunächst nur ein schwarzes Bild an.

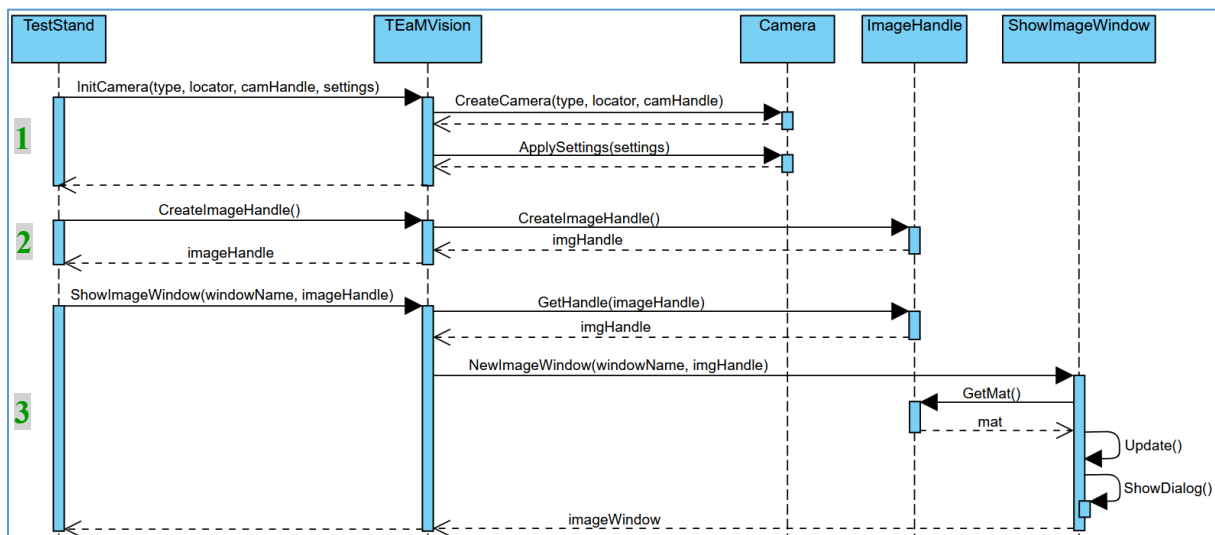


Abbildung 60: Programmablauf Init-Bereich Testprogramm

Abbildung 61 zeigt den Programmablauf für den Main-Abschnitt der TestStand Sequenz. Dieser Programmteil läuft in einer Schleife, somit wird kontinuierlich ein Bild aufgenommen, verarbeitet und das neue Ergebnis angezeigt. Das Programm läuft, bis der Anwender den Bilddialog manuell schließt.

1. In *CameraTakeImage* werden für die Integer-Handles zunächst die Objektinstanzen von Camera und ImageHandle abgerufen. Anschließend wird über *TakeImage* ein Kamerabild aufgenommen. Das Bild wird via *SetMat* in dem übergebenen ImageHandle gespeichert und ist nun für weitere Operationen verfügbar.
2. Via *DmtxDetection* wird eine DataMatrix-Code Erkennung auf dem Bild ausgeführt. Für jeden Aufruf wird eine neue Instanz der Erkennungsklasse erstellt. Der Integer-Bildhandle wird wieder in eine Objektinstanz übersetzt und mit den Erkennungseinstellungen an den Konstruktor der DmtxDetection-Klasse übergeben. Beim Aufruf von *Run* wird das Bild aus dem ImageHandle ausgelesen und darauf die Erkennung ausgeführt. Die Ergebnisse sind

danach in der Eigenschaft *Results* gespeichert und werden von TEaMVision ausgelesen und an das TestStand Programm zurückgegeben.

3. *Process Results* steht hier für eine anwendungsspezifische Auswertung der Erkennungsergebnisse direkt in TestStand. Dabei werden auch Ergebnisannotationen generiert.

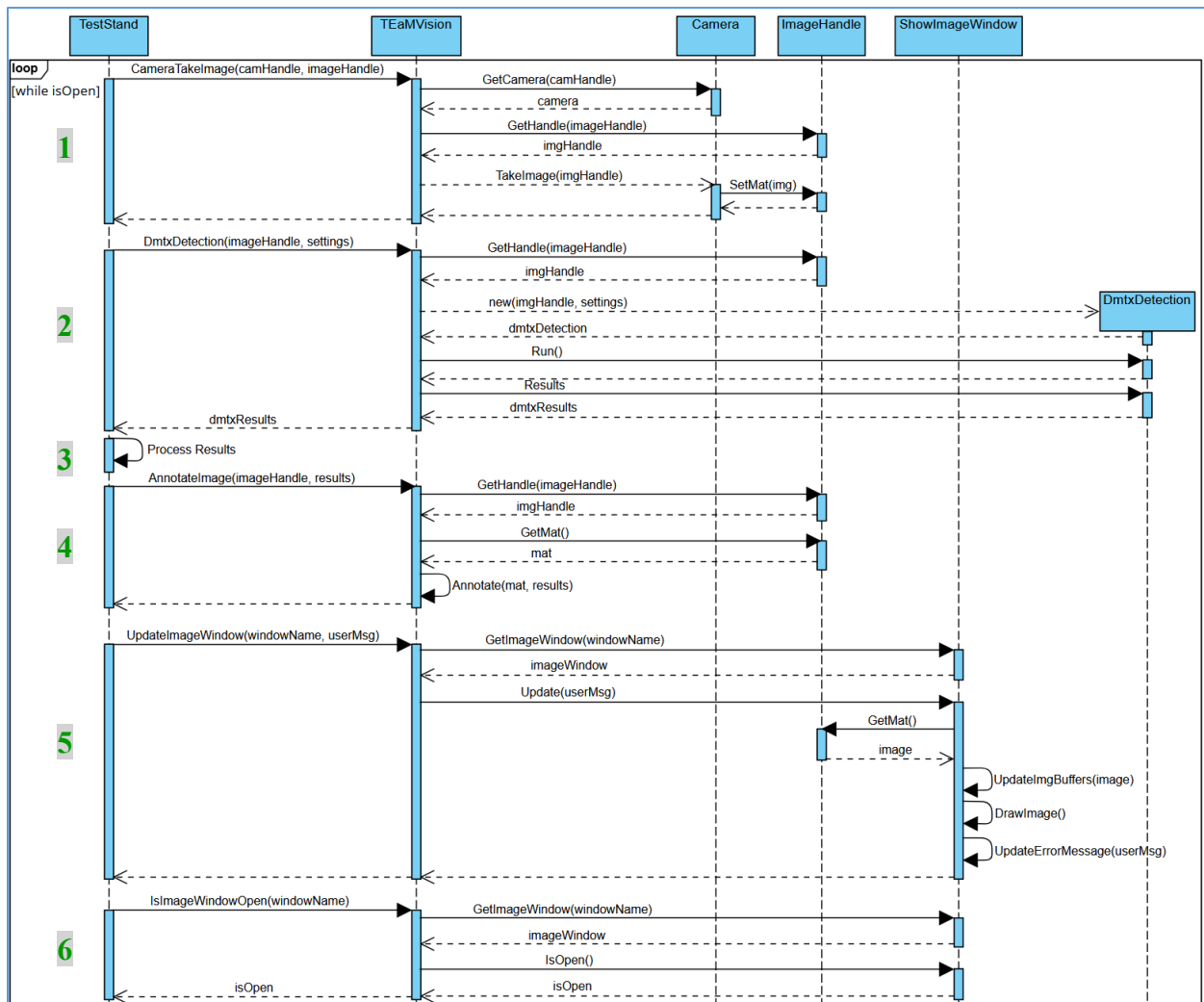


Abbildung 61: Programmablauf Main-Bereich Testprogramm

4. Die Annotationen werden an *AnnotateImage* übergeben. Die Funktion ruft über den Integer-Handle den ImageHandle ab, holt sich über *GetMat* die Bildinstanz und fügt dort die Annotationen ein. Das Originalbild wird für die Ergebnisanzeige weiterverwendet, es muss keine Kopie angelegt werden.
5. *UpdateImageWindow* zeigt das nun annotierte Bild auf der Oberfläche an. Die Funktion holt die ShowImageWindow -Instanz zum übergebenen Fensternamen und ruft die *Update* Methode auf. Darin wird das Bild aus dem anfangs gesetzten ImageHandle ausgelesen, konvertiert und im Dialog angezeigt. Über *UpdateErrorMessage* wird die Textnachricht unten im Dialog aktualisiert.
6. Am Ende jedes Zyklus wird über *IsImageWindowOpen* abgefragt, ob das Dialogfenster vom Anwender geschlossen wurde. Wenn nein wird der nächste Schleifendurchlauf ausgeführt, wenn ja wird die Schleife verlassen.

Abbildung 62 zeigt die Schritte im Cleanup-Bereich der TestStand Sequenz.

1. Nach Verlassen der Schleife wird über *GetImageWindowDialogResult* die Information abgeholt, ob der Anwender den Dialog über den Button *Ok* oder eine andere Aktion geschlossen hat. Diese Information kann vom Testprogramm je nach Anwendungsfall weiterverarbeitet werden, etwa um zu entscheiden, ob ein Testreport erstellt und gespeichert werden soll oder nicht.
2. Zuletzt werden die zuvor angelegten Ressourcen wieder freigegeben. Der Aufruf von *CloseImageWindow* stellt sicher, dass auch im Fall eines unerwarteten Fehlers oder eines Programmabbruches alle geöffneten Fenster wieder geschlossen werden.
3. Die Funktion *ReleaseImageHandle* gibt den Bildhandle und somit den reservierten Speicher wieder frei, um Speicherlecks zu vermeiden.

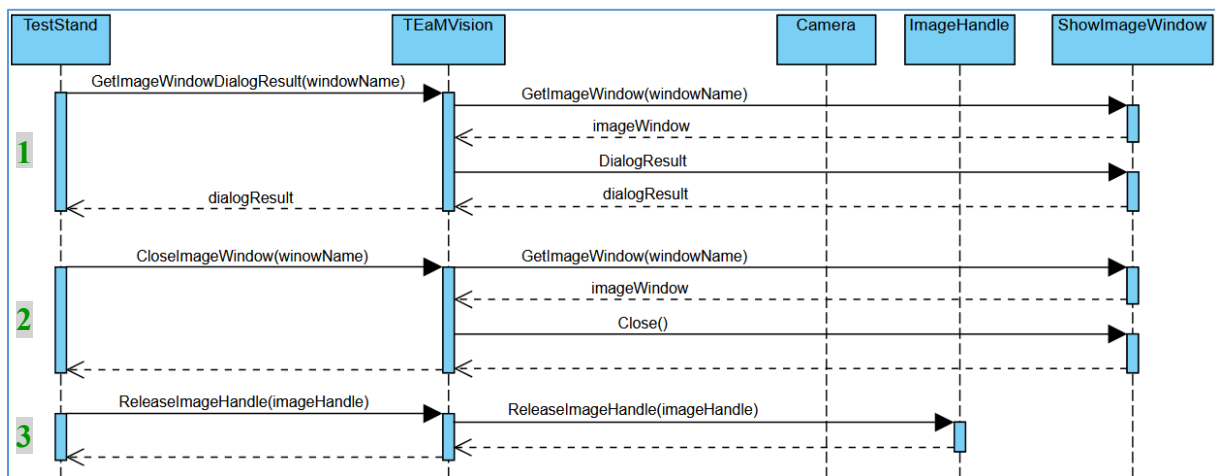


Abbildung 62: Programmablauf Cleanup-Bereich Testprogramm

Zur Veranschaulichung zeigt Abbildung 63 den gesamten Programmablauf, wie er in TestStand aussehen würde. Die Trennung von Bildaufnahme, Erkennungsmethode, Annotierung und Bildanzeige in separate Schritte erlaubt es, beliebige Erkennungen auf demselben Bild auszuführen und gibt dem Applikationsentwickler volle Kontrolle über die Verarbeitung und Anzeige der Ergebnisse.

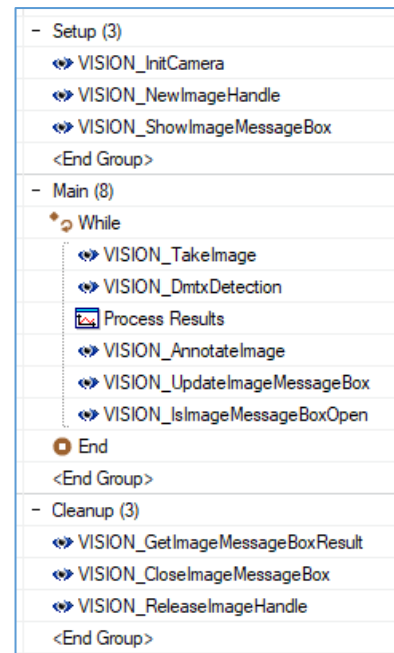


Abbildung 63: Programmablauf in TestStand

4.6 Zusammenfassung

In diesem Kapitel wurden die Anforderungen an eine Vision-Lösung für den Einsatz im Fertigungsumfeld bei Festo und die Integration in TestStand und das TEaM-Framework gesammelt. Anschließend wurde ein Konzept für den Aufbau und die Bedienung des TEaM-Moduls *TEaMVision* vorgestellt. Mit den Live-Preview Dialogen wurde ein neuartiger Mechanismus eingeführt, mit dem die verschiedenen Erkennungsmethoden einem breiten Kreis von potenziellen Anwendern verfügbar gemacht und die Entwicklung von effektiven Prüfprogrammen deutlich beschleunigt werden kann. Anhand ihrer spezifischen Live-Preview Dialoge wurden die Funktionsweisen der ersten vier Erkennungsmethoden beschrieben, mit denen bereits ein breites Spektrum von Anwendungsfällen umgesetzt werden kann.

Daraufhin wurden die Softwarearchitektur und die Funktion der wesentlichen Komponenten des Funktionsmoduls von *TEaMVision* definiert. Das Zusammenspiel der einzelnen Komponenten miteinander und im Einsatz in der TestStand Umgebung wurde anhand von mehreren beispielhaften Programmabläufen erläutert.

Die saubere Trennung durch klar definierte Schnittstellen zwischen den Komponenten und zwischen dem Funktionsmodul und TestStand erlauben Änderungen und Korrekturen im Rahmen der Weiterentwicklung, ohne bestehende TestStand Applikationen zu beeinflussen. Durch die Abstraktion von Kameras und Live Preview Komponenten können neue Kameravarianten, Erkennungsmethoden und sonstige denkbare Vorschau- und Bilddialoge sehr einfach und effizient ergänzt werden.

Das Gesamtkonzept für *TEaMVision* orientiert sich an bekannten Paradigmen aus dem TEaM-Framework. Jeder StepType ist eine atomare und klar verständliche Funktion, die über Edit-Dialoge konfiguriert wird. Ressourcen wie Kameras und Bildinstanzen werden initialisiert, über einen Handle angesprochen und wieder sauber deinitialisiert. Verschiedene Kameraimplementierungen werden abstrahiert und nach der Initialisierung gleichartig angesprochen. Parametrierungen werden für den Anwender gut nachvollziehbar in einer Einstellungsstruktur in den Eigenschaften einer Step-Instanz gespeichert, jede Erkennungsmethode gibt eine klar definierte Datenstruktur mit allen relevanten Informationen zurück. Auf Basis der Erkennungsergebnisse ist der Anwender völlig frei in der Weiterverarbeitung und Interpretation der Ergebnisse direkt in der TestStand Umgebung. Das potenziert noch einmal die Anzahl möglicher Anwendungsfälle.

Das Konzept für das *TEaMVision*-Modul beruht primär auf den Live-Preview Dialogen, für eine erfolgreiche Umsetzung ist es somit essenziell wichtig, dass diese Dialoge performant, zuverlässig und stabil funktionieren. Deshalb müssen für die Implementierung Technologien, Lösungen und Programmiertechniken eingesetzt werden, mit denen schnelle Zykluszeiten für Erkennungsmethoden und Bildverarbeitung, sowie effiziente Ressourcen- und Speicherverwaltung gewährleistet werden können.

5 Verwendete Technologien

Das Funktionsmodul hinter TEaMVision wird mit dem .Net Framework in der Programmiersprache C# umgesetzt. In diesem Kapitel werden kurz die wichtigsten Frameworks und Bibliotheken für die Implementierung der TEaMVision-Bibliothek eingeführt.

5.1 WPF

Zum Erstellen von GUI-Dialogen wird mit dem *Windows Presentation Framework* (WPF) die Standard C#-Bibliothek von Microsoft für grafische Oberflächen unter Windows eingesetzt.

Zur Anzeige einer grafischen Oberfläche wird ein Fenster, das *Window*, benötigt. Ein Window beinhaltet mehrere Kontrollelemente, die *Controls*. WPF bringt eine Palette von vordefinierten Controls mit. Das sind zum Beispiel die ganzen typischen UI-Elemente wie Textfelder, Buttons, Comboboxen (Dropdown-Menüs) oder Schieberegler. Es gibt auch Strukturierungs- und Gruppierungselemente. Ein *StackPanel* zum Beispiel kann mehrere weitere Controls enthalten und zeigt diese als horizontale oder vertikale Liste mit gleichmäßigem Abstand an. Mit solchen Elementen muss nicht jedes Control manuell auf der Oberfläche platziert werden.

Windows und Controls sind als Klassen definiert. Es können auch eigene Controls definiert, mit einer Reihe vordefinierter Controls gefüllt und anschließend in mehreren Fenstern wiederverwendet werden. Eine WPF-Applikation erstellt zum Anzeigen einer GUI eine neue Instanz eines Windows und öffnet das Fenster über die Methode *ShowDialog*.

Grafische Oberflächen in WPF werden primär in der Sprache XAML (Extensible Application Markup Language) definiert. Eine Window- oder Control-Klasse besteht aus einer XAML- und eine C#-Datei. Beim Kompilieren wird die XAML-Definition in C#-Code umgesetzt. Die Syntax von XAML ist abgeleitet von XML. Entsprechend wird hier eine hierarchische Struktur aus Elementen und Unterelementen definiert. Jedes Element kann optionale Eigenschaften in Form von Key-Value Paaren erhalten, zum Beispiel Name, Größe, Position des Elements.

GUI-Elemente werden in XAML definiert, das Verhalten der Elemente wird in C# festgelegt. Über Eventhandler können C#-Methoden mit verschiedenen Events der Controls verknüpft werden. Mit dem *Click* Event zum Beispiel kann eine Methode so bei Anklicken eines Buttons eine Aktion ausführen. Über die *Name* Property können alle GUI-Elemente im C#-Code als reguläre Objekte angesprochen werden.

```
<StackPanel Orientation="Vertical" Margin="20,0,0,0" >
    <StackPanel Orientation="Horizontal" Margin="1">
        <Border x:Name="ColorPreview" Width="58" Height="20" Background="Black" Margin="0,0,10,0"/>
        <Button x:Name="btnSelectColor" Height="20" Click="OnBtnSelectColor" Content=" Select " />
    </StackPanel>
    <Slider x:Name="HueSlider" Minimum="0" Maximum="359" Value="0" Margin="1" ValueChanged="HueSlider_ValueChanged"
        TickFrequency="1" IsSnapToTickEnabled="True" />
</StackPanel>
```

Abbildung 64: XAML Beispielcode

Abbildung 64 zeigt beispielhaft einen XAML-Ausschnitt der GUI von FindShapesLivePreview. Hier wird das Menü mit dem Schieberegler zur Farbauswahl definiert. Das Element *ColorPreview* zeigt die aktuell gewählte Farbe an. Der *HueSlider* erlaubt das manuelle Setzen der gewählten Farbe, mit dem Button *btnSelectColor* kann die Farbe direkt im Bild gewählt werden. Der C#-Code in Abbildung 65 zeigt die Eventhandler Methoden für den Button und den Slider. *HueSlider_ValueChanged* wird bei jeder Wertveränderung ausgelöst. Die Methode aktualisiert den Anzeigewert von *ColorPreview* und überschreibt den entsprechenden Wert in den Einstellungen der Erkennungsmethode. *OnBtnSelectColor* aktiviert über einen weiteren Aufruf den Auswahlmodus in *PreviewBaseControl*.

```

private void HueSlider_ValueChanged(object sender, RoutedPropertyChangedEventArgs<double> e)
{
    ColorPreview.Background = new SolidColorBrush(HSVToRGB(HueSlider.Value));
    _findShapes.Settings.ColorHue = (int)HueSlider.Value;
}

private void OnBtnSelectColor(object sender, RoutedEventArgs e)
{
    pBase.StartSelectPixel();
}

```

Abbildung 65: Eventhandler in C#

Das sind die wesentlichen Mechanismen, mit denen die Einstellungsmenüs in den Preview-Dialogen der Erkennungsmethoden aufgebaut werden.

Die Anzeige und die Verarbeitung der Benutzereingaben laufen in WPF in einem dedizierten GUI-Thread. Auch die Eventhandler Methoden werden in diesem Thread ausgeführt. Länger laufende Operationen blockieren somit den GUI-Thread, wodurch die Anzeige einfriert und nicht mehr auf Benutzereingaben reagiert. Aufwändigere Abläufe und Hintergrundaufgaben sollten somit immer außerhalb des GUI-Threads ausgeführt werden. Auf die Elemente der Dialoge kann nur innerhalb des GUI-Threads zugegriffen werden, ein Zugriff von außerhalb führt zu einem Fehler. Dafür stellt WPF die Funktion *Dispatcher.Invoke* bereit. Sie nimmt eine Inline-Funktion als Parameter und führt diese im GUI-Thread aus. So können auch Hintergrundaufgaben die Anzeige in der Oberfläche manipulieren.

5.2 OpenCV

Zur Bildverarbeitung wird OpenCV eingesetzt. Die Bibliothek ist in C++ implementiert und wird über einen Wrapper in C# verfügbar gemacht. Zentral in OpenCV ist der Datentyp *Mat*. Ein Mat-Objekt repräsentiert abstrakt eine N-Dimensionale Matrix von Datenpunkten, primär wird Mat für Bilder eingesetzt. Mat unterstützt verschiedenste Repräsentationsformen, Pixel können als Integer oder Gleitkommazahlen unterschiedlicher Größe abgebildet werden. In den meisten Fällen, so auch in TEaMVision, werden für die Pixel 8-Bit Integer verwendet. Ein Mat-Objekt kann mehrere Kanäle haben. So hat ein Graustufenbild einen Kanal, RGB-Farbbilder haben drei Kanäle.

OpenCV stellt eine Vielzahl von Funktionen bereit, die Mat-Objekte als Eingabe- und Ausgabeparameter nehmen. Dabei kann in der Regel auch das Eingabebild als Ausgabe verwendet werden, um keine Kopie anlegen zu müssen. Die Funktionen von OpenCV stellen viele Grundmechanismen der Bildverarbeitung bereit, so etwa Morphologische Operationen, Bildbinarisierung, Farbkonvertierung, Konturen finden, Kopieren von Bildern, Setzen von Pixeln und vieles mehr.

Mat-Instanzen belegen mit den Bilddaten große Blöcke im Arbeitsspeicher, deshalb müssen die Instanzen immer korrekt freigegeben werden. Mat implementiert in C# das *IDisposable*-Interface und kann somit über den Using-Dispose Mechanismus freigegeben werden. Mat-Instanzen sollten innerhalb einer Funktion immer in einer Using-Anweisung angelegt werden. Nach Verlassen des Using-Blocks werden die Instanzen und der Speicher so automatisch wieder freigegeben.

5.3 Kamertreiber

Zur Anbindung der Ethernetbasierten Baumer-Kameras wird das GigE-Vision Protokoll verwendet. Derzeit gibt es aber keine etablierte und frei verfügbare Implementierung des Protokolls für C#. Anfänglich wurde eine C#-Implementierung aus einem Community-Projekt verwendet, damit konnte aber keine erfolgreiche Verbindung zu den Baumer Kameras aufgebaut

werden. Deshalb wird vorerst der *neoAPI*-Treiber von Baumer verwendet. Dieser nutzt ebenfalls das GigE-Vision Protokoll, unterstützt aber nur die Anbindung von Baumer Kameras und keine anderen Hersteller.

5.4 Dmtx-Erkennung

Für die Dekodierung von Datamatrix Codes wird nicht OpenCV eingesetzt, hierfür kommen dedizierte Bibliotheken zum Einsatz. Die Bibliotheken sind sehr einfach einzusetzen. Das Mat-Bild kann entweder direkt übergeben werden oder muss in ein Bildformat wie Bitmap o.Ä. konvertiert werden. Die Dmtx-Bibliothek analysiert anschließend das Bild und gibt eine Liste der gefundenen Ergebnisse inklusive dekodiertem Text zurück.

Als erstes wurde die Bibliothek *LibDmtx* evaluiert. Die Leistungsfähigkeit ist allerdings nicht zufriedenstellend. Codes mit schlechter Qualität werden oft nicht erkannt und in vielen weiteren Fällen werden fehlerhafte Ergebnisse zurückgegeben, in denen der Inhalt nicht korrekt dekodiert wurde. Die Erkennungsleistung ist leider deutlich schlechter als mit Handscannern oder kommerziellen Vision-Systemen, deshalb ist die Lösung für den Praxiseinsatz nicht geeignet.

Als Alternative wird die *Barcode Scanner* Bibliothek vom Hersteller *Dynamsoft* eingesetzt. Hierbei handelt es sich um ein kommerzielles Produkt, glücklicherweise bietet Dynamsoft ein Lizenzmodell, mit dem die Bibliothek Festo-Intern freizügig verteilt werden kann. Für 9000€ wurde eine Volumenlizenz für 100 gleichzeitige Anwender angeschafft. Dabei wird TEaMVi-sion einfach mit einem Lizenzschlüssel im Quellcode versehen, es ist keine Verbindung zu einem Lizenzserver o.Ä. vonnöten. Für 100 potenzielle Anwender ist dieses einmalige Investment mehr als gerechtfertigt und durch das Lizenzmodell entstehen beim Anwender keinerlei direkten Kosten oder Aufwände durch Anbindung an einen Lizenzserver.

Die Performance der Dynamsoft Bibliothek ist ausgesprochen gut. Die Dekodierung eines Codes dauert Millisekunden und bei vertretbarer Bildqualität konnten bisher alle Codes korrekt dekodiert werden.

5.5 Tesseract-OCR

Zur Erkennung von Text wird die freie Bibliothek Tesseract-OCR eingesetzt. Die Bibliothek nimmt ein Bild und kann in diesem selbständig alle Bereiche mit Text identifizieren und dekodieren. Abbildung 66 zeigt den Code zur Verarbeitung aller Textblöcke in einem Bild, eine gesonderte Vorverarbeitung o.Ä. ist nicht notwendig.

```
_tEngine = new TesseractEngine(TESSDATAPATH, "eng", EngineMode.Default);
using (var page = _tEngine.Process(img))
using (var it = page.GetIterator())
{
    do
    {
        it.TryGetBoundingBox(PageIteratorLevel.TextBlock, out var rect);
        var text = it.GetText(PageIteratorLevel.TextBlock);
        /* use text/rect */
    }
    while (it.Next(PageIteratorLevel.TextBlock));
}
```

Abbildung 66: Code zum Einsetzen von Tesseract-OCR

Über einen *PageIterator* kann dann in einer Schleife jedes Textelement im Bild einzeln betrachtet werden. Über das *PageIteratorLevel* können einzelne Wörter, Zeilen, Absätze oder ganze Seiten betrachtet und dekodiert werden. Die *BoundingBox* gibt Informationen zu Positionen und Größe eines Textelements zurück.

6 Implementierung

Nachdem nun die eingesetzten Technologien und Bibliotheken eingeführt wurden, soll dieses Kapitel detailliert die Implementierung der TEaMVision Funktionsbibliothek beleuchten. Die Kernkomponenten, ihre öffentlichen Eigenschaften und die Interaktion untereinander wurden bereits im Abschnitt zur Softwarearchitektur erläutert. Dort wurde gezeigt, welche Methoden und Eigenschaften der Komponenten wie ineinandergreifen, um die notwendige Funktionalität abzubilden. Ergänzend dazu soll hier anhand ihrer konkreten Implementierung vorgestellt werden, wie diese Mechanismen genau funktionieren, auch um verständlich zu machen wie und mit welchem Aufwand weitere Erkennungen, Dialoge oder Kameras ergänzt werden können. Darüber hinaus wird die Implementierung einiger wesentlicher Schlüsselstellen vorgestellt, ohne welche die Funktionsfähigkeit, Performance und Stabilität des Moduls in dieser Form nicht gegeben wären. Und schließlich werden auch Umsetzung und Funktionsweise der einzelnen Erkennungsmethoden erläutert.

6.1 TEaMVision

Die Klasse *TEaMVision* ist für alle externen Aufrufe der zentrale Einstiegspunkt, jede umgesetzte Funktionalität wird hier gebündelt angeboten. Als Schnittstelle zwischen der C#-Bibliothek und der TestStand Umgebung sichert sie die reibungslose Integrierbarkeit, Interaktion und Fehlerbehandlung ab.

6.1.1 Statische Funktionen

Abbildung 67 zeigt beispielhaft eine der statischen Funktionen von TEaMVision. Hier ist das einheitliche Interface mit Integer Rückgabewert und dem *msg* Out-Parameter zu sehen. Daran wird die Aufgabe der TEaMVision Klasse deutlich. Die Funktionen stellen einen Try-Catch Wrapper um die interne Logik bereit, der verhindert, dass Exceptions zu Ausnahmefehlern in TestStand führen. Anstatt dessen werden ein entsprechender Return Code und eine Fehlermeldung zurückgegeben.

```
public static int SaveImage(int imageHandle, string filePath, out string msg)
{
    try
    {
        msg = "";
        if (!ImageHandle.HandleExists(imageHandle))
        {
            msg = "Invalid image handle!";
            return 1;
        }
        var mat = ImageHandle.GetHandle(imageHandle).Mat;
        if (mat == null)
        {
            msg = "Image handle is empty!";
            return 2;
        }
        Cv2.ImWrite(filePath, mat);
        return 0;
    }
    catch (Exception ex)
    {
        msg = ex.Message;
        return -1;
    }
}
```

Abbildung 67: Codebeispiel statische Funktion von Klasse TEaMVision

Die Eingabeparameter werden hier vor dem Ausführen der Funktionslogik überprüft und es wird gegebenenfalls ein Fehlercode und eine Fehlermeldung zurückgegeben. Somit agiert TEaMVision in beide Richtungen als „Schutzschicht“. Nach oben werden keine unerwarteten Ausnahmefehler propagiert und nach unten kann die weitere Funktionslogik sich auf definierte und erwartete Parametrierung und Szenarien verlassen.

6.1.2 Interaktion mit TestStand Context

Die Funktionen für die Live-Preview Dialoge sind eine Ausnahme von der zuvor aufgestellten Regel, denn diese werden zur Programmierzeit aufgerufen. Eine Herausforderung ist die Übergabe der Step-Parameter, denn diese werden durch die Edit-Dialoge meist nicht als Literalwert, sondern in Form von TestStand Ausdrücken in der Pre-Expression eines Schrittes definiert. Die Ausdrücke werden zur Laufzeit evaluiert. Somit enthalten die Parametervariablen des Steps auch nur dann die korrekten Werte für Kameratyp, Handle und Locator. Deshalb lesen die Live-Preview Funktionen die Pre-Expression über die TestStand API direkt aus dem Step aus und lösen die Ausdrücke bereits zur Programmierzeit selbstständig auf.

```
public static void ConfigureCameraLivePreviewTS(SequenceContext thisContext, CameraTypes cameraType,
    string cameraLocation, int cameraHandle, CameraSettings settings, out CameraSettings updatedSettings)
{
    cameraType = GetCameraTypeFromSeqContext(thisContext, cameraType);
    cameraLocation = GetCameraLocationFromSeqContext(thisContext, cameraLocation);
    cameraHandle = GetCameraHandleFromSeqContext(thisContext, cameraHandle);

    if (ConfigureCameraLivePreview(cameraType, cameraLocation, cameraHandle, settings, out updatedSettings))
        thisContext.SequenceFile.IncChangeCount();
}

private static int GetCameraHandleFromSeqContext(SequenceContext thisContext, int defaultValue)
{
    var ex = thisContext.Step.PreExpression
        .Split('\n')
        .FirstOrDefault(s => s.StartsWith("Step.Parameters.CameraHandle = "))
        ?.Replace("Step.Parameters.CameraHandle = ", "");

    if (ex == default) return defaultValue;
    if (ex.EndsWith(", ")) ex.Replace(", ", "");
    if (ex.EndsWith("; ")) ex.Replace("; ", "");

    var result = thisContext.AsPropertyObject().EvaluateEx(ex, 0);
    return result.GetValNumber("", 0);
}
```

Abbildung 68: Codebeispiel Auflösen von TestStand Pre-Expression in TEaMVision

Abbildung 68 zeigt die Funktion für das Live-Preview zum Konfigurieren einer Kamera. Diese bekommt aus TestStand eine Referenz auf den *SequenceContext* übergeben. Unterfunktionen ermitteln daraus die korrekten Werte für Kameratyp, Handle und Location. Damit wird anschließend die reguläre Live-Preview Funktion aufgerufen, die mit dieser Trennung auch unabhängig von TestStand genutzt werden kann. Diese gibt als Boolean zurück, ob die Einstellungen aktualisiert wurden oder ob der Dialog über *Abbrechen* geschlossen wurde. Die letzte Aktion *SequenceFile.IncChangeCount* teilt TestStand im Positivfall mit, dass Änderungen an der Sequenzdatei vorgenommen wurden, woraufhin diese im Sequence Editor als *hat ungespeicherte Änderungen* markiert wird.

Die Methode *GetCameraHandleFromSeqContext* zeigt das Parsen der Pre-Expression. Der Edit-Dialog gewährleistet ein einheitliches Format des Ausdrucks, deshalb kann die relevante Zeile für den jeweiligen Parameter zuverlässig ermittelt werden. Der zweite Teil der Zeile ist in der Regel ebenfalls ein Ausdruck nach dem Muster *Locals.CameraHandle*. Dieser kann über die *EvaluateEx* Methode im Kontext der aktuellen Sequenz aufgelöst werden und der Wert wird dann zurückgegeben.

Anstatt eines Variablenverweises kann im Edit-Dialog auch direkt ein Literalwert eingetragen sein. Dieser wird dann direkt im Parameterfeld des Steps und nicht in einer Expression gespeichert. Somit gibt es in der Pre-Expression auch keinen Eintrag für den Parameter, deshalb werden für diesen Fall neben dem *SequenceContext* immer auch die direkten Parameter des Steps mitübergeben. Falls keine passende Expression gefunden wird, wird dieser Default-Wert verwendet.

6.1.3 Laden von weiteren Abhängigkeiten

Das TEaMVision Modul ist abhängig von weiteren externen Codemodulen. Diese haben auch Abhängigkeiten, die nicht als .Net Modul, sondern als native DLL vorliegen. Die Standardmechanismen zum automatischen Lokalisieren von nativen DLL-Abhängigkeiten suchen primär im Verzeichnis des aktuell ausgeführten Prozesses. Da die TEaMVision Bibliothek nicht im gleichen Verzeichnis wie der TestStand Sequence Editor liegt, führt das hier zu Problemen. Deshalb fügt TEaMVision den Pfad zu den nativen DLLs manuell zu den Suchpfaden hinzu, womit das Betriebssystem in der Lage ist, die Abhängigkeiten zu lokalisieren. Abbildung 69 zeigt den Code dafür.

```
public static void SetDllSearchDirectory()
{
    if (_dllDirectorySet) return;
    var dllDirectory = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
    SetDllDirectory(Path.Combine(dllDirectory, Environment.Is64BitProcess ? "x64" : "x86"));
    _dllDirectorySet = true;
}

[DllImport("kernel32.dll", CharSet = CharSet.Auto, SetLastError = true)]
private static extern bool SetDllDirectory(string lpPathName);
```

Abbildung 69: DLL-Suchpfade ergänzen in TEaMVision

Als Basispfad wird das Verzeichnis ermittelt, in dem sich das aktuell ausführende Modul, also die *TEaMVision.dll* befindet. Diese ist für *Any CPU* kompiliert und kann somit von 32- und 64-Bit Prozessen eingebunden werden. Die nativen Abhängigkeiten sind entweder für 32- oder 64-Bit kompiliert, deshalb befinden sich die jeweiligen Ausführungen der Abhängigkeiten in entsprechenden Unterordnern. Abhängig von der Architektur des aktuellen Prozesses wird somit der *x86* oder der *x64* Unterordner eingebunden. Zum Setzen des Suchpfades wird aus der nativen Windows API die Funktion *SetDllDirectory* der *kernel32.dll* eingebunden und mit dem Pfad aufgerufen.

Die Funktion *SetDllSearchDirectory* muss genau einmal nach Laden der *TEaMVision.dll* aufgerufen werden, damit weitere Funktionen genutzt werden können. Um das sicherzustellen, wird die Funktion unter Anderem in den *InitCamera* Methoden aufgerufen. Ohne eine Kamera kann die weitere Funktionalität von TEaMVision nicht sinnvoll genutzt werden, deshalb ist ein *InitCamera* Aufruf in der Regel die erste ausgeführte Aktion. So wird sichergestellt, dass immer alle Abhängigkeiten erfolgreich geladen werden können.

6.2 ImageHandle

Die Klasse *ImageHandle* dient zur Verwaltung von Bildinstanzen, die von mehreren Komponenten verwendet und zwischen TestStand-Schritten übergeben werden. Instanzen werden über eindeutige Integer-Handles referenziert, dafür wird das Multiton-Pattern eingesetzt. Der Ausschnitt der Implementierung von *ImageHandle* in Abbildung 70 zeigt die konkrete Umsetzung des Entwurfsmusters.

Der Konstruktor der Klasse ist privat und wird nur von *CreateImageHandle* aufgerufen. Für die eindeutigen Handles wird bei jedem Aufruf ein statischer Counter inkrementiert und der Wert als Handle der neuen Instanz gesetzt. Die verwalteten Instanzen von *ImageHandle* werden in einem statischen Dictionary¹⁶ gehalten. *GetImageHandle* ruft über den Integer-Schlüssel existierende Instanzen ab. *ReleaseImageHandle* gibt die Bilddaten einer Instanz frei und entfernt den Eintrag aus dem Dictionary.

¹⁶ Dictionary ist die C#-spezifische Implementierung einer Hashmap


```

public class ImageHandle
{
    private static Dictionary<int, ImageHandle> ImageHandles = new Dictionary<int, ImageHandle>();
    private static int ImageHandlesCtr = 1;
    public int Handle { get; private set; }
    public Mat Mat { get; private set; }

    private ImageHandle(int handle)
    {
        Handle = handle;
        Mat = new Mat();
    }

    public static ImageHandle CreateImageHandle()
    {
        var ih = new ImageHandle(ImageHandlesCtr);
        ImageHandles.Add(ImageHandlesCtr, ih);
        ImageHandlesCtr++;
        return ih;
    }

    public static ImageHandle GetImageHandle(int handle)
    {
        if (ImageHandles.ContainsKey(handle))
            return ImageHandles[handle];
        else return null;
    }

    public static void ReleaseImageHandle(int handle)
    {
        if (ImageHandles.ContainsKey(handle))
        {
            var ih = ImageHandles[handle];
            ImageHandles.Remove(handle);
            ih.Mat.Dispose();
        }
    }

    public void SetMat(Mat mat)
    {
        if (Mat != null && (mat.Width != Mat.Width || mat.Height != Mat.Height))
            Mat.Dispose();
        if (Mat == null || Mat.IsDisposed)
            Mat = new Mat();
        mat.CopyTo(Mat);
    }
}

```

Abbildung 70: Implementierung des Multiton-Patterns bei ImageHandle

Die Bildinstanz wird in der Property *Mat* gehalten. Die Property kann öffentlich gelesen aber nur privat geschrieben werden. Eine Get-Methode ist nicht notwendig, diese würde nur den Wert von *Mat* zurückgeben.

Ein *ImageHandle* soll immer die Hoheit über die in ihm enthaltene *Mat*-Instanz behalten. Deshalb ist es nicht erlaubt, die Instanz von außen neu zu setzen. *SetMat* kopiert den Inhalt einer übergebenen Instanz in die eigene. Ansonsten bestünde die Gefahr, dass beim Freigeben eines *ImageHandles* auch ein Bild freigegeben wird, das noch anderweitig genutzt wird. Es wäre somit nur schwer zu überblicken, wer für das Freigeben des Bildes zuständig ist. Gleichermäßen sollte der Empfänger eines *ImageHandles* den Inhalt immer in ein eigenes Bild kopieren oder sich bewusst sein, dass die Instanz freigegeben werden könnte. Das passiert zum Beispiel, wenn bei *SetMat* ein Bild mit neuen Dimensionen übergeben wird, dafür muss die alte Instanz freigegeben und durch eine neue ersetzt werden.

6.3 Camera

Die abstrakte Klasse *Camera* vereint mit ihrer statischen Funktion *CreateCamera* das Multiton-Pattern mit einem Factory-Mechanismus. Abbildung 71 zeigt, wie hier zuerst die Eingabeparameter validiert und anschließend eine Instanz der konkreten Implementierung für die jeweilige Kameravariante erzeugt wird.

Da der Integer-Handle hier nicht automatisch vergeben, sondern vom TestStand-Anwender vorgegeben wird, muss hier die Einmaligkeit des Handles sichergestellt werden. Ist der Handle

bereits für eine andere Kamera registriert, wird die alte Kamera verworfen und der Handle einer neuen Kamera zugewiesen. Falls für die gegebene Paarung aus Locator und Kameratyp bereits eine Instanz existiert, wird dieser Instanz der neue Handle zugewiesen, anstatt eine neue Instanz zu erzeugen.

Dieses Verhalten ist dem Auslösen einer Fehlermeldung an dieser Stelle vorzuziehen, denn die Lebenszeit und Gültigkeit des Camera Multiton geht über die eines einzelnen TestStand Testprogramms hinaus. Wenn der Anwender im Sequence-Editor zwischen mehreren Programmen wechselt, sollten daraus keine lästigen Seiteneffekte entstehen.

```
public static Camera CreateCamera(CameraTypes mode, string sourceLocator, int handle)
{
    var cam = _cameras.FirstOrDefault(m => m.Handle == handle);
    if (cam != default && (cam.Mode != mode || cam.SourceLocation != sourceLocator))
        _cameras.Remove(cam);

    cam = _cameras.FirstOrDefault(m => m.SourceLocation == sourceLocator && m.Mode == mode);
    if (cam != default)
    {
        cam.Handle = handle;
        return cam;
    }

    switch (mode)
    {
        case CameraTypes.BaumerIpCamera:
            cam = new BaumerCamera(mode, sourceLocator, handle);
            break;
        case CameraTypes.File:
            cam = new FileCamera(mode, sourceLocator, handle);
            break;
        case CameraTypes.UsbCamera:
        default:
            throw new Exception("Unsupported Source Mode!");
    }

    _cameras.Add(cam);
    return cam;
}
```

Abbildung 71: Methode CreateCamera der Klasse Camera

Um eine weitere Kameraimplementierung hinzuzufügen, muss diese im Enumerationstyp *CameraTypes* ergänzt und anschließend in der Factory in *CreateCamera* eingetragen werden. Hier ist bereits ein neuer angedachter Typ *UsbCamera* eingetragen, der aber noch keine Implementierung besitzt.

6.3.1 FileCamera

Abbildung 72 zeigt die Implementierung der Kameravariante *FileCamera*. Hier wird ersichtlich, wie leichtgewichtig eine weitere Kameravariante hinzugefügt werden kann. Die Variante *FileCamera* liest zum Erstellungszeitpunkt ein Bild aus einer Datei aus und gibt dieses beim Abrufen eines neuen Kamerabildes zurück. *FileCamera* unterstützt keine weiteren generischen Parameter, deshalb müssen keine Implementierungen für *SetParameter* und auch keine *SupportedParameters* bereitgestellt werden.

```
internal class FileCamera : Camera
{
    private Mat _fileImage;
    public FileCamera(CameraTypes mode, string locator, int handle) : base(mode, locator, handle)
    {
        _fileImage = Cv2.ImRead(SourceLocation);
    }

    protected override void GetImageInternal()
    {
        _fileImage.CopyTo(_cameraImage);
    }
}
```

Abbildung 72: Implementierung der Kameravariante FileCamera

Eine minimale Kameraimplementierung muss nur die abstrakte Methode *GetImageInternal* implementieren und darin ein aktuelles Kamerabild übergeben. *GetImage* von *Camera* ruft diese Methode auf, wendet Rotation und Region of Interest auf das Bild an und überträgt das Bild anschließend in den *ImageHandle*.

Für die Übergabe des Kamerabildes hat *Camera* eine *Mat*-Instanz *_cameraImage*. In diese schreibt eine Kameraimplementierung das rohe aktuelle Kamerabild, das dann von *Camera* in mehreren Schritten weiterverarbeitet wird. Die gesamte Logik für Rotation, ROI und eventuelle zukünftige Funktionen ist komplett in der Basisklasse lokalisiert. So kann eine neue Kamera-variante mit wenigen Zeilen Code den vollen Funktionsumfang von *Camera* bereitstellen.

6.3.2 BaumerCamera

Die *BaumerCamera* nutzt den spezifischen Treiber der Baumer Industriekameras, um diese per Ethernet/IP einzubinden. Abbildung 73 zeigt, wie in *GetImageInternal* ein Kamerabild abgefragt und daraus eine OpenCV *Mat* generiert wird.

Über *GetImage* des Baumer Treibers wird ein Bild in einem Baumer-spezifischen Bildformat abgeholt. Bei der Initialisierung der Kamera wird ein Pixelformat konfiguriert, das mit OpenCV kompatibel ist. So kann über *FromPixelData* ein *Mat*-Objekt aus dem rohen Speicher des Baumer Bildes erstellt werden, ohne eine Kopie anzulegen. Das *Mat*-Objekt kann dann nach *_cameraImage* kopiert werden. Das Baumer Bild und somit auch der Speicher hinter dem hier erstellten *Mat*-Objekt werden beim Verlassen des *Using*-Blocks freigegeben.

```
protected override void GetImageInternal(ImageHandle imageHandle)
{
    try
    {
        if (_connectionLost)
        {
            InitBaumerCamera();
            _connectionLost = false;
        }

        using (var img = _baumerCamera.GetImage(4000))
        {
            if (img.Size == 0)
            {
                throw new Exception("Camera GetImage Timeout. Try again...");
                Mat.FromPixelData((int)image.Height, (int)image.Width, MatType.CV_8UC3, image.ImageData)
                    .CopyTo(_cameraImage);
            }
        }
    }
    catch (Exception ex)
    {
        _connectionLost = true;
        throw;
    }
}
```

Abbildung 73: *GetImageInternal* von *BaumerCamera*

Hier zeigt sich, warum der Umweg über das *Mat*-Objekt *_cameraImage* gewählt wurde. *GetImageInternal* könnte auch einfach ein *Mat*-Objekt zurückgeben. Das würde aber zu Komplikationen beim Freigeben des Speichers führen. Das hier erstellte *Mat*-Objekt wird durch den *Using*-Block spätestens beim Verlassen der Methode freigegeben, eine darüberliegende Methode könnte nicht mehr auf den Speicher zugreifen. Zusätzlich wäre das Freigeben des *Mat*-Objektes auf der Ebene von *Camera* je nach Implementierung eventuell ungenügend, da das *Mat*-Objekt hier lediglich den Speicher hinter einem anderen Objekt referenziert.

Verbindungsfehler werden hier als *Exception* nach außen propagiert und in *TEaMVision* oder den *Live-Preview* Dialog gehandhabt. Bei einer *Exception* in *GetImageInternal* würde ein Mechanismus zur Freigabe eines *Mat*-Objektes in *Camera* überhaupt nicht greifen, da das *Mat*-Objekt an dieser Stelle noch gar nicht erstellt oder zurückgegeben wurde.

Daran wird deutlich, wieso die klare Teilung der Zuständigkeit für einen Speicherbereich so wichtig ist, um Speicherlecks und Zugriffsverletzungen zu vermeiden. Die Kameraimplementierung behält die Kontrolle über den in ihr reservierten Speicher und übergibt Bilddaten nur strikt als Kopie in einen Bildpuffer, dessen Zuständigkeit ebenfalls klar definiert ist. Durch solche Maßnahmen wird hier und an vielen anderen Stellen eine saubere Verwaltung von Speicher und somit eine Stabilität auch bei längeren Laufzeiten gewährleistet.

6.3.3 Kameraparameter

Abbildung 74 zeigt Teil der Implementierung für die zusätzlichen Parameter von Baumer-Camera, zur besseren Darstellbarkeit wurden hier einige Parameter entfernt.

```
public override List<CameraParameter> SupportedParameters = new List<CameraParameter>()
{
    new CameraParameter
    {
        Name = "gainauto",
        Type = CameraParameterTypes.Bool,
        Value = "False",
    },
    new CameraParameter
    {
        Name = "exposure",
        Type = CameraParameterTypes.Double,
        MinValue = "1",
        MaxValue = "1000000",
        Value = "100000",
    },
    new CameraParameter
    {
        Name = "targetbrightness",
        Type = CameraParameterTypes.Double,
        MinValue = "1",
        MaxValue = "100",
        Value = "1",
    },
};

protected void SetParameter(string name, double value)
{
    switch (name)
    {
        case "exposure":
            SetExposure(value);
            break;
        case "targetbrightness":
            SetTargetBrightness(value);
            break;
        default:
            break;
    }
}

protected void SetGainAuto(bool value)
{
    if (value)
    {
        _baumerCamera.f.GainAuto.Set(GainAuto.Continuous);
        _baumerCamera.f.BrightnessAutoPriority.Set(BrightnessAutoPriority.GainAuto);
    }
    else
    {
        _baumerCamera.f.GainAuto.Set(GainAuto.Off);
    }
}
```

Abbildung 74: Teil der Parameterdefinition von BaumerCamera

Die unterstützten Parameter werden durch Überschreiben des Feldes `SupportedParameters` von Camera definiert. Jeder Parameter definiert Name, Datentyp und einen Defaultwert. Numerische Parameter unterstützen einen zulässigen Wertebereich. Die Basisklasse überprüft für alle Kameravarianten, ob der übergebene Parameter von der Kameravariante unterstützt wird, ob der Datentyp stimmt und ob der zulässige Wertebereich verletzt wurde. Die Kameravariante erhält in `SetParameter` ausschließlich validierte Parameterwerte und muss somit nur Handler zum Verarbeiten für jeden unterstützten Parameter definieren.

Die Methode *SetGainAuto* zeigt, dass hinter dem Setzen eines Parameters durchaus mehr als eine einzige Aktion stehen kann. Der Wert wird nicht einfach in ein Feld der Baumer Kamera übertragen. Oft sind eine Fallunterscheidung und das Setzen mehrerer Felder notwendig.

Die generischen Kameraparameter bieten somit eine gute Möglichkeit, trotz der Abstraktion durch die einfach gestaltete Camera Basisklasse auch komplexere Spezialfunktionen diverser Kameraimplementierungen abbilden zu können.

6.4 FindShapes

Die Erkennungsmethode *FindShapes* binarisiert ein Bild nach Helligkeit oder Farbe und sucht in der Binärmaske anschließend nach Konturen mit einer vorgegebenen Form und Dimension. Der Anwender von *FindShapes* initialisiert die Instanz der Klasse mit Einstellungen und einem *ImageHandle*, führt die Methode *Run* aus und kann anschließend aus *FoundResults* die gefundenen Ergebnisse auslesen. Zusätzlich können das Maskenbild und ein Originalbild mit Annotationen aller Ergebnisse über *MaskImage* und *AnnotatedImage* ausgelesen und weiterverwendet werden.

Der vereinfachte Ablauf der Methode *Run* ist in Abbildung 75 zu sehen. Bei jedem Durchlauf wird das aktuelle Bild aus dem *ImageHandle* ausgelesen, der im Konstruktor übergeben wurde. Anschließend wird die Binärmaske aktualisiert und damit die Erkennung ausgeführt. Zuletzt werden die Ergebnisbilder generiert.

```
public void Run()
{
    UpdateImage();
    UpdateMask();
    RunDetection();
    AnnotateImage();
}
```

Abbildung 75: Schematischer Ablauf der Methode *Run* von *FindShapes*

In den nächsten Abschnitten werden die Bildung der Maske und die Funktionsweise der Erkennung genauer erläutert.

6.4.1 Maskenbildung

Das Erstellen der Binärmaske für einen gegebenen Intensitätsgrenzwert in Abbildung 76 ist nicht sonderlich kompliziert. Das Feld *_gray* wird bereits in *UpdateImage* gesetzt und enthält das Graustufenbild zum Eingabebild. Mit *GetRoiRect* wird der Bildbereich für die Region of Interest extrahiert. Je nach Einstellung wird dann über *InRange* das Binärbild für alle Pixel, entweder über oder unter dem Helligkeitsgrenzwert erstellt.

```
private void GenerateIntensityMask(Mat mask, int intensity)
{
    var roiGray = new Mat(_gray, GetRoiRect());

    if (Settings.Dark)
        Cv2.InRange(roiGray, 0, intensity, mask);
    else
        Cv2.InRange(roiGray, intensity, 255, mask);
}
```

Abbildung 76: Bildung der Intensitätsmaske in *FindShapes*

Für die Bildung der Farbmaske in Abbildung 77 muss etwas mehr Aufwand betrieben werden. Die originalen Farbbilder liegen im RGB-Format vor, für die Bildung der Farbmaske wird das Bild in den HSV-Farbraum konvertiert. Dadurch befinden sich typische Signal- und Designfarben wie Rot, Gelb, Grün, Blau, Violett etc. auf einer eindimensionalen Skala und sind so leichter über einen Grenzwert zu definieren.

```

private void GenerateColorMask(Mat mask)
{
    using (var roi = Image.SubMat(GetRoiRect()))
    using (var hsvImg = roi.CvtColor(ColorConversionCodes.BGR2HSV))
    using (var h = new Mat())
    using (var s = new Mat())
    using (var v = new Mat())
    using (var tmpMask = new Mat())
    {
        Cv2.MedianBlur(hsvImg, hsvImg, 3);
        Cv2.ExtractChannel(hsvImg, h, 0);
        Cv2.ExtractChannel(hsvImg, s, 1);
        Cv2.ExtractChannel(hsvImg, v, 2);
        var hueMargin = Settings.HueMargin;
        var targetHue = Settings.ColorHue / 2;

        if (targetHue < hueMargin)
        {
            Cv2.InRange(h, 0, targetHue + hueMargin, mask);
            Cv2.InRange(h, 179 - (hueMargin - targetHue), 179, tmpMask);
            mask.SetTo(new Scalar(255), tmpMask);
        }
        else if (targetHue > 179 - hueMargin)
        {
            Cv2.InRange(h, 0, 179 - (targetHue - hueMargin), mask);
            Cv2.InRange(h, targetHue - hueMargin, 179, tmpMask);
            mask.SetTo(new Scalar(255), tmpMask);
        }
        else
        {
            Cv2.InRange(h, targetHue - hueMargin, targetHue + hueMargin, mask);
        }

        Cv2.InRange(s, 0, Settings.ColorMinSaturation, s);
        Cv2.InRange(v, 0, Settings.ColorMinBrightness, v);
        mask.SetTo(new Scalar(0), s);
        mask.SetTo(new Scalar(0), v);
    }
}

```

Abbildung 77: Bildung der Farbmaske in FindShapes

Aus dem HSV-Bild werden die einzelnen Farbkanäle h , s und v extrahiert, die dadurch jeweils als Graustufenbild vorliegen. Über *InRange* wird erneut ein Binärbild erstellt, diesmal für den Zielfarbtone *ColorHue*. Der Parameter *HueMargin* gibt eine Maximalabweichung in beide Richtungen vor. Die Hue-Skala ist üblicherweise definiert von 0-359, in OpenCV wird pro Pixel aber nur ein Byte genutzt. Die Skala ist hier 0-179, deshalb wird der Parameterwert halbiert. Die Hue-Skala bildet einen Farbkreis ab, dabei liegen die roten Farbbereiche um die Grenzwerte 0 und 179. Deshalb müssen bei einem Zielwert nahe des Grenzbereiches auch die Werte am anderen Ende des Wertebereiches mit einbezogen werden.

Über die Filter auf *ColorMinSaturation* und *ColorMinBrightness* werden sehr dunkle Bereiche und Bereiche mit sehr niedriger Farbsättigung ausgeblendet. Das sind weiße, graue und schwarze Pixel, die nicht wirklich einem Farbton zuzuordnen sind. Ohne diesen Filter würde die Maske signifikantes Rauschen enthalten. Mit diesen Filtern stellt die Maske nur die Bereiche klar heraus, die auch für einen Menschen deutlich als eingefärbt erkennbar sind.

Unabhängig davon, ob eine Farb- oder Intensitätsmaske generiert wurde, wird in beiden Fällen im Anschluss dieselbe Nachverarbeitung auf die Maske (beziehungsweise Vorverarbeitung für die Erkennung) angewandt. Der Zweck der Nachverarbeitung in Abbildung 78 ist die Eliminierung von Rauschen im Hintergrund und Artefakten am Rand von Konturen, sowie das Schließen von kleinen Löchern in großen Objekten.

Die Kombination von Erosion und Dilation entfernt kleine Flecken und Rauschen im Bild und glättet die Kanten von größeren Objekten. Die morphologischen Operationen können nur kleine Löcher in Konturen zuverlässig schließen. Zu starke Dilation könnte die Form einer Kontur zu

sehr verfälschen. Deshalb werden im mittleren Teil die verbleibenden Löcher in Konturen ermittelt und manuell ausgefüllt.

Dazu wird das Bild invertiert und darin nach Konturen mit einer kleinen Fläche (unter 500 Pixeln) gesucht. Die Konturen im invertierten Bild sind der Hintergrund, sowie alle Löcher in Konturen des Originalbildes. Die kleinen Konturen werden über *DrawContours* mit dem Wert 0 im invertierten Bild entfernt. Anschließend wird das invertierte Bild erneut invertiert und als Maske weiterverwendet. Die Löcher sind nun geschlossen.

```
private void ProcessMask(Mat mask)
{
    using (var kernel = Mat.Ones(3, 3, MatType.CV_8UC1).ToMat())
    using (var tmp = new Mat())
    {
        Cv2.Erode(mask, mask, kernel, iterations: 3);
        Cv2.Dilate(mask, mask, kernel, iterations: 5);

        Cv2.BitwiseNot(mask, tmp);
        Cv2.FindContours(tmp, out var contours, out _,
            RetrievalModes.CComp, ContourApproximationModes.ApproxSimple);

        foreach (var contour in contours)
        {
            var area = Cv2.ContourArea(contour);
            if (area < 500)
                Cv2.DrawContours(tmp, new[] { contour }, 0, new Scalar(0), 1);
        }

        Cv2.BitwiseNot(tmp, mask);
        Cv2.Erode(mask, mask, kernel, iterations: 2);

        //Attempt to split contours into single objects if they are connected by a small bridge
        SplitContours(mask);
    }
}
```

Abbildung 78: Nachverarbeitung der Binärmaske in FindShapes

Am Anfang der Methode wurde die Erosion mit 3 Iterationen und die Dilation mit 5 Iterationen ausgeführt. Am Ende folgt eine erneute Erosion mit weiteren 2 Iterationen. Durch die gleiche Gesamtsumme von Iterationen pro Operation haben die Konturen am Ende die gleiche Dimension wie zuvor. Die Kombination von Verarbeitungsschritten muss immer in der Praxis erprobt und angepasst werden, der Ablauf hier hat sich als zuverlässig für verschiedene Fälle erwiesen.

6.4.2 Konturen in Maske zerschneiden

Bei der Maskenbildung passiert es häufig, dass zwei Objekte die nah beieinander liegen durch eine kleine Brücke miteinander verbunden bleiben und somit als einzelne große Kontur gefunden werden. Die Methode *SplitContours* sucht solche großen Konturen und versucht sie zu zerschneiden, sodass sie bei der Erkennung als zwei einzelne Objekte erkannt werden.

```
private void SplitContours(Mat mask)
{
    Cv2.FindContours(mask, out var contours, out _,
        RetrievalModes.External, ContourApproximationModes.ApproxSimple);

    foreach (var contour in contours)
    {
        var objectMat = new Mat(mask, Cv2.BoundingRect(contour));

        if (objectMat.Width > mask.Width * 0.8) continue;
        if (objectMat.Height > mask.Height * 0.8) continue;
        if (objectMat.Height < Settings.MinH || objectMat.Width < Settings.MinW) continue;

        CreateDepthProfiles(objectMat, out var heightProfile, out var depthProfile);
        DepthProfileConnectValleys(objectMat, heightProfile, depthProfile);
    }
}
```

Abbildung 79: Methode SplitContours von FindShapes

Der Ablauf in Abbildung 79 sucht nach allen Konturen in der Maske und filtert alle zu großen und zu kleinen Ergebnisse aus. Anschließend wird für jede verbleibende Kontur ein Höhen-

und ein Tiefenprofil erstellt. Diese geben für jeden X-Wert die Tiefe von Tälern, betrachtet von oben oder von unten an. Im nächsten Schritt wird überprüft, ob sich zwei Täler direkt übereinander und nah beieinander befinden. In dem Fall werden die Täler durch einen schwarzen Strich verbunden und somit die Kontur getrennt.

Die Methode *CreateDepthProfiles* in Abbildung 80 ist als *unsafe* deklariert. Dadurch kann hier über echte Pointer direkt im ungeschützten Arbeitsspeicher des Prozesses gearbeitet werden. Das erhöht deutlich die Geschwindigkeit des Algorithmus, gleichzeitig sind alle üblichen Schutzmechanismen der .Net-Laufzeit außer Funktion. Ein Zugriffsfehler hier führt in der Regel zu einem nicht handhabbaren Programmabsturz.

```
private unsafe void CreateDepthProfiles(Mat objectMat, out int[] heightProfile, out int[] depthProfile)
{
    depthProfile = new int[objectMat.Width];
    heightProfile = new int[objectMat.Width];
    byte* dataPtr = objectMat.DataPointer;
    int width = objectMat.Width;
    int height = objectMat.Height;
    long step = objectMat.Step();

    for (int x = 0; x < width; x++)
        for (int y = 0; y < height; y++)
            if (dataPtr[y * step + x] > 0)
            {
                depthProfile[x] = y;
                break;
            }

    for (int x = 0; x < width; x++)
        for (int y = height - 1; y >= 0; y--)
            if (dataPtr[y * step + x] > 0)
            {
                heightProfile[x] = y;
                break;
            }
}
```

Abbildung 80: Methode CreateDepthProfiles von FindShapes

Die Methode greift direkt auf den Speicherbereich des Mat-Objektes zu. Im Speicher liegen die Zeilen eines Bildes hintereinander, das heißt die Adresse eines Pixels (x,y) im Speicher berechnet sich als $\text{Startadresse} + y \cdot \text{Zeilenlänge} + x$. Die Variable *objectMat* verweist intern aber auf einen Ausschnitt des originalen Maskenbildes. Somit kann die Zeilenlänge von *objectMat* nicht als Offset für die Pixelbestimmung verwendet werden. Hierfür stellt OpenCV mit *Step* das korrekte Offset für den zeilenweisen Speicherzugriff bereit.

Neben der Schrittweite werden hier auch für Breite und Höhe eigene lokale Variablen angelegt. Tatsächlich ist der Zugriff auf diese lokalen Variablen schneller als der Zugriff auf die Eigenschaften von *objectMat*. Die lokalen Variablen werden direkt aus dem Speicher im aktuellen Kontext ausgelesen, bei jedem Zugriff auf die Eigenschaften von *objectMat* müssen hingegen mehrere Abstraktionsschichten durch die .Net Laufzeit und den darunterliegenden nativen Code aufgelöst werden. Bei einer großen Anzahl von Zugriffen ergibt sich dadurch ein spürbarer Performanceunterschied.

Der Algorithmus läuft über die gesamte Breite der Kontur für jede X-Position einmal von oben und von unten hin zur Mitte, bis das erste weiße Pixel gefunden wurde und speichert diese Y-Position in dem Höhen- beziehungsweise Tiefenprofil. Somit werden für ein Bild viele tausende einzelne Speicherzugriffe ausgeführt. Deshalb addieren sich alle kleinen Optimierungen schnell zu einer deutlichen Performanceverbesserung auf. Ohne die Optimierungen durch direkte und effiziente Speicherzugriffe wäre die Implementierung eines solchen Mechanismus nicht praktikabel möglich.

Die Methode *DepthProfileConnectValleys* durchläuft anschließend Höhen- und Tiefenprofil und ermittelt Positionen, an denen Höhe und Tiefe sehr nah beieinanderliegen. An diesen Positionen wird ein vertikaler schwarzer Strich in der Binärmaske eingezeichnet. Abbildung 81 zeigt den Effekt von *SplitContours* an einem Beispielbild für eine Farbmaske in *FindShapes*.

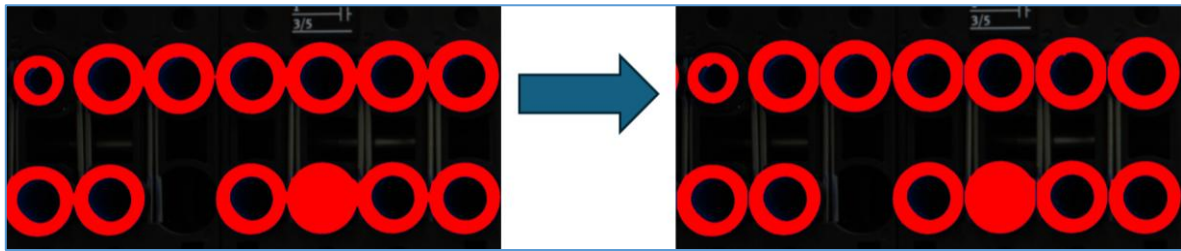


Abbildung 81: Effekt von *SplitContours* auf die Binärmaske in *FindShapes*

Im linken Bild sind die meisten Kreise durch einen winzigen Bereich miteinander verbunden, dadurch würde *FindShapes* hier nur vier Objekte erkennen. Im rechten Bild wurden die Objekte erfolgreich aufgeteilt, wodurch alle Kreise korrekt erkannt und klassifiziert werden können.

6.4.3 Erkennung in Maske

Nachdem nun eine saubere Binärmaske erstellt ist, wird in dieser nach Objekten gesucht. Dazu zeigt Abbildung 82 den Ablauf in *FindCircles*, der Methode zum Ermitteln von kreisförmigen Objekten. Der Ablauf für Rechtecke ist weitestgehend äquivalent, logischerweise bis auf die kreisspezifischen Berechnungen. Über *FindContours* werden hier nun alle potenziellen Objekte in der Maske gesucht und anschließend anhand der Erkennungseinstellungen gefiltert. Übrig bleiben die Objekte, die den Einstellungen entsprechen. Für jedes dieser Objekte wird ein *FindShapesResult* mit Position und Dimensionen des gefundenen Objekts erzeugt. Die Liste der Ergebnisse wird am Ende zurückgegeben.

```
private List<FindShapesResult> FindCircles(Mat mask)
{
    var results = new List<FindShapesResult>();

    Cv2.FindContours(mask, out var contours, out _,
        RetrievalModes.List, ContourApproximationModes.ApproxSimple);

    foreach (var contour in contours)
    {
        Cv2.MinEnclosingCircle(contour, out var center, out var radius);
        if (!CircleCheckDimensions(contour, radius))
            continue;

        var region = mask.SubMat(contour.BoundingRect());
        var minArea = Math.PI * radius * radius;

        if (Settings.FillMode == FillModes.Contour)
        {
            var innerRadius = radius - Settings.Thickness * 2;
            if (!CircleCheckContour(region, radius, innerRadius))
                continue;
            minArea -= Math.PI * innerRadius * innerRadius;
        }

        if (region.CountNonZero() < minArea * Settings.FillRatio / 100.0)
            continue;

        results.Add(new FindShapesResult
        {
            X = center.X + RoiX,
            Y = center.Y + RoiY,
            Height = radius * 2,
            Width = radius * 2,
            ShapeType = ShapeType.Circle
        });
    }

    return results;
}
```

Abbildung 82: Methode *FindCircles* von *FindShapes*

Im ersten Schritt wird der kleinste Kreis ermittelt, der die Kontur komplett einschließt. *CircleCheckDimensions* vergleicht den Radius des Kreises mit den Einstellungen und stellt eine annähernd kreisförmige Form sicher, indem Flächen/Radius- und Umfangs/Radius-Verhältnis betrachtet werden. Zu große, zu kleine und nicht ausreichend kreisförmige Objekte werden verworfen.

Als nächstes wird der Füllgrad des Objektes überprüft. Dafür wird die Anzahl weißer Pixel im Objekt aus der Maske mit der Fläche des idealen Kreises verglichen. Dabei muss aber auch der *FillMode* mit einbezogen werden, denn die Fläche eines Rings mit einer Öffnung in der Mitte kann nicht sinnvoll mit der Fläche eines gefüllten Kreises verglichen werden. Bei aktivem Konturmodus wird deshalb die Fläche des inneren Kreises von der Gesamtsollfläche abgezogen. *CircleCheckContour* stellt zusätzlich sicher, dass der Bildbereich im inneren Kreis nicht oder nur leicht gefüllt ist und es sich somit tatsächlich um einen Ring handelt. Durch den niedrigen Füllgrad in der Mitte wird garantiert, dass die reduzierte Pixelanzahl sich auf den äußeren Bereich der Kontur konzentriert. Somit kann der Füllgradparameter aus den Einstellungen akkurat angewandt werden.

Bei der Generierung der Ergebnisse wird auf die X- und Y-Koordinaten noch die Verschiebung durch die ROI angewandt, wodurch die Koordinaten eines Ergebnisses wieder zum Bezugssystem des Originalbildes passen.

6.4.4 Dynamische Intensitätsgrenzwertermittlung

Zuletzt wird noch der Mechanismus für den Automatikmodus zur Intensitätsmaskenbildung erläutert. Der Ablauf führt Maskenbildung und Erkennung für mehrere Intensitätswerte im Bereich bis zum definierten Limit aus und wählt den Wert mit den meisten gefundenen Ergebnissen.

```
private List<Shape> RunDetectionVariableIntensity()
{
    var foundRecord = new ConcurrentBag<(int intensity, List<Shape> shapes, Mat mask)>();
    var steps = Generate1stPassSteps();

    Parallel.ForEach(steps, i =>
    {
        var mask = new Mat();
        var found = RunDetection(mask, i, false);
        foundRecord.Add((i, found, mask));
    });

    steps = Generate2ndPassSteps(foundRecord);

    Parallel.ForEach(steps, i =>
    {
        var mask = new Mat();
        var found = RunDetection(mask, i, false);
        foundRecord.Add((i, found, mask));
    });

    numFoundMax = foundRecord.Max(r => r.shapes.Count);
    foundMax = foundRecord.First(r => r.shapes.Count == numFoundMax);

    UpdateMaskImage(foundMax.mask);

    foreach (var fr in foundRecord)
        fr.mask.Dispose();

    return foundMax.shapes;
}
```

Abbildung 83: Ablauf der dynamischen Intensitätsgrenzwertermittlung in FindShapes

Dafür wird zuerst ein Durchlauf mit größeren Schritten über den gesamten zulässigen Bereich gemacht. In einem kleinen Bereich um den Intensitätswert mit den meisten Ergebnissen wird anschließend ein zweiter durchlauf für jeden einzelnen Intensitätswert ausgeführt.

Die Erkennungen eines Durchlaufs werden parallel in mehreren Threads ausgeführt, um die Laufzeit nicht übermäßig zu verlangsamen. Die Skalierung ist begrenzt durch die Anzahl von Prozessorkernen und dem verfügbaren Arbeitsspeicher, deshalb wird die Anzahl der parallel ausgeführten Erkennungen durch die groben Schritte limitiert.

Die generierten Masken aller Erkennungen wurden gespeichert, damit zum Schluss über *UpdateMaskImage* das Maskenbild für die Vorschau generiert werden kann. Daraufhin werden alle Masken wieder freigegeben und das ermittelte Ergebnis wird zurückgegeben.

6.5 Template Matching

Der Aufbau und der Grundablauf der TemplateMatching Erkennungsmethode sind sehr ähnlich zu FindShapes. Es wird ein Bild eingelesen, eine Vorverarbeitung ausgeführt, auf dem vorverarbeiteten Bild eine Erkennung durchgeführt und es werden Ergebnisse und annotierte Ergebnisbilder zurückgeliefert. Zusätzlich zu den Einstellungen benötigt TemplateMatching noch ein Template, dieses wird zum Start über den Dateipfad in der Settings-Struktur aus einer Datei geladen.

6.5.1 Vorverarbeitung

Durch die Vorverarbeitung von TemplateMatching werden unwesentliche Details wie Flächen-texturen, Rauschen und Helligkeitsunterschiede ausgeblendet. Es sollen nur die markanten Formen und Kanten von Vorlage und Bild verglichen werden.

```
private void Preprocess(Mat image, Mat resultImage)
{
    using (var edges = new Mat())
    {
        Cv2.CvtColor(image, resultImage, ColorConversionCodes.BGR2GRAY);
        Cv2.GaussianBlur(resultImage, resultImage, new Size(5, 5), 0);
        Cv2.Laplacian(resultImage, edges, MatType.CV_16S, 3);
        Cv2.ConvertScaleAbs(edges, resultImage);
    }
}
```

Abbildung 84: Vorverarbeitungsmethode von TemplateMatching

Die Vorverarbeitung hier ist deutlich einfacher als bei FindShapes. Das Farbbild wird in ein Graustufenbild konvertiert, anschließend werden ein Unschärfefilter und ein Laplace-Filter genutzt, um Rauschen zu reduzieren und die Kanten im Bild herauszustellen. Der Laplace-Filter produziert sowohl positive als auch negative Werte, deshalb muss über *ConvertScaleAbs* das das Ergebnisbild wieder in einen rein positiven Wertebereich überführt werden.

6.5.2 Template Matching

Die Methode *MatchTemplate* in Abbildung 85 nimmt das vorverarbeitete Bild und Template entgegen und gibt eine Liste gefundener Ergebnisse zurück. Die Methode *MatchTemplate* von OpenCV nimmt die zwei Bilder als Parameter und gibt ein Ergebnisbild zurück, in dem jedes Pixel den Übereinstimmungsgrad an dieser Position enthält.

Die TemplateMatching Erkennungsmethode unterstützt die Erkennung von rotierten Ergebnissen. Der Ablauf hier wird mehrfach in parallelen Threads ausgeführt, um alle Rotationswinkel im erlaubten Bereich abzudecken. Deshalb wird das Template direkt am Anfang des Ablaufs über die Methode *RotateImage* gedreht. Bei der Rotation vergrößern sich zwangsweise die Dimensionen des Templates, der Bereich um das Originalbild wird dabei schwarz aufgefüllt. Dieser schwarze Bereich würde das Ergebnis verfälschen, deshalb wird zusätzlich ein weißes Bild mit den Dimensionen des Templates um den gleichen Wert rotiert und bei *MatchTemplate* als

Maske mit übergeben. So wird nur der Bereich des eigentlichen Templates für den Vergleich genutzt.

```
private List<OrientedBoundingBox> MatchTemplate(Mat image, Mat template, int rot, double minScore)
{
    using (var rotatedTemplate = RotateImage(template, rot))
    using (var templateOnes = Mat.Ones(template.Size(), MatType.CV_8UC1))
    using (var templateMask = RotateImage(templateOnes, rot))
    using (var result = new Mat())
    using (var mask = new Mat())
    using (var dilated = new Mat())
    using (var resultPoints = new Mat<Point>())
    {
        Cv2.MatchTemplate(image, rotatedTemplate, result, TemplateMatchModes.CCNormed, templateMask);
        Cv2.InRange(result, 0, minScore, mask);
        result.SetTo(0, mask);
        mask.SetTo(new Scalar(0));

        Cv2.Dilate(result, dilated, new Mat());
        Cv2.Compare(result, dilated, mask, CmpType.NE);
        result.SetTo(new Scalar(0), mask);

        Cv2.FindNonZero(result, resultPoints);
        var obbs = resultPoints.ToArray()
            .Select(p => new OrientedBoundingBox(p, template.Size(), rot, result.At<float>(p.Y, p.X)));

        var results = new List<OrientedBoundingBox>();
        foreach (var obb in obbs)
            obb.InsertAndRemoveIntersecting(results);

        return results;
    }
}
```

Abbildung 85: Methode MatchTemplate von TemplateMatching

Im Ergebnis von MatchTemplate werden zunächst alle Punkte auf 0 gesetzt, die unter dem Mindestübereinstimmungsgrad liegen. Danach gibt es im Bereich rund um die besten Ergebnisse aber immer noch unzählige Punkte, an denen die Übereinstimmung über dieser Schwelle liegt. Für einen Bildbereich kann es nur ein Ergebnis geben, deshalb muss der Punkt mit der höchsten Übereinstimmung ermittelt werden.

Zum Ermitteln aller lokalen Maxima im Ergebnisbild wird die Dilate-Funktion angewandt, dadurch dehnen sich die hellen Bildbereiche rund um ein lokales Maximum aus. Das Ergebnis wird mit dem Originalbild verglichen. Die Punkte, die ihren Wert nicht verändert haben, sind lokale Maxima, alle anderen Punkte werden somit ebenfalls auf 0 gesetzt. Übrig bleiben nur die lokalen Maxima. Abbildung 86 zeigt links ein Ergebnisbild vor und rechts nach der Ermittlung der lokalen Maxima. Es ist zu erkennen, dass rechts nur noch einzelne Punkte übrigbleiben, wo es vorher Punktwolken von Ergebnissen gab.

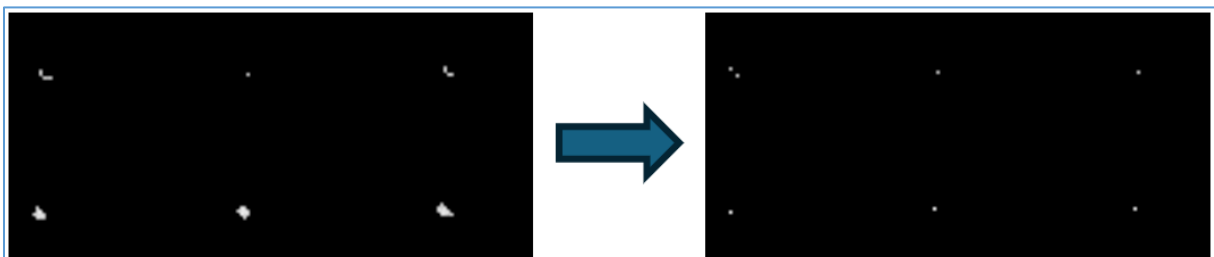


Abbildung 86: Vergleich Template Matching Ergebnisse vor und nach Ermittlung lokaler Maxima

Für die verbleibenden Punkte wird jeweils ein Ergebnis generiert. Diese können sich auch jetzt noch überlappen, um das aufzulösen wurde die Hilfsklasse *OrientedBoundingBox* (OBB) erstellt. Eine Instanz von OBB repräsentiert ein Ergebnis an einer Position mit einer gegebenen Dimension und einem Übereinstimmungsgrad. Zwei OBBs können auf Überschneidung geprüft werden. Damit kann ein OBB-Objekt mit einer Liste weiterer OBBs verglichen werden,

wobei alle Überschneidungen festgestellt und nur die OBB mit dem höchsten Übereinstimmungsgrad beibehalten wird. So werden alle OBBs zu einer Liste hinzugefügt, die am Ende nur die überschneidungsfreien Ergebnisse mit der besten Übereinstimmung enthält. Diese Liste wird zur weiteren Verarbeitung zurückgegeben.

Die OBB unterstützt auch unterschiedlich rotierte Ergebnisse. So wird im darüberliegenden Ablauf auch die Überschneidung der Ergebnisse aus den Durchläufen zu verschiedenen Rotationswinkeln bestimmt. Auch hier werden nur die überschneidungsfreien Ergebnisse mit der besten Übereinstimmung beibehalten. Daraus wird die finale Liste von Erkennungsergebnissen generiert.

6.6 DmtxDetection

Die DataMatrix-Code Erkennung nutzt eine Vorverarbeitung, um Bereiche zu extrahieren, in denen wahrscheinlich ein Dmtx-Code enthalten ist. Ein solcher Code ist rechteckig und zu einem signifikanten Teil schwarz gefüllt. Deshalb ist das Ziel, diese teilweise schwarzen Rechtecke als abgetrennte Bildbereiche herauszustellen, sodass diese über einen FindContours-Schritt gefunden werden.

Dafür wird das Bild mit der OpenCV Methode *AdaptiveThreshold* binarisiert. Die Funktion betrachtet die direkte Umgebung eines jeden Pixels, vergleicht, ob das Pixel deutlich heller oder dunkler als das Umfeld ist und setzt im Ergebnisbild dann entweder ein weißes oder ein schwarzes Pixel. Die Dmtx-Codes werden dadurch im Ergebnisbild bereits als recht gut erkennbare weiße Bereiche herausgestellt.

```
private void Preprocess()
{
    using (var kernel = Cv2.GetStructuringElement(MorphShapes.Rect, new Size(3, 3)))
    {
        Cv2.GaussianBlur(_gray, _mask, new Size(9, 9), 0);
        Cv2.AdaptiveThreshold(_mask, _mask, 255, AdaptiveThresholdTypes.MeanC,
            ThresholdTypes.BinaryInv, 7, 2);
        Cv2.Erode(_mask, _mask, kernel, iterations: 2);
        Cv2.Dilate(_mask, _mask, kernel, iterations: 1);
    }
}
```

Abbildung 87: Vorverarbeitung für DataMatrix-Code Erkennung

Da die einzelnen Felder in einem Dmtx-Code nicht alle miteinander verbunden sind, bestehen diese Bereiche aber oft noch aus mehreren Teilen. Dadurch würde FindContours mehrere Teilbereiche des Codes extrahieren und somit kein zufriedenstellendes Ergebnis erzielen. Zusätzlich verläuft der Bereich des Codes oft noch mit dem Hintergrund, wodurch der eigentliche Bildbereich des Codes überhaupt nicht zuverlässig gefunden wird.

Durch die Kombination von einem Erosions- und einem Dilationsschritt wird der Code zum einen deutlich besser von der Umgebung getrennt, zusätzlich wachsen dadurch die Felder im Code so weit zusammen, dass diese ineinander überlaufen und somit der Code als eine einzelne Kontur erkannt wird.

Nach der Vorverarbeitung werden die Codebereiche über einen FindContours-Schritt ermittelt. Diese Bildbereiche werden extrahiert und einzeln an eine Bibliothek zur DataMatrix-Dekodierung übergeben. Das Vorgehen kann gleichermaßen mit LibDmtx oder der Dynamsoft Barcode Reader Library angewandt werden, aktuell wird nur noch die Dynamsoft Bibliothek genutzt.

Auch in DmtxDetection werden alle Codes in parallelen Threads dekodiert. Für alle erfolgreich dekodierten Bildbereiche wird ein Ergebnis mit Position, Größe und Inhalt des Dmtx-Codes generiert und die Liste der Ergebnisse zurückgegeben.

6.7 Preview-Dialoge

Dieser Abschnitt zeigt die Implementierung der Preview-Dialoge, mit Fokus auf PreviewBaseControl und die Interaktion mit den Dialogfenstern.

6.7.1 Definition GUI-Elemente

Abbildung 88 zeigt Teil der GUI-Definition von PreviewBaseControl in XAML.

Das wichtigste Element ist das Bildanzeigeelement *image*. Es füllt den Hauptteil der Oberfläche aus. PreviewBaseControl enthält auch die Buttons zum Schließen des Dialogs und weitere Elemente in der Fußzeile. PreviewBaseControl füllt immer den gesamten Raum des Dialogs aus, die weiteren Elemente des Dialogfensters liegen als über PreviewBaseControl und nicht daneben. Somit muss das Anzeigebild genügend Platz lassen, dass die weiteren dialogspezifischen Einstellungselemente angezeigt werden können. Deshalb müssen Breite und Höhe von *image* unabhängig von der Größe des gesamten Controls steuerbar sein.

```
<UserControl x:Class="Festo.TEaM.Vision.Controls.PreviewBaseControl" MinHeight="600" MinWidth="800"
    HorizontalAlignment="Stretch" VerticalAlignment="Stretch" MouseUp="Window_LeftMouseUp">
    <Grid d:Background="White">
        <Border x:Name="ImageBorder" BorderBrush="Black" BorderThickness="2" CornerRadius="0" Padding="0"
            Margin="{Binding ImageBorderMargin, RelativeSource={RelativeSource AncestorType=UserControl}},
            FallbackValue='5,5,5,30'"/>
            <Image Name="image" Width="Auto" Height="Auto" MouseLeftButtonDown="Image_LeftMouseDown"
                MouseMove="Image_MouseMove" />
        </Border>
    ...
    <StackPanel Orientation="Horizontal" HorizontalAlignment="Right" VerticalAlignment="Bottom">
        <Button x:Name="ButtonOk" Content="Ok" Width="70" Margin="0,0,10,0" Click="ButtonOk_Click"/>
        <Button x:Name="ButtonCancel" Content="Cancel" Width="70" Click="ButtonCancel_Click"/>
    </StackPanel>
    </Grid>
</UserControl>
```

Abbildung 88: XAML-Definition der PreviewBaseControl Oberfläche

Dafür ist das Image-Element in einer Border eingefasst, deren Margin dynamisch über eine Eigenschaft des Controls gesteuert wird. Standardmäßig füllt das Bild somit den gesamten verfügbaren Bereich aus und optional kann der einbindende Dialog über die Eigenschaft *ImageBorderMargin* das Bild verkleinern. Die Buttons und alle weiteren Elemente von PreviewBaseControl bleiben dabei unverändert an derselben Stelle.

Abbildung 89 zeigt Teil der XAML-Definition von *FindShapesPreview*. Hier ist gut zu sehen, wie PreviewBaseControl als Element mit dem Namen *pBase* eingebunden wird. Die ImageBorderMargin wird dabei so gesetzt, dass genug Platz für die weiteren Elemente bleibt.

```
<Window x:Class="Festo.TEaM.Vision.Dialogs.FindShapesPreview" Height="714" Width="1000"
    MinHeight="714" MinWidth="1000" Title="Find Shapes Preview">
    <Grid>
        <localcontrols:PreviewBaseControl x:Name="pBase" ImageBorderMargin="5,25,200,30" Margin="0"/>
        <StackPanel Orientation="Horizontal" VerticalAlignment="Top" HorizontalAlignment="Center"
            Margin="5,5,200,5">
            <RadioButton x:Name="btnUpdateImage" Checked="OnBtnUpdateImage" GroupName="ImagePreviewMode"
                Content="Show Raw Image" IsChecked="True"/>
            <RadioButton x:Name="btnUpdateMask" Checked="OnBtnUpdateMask" GroupName="ImagePreviewMode"
                Content="Show Preprocessing Mask"/>
            <RadioButton x:Name="btnShapes" Checked="OnBtnShapes" GroupName="ImagePreviewMode"
                Content="Show Found Shapes"/>
        </StackPanel>
        <StackPanel Orientation="Vertical" Width="190" Margin="0,10,5,0" VerticalAlignment="Top"
            HorizontalAlignment="Right">
            <localcontrols:RoiSettingsControl x:Name="RoiSettings" />
        </StackPanel>
    </Grid>
</Window>
```

Abbildung 89: XAML-Definition der GUI von FindShapesPreview

Am oberen Rand des Dialogs werden die Radio Buttons zur Auswahl des Vorschaubildes angezeigt, deshalb lässt pBase nach oben 25 Pixel Platz, statt standardmäßig 5. Am seitlichen Rand befindet sich der Bereich für die Einstellungen. Dieser hat eine Breite von 190 Pixeln, hierfür lässt pBase 200 Pixel Platz. Hier ist auch zu sehen, wie RoiSettingsControl unter dem Namen *RoiSettings* eingebunden und in dem Einstellungsbereich eingeordnet wird.

Abbildung 90 zeigt die vollständige XAML-Definition von *ShowImageWindow*. Hier wird deutlich, wie effizient neue Vorschaudialoge mit *PreviewBaseControl* erstellt werden können.

```
<Window x:Class="Festo.TEaM.Vision.Dialogs.ShowImageWindow" Height="714" Width="1000" MinHeight="714"
        MinWidth="1000">
    <Grid>
        <localcontrols:PreviewBase x:Name="pBase" ImageBorderMargin="5,5,5,30" Margin="0,0,0,0"/>
    </Grid>
</Window>
```

Abbildung 90: XAML-Definition der GUI von *ShowImageWindow*

Ein einziges GUI-Element liefert das Vorschaubild, die Buttons zum Schließen, Informationen zum Pixel unter dem Cursor, Abspeichern des angezeigten Bildes und Anzeigen einer Textmeldung an den Anwender. Die Definition weiterer GUI-Elemente ist hier nicht notwendig.

6.7.2 Dynamische GUI-Elemente

ConfigureCameraLivePreview besitzt variable Eingabefelder für die Parameter der verschiedenen Kameravarianten. Diese Parameterfelder können nicht statisch in XAML definiert werden, deshalb werden sie beim Öffnen des Dialogs dynamisch im C#-Code generiert.

```
private void AddParameterControls()
{
    foreach (var parameter in _camera.Settings.Parameters)
    {
        var index = SettingsStackPanel.Children.IndexOf(SeparatorAfterParameters);
        var sp = new StackPanel() { Orientation = Orientation.Horizontal };
        sp.Children.Add(new TextBlock() { Width = 130, Text = parameter.Name });

        if (parameter.Type == CameraParameterTypes.Bool)
        {
            var cb = new CheckBox()
            {
                Width = 60,
                IsChecked = parameter.Value == "True",
                Name = $"ParamCheckBox_{parameter.Name}"
            };
            cb.Click += ParameterCheckBox_Checked;
            sp.Children.Add(cb);
        }
        else if (parameter.Type == CameraParameterTypes.Double)
        {
            var tb = new TextBox()
            {
                Width = 60,
                Text = parameter.Value,
                Name = $"ParamTextBox_{parameter.Name}"
            };
            tb.PreviewTextInput += ParameterTextBox_PreviewInput;
            tb.KeyDown += ParameterTextBox_EnterKeyDown;
            sp.Children.Add(tb);
        }

        SettingsStackPanel.Children.Insert(index, sp);
    }
}
```

Abbildung 91: Methode *AddParameterControls* von *ConfigureCameraLivePreview*

Abbildung 91 zeigt die Methode zum Hinzufügen der dynamischen Parameterfelder. In der statischen GUI-Definition ist nur ein leeres Element *SettingsStackPanel* definiert, in das für jeden Kameraparameter ein GUI-Element eingefügt wird. Es sind verschiedene Kontrollelemente für die unterstützten Datentypen definiert, eine *CheckBox* für Booleans und eine *TextBox* für Zahlenwerte. Die Elemente werden mit den aktuellen Werten der Parameter initialisiert.

Die GUI-Elemente sind auf eine Eventhandler-Methode registriert, die bei Eingabe eines Wertes aufgerufen wird. Dabei wird eine Referenz auf das GUI-Element mit übergeben. Der Name eines Elementes enthält den Namen des jeweiligen Parameters. So erkennt die Methode, zu welchem Parameter das Element gehört. Die Methode liest den neuen Wert aus und übergibt ihn via `SetParameter` an die Kamera.

6.7.3 Bildanzeige

Zur Anzeige eines Vorschaubildes müssen die Bilder der Erkennungsmethode in das Bildanzeigeelement von `PreviewBaseControl` übertragen werden. Das Image-Element benutzt ein *WriteableBitmap*¹⁷ als Datenquelle. Das Bitmap wird im Konstruktor von `PreviewBaseControl` und bei Größenveränderung des Bildes über die Methode *SetNewBitmap* initialisiert. Bitmap und Mat haben unterschiedliche Speicherstrukturen und Pixelformate. Die Methode *UpdateImageBuffers* konvertiert die OpenCV-Bilder zur Anzeige in das korrekte Format.

```
public PreviewBaseControl()
{
    InitializeComponent();
    SetNewBitmap(1000, 1000);
    image.Source = _bitmap;
}

private void SetNewBitmap(int w, int h)
{
    _bitmap = new WriteableBitmap(w, h, 96, 96, PixelFormats.Bgr32, null);
    _imgWidth = w;
    _imgHeight = h;
}

public void UpdateImageBuffers(List<Mat> mats)
{
    lock (_bufLock)
    {
        if (mats.Any(m => m.Width != mats[0].Width || m.Height != mats[0].Height))
            throw new Exception("Not all mats have the same dimension");

        if (mats[0].Width != _imgWidth || mats[0].Height != _imgHeight)
        {
            DispatcherInvokeWithTimeout(() => SetNewBitmap(mats[0].Width, mats[0].Height));

            for (int i = 0; i < _imageBuffers.Count; i++)
            {
                _imageBuffers[i] = null;
            }
        }

        Parallel.ForEach(mats, mat =>
        {
            if (_imageBuffers[mats.IndexOf(mat)] == null)
                _imageBuffers[mats.IndexOf(mat)] = new int[mat.Width * mat.Height];

            ConvertMatData(mat, _imageBuffers[mats.IndexOf(mat)]);
        });
    }
}
```

Abbildung 92: Initialisierung `PreviewBaseControl` mit leerem `WriteableBitmap`

`PreviewBaseControl` besitzt einen Zwischenspeicher *_imageBuffer*, der eine konvertierte Kopie jedes Anzegebildes enthält. Der *_imageBuffer* ist eine Liste von Integer-Arrays mit einem Eintrag pro verfügbarem Vorschaubild. Ein Integer-Wert repräsentiert ein Pixel. Ein Eintrag kann bei Bedarf sehr schnell in das Bitmap kopiert werden.

Der Buffer wird aktualisiert, wenn sich das Quellbild ändert. Das Konvertieren passiert außerhalb des GUI-Threads, so wird die Oberfläche nicht unnötig durch das Konvertieren blockiert. Zusätzlich muss nicht bei jedem Umschalten des Anzegebildes neu konvertiert werden. Das verbessert die Performance der Oberfläche und entkoppelt die Quellbilder von der Bildanzeige.

¹⁷ Die Implementierung *WriteableBitmap* erlaubt es, den Speicher hinter einem Bild direkt zu beschreiben, was eine speichereffiziente und schnelle Aktualisierung des Livebildes erlaubt.

Der Ablauf findet in einem Lock-Block statt. Das Lock-Objekt *_bufLock* wird bei jedem kritischen Zugriff auf die Bildpuffer überprüft, um sicherzustellen, dass nicht zwei konkurrierende Zugriffe gleichzeitig auf dem Puffer stattfinden.

Alle Bilder werden gleichzeitig in parallelen Threads konvertiert, um die Ausführung noch einmal zu beschleunigen. Die Methode *ConvertMatData* setzt ein einzelnes Mat-Bild in das korrekte Format um. Die Methode ist als *unsafe* deklariert und kann somit direkt auf den rohen Speicher hinter beiden Datenstrukturen zugreifen.

```
internal static unsafe void ConvertMatData(Mat mat, int[] intArray)
{
    if (mat == null || mat.IsDisposed)
        return;

    int length = intArray.Length * 3;

    fixed (int* pIntArray = intArray)
    {
        byte* pMatData = (byte*)mat.Data;
        int* pInt = pIntArray;

        for (int i = 0; i < length; i += 3)
        {
            *pInt++ = (pMatData[i + 2] << 16) | (pMatData[i + 1] << 8) | pMatData[i];
        }
    }
}
```

Abbildung 93: Funktion *ConvertMatData* von *PreviewBaseControl*

Im Bitmap hat jedes Pixel eine Größe von 4 Byte und wird somit durch einen 32-Bit Integer repräsentiert. Mat nutzt 3 Byte pro Pixel. Folglich wird der Integer-Zeiger *pInt* für jedes Pixel um ein Element, also um 4 Byte erhöht, der Zähler *i* für den Speicher hinter *mat* wird für jedes Pixel um 3 Byte erhöht. Ein Pixel setzt sich zusammen aus den drei Bytewerten für den Rot-Grün- und Blaukanal des Bildes, die um 16, 8 oder 0 Bit verschoben und anschließend per bitweisem oder zusammengefügt werden.

Der Ablauf ist sehr gut optimiert und benötigt für die Konvertierung eines 20 Megabyte großen Bildes nur etwa eine Millisekunde.

Die Methode *DrawImageInternal* stellt ein Bild aus dem Bildpuffer auf der Oberfläche dar. Mit einem weiteren Lock-Block auf *_bufLock* wird verhindert, dass der Bildpuffer während dem Kopieren neu beschrieben wird. Das würde zu Anzeigefehlern führen, weil Teile vom neuen und vom alten Bild in dasselbe Ziel kopiert werden (Tearing). In ungünstigen Fällen kann es auch zu einem Zugriffsfehler kommen, zum Beispiel wenn die Bildgröße sich geändert hat. Ein Programmabsturz wäre die Folge.

```
private void DrawImageInternal()
{
    lock (_bufLock)
    {
        var imgBuf = GetBuf();

        if (_bitmap != image.Source)
            image.Source = _bitmap;

        _bitmap.Lock();
        Marshal.Copy(imgBuf, 0, _bitmap.BackBuffer, imgBuf.Length);
        _bitmap.AddDirtyRect(new Int32Rect(0, 0, _bitmap.PixelWidth, _bitmap.PixelHeight));
        _bitmap.Unlock();
    }
}
```

Abbildung 94: Methode *DrawImageInternal* von *PreviewBaseControl*

Die Methode *GetBuf* gibt den Puffereintrag für das ausgewählte Anzeigebild zurück. Der Inhalt wird mit *Marshal.Copy*¹⁸ in den Speicher des Bitmaps geschrieben.

Lock sperrt den unsicheren Datenpuffer des Bitmaps, als Vorbereitung auf das Schreiben. *AddDirtyRect* teilt dem Bitmap mit, welcher Bildbereich aktualisiert wurde und somit neu gezeichnet werden muss. *Unlock* entsperrt das Bitmap und stößt gleichzeitig die Aktualisierung des Bildes in der Oberfläche aus dem Puffer des Bitmaps an. [Microsoft, 2026]

Abbildung 95 zeigt, wie der Live-Preview Dialog den Image Buffer zur Bildanzeige verwendet. Im Konstruktor ruft der Dialog die Methode *Initialize* von *PreviewBaseControl* auf und übergibt die benötigte Anzahl an Puffereinträgen für die angebotenen Vorschaubilder. Hier werden auch *RoiSettingsControl* initialisiert und die Eventhandler für ROI- und Pixelauswahl registriert. Mit dem Aufruf von *StartUpdateTask* wird die Methode *UpdateTask* ab jetzt zyklisch ausgeführt, die Live-Updates der Oberfläche sind jetzt aktiv.

```
public FindShapesPreview(FindShapes findShapes, Camera cam, ImageHandle imageHandle)
{
    InitializeComponent();
    _findShapes = findShapes;
    _camera = cam;
    _imageHandle = imageHandle;
    pBase.Initialize(this, 3);
    RoiSettings.Initialize(pBase, GetRoiSettings());
    pBase.OnRoiUpdated += _findShapes.UpdateRoi;
    pBase.OnPixelSelected += OnPixelSelected;
    pBase.StartUpdateTask(UpdateTask);
}

private void OnBtnUpdateImage(object sender, RoutedEventArgs e)
{
    pBase.SetActiveImageBuffer(0);
    pBase.DrawImage();
}

private void UpdateTask()
{
    _camera.GetImage(_imageHandle);
    _findShapes.Run();

    pBase.UpdateImageBuffers(new List<Mat>
    {
        _findShapes.Image,
        _findShapes.MaskImage,
        _findShapes.AnnotatedImage
    });

    pBase.DrawImage();
}
```

Abbildung 95: Konstruktor von FindShapesPreview

Der Update Task einer Erkennungsmethode ruft ein Kamerabild ab, führt die Erkennung aus und übergibt die Ergebnisbilder in *UpdateImageBuffers* an *PreviewBaseControl*. *DrawImage* aktualisiert das angezeigte Bild in der Oberfläche mit dem ausgewählten Eintrag des Bildpuffers.

Zur Auswahl des Vorschaubildes ruft der Dialog *SetActiveImageBuffer* mit dem gewünschten Index auf, wie hier beispielhaft in *OnBtnUpdateImage* zu sehen. Die Methode ist mit dem entsprechenden Kontrollelement in der Oberfläche verknüpft. *DrawImage* aktualisiert das Anzeigebild aus dem Puffer, ohne dass der Puffer aktualisiert werden muss.

¹⁸ Die Funktion *Marshal.Copy* erlaubt das Kopieren von verwaltetem Speicher, also dem Integer-Array, über eine Speicheradresse in unverwalteten Speicher, also dem Puffer des Bitmaps, ohne einen Unsafe-Block. Die Methode selber ist weiterhin unsafe, kann also trotzdem zu einem Programmabsturz führen. [Microsoft, 2026]

6.8 Zusammenfassung

In diesem Kapitel wurden Kernaspekte für die Implementierung der TEaMVision Bibliothek vorgestellt.

Die Klasse TEaMVision dient als zentraler Einstiegspunkt für alle externen Aufrufe. Hier werden alle externen Abhängigkeiten geladen und korrekte Parametrierung und Fehlerhandling sichergestellt.

Anhand der Klasse ImageHandle wurde die Wichtigkeit der klaren Zuständigkeit für die Verwaltung von Speicher eingeführt und die konkrete Umsetzung des Multiton-Patterns gezeigt.

Die abstrakte Klasse Camera erweitert das Multiton-Pattern um einen Factory-Mechanismus zur Abbildung von verschiedenen Kameravarianten. Hier wurde ersichtlich, wie leichtgewichtig weitere Varianten in Zukunft ergänzbar sind. Eine neue Kamera erbt automatisch die gesamte Funktionalität der Basisklasse und muss lediglich in einer Methode ein Kamerabild bereitstellen. Zusätzlich können über die variablen Kameraparameter diverse Spezialfunktionen effizient abgebildet werden.

Anhand der Erkennungsklasse FindShapes wurden Aufbau und Grundablauf aller Erkennungsmethoden gezeigt. Für die Klassen FindShapes, TemplateMatching und DmtxDetection wurde erläutert, wie mit den Grundmechanismen der Bildverarbeitung das gewünschte Ergebnis erzielt werden konnte.

Für PreviewBaseControl und die Dialoge wurde das Zusammenspiel zur Integration, Bildanzeige und Ausführung von Update Tasks gezeigt.

Die Speicherverwaltung ist bei der Verarbeitung von vielen und großen Bildern ein Thema, das sich durch die gesamte Implementierung zieht. Komponenten besitzen eigene, mehrfach wiederverwendete Bildinstanzen, auf denen sie ihre Operationen ausführen.

Die Übergabe von Bildern zwischen Komponenten geschieht über dedizierte Felder, für die die Zuständigkeit der Initialisierung und Freigabe klar definiert ist. Dadurch werden Zugriffsfehler, Speicherlecks und Anzeigefehler vermieden, was die Stabilität auch bei längeren Laufzeiten gewährleistet.

Die Performanceoptimierung ist ein weiteres Kernthema. Mit Durchlaufzeiten von mehreren Sekunden für Bildaufnahme, Erkennungsmethode, Ergebniserzeugung und Bildanzeige wären die Live-Preview Dialoge nicht praktikabel einsetzbar. Verschiedene Maßnahmen optimieren die Laufzeit so weit, dass livebildtaugliche Durchlaufzeiten auch mit großen Bildern gewährleistet werden. Optimierte Algorithmen, direkte Speicherzugriffe in unsicherem Code und parallele Threads werden an vielen Stellen zielgerichtet eingesetzt.

Auch die gute Erweiterbarkeit wurde an der Implementierung deutlich. Aufbau und Ablauf der Erkennungsmethoden können leicht auf neue Erkennungen übertragen werden. Durch das Bündeln von Kernfunktionen in Basiskomponenten können neue Kameras und Dialoge schnell erstellt werden.

Durch die meist lose Kopplung der verschiedenen Komponenten über definierte Schnittstellen können nicht nur neue Varianten, sondern auch ganz neue Komponenten in Zukunft ergänzt werden. Bereits während der Entwicklung des Moduls zeigte sich, wie gut weitere Erkennungen, aber auch komplett neue Komponenten wie das ShowImageWindow auf Basis der Preview-Dialoge umgesetzt werden können.

Die erfolgreiche Implementierung des Konzeptes stimmt zuversichtlich, dass reale Anwendungsfälle dadurch abbildbar sind und Anwender in die Lage versetzt werden können, damit verschiedene Prüfstandsapplikationen umzusetzen.

7 Praxiseinsatz & Anwender-Feedback

Das TEaMVision-Modul wurde nach der Implementierung von mehreren Anwendern im Produktionsumfeld mit unterschiedlichen TestStand- und Programmierkenntnissen eingesetzt, um drei reale Prüfstandsapplikationen umzusetzen und weitere Anwendungsfälle zu untersuchen. In diesem Kapitel werden diese Projekte vorgestellt. Anhand der Ergebnisse und dem Anwenderfeedback wird beurteilt, wie gut sich die Anwendungsfälle mit der TEaMVision-Lösung umsetzen lassen und inwieweit die Anforderungen erfüllt wurden.

7.1 Entwicklung Testapplikation für Fixdrosseln in der VTUX-Montage

Das Pilotprojekt für TEaMVision ist Integration in die Fertigungslinie der **Ventilinselfamilie VTUX**. An verschiedenen Stationen sollen mehrere Merkmale erkannt und mit dem vorgesehenen Bauzustand abgeglichen werden.

Als erster Anwendungsfall wird dabei ein neues Produktfeature getestet, die sogenannten *Fixdrosseln*. Das sind kleine Einsätze in den Luftkanälen unter den Ventilen. Es gibt mehrere Varianten mit verschiedenen Funktionen, etwa Drosseln mit unterschiedlichem Durchfluss, Verschlussstopfen und Rückschlagventile.

Die Varianten haben zur klaren Unterscheidbarkeit und auch als Vorbereitung für kamerabasierte Prüfung unterschiedliche Farben erhalten.

Die erste Fixdrosselvariante, die in der Fertigung verbaut werden soll, hat eine weiße Farbe. Somit setzt die erste Iteration dieser Pilotapplikation die Erkennung weißer Einsätze um. Die Erkennung andersfarbiger Muster wird ebenfalls getestet, aber erst später in die Prüfapplikation integriert.

7.1.1 TestStand Kernprogramm

Das Prüfprogramm muss alle verbauten Fixdrosseln erkennen und ihre Position auf der Insel mit dem Fertigungsauftrag abgleichen. Jeder Ventilplatz hat 5 Luftkanäle, die Fixdrosseln müssen im richtigen Ventilplatz und im richtigen Luftkanal verbaut sein.

Abbildung 97 zeigt den Ablauf des Kernprogramms in TestStand. Der Ablauf wurde noch als Teil dieser Arbeit entwickelt und anschließend von einem Prüfsystemtechniker in eine reale Testapplikation integriert.

Zunächst wird die Verbindung zur Kamera aufgebaut, anschließend werden Bildhandles initialisiert und eine Image Message Box für die Bedieneroberfläche geöffnet. Es wird ein

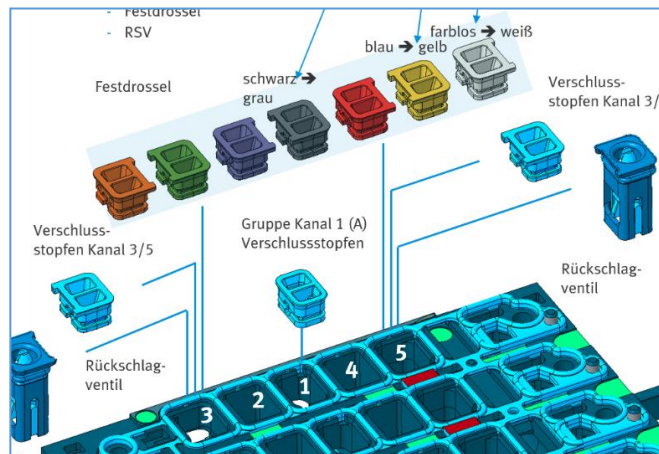


Abbildung 96: Illustration Fixdrosseleinsätze für VTUX-Ventilinselfamilie

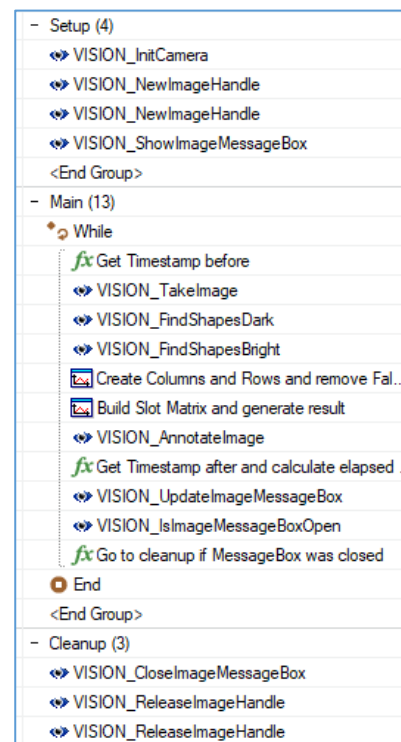


Abbildung 97: TestStand Programmablauf Fixdrosselerkennung

Bild aufgenommen und damit werden zwei FindShapes Erkennungsschritte ausgeführt.

FindShapesDark erkennt alle leeren Luftkanäle ohne Fixdrosseleinsatz, im Folgenden auch *Slots* genannt. Mit der Menge an erkannten Slots wird ein vereinfachtes Koordinatensystem aufgespannt. Auf einer Ventilinsel mit 20 Ventilplätzen und 5 Kanälen pro Ventil können die Slots so mit Koordinaten von (0, 0) bis (19, 4) adressiert werden. Die Slot-Koordinaten sind immer relativ zum ersten Luftkanal im ersten Ventilplatz.

FindShapesBright erkennt alle weißen Fixdrosseln. Die Ergebnisse werden ebenfalls in das Slot-Koordinatensystem übertragen und anschließend mit der Definition aus dem Fertigungsauftrag abgeglichen.

AnnotateImage zeichnet im aufgenommenen Bild die gefundenen Fixdrosseln ein. Drosseln an der korrekten Position werden grün und an der falschen Position rot eingezeichnet. Positionen, an denen eine Fixdrossel erwartet aber nicht gefunden wurde, werden ebenfalls rot markiert.

UpdateImageMessageBox zeigt das annotierte Bild auf der Oberfläche an. In Abbildung 98 zu sehen ist das Bild, das dem Bediener des Prüfstands angezeigt wird.

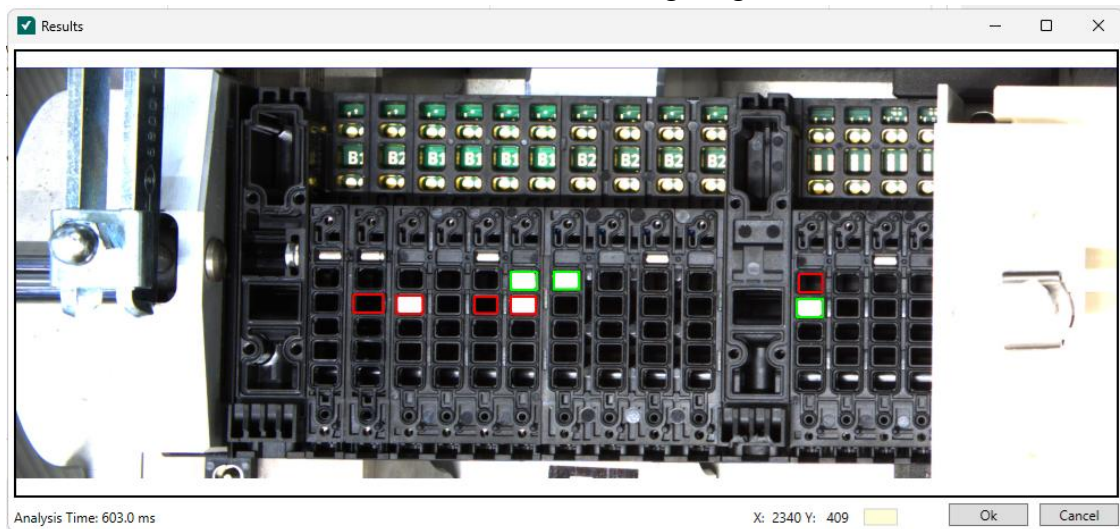


Abbildung 98: Ansicht Bedienerhilfe Fixdrosselmontage

Der Ablauf wird in einer Schleife ausgeführt, die Zykluszeit bewegt sich im dreistelligen Millisekunden Bereich. Die Bildwiederholrate im einstelligen Hz-Bereich ist tatsächlich ausreichend, um dem Bediener ein Live-Feedback für die Montage der Fixdrosseln zu geben.

Initial wird ein Bild der leeren Insel angezeigt, in dem die Slots markiert sind, in denen eine Fixdrossel verbaut werden muss. Nach dem Platzieren einer oder mehrerer Drosseln kann der Bediener auf den Bildschirm schauen und bekommt sofortige Rückmeldung, ob er die richtigen Positionen getroffen hat. Wenn nicht, kann er die Falschplatzierungen leicht korrigieren, denn jetzt muss er nur den Abstand zwischen Ist- und Sollposition abzählen, nicht den Abstand vom Anfang der Insel.

Beide FindShapes-Schritte nutzen den Intensity-Modus zur Suche nach gefüllten Rechtecken. Die hellen Fixdrosseln auf schwarzem Hintergrund sind damit sehr einfach zu erkennen, die Erkennung der leeren Slots ist etwas schwieriger. Hier wird nach dunklen Bereichen auf hellem Hintergrund gesucht. Der Kontrast der schwarzen Löcher zum schwarzen Vordergrund ist nicht gerade hoch, deshalb wird hier über die Beleuchtung des Arbeitsbereiches ein Schattenwurf in den Luftkanälen erzeugt. Damit lässt sich ein sehr zuverlässiger Kontrast zwischen dem dunk-

len Vordergrund der Ventilinsel und dem noch dunkleren Luftkanal erzeugen. Mit dem dynamischen Grenzwert von FindShapes werden die Slots so auch bei veränderbaren Umgebungslichtverhältnissen sehr zuverlässig gefunden.

Für das Aufspannen des Slot-Koordinatensystems müssen nicht alle leeren Luftkanäle erkannt werden. Als Tabelle betrachtet, wird lediglich ein Ergebnis pro Spalte und pro Zeile benötigt. Die Pixelkoordinaten der restlichen Slots lassen sich aus den vorhandenen Stützpunkten berechnen.

Abbildung 99 zeigt den Algorithmus zum Gruppieren der Ergebnisse in Spalten. Eine Spalte ist eine Liste von Shapes. Für jedes Shape S wird geprüft, ob es eine Spalte C gibt, deren erstes Element eine X-Abweichung von weniger als 5 Pixeln hat. Wenn ja, füge S zu C hinzu, wenn nein erstelle neue Spalte mit S als erstem Element. So wird jedes S einer Spalte zugeordnet. Die Spalten werden anschließend jeweils nach den Y-Positionen der Elemente sortiert. Die Liste der Spalten wird nach den X-Positionen des jeweils ersten Elementes sortiert.

Analog werden die Shapes in Zeilen gruppiert. Der Ablauf ist dabei gleich, logischerweise bis auf die getauschten X- und Y-Koordinaten in den jeweiligen Bedingungen.

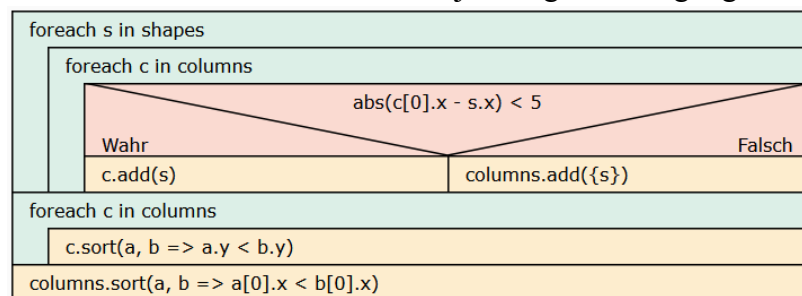


Abbildung 99: Programmablauf zum Gruppieren von Shapes in Spalten

Die Erkennung falsch-positiver Ergebnisse von FindShapes ist nicht auszuschließen. Das hätte zur Folge, dass zusätzliche fehlerhafte Spalten oder Zeilen generiert werden. Abbildung 100 zeigt den Algorithmus zum Entfernen dieser fehlerhaften Spalten. Dabei werden Zeilen und Spalten entfernt, bis die Anzahl der aus der Definition entspricht.

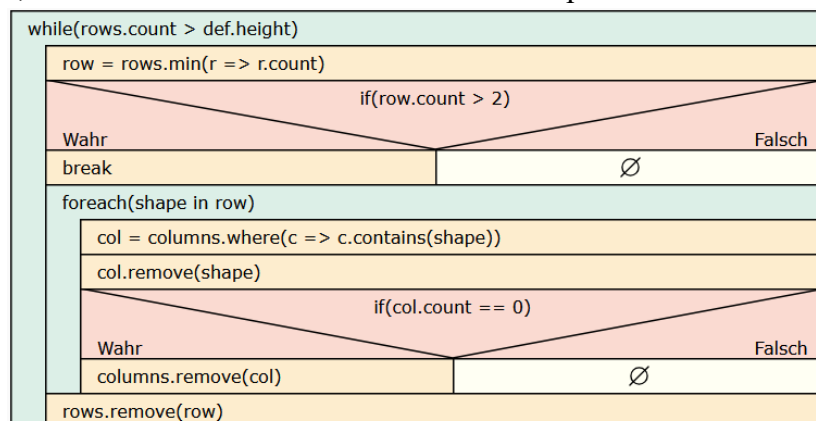


Abbildung 100: Programmablauf zur Entfernung von falsch positiven Ergebnissen

Die Slots liegen in der Regel alle auf einer geraden Linie mit vielen weiteren Elementen. Die falsch-positiven Ergebnisse sind selten perfekt mit den anderen Elementen eingereiht. Deshalb wird die Zeile mit den wenigsten Elementen ermittelt, hierin befindet sich sehr wahrscheinlich ein falsch-positives Ergebnis. Die Zeile wird eliminiert und die Elemente der Zeile werden auch aus allen Spalten entfernt. Falls das falsch-positive Ergebnis doch in einer Zeile mit korrekten

Ergebnissen eingereiht ist, würde dieses Vorgehen eine Zeile mit korrekten Ergebnissen löschen. Deshalb werden nur Zeilen mit 1-2 Elementen entfernt.

Der Ablauf entfernt also alle falsch-positiven Ergebnisse, die nicht in einer Zeile von korrekten Ergebnissen liegen. Um die verbleibenden Fehler zu eliminieren, wird der Algorithmus erneut ausgeführt, aber für die Spalten anstatt der Zeilen. Danach können nur noch Ergebnisse verbleiben, die in einer Spalte und Zeile mit korrekten Ergebnissen liegen. Das kann nur für die Slots der Fall sein, somit wurden alle falsch-positiven Ergebnisse erfolgreich entfernt.

Jetzt sind die Pixelpositionen aller Slots auf der Insel bekannt. Die Ergebnisse von FindShapes-Bright werden nun in das Slot-Koordinatensystem eingeordnet. Die Ist-Positionen der gefundenen Drosseln werden mit den Soll-Positionen verglichen und dafür Ergebnisannotationen generiert.

7.1.2 Integration in Produktionslinie

Der Ablauf wurde anschließend von einem erfahrenen Prüfsystemtechniker in eine vollumfängliche TestStand Prüfapplikation integriert. Dafür wurde in der VTUX-Montagelinie an einem bestehenden Arbeitsplatz eine Kamera montiert und mit dem dort vorhandenen PC verbunden. Über den Product Key der Insel wird die Produktkonfiguration aus dem SAP-System geladen. Daraus wird die Definition für das Fixdrosselprogramm generiert und an den vorher beschriebenen Ablauf übergeben. Nach Montage der Drosseln werden Testergebnis und ein Bild der Insel zusammen in einem Testbericht als TreX-Container abgelegt und archiviert.

Abbildung 102 zeigt den Aufbau in der VTUX-Montagelinie. Die Insel wird unten von einer Spannvorrichtung gehalten. Auf dem Monitor im Bild ist der Sichtbereich der Kamera zu erahnen, es werden auch deutlich längere als die hier montierte Insel unterstützt. Die ROI der Erkennungen wird mit den Daten aus dem Fertigungsauftrag dynamisch auf die Breite der Insel angepasst, um falsch-positive Ergebnisse im ungenutzten Arbeitsbereich zu vermeiden.

Die Fixdrosseln sind aktuell noch nicht bestellbar, die Testapplikation wurde in Vorbereitung auf den Produktstart erstellt. Das Programm wurde dafür über einen Zeitraum von mehreren Wochen im Produktionsprozess für viele VTUX-Exemplare ausgeführt, um die Erkennung der Slots zu testen und allgemeine Erfahrungen zum Produktiveinsatz von TEaMVision zu sammeln. Für jeden Prüfling wurde ein Report mit Bildern und der Anzahl der gefundenen Ventilplätze erzeugt.

In der Testphase konnten der Arbeitsplatz und die Parametrierung des Testprogramms so weit optimiert werden, dass die Quote für positive Prüfungen zuletzt über 95% lag. Für das Live-Feedback läuft der Algorithmus in einer Schleife, wodurch Fehler nur kurz auftreten und im nächsten Durchlauf wieder verschwinden. Somit ist die Erkennungsquote hier mehr als zufriedenstellend.

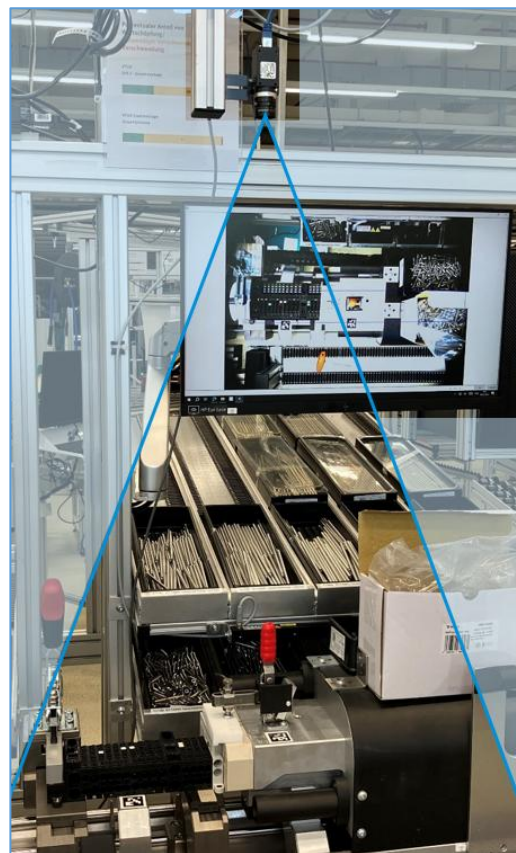


Abbildung 101: Fixdrosselerkennung in VTUX-Montagelinie

Die größte Herausforderung war die Belichtung des Arbeitsplatzes, die Erkennung benötigt einen zuverlässigen Schattenwurf in den Slots. Eine einfache Abdeckung als Dach über dem Arbeitsplatz konnte hier das Umgebungslicht von oben so weit eliminieren, dass jetzt in allen getesteten Lichtbedingungen ein sehr zuverlässiger Kontrast erzeugt wird.

Bei einem Test wurde nachts die Hallenbeleuchtung komplett abgeschaltet. Im Vorschaubild war nur eine schwarze Fläche zu sehen, lediglich die Fixdrosseln waren als graue Objekte erkennbar. Dennoch konnte FindShapes zuverlässig die Slots und die Fixdrosseln erkennen. Das Testprogramm funktionierte weiterhin ordnungsgemäß.

7.1.3 Fazit

Das erste reale Applikationsbeispiel zeigt, dass die FindShapes-Methode auch zur Umsetzung weiterer Anwendungsfälle eingesetzt werden kann. Zusätzlich werden die Vorteile der Integration in die TestStand Umgebung deutlich, **ein komplexer Algorithmus wie der hier in TestStand wäre mit einem dedizierten Vision-System nicht möglich.**

Der FindShapes Schritt ermittelt nur die Positionen von schwarzen und weißen Rechtecken im Kamerabild. Durch die Integration und Weiterverarbeitung per Skripting in TestStand kann der Anwender aus diesen einfachen Ergebnissen mächtige Erkenntnisse über den Zustand des Prüfings gewinnen. Die Performance aller Komponenten ist gut genug, dass Livebilder nicht nur zur Parametrierung, sondern auch als Ergebnisanzeige realisiert werden können. Die Stabilität von FindShapes beeindruckt damit, dass selbst schwarz-schwarz Kontraste unter veränderbaren Lichtverhältnissen zuverlässig erkannt werden.

Der Kernablauf wurde als Teil dieser Arbeit entwickelt, deshalb gibt es hier noch kein ausführliches Anwenderfeedback. Nach der Übergabe an den Prüfsystementwickler konnte dieser den Ablauf aber problemlos in das Applikationsprogramm integrieren. Bei der Erweiterung gab es auch keine Probleme, etwa beim dynamischen Setzen der ROI. Auch die nachträglichen Anpassungen der Parametrierung konnten die Anwender selbstständig vornehmen. Zusammengefasst war das erste, wenn auch limitierte Anwenderfeedback durchwegs positiv.

Mit dem Einsatz und der Vorstellung der Applikation in verschiedenen Kreisen konnten bereits großes Interesse und Erwartungen für weitere Lösungen geschaffen werden. Die Erfahrungen aus dem ersten Pilotprojekt stimmen zuversichtlich für die Umsetzung der nächsten Anwendungsfälle.

7.2 Überprüfung des Bauzustands von VTUX-Ventilinsel

Die nächste Applikation ist ebenfalls eine Station in der VTUX-Fertigungslinie. Hier wird der Bauzustand einer fertig montierten Insel überprüft und dokumentiert. Dazu werden mehrere Merkmale erkannt und mit dem Fertigungsauftrag abgeglichen. An der Entwicklung sind mehrere verschiedene Anwender beteiligt, das Projekt eignet sich somit gut, um Feedback zur Bedienbarkeit zu sammeln.

7.2.1 Erkennung der Merkmale

Der Umfang der Prüfungen an dieser Station wird in mehreren Stufen umgesetzt und erweitert. Für die erste Stufe wurden fünf Merkmale definiert, drei auf der Vorderseite und zwei auf der Oberseite der Insel.

Abbildung 102 zeigt ein Ergebnisbild zur Vorderseite der Insel. Hier werden die *Fittinge*, blaue *Blindstopfen* und weiße *Schalldämpfer* erkannt. Alle drei Merkmale werden über FindShapes gesucht. Die Fittinge werden als blauer Kreis im Konturmodus, die Blindstopfen als gefüllter blauer Kreis und die Schalldämpfer als weißer Kreis erkannt.

Fittinge und Blindstopfen können in drei verschiedenen Größen vorkommen. Mit den passenden Grenzwerten für minimale und maximale Größe werden jeweils alle Varianten mit einem einzelnen Schritt erkannt. In TestStand werden die Ergebnisse anhand ihrer Größe klassifiziert.



Abbildung 102: Ergebnisbild Vorderseite VTUX-Bauzustand

Abbildung 103 zeigt ein Ergebnisbild zur Oberseite der Insel. Hier werden die *Handhilfsbetätigungen* (HHBs), das sind Knöpfe zur manuellen Betätigung des Ventils, und die *Abdeckkappen* für HHBs gesucht. Beide Komponenten können an einer von zwei festen Positionen auf jedem Ventil vorkommen. Für jede Position wird überprüft, ob dort eine HHB, eine Abdeckkappe oder keines von Beiden vorhanden ist.



Abbildung 103: Ergebnisbild Oberseite VTUX-Bauzustand

Beide Merkmale können wieder mit FindShapes erkannt werden. Die HHBs als kleine blaue Kreise und die Abdeckkappen als blaues Rechteck über den blauen Strich auf ihrer Oberseite.

Die Beleuchtung ist auch bei dieser Applikation wieder eine Herausforderung. Die Farbe des blauen Kunststoffes kann sich bei Überbelichtung in Richtung Hellblau bis hin zu Weiß verschieben und je nach Helligkeit verläuft der Schalldämpfer mit dem weißen Hintergrund, wodurch er nicht zuverlässig erkannt werden kann.

Für beide Fälle hat sich eine Tendenz zur Unterbelichtung in den Kameraeinstellungen als optimal erwiesen. Dadurch verschiebt sich der blaue Farbton nicht zu weiß und der Schalldämpfer setzt sich deutlich besser vom Hintergrund ab.

Die Fittinge und Blindstopfen können je nach Größe sehr nah beieinanderliegen. Der SplitContours-Mechanismus von FindShapes kann die einzelnen Objekte gut voneinander trennen, auch wenn sie sich teilweise berühren. Um das zu unterstützen haben sich hohe Grenzwerte für minimale Sättigung und Helligkeit als ideal erwiesen. Dadurch fallen an den Rändern der Konturen einige Pixel weg, wodurch auch die Brücken zwischen den Objekten dünner ausfallen und zuverlässig getrennt werden können.

7.2.2 Testapplikation

In der Testapplikation werden die Merkmale detektiert und anschließend der Ist- mit dem Sollbauzustand verglichen. Dafür wird wieder der Fertigungsauftrag abgefragt und daraus die Liste der zu verbauenden Komponenten extrahiert. Für alle Komponenten sind Breite und Reihenfolge bekannt. Der Arbeitsplatz hat einen festen Anschlag für den Prüfling und durch die fixe Kameraposition gibt es einen gleichbleibenden Umrechnungsfaktor von Pixeln zu Millimetern im Arbeitsbereich. So kann die Soll-Position jedes Merkmals im Kamerabild bereits sehr genau vorgegeben werden. Abbildung 104 zeigt das Ergebnisbild vor Platzieren der Insel im Arbeitsbereich. Die Positionen aller Merkmale sind hier bereits eingezeichnet.

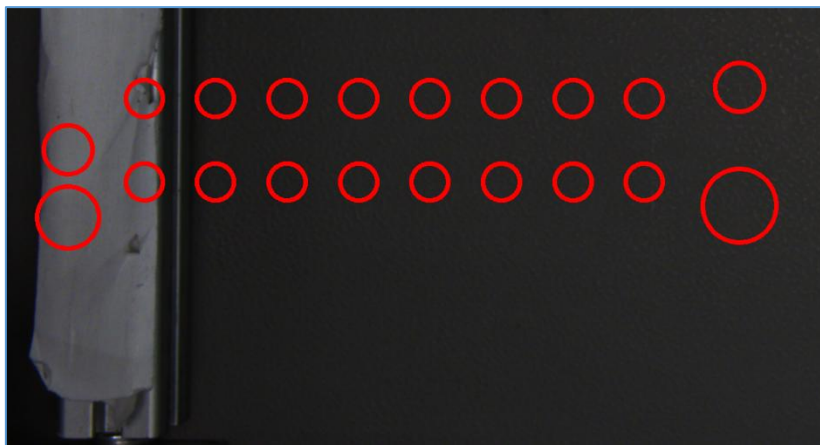


Abbildung 104: Ergebnisbild VTUX-Soll-Bauzustand ohne Ventilinsel

Ähnlich wie bei der Fixdrosselerkennung wird hier ein Ablauf in einer Schleife ausgeführt, der ein Bild aufnimmt, Merkmale erkennt, Ergebnisse mit der Vorgabe abgleicht und das Anzeigebild mit roten und grünen Ergebnissen aktualisiert.

Die Zykluszeit liegt im dreistelligen Millisekunden-Bereich. Damit hat die Ergebnisanzeige hier wieder Livebildcharakter. Beim Einlegen der Ventilinsel kann in Echtzeit beobachtet werden, wie die Merkmal-Annotationen sukzessive von Rot auf Grün umspringen. Der Bediener kann den Ablauf über *Ok* oder *Cancel* in der Bildanzeige beenden, im *Ok*-Fall wird ein Testreport erzeugt und abgespeichert.

Die Applikation wird aktuell im Produktivbetrieb der Produktionslinie getestet. Die Testberichte werden zunächst noch verwendet, um die Funktion und Stabilität der Applikation zu

überwachen. Das Testergebnis wird also bisher noch als Qualitätsmerkmal der Applikation und nicht des Prüflings interpretiert.

Es wurden bereits weit über 100 unterschiedliche reale Prüflinge an verschiedenen Tagen und zu unterschiedlichen Uhrzeiten getestet. Die allermeisten Ventilinseln durchlaufen die Prüfung ohne Probleme, Fehler treten nur noch vereinzelt auf. Anfängliche höhere Fehlerraten konnten durch Feintuning der Grenzwerte optimiert werden. Bei über 90% aller Prüflinge werden mittlerweile alle Merkmale korrekt erkannt.

Der Spezialfall in Abbildung 105 erfordert noch Korrektur auf der Seite von TEaMVision. SplitContours zerschneidet nur horizontal verbundene Konturen. In der Endplatte können ein Blindstopfen und ein Fitting direkt übereinandersitzen und sich berühren, dieser Fall wird von SplitContours nicht abgedeckt. Der Ablauf von FindShapes muss also so erweitert werden, dass SplitContours horizontal und vertikal auf alle Konturen angewandt wird. Der Algorithmus ist performant genug, dass die Erkennungsmethode dadurch nicht nennenswert verlangsamt werden dürfte.

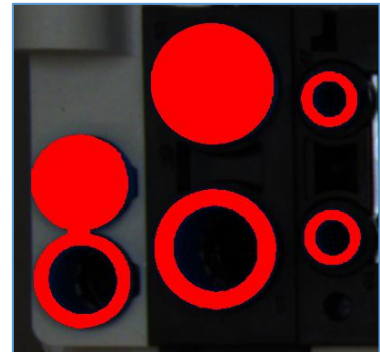


Abbildung 105: SplitContours Problem

Die Erfahrungen stimmen positiv, dass die Prüfapplikation den als nächstes anstehenden Qualifikationsprozess erfolgreich durchlaufen und anschließend voll in den Produktionsprozess integriert werden kann.

7.2.3 Erkennung weiterer Merkmale

Erkennungen für weitere Merkmale sollen in zukünftigen Arbeitspaketen sukzessive ergänzt werden. Im Labor werden dafür bereits jetzt die am besten geeigneten Erkennungsmethoden, Parameter und Beleuchtungseinstellungen erprobt.

Mit FindShapes können im Labor bereits einige weitere Merkmale erkannt werden.

Die *Hutschienelemente* unten links und rechts in Abbildung 106 dienen zur Befestigung auf einer Metallschiene, etwa in einem Schaltschrank.

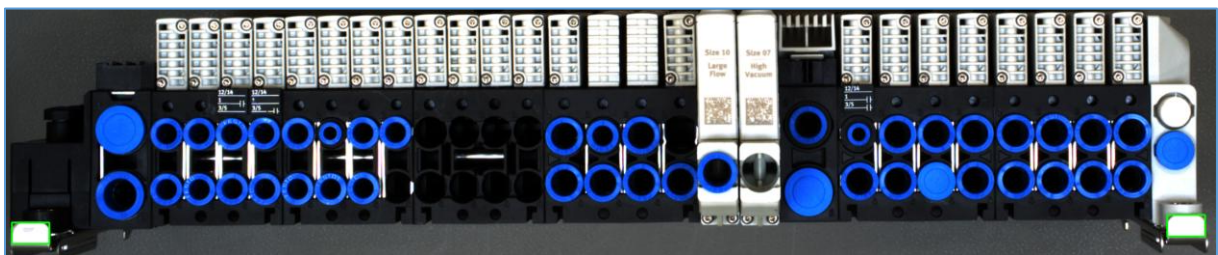


Abbildung 106: Erkennung Hutschienelemente

Mit passender Belichtung wird ein guter Hell-Dunkel Kontrast erzeugt und die Metallklammern können als weiße Rechtecke erkannt werden. Ob der Kontrast im wechselhaften Produktionsumfeld konsistent reproduziert werden kann, muss noch weiter erprobt werden. Zusätzlich muss betrachtet werden, ob die notwendige Ausleuchtung die Erkennung anderer Merkmale negativ beeinflusst.

Die *Klemmbügel* in Abbildung 107 fixieren die Fittings und Blindstopfen in den Verkettungsplatten. Sie können mit FindShapes gut als helle Rechtecke erkannt werden.

Die Klemmbügel sind in der Frontalansicht nicht sichtbar und auch von oben nur schwer zu erkennen, da sie durch die Perspektive von den Ventilen verdeckt werden.



Abbildung 107: Erkennung Klemmbügel

Im Labor konnte die Erkennung über einen schrägen Kamerawinkel realisiert werden. Die einzelnen Klemmbügel werden gut und reproduzierbar erkannt, auch wenn diese sehr nah beieinanderliegen. Für den Praxiseinsatz muss noch genauer untersucht werden, inwiefern sich die verzerrte Perspektive auf die Berechnung der Soll-Positionen im Bild auf Basis der Bauteilbreiten auswirkt. Gegebenenfalls muss hier ein Skalierungsfaktor in die Berechnung miteinbezogen werden.

Neben den Einzelventilen gibt es noch weitere Komponenten auf der Oberseite der Ventilinsel. Die *Leerplatten* in Abbildung 108 können anstatt eines Ventils verbaut werden. Abgesehen von der Abdichtung des Luftkanals besitzen sie keine weitere Funktion. Sie können mit FindShapes als große weiße Rechtecke erkannt werden.



Abbildung 108: Erkennung Leerplatten



Abbildung 109: Erkennung Flächenschalldämpfer

Die *Flächenschalldämpfer* in Abbildung 109 dienen zur Entlüftung eines Luftkanals nach oben, anstatt nach vorn. Hier wurden zwei Varianten für die Erkennung erprobt.

Die schwarz-weiß gestreifte Textur kann nicht einfach mit FindShapes erkannt werden. Hier wurde versucht, nur die weißen Stege eines Schalldämpfers zu erkennen. Eine Gruppierung von mindestens 5 Stegen würde dann als Flächenschalldämpfer interpretiert. Dieser Ansatz funktioniert vermutlich, ist aber sehr aufwändig.

Stattdessen wird die Erkennung via TemplateMatching bevorzugt. Hier kann das Ergebnis eines einzelnen Erkennungsschrittes direkt als Flächenschalldämpfer weiterverarbeitet werden. Die Erkennung im Labor funktioniert so weit zuverlässig, muss aber noch ausführlich unter verschiedenen Bedingungen getestet und evaluiert werden.

Die *Schilderträger* in Abbildung 110 zeigen Piktogramme mit pneumatischen Funktionsbeschreibungen. Die Erkennung wurde bereits in Kapitel 4 als Beispielanwendung für Template-Matching herangezogen. Entsprechend wird die Erkennung und Klassifizierung der verschiedenen Schildervarianten aktiv unter verschiedenen Bedingungen erprobt und in einem der nächsten Arbeitspakete in einer realen Prüfstandsapplikation integriert.

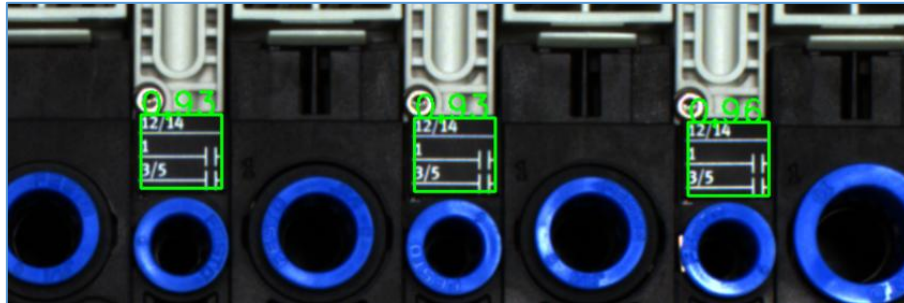


Abbildung 110: Erkennung Schilderträger

Besonders interessant wird hierbei sein, wie zuverlässig die verschiedenen Varianten voneinander unterschieden werden können. Oft unterscheiden diese sich nur in kleinen Details. Die ersten Untersuchungen fallen hier bisher positiv aus.

In einer weiteren Applikation soll bereits vor Montage der Ventile und Fittings überprüft werden, ob die *Verkettungsplatten* korrekt zusammengestellt wurden.

In Abbildung 111 sind unten die unterschiedlichen Einkerbungen in den Kunststoffelementen zu sehen. Zur Unterscheidung der Varianten wird ebenfalls das Template-Matching eingesetzt. Das funktioniert im Labor bisher noch nicht zuverlässig, weil die Vorverarbeitung den Kontrast zwischen dem recht dunklen Vordergrund und der Einkerbung nicht deutlich genug herausstellt. Hier ist weitere Entwicklungsarbeit an der Vorverarbeitung vonnöten.

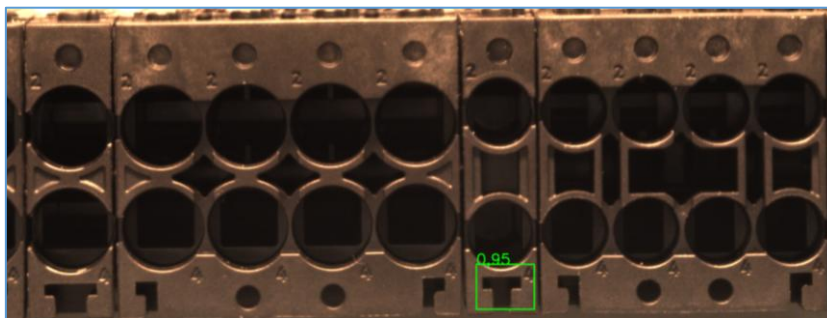


Abbildung 111: Erkennung Varianten Verkettungsplatten

In Abbildung 112 sind die *Platinen* zu sehen, die zur Kontaktierung der Ventile in die Verkettungsplatten eingesetzt werden. Über die Beschriftung und die Anzahl der Kontaktierungsflächen sind diese eindeutig unterscheidbar.



Abbildung 112: Erkennung Platinenvarianten

Mit dem TemplateMatching ist die Erkennung der Beschriftung und auch der verschiedenen Kontaktierungsflächen grundsätzlich möglich. Durch die Perspektive wird die Erkennung an den Randbereichen allerdings deutlich erschwert, denn durch Schattenwürfe und teilweise Verdeckung wird die Übereinstimmung mit dem Template so weit beeinflusst, dass eine zuverlässige Unterscheidung nicht realistisch umsetzbar ist. Hier muss eine andere Lösung auf Seite der Applikation gefunden werden, um gute Sichtverhältnisse für die kamerabasierte Erkennung zu gewährleisten.

7.2.4 Zukünftiges Projekt - DataMatrix Massenscanner

Bei der Fertigung von Ventilinseln verschiedener Baureihen werden die DataMatrix-Codes auf allen Einzelventilen dokumentiert und mit dem Product Key der gesamten Ventilinsel verknüpft. Dafür müssen in den meisten Fällen alle Ventile einzeln mit einem Handscanner der Reihe nach abgescannt werden. Dafür wird, wie in Abbildung 113 zu sehen, eine Schablone mit einem Ausschnitt der Größe eines einzelnen Codes über die Insel bewegt, um nicht aus Versehen den falschen Code zu scannen. Dieser Ablauf ist zeitaufwändig und fehleranfällig. Hier wurde schon länger nach einer adäquaten automatisierten Lösung gesucht.



Abbildung 113: Einscannen Dmtx-Codes mit Handscanner und Schablone

An einigen Stationen werden seit kurzem spezialisierte automatische Scanner von Keyence eingesetzt, die in einem Arbeitsbereich von 50cm Breite alle Dmtx-Codes auf einmal einscannen und in der korrekten Reihenfolge dokumentieren können. Diese Scanner sind mit einem Preis im mittleren 5-stelligen Bereich allerdings sehr teuer und werden entsprechend nur an wenigen Stationen mit besonders hohem Durchsatz eingesetzt, ein Ausrollen auf viele weitere Arbeitsplätze ist unwahrscheinlich.

Bei Versuchen im Labor wurde die Leistungsfähigkeit der DmtxDetection-Erkennung in Verbindung mit einer Baumer Kamera getestet. Hier konnten in einem Arbeitsbereich von 50cm Breite ebenfalls alle Dmtx-Codes auf verschiedenen Ventilinselvarianten erfolgreich erkannt werden, wenn Belichtungseinstellungen und Kamerafokus optimal gewählt wurden.



Abbildung 114: Erkennung aller DMTX-Codes auf einer Ventilinsel

Die Abbildung oben zeigt eine große VTUX-Ventilinsel, bei der in einem einzelnen Bild fast alle Dmtx-Codes erfolgreich dekodiert werden. In der Live-Vorschau flackern einige Codes sporadisch, aber mit zwei bis maximal fünf Durchläufen können alle Codes zuverlässig ausgelesen werden. Somit tut sich hier großes Potenzial auf, das manuelle Einscannen der Dmtx-Codes auch an vielen weiteren Arbeitsplätzen zu automatisieren und ein teures Zukaufprodukt für über 30.000€ mit einer eigenen Lösung zu ersetzen, die nur eine Hardware-Investition im Bereich von 1.000€ benötigt.

7.2.5 Benutzerfreundlichkeit und Anwenderfeedback

Mehrere Anwender waren in verschiedenen Rollen an der Umsetzung dieser Applikation beteiligt. Die Ansätze für die Erkennung jedes Merkmals konnten im Labor von einem Projektmanager ohne Informatikhintergrund und ohne Einblick in die TEaMVision-Implementierung erarbeitet werden.

Auf Basis dieser Grundparametrierungen konnte ein Auszubildender der Fachrichtung Informationstechnik die Erkennungen ausführlich testen und die optimalen Einstellungen für verschiedenen Lichtbedingungen und unterschiedliche Prüflinge erarbeiten. Die Testapplikation konnte von einem erfahrenen TestStand-Programmierer bereits parallel dazu entwickelt werden. Dazu wurden zunächst statische Bilder mit der FileCamera und die Grundparameter für die Erkennungsmethoden verwendet. Für die Entwicklung des Testprogramms waren keinerlei Hardware oder reale Prüflinge notwendig. Bei der Inbetriebnahme in der Fertigungslinie wurden anschließend die finalen Parametrierungen für Erkennungen und Kamera integriert.

Alle Anwender konnten nach kurzer Einweisung selbstständig funktionsfähige Erkennungen erstellen. Auch ohne genaues Verständnis über die interne Funktion der Methoden konnten die Anwender durch Verstellen der verschiedenen Einstellungen zuverlässig Lösungen für die allermeisten Zielsetzungen finden.

Das zeigte sich auch an einem besonders schönen Beispiel. Bei einer Station eines Praktikums durften zwei 14-jährige Schüler einen Tag lang mit TEaMVision und einer Ventilinsel experimentieren. Nach einer kurzen Einführung waren sie eigenständig in der Lage, komplett eigenständig Erkennungen für mehrere der oben beschriebenen Merkmale auszuarbeiten.

Der TestStand-Programmierer hatte keine Schwierigkeiten bei der Integration der verschiedenen Testschritte in die Applikation. Die final konfigurierten Erkennungsschritte konnten einfach in das fertige TestStand-Programm kopiert werden, um die neuen Einstellungen zu übernehmen. Auch bei der Weiterverarbeitung der Erkennungsergebnisse und der Generierung der Ergebnisbilder traten keine Probleme auf.

Die Rückmeldungen aller Anwender waren überwiegend sehr positiv. Die Konfiguration über Preview-Dialoge stieß auf sehr gute Resonanz. Die Anwender und die Bediener im Testbetrieb waren beeindruckt von der Erkennungsgeschwindigkeit und dem Livebildcharakter der Ergebnisanzeige.

Die Arbeit mit Kamera-, Bild- und Dialoghandles hat sich für die Anwender als weitestgehend intuitiv erwiesen. Auch die Inkonsistenz bei der Vergabe von Kamera- und Bildhandles wurde nach kurzer Erklärung wohlwollend angenommen.

Die Annotation der FindShapes-Ergebnisse im Outline-Modus führte vereinzelt zu Verwirrung. Das Ergebnis wird grün eingezeichnet, die Outline wird zusätzlich blau markiert. Die Anwender haben das teilweise als zwei Ergebnisse interpretiert, kurze Aufklärung sorgte aber sofort für Verständnis und Klarheit. Die Farbe und Art der Annotationen müssen noch einmal überdacht und dafür eine klarere Lösung gefunden werden.

Die Preview-Dialoge und die Anwendungen laufen sehr stabil. Erkennungsergebnisse sind auch an verschiedenen Tagen und mit unterschiedlichen Umgebungsbedingungen reproduzierbar. Testprogramme laufen auch längere Zeit und oft hintereinander störungsfrei. Bei der Bedienung der Preview-Dialoge kommt es sporadisch zu Fehlermeldungen und selten auch zu Programmabstürzen. Meist ist das auf eine unerwartete Kombination von Benutzereingaben oder Probleme mit dem Netzwerk zurückzuführen. Diese Fehler müssen über die Zeit gesammelt und korrigiert werden.

7.3 Integration TEaMVision in bestehendes Prüfsystem

Im nächsten Projekt wurden mit TEaMVision visuelle Prüfungen in einem existierenden Prüfstand hinzugefügt. Das Kontrollmodul *PCFU* wird während dem Belichtungsprozess in der Halbleiterfertigung zum kontrollierten Klemmen von Wafern eingesetzt. Der Prüfstand führt elektrische und pneumatische Prüfungen an dem Kontrollmodul durch. PCFU hat 9 verschiedene Varianten, die alle mit demselben Prüfsystem getestet werden. Die Varianten haben intern verschiedene Funktionen und unterscheiden sich extern in der Art, Anzahl und Position der Druckluftanschlüsse.

Beim Start des Testprogramms wird die Produktvariante ausgewählt. Je nach Variante unterscheiden sich die Ansteuerung und die Prüfungen. Wird die falsche Variante ausgewählt, können dadurch Prüfling und Prüfstand beschädigt werden oder falsche Messwerte ermittelt werden. Mit der visuellen Prüfung sollen alle Varianten unterschieden werden, um so das Einlegen des falschen Prüflings zu verhindern.

Abbildung 115 zeigt das Ergebnisbild mit den Anschlüssen des Prüflings. Die drei Varianten werden via FindShapes erkannt. Die erste Variante wird als messingfarbener Ring, die zweite als messingfarbener gefüllter Kreis und die dritte als schwarz gefüllter Kreis erkannt. Anhand der Kombination und Position der verschiedenen Anschlüsse können alle Varianten zuverlässig unterschieden werden.

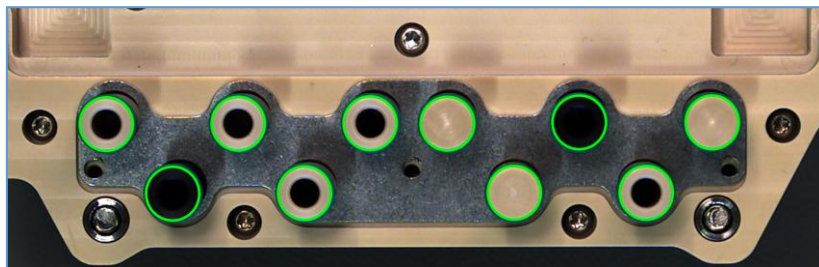


Abbildung 115: Ergebnisbild PCFU-Prüfstand mit Klassifizierung der Anschlussvarianten

Auf der Oberseite des Moduls in Abbildung 116 befinden sich mehrere Einzelkomponenten mit einem eigenen Product Key. Zur Rückführbarkeit werden die Product Keys an diesem Prüfstand mit dem PCFU-Modul verknüpft. Bisher wurde dafür ein Handscanner genutzt, mit den Ergänzungen kommt dafür nun DmtxDetection zum Einsatz. Die Product Keys jeder Komponente werden im Testreport zusammen mit den Testergebnissen und den Bildern des Prüflings dokumentiert und archiviert.



Abbildung 116: Ergebnisbild PCFU-Prüfstand mit DMTX-Erkennung

Auf der Oberseite des Prüflings sollen mit TextDetection zukünftig noch einige Beschriftungen erkannt werden. Die Erkennung funktioniert aber noch nicht zuverlässig, deshalb ist sie aktuell nicht in den Prüfablauf integriert.

Ansonsten traten bei der Integration von TEaMVision in den Prüfstand keine nennenswerten Probleme auf. Die Anwender waren nach kurzer Einführung selbstständig in der Lage, die neuen Schritte hinzuzufügen und zu konfigurieren. Mit der Farbauswahl im Bild konnten sehr

schnell effektive Erkennungen für die verschiedenen Anschlüsse erstellt werden. Die Stabilität und Performance von FindShapes und DmtxDetection sind mehr als ausreichend. Der Prüfstand wird bereits einige Zeit mit den neuen Ergänzungen produktiv eingesetzt und die Anwender sind zufrieden mit der Lösung. Im Umfeld um das Produkt PCFU gibt es noch weitere ähnliche Prüfstände, die in Zukunft ebenfalls mit TEaMVision erweitert werden sollen.



Abbildung 117: Prüfstand zum Test des Kontrollmoduls PCFU

Abbildung 117 zeigt den Prüfstand von der Seite und von vorne. Der Prüfling wird unten eingelegt und über eine spezielle Klemmvorrichtung fixiert. Oben ist die Baumer Kamera hinter einer Abdeckung angebracht. Auch hier wurde ein Dach über dem Arbeitsbereich angebracht, um vom Umgebungslicht abzuschirmen und reproduzierbaren Schattenwurf in den Anschlüssen zu erzeugen. Rechts zu sehen ist die Anwenderoberfläche des Prüfstands. Der Prüfling muss zum Start des Programms mit den Anschlüssen zur Kamera gedreht werden. Nur nach erfolgreicher Klassifikation kann das Programm fortgesetzt werden.

Durch die Erweiterung mit visuellen Prüfungen wurden die Fehlerquote deutlich reduziert und die Produktivität durch höhere Automatisierung verbessert.

7.4 Speicherfragmentierung

Um die Stabilität von Testprogrammen auch bei längeren Laufzeiten zu gewährleisten, werden Bildinstanzen und sonstige große Speicherblöcke immer explizit freigegeben, nachdem diese nicht mehr benötigt werden. Bei der Arbeit mit TEaMVision in TestStand bleibt der Arbeitsspeicherverbrauch laut Task Manager auch bei langer Programmier- und Testprogramm Laufzeit sehr konstant, Speicherlecks sind nicht beobachtbar. Allerdings kommt es mit der 32-Bit Version von TestStand bei sehr langer Entwicklungszeit von vielen Stunden vor, dass irgendwann regelmäßig OutOfMemory-Fehler auftreten, auch wenn der Prozess einen Arbeitsspeicherverbrauch von nur 2 Gigabyte oder weniger angibt.

Das ist sehr wahrscheinlich auf Speicherfragmentierung zurückzuführen. Das tritt auf, wenn in unverwaltetem Code hochfrequent große Speicherblöcke reserviert und wieder freigegeben werden. Dadurch kann dann irgendwann für neue große Speicherblöcke kein ausreichend langer zusammenhängender Speicherbereich im virtuellen Adressraum des Prozesses mehr gefunden werden. [Microsoft, 2026]

Ein einfacher Neustart des Sequence Editors behebt dieses Problem, jedoch darf das keine reale Lösung für derartige Herausforderungen sein. Beim Einsatz der 64-Bit Version von TestStand würde dieser Fehler aufgrund des viele Größenordnungen größeren Adressbereiches sehr sicher

niemals auftreten, aber die 32-Bit Version ist bei Festo der weit verbreitete Standardfall und somit ist die Stabilität auch im 32-Bit Adressraum eine nicht verhandelbare Anforderung. Das Betriebssystem bietet leider keinerlei Handhabe, den Speicher zur Laufzeit zu defragmentieren. Somit ist es unvermeidbar, die Programmstruktur so anzupassen, dass große Speicherblöcke weniger frequent reserviert und freigegeben werden.

Die Ursache der Fragmentierung ist die Programmierpraxis, dass Speicherpuffer für OpenCV Mat-Objekte in vielen Methoden freizügig angelegt und wieder freigegeben werden. Oberflächlich ist das eine saubere Programmiertechnik, wenn alle Mat-Objekte durch Using-Blöcke zuverlässig freigegeben werden. Die Nachteile werden im Produktiveinsatz nun sichtbar.

Ein Lösungsansatz könnte die Arbeit mit Memorypools sein, wobei Mat-Objekte von oft verwendeter Größe nicht direkt freigegeben, sondern nach Möglichkeit wiederverwendet werden. Im Anschluss an diese Arbeit muss das Speicherfragmentierungsproblem weiter untersucht und dafür eine adäquate Lösung implementiert werden. Bis dahin besteht dadurch aber keine Blockade für den Produktiveinsatz, das Problem tritt primär bei langen und ausführlichen Entwicklungssitzungen auf und kann durch einen TestStand-Neustart sehr schnell behoben werden.

7.5 Erfüllung der Anforderungen

Mit den Erfahrungen aus den Pilotprojekten kann nun die Erfüllung der Anforderungen aus Kapitel 4 bewertet werden.

7.5.1 A1 – Erkennungen

Die Anforderungen A1.1 und A1.2 sind zu großer Zufriedenheit erfüllt. Für A1.3 ist noch nicht vollständig erfüllt, es muss eine zuverlässigere Erkennung von Textblöcken gewährleistet werden.

A1.1 – Komponenten: Mit FindShapes konnten diverse Produktkomponenten erfolgreich erkannt werden. Mit TemplateMatching werden auch einige komplexere Merkmale bereits erfolgreich erkannt. Die Methoden geben ein Ergebnis pro gefundenes Objekt zurück, mit Position, Größe und Form bzw. Übereinstimmungsgrad. Die Präzision von Position und Größe ist sehr hoch, im Praxiseinsatz waren keine sichtbaren Abweichungen zwischen Erkennungsergebnis und Merkmal beobachtbar. Die Maximalabweichung von 10% wird eingehalten und in der Regel deutlich untertroffen. Die Erkennungsrate ist regelmäßig besser als erwartet, in den Praxistests werden für 90% aller Prüflinge alle Merkmale bereits mit einem einzigen Bild erkannt. Die Anforderung ist zu großer Zufriedenheit erfüllt.

A1.2 – DataMatrix-Codes: DmtxDetection erkennt sehr zuverlässig auch kleine und schwer lesbare Codes. In einem einzelnen Bild können problemlos 30+ Codes erkannt und dekodiert werden. Jedes Einzelergebnis umfasst Größe, Position und Textinhalt. Mit der ersten LibDmtx-Implementierung wurden Codes regelmäßig fehlerhaft dekodiert, die Dynamsoft Implementierung hat bisher immer nur korrekt dekodierte Ergebnisse zurückgegeben. Die Erkennungsmethode ist so leistungsfähig, dass damit auch teure kommerzielle Vision-Systeme ersetzt werden können. Die Anforderung ist ebenfalls voll erfüllt.

A1.3 – Texte: Bei guten Bedingungen funktioniert TextDetection zufriedenstellend. Texte werden korrekt dekodiert und die Position und Größe der Textblöcke entspricht der Erwartung. Im Produktiveinsatz ist die Erkennung von Texten noch nicht ausreichend stabil, oft werden Textblöcke noch nicht zuverlässig als solche identifiziert. Hier ist Nacharbeit vonnöten.

7.5.2 A2 – Applikation

Die Anforderungen A2.1 – A2.3 sind durch das Bedienkonzept vollständig erfüllt.

A2.1 - Interface: Die Integration in TestStand entspricht vollständig den Erwartungen. Die Parameter der Funktionen sind in den Eigenschaften der Steps hinterlegt. Dadurch ist auch die dynamische Modifikation von Parametern zur Laufzeit des Testprogramms möglich. Der Applikationsentwickler hat die volle Kontrolle. Auch die zurückgegebenen Ergebnisse bringen alles mit, was für eine detaillierte Auswertung und Weiterverarbeitung im Testprogramm benötigt wird.

A2.2 - Kameras: Die Baumer Industriekamera wurde erfolgreich eingebunden. Mit den Kameraparametern ist auch eine gute gerätespezifische Konfigurierbarkeit gewährleistet. Mit der FileCamera können Erkennungen und Applikationen auch effektiv offline und ohne jegliche Hardware entwickelt werden. Die beiden Kameravarianten sind in TestStand und intern im TEaMVision Modul durch ihre Schnittstellen problemlos gegeneinander austauschbar. Durch Enumerationen, Auswahllisten und die Camera Basisklasse können weitere Kameravarianten in Zukunft leicht ergänzt werden.

A2.3 - Bildhandhabung: Mit ImageHandle werden Bildinstanzen in TestStand gut und nachvollziehbar gehandhabt. Dieselbe Bildinstanz kann für mehrere Erkennungen und anschließend für die Benutzeroberfläche verwendet werden. AnnotateImage erlaubt das Einzeichnen von beliebigen Ergebnissen und sonstigen Markierungen, ShowImageWindow zeigt das Bild während der Laufzeit auf der Oberfläche an und SaveImage kann das Bild abspeichern und so auch zu einem Testbericht hinzufügen. Über den Button in jedem Bilddialog kann zusätzlich jedes aktuell angezeigte Bild bei Bedarf manuell gespeichert werden.

7.5.3 A3 – Bedienung

Auch die Anforderungen A3.1 – A3.3 sind mit erfolgreicher Umsetzung des Bedienkonzeptes vollständig erfüllt, was durch die positive Rückmeldung der Pilotanwender bestätigt wird.

A3.1 – Step Types: Alle Funktionen sind erwartungsgemäß als Step Types in die Palette von TestStand integriert. Die Rückmeldung der Anwender bestätigt, dass die Step Types jeweils eine atomare Funktion mit gut verständlicher Funktion abbilden. Erfahrene Anwender des TEaM-Frameworks finden sich schnell zurecht. Die Edit-Dialoge enthalten alle wichtigen Parameter, die Live-Preview Dialoge erweitern das Paradigma und machen auch komplexere Parametrierung zugänglich.

A3.2 - Handles: Die drei eingeführten Typen von Handles für Kameras, Bilder und Dialogfenster werden von den Anwendern wohlwollend angenommen. Bei den Pilotprojekten traten hier keine Probleme auf. Die Ressourcen hinter den Handles werden, wie üblich, über Init-Methoden angelegt, anschließend über den Handle angesprochen und am Ende über Close/Release-Schritte freigegeben. Nur die Kameras besitzen keine Freigabe-Funktion, das ist zur Unterstützung der Live-Preview Dialoge aber so vorgesehen und notwendig.

A3.3 – Livebilder: Die Live-Preview Dialoge erzeugen einen großen Mehrwert bei der Entwicklung von Erkennungen. Damit konnten bereits viele Anwender ohne entsprechenden Hintergrund erfolgreich Merkmale erkennen und Feintuning vornehmen. Die Entwicklung der Vi-

sion-Applikationen wird dadurch massiv beschleunigt. Die Rückmeldungen zu den Live-Preview Dialogen sind überwältigend positiv. Zusätzlich positiv aufgenommen wird, dass viele der umgesetzten Applikationen auch zur Laufzeit Livebildcharakter haben.

7.5.4 A4 - Nichtfunktionale Anforderungen

Die Anforderungen A4.1 – A4.3 sind grundsätzlich erfüllt. Lediglich bei der Performance von TemplateMatching und beim Thema Speicherfragmentierung ist Nacharbeit vonnöten.

A4.1 - Performance: Die Performance von TEaMVision ist in vielen Fällen deutlich besser als erwartet. Auch mit mehreren FindShapes-Schritten sind Beurteilung und Ergebnisanzeige für mehrere Merkmale in deutlich unter einer Sekunde möglich. Auch Dmtx- und TextDetection erreichen dieses Niveau. Somit ist hier auch die Wunschbedingung der deutlich geringeren Durchlaufzeiten erfüllt.

Die Performance von TemplateMatching ist noch nicht in diesem Bereich. Mit restriktiven Parametern können hier Durchlaufzeiten von unter einer Sekunde erreicht werden, aber mit großen Bildern, Rotation und einem Mindestübereinstimmungsgrad unterhalb von 80% kann ein Durchlauf auch über 10 Sekunden dauern. Mit entsprechenden Einstellungen kann aber zumindest die Minimalanforderung (5-10 Merkmale in 15-20 Sekunden) erfüllt werden.

A4.2: Langzeitstabilität: Die Langzeitstabilität ist weitestgehend gegeben. Speicherlecks sind nicht beobachtbar, die Freigabe von Ressourcen arbeitet wie vorgesehen und der Arbeitsspeicherverbrauch bleibt auch bei längerer Laufzeit konstant. Bei 32-Bit Prozessen kommt es mit der Zeit zu Speicherfragmentierung, das muss zeitnah untersucht und gelöst werden. Die Anforderung ist somit noch nicht vollständig erfüllt.

A4.3: Erweiterbarkeit: Die gute Erweiterbarkeit hat sich bereits während der Entwicklung gezeigt. Weitere Erkennungsmethoden konnten auf Basis von FindShapes schnell entwickelt werden. Mit PreviewBaseControl war auch für die dazugehörigen Live-Preview Dialoge kein großer Aufwand vonnöten und das ShowImageWindow konnte auf dieser Basis sehr leicht ergänzt werden.

Durch die Kapselung mit der TEaMVision Klasse sind Änderungen an der Implementierung des Moduls problemlos auf bestehende Systeme ausrollbar. Auch neue Funktionen können als weitere StepTypes einfach verteilt und hinzugefügt werden.

7.6 Fazit

Das Potenzial von TEaMVision wurde im Piloteinsatz erfolgreich demonstriert. Mit dem Modul wurden mehrere reale Prüfapplikationen umgesetzt, die sich bereits im Produktiveinsatz befinden oder dort zumindest erfolgreich getestet werden.

Mit den visuellen Prüfungen können Anwendungen umgesetzt werden, für die es mit der etablierten pneumatischen und elektrischen Prüftechnik bisher keine gangbare Lösung gab. Auch komplexe Applikationen, bei denen die Umsetzung mit kommerziellen Vision-Systemen an der eingeschränkten Integration scheiterte, können mit TEaMVision problemlos umgesetzt werden. Das Bedienkonzept mit den Live-Preview Dialogen wird von den Anwendern sehr gut angenommen. Es zeigt sich klar, dass dadurch ein realer Mehrwert entsteht. Die ersten Rückmeldungen sind sogar positiver als erwartet, die gute Annahme von TEaMVision lässt auf viele Folgeprojekte hoffen, mit denen weiterer Mehrwert geschaffen werden kann.

8 Reflexion und Ausblick

Computerbasierte visuelle Prüfungen sind schon lange ein wichtiges und interessantes Thema für automatisierte Produktions- und Testing-Abläufe. Die großflächige Implementierung scheiterte aber immer wieder an hohen Kosten, großen Entwicklungsaufwänden und den mangelnden Schnittstellen für die Umsetzung komplexer Anwendungen.

TEaMVision versucht, diese Lücken durch den Einsatz freier Software, das Bereitstellen wiederverwendbarer Erkennungsbausteine und die Integrierbarkeit als Softwaremodul zu schließen.

8.1 Diskussion der Ergebnisse

Als erstes wurden etablierte und bekannte Vision-Lösungen analysiert, um ihre Funktionsweise zu verstehen und wünschenswerte Eigenschaften für TEaMVision zu identifizieren. Als größter Vorteil kommerzieller Systeme wurde die Konfiguration von Erkennungsschritten per GUI mit Liveanzeige der Ergebnisse identifiziert. Als nächstes wurde mit TestStand und dem TEaM-Framework die Zielumgebung für das Modul betrachtet. Wesentlicher Bestandteil hier sind die atomaren StepType-Funktionsbausteine, die per GUI-Dialog konfiguriert werden.

Das Bedienkonzept für TEaMVision greift die TEaM-Konfigurationsdialoge und die Livebildanzeige der kommerziellen Vision-Systeme auf. Hieraus werden die Live-Preview Dialoge abgeleitet, mit denen Kameras und Erkennungsmethoden in einem GUI-Dialog mit Live-Kamerabild konfiguriert werden. Bei der Implementierung stellen die Kernaspekte Performance, Stabilität und Erweiterbarkeit sicher, dass das Modul effektiv und produktiv eingesetzt werden kann. Anschließend wurden mit TEaMVision erfolgreich mehrere Testapplikationen umgesetzt. **Das Konzept geht auf, die Anwender sind zufrieden mit der Lösung.**

Das Projekt wurde anfangs mit einer vorsichtigen Erwartungshaltung gestartet. Die Ambition bei der Entwicklung der Erkennungsmethoden war immer, dass diese für verschiedenste Anwendungsfälle eingesetzt werden können. Es wurde aber als unrealistisch angesehen, mit einer eigenentwickelten Lösung ansatzweise die Leistungsfähigkeit teurer kommerzieller Systeme erreichen zu können. Die Erprobung jedes weiteren Anwendungsfalles führte jedoch immer wieder erneut zu positiver Überraschung. Bereits mit der FindShapes Methode lassen sich ohne nachträgliche Anpassungen mehr Anwendungsfälle umsetzen, als anfänglich erwartet.

Mit den Live-Preview Dialogen konnten mehrere Anwender auch ohne Vorerfahrung sehr schnell produktiv Erkennungen entwickeln. Allein durch das Experimentieren mit den verschiedenen Einstellungen fanden die Anwender dabei mehrmals auch Lösungsansätze, die vorher niemand in Betracht gezogen hätte.

Auch die Performance der Applikationen übertrifft jede Erwartungshaltung. Anwendungen mit Livebildcharakter, in denen mehrere Merkmale bei einem konfigurierbaren Produkt sofort erkannt und klassifiziert werden, konnten auch mit kommerziellen Systemen bisher nicht umgesetzt werden.

Das Ergebnis des Projektes ist mehr als zufriedenstellend. Der angestrebte Funktionsumfang wurde erreicht, die Ziele wurden teilweise deutlich übertroffen und es wurden erfolgreich mehrere Applikationen umgesetzt.

Metrik / Quantifizierung fehlt - sehr weiche Beurteilung,

8.2 Erweiterungspotenzial

TEaMVision stellt primär die Grundinfrastruktur zum Aufbau von Vision-Systemen bereit, sodass kein wiederholter Entwicklungsaufwand für das Einbinden von Kameras, Handhaben von Bildern und Anzeigen von Ergebnissen anfällt.

Die ersten Erkennungsmethoden decken bereits einige Bereiche ab, der Funktionsumfang kann in Zukunft aber noch deutlich ausgebaut werden. Bisher lag der Fokus auf der Erkennung von Objekten. Ein nächster Baustein könnte der Beurteilung von Kanten dienen, dazu gab es bereits Anfragen, etwa um die Gewindesteigung einer Schraube zu bestimmen.

Neben den klassischen Verarbeitungsmethoden ist die Integration von KI-basierten Erkennungen ein sehr interessantes Feld für zukünftige Erweiterungen. Hierzu ist bereits viel Know-How im Unternehmen vorhanden, mit geschickter Integration in TEaMVision können sicherlich leistungsstarke Erkennungsmethoden entwickelt werden.

Bei der Abdeckung großer Arbeitsbereiche kommen einzelne Industriekameras schnell an ihre Grenzen. Hier könnte ein Mechanismus interessant sein, um mehrere Kamerabilder zu vereinen und als einzelnes Bild analysieren zu können.

Vision-Anwendungsfälle wollen nicht immer nur ein statisches, unbewegliches Objekt betrachten. So gibt es zum Beispiel Applikationen, in denen Frequenz und Schema von blinkenden LEDs getestet werden. Für die visuelle Analyse müsste TEaMVision schnelle Bildfolgen oder Videos aufnehmen, bei denen jedes Bild einem genauen Zeitstempel zuzuordnen ist. Die Einzelbilder könnten dann extrahiert und mit den bekannten Erkennungsbausteinen analysiert werden.

8.3 Fortführung des Projekts

Nach Abschluss dieser Arbeit muss als erstes das Problem mit der Speicherfragmentierung gelöst werden. Sowohl als Vorbereitung auf zukünftige Erweiterungen mit größeren und hochfrequenten Bildern, als auch um für den kommenden Produktiveinsatz gewappnet zu sein.

Die entwickelten VTUX-Applikationen sollen zeitnah als fester Teil der Fertigungsline starten und weitere Applikationen sind bereits eingeplant. Wenn sich hier auch im Produktivbetrieb die ersten messbaren Erfolge zeigen, wird der Einsatzbereich voraussichtlich auch auf die anderen Ventilinselfamilien und gänzlich andere Produktfamilien erweitert.

Diverse Anwender und sonstige Stakeholder sind bereits jetzt beeindruckt von den Ergebnissen und zeigen große Bereitschaft und Erwartungen, dass zeitnah weitere Anwendungsfälle mit TEaMVision umgesetzt werden.

8.4 Fazit

Beim Thema Computer Vision wird meist ein zu großer Fokus auf die immer bessere Erkennung neuer, noch komplizierterer Merkmale gelegt. Dabei wird oft vernachlässigt, wie diese Mechanismen tatsächlich in der Realität eingesetzt werden können. Diese Arbeit zeigt eindrucksvoll, dass bereits sehr einfache und lange etablierte Methoden viele praktische Anwendungen abdecken können. **Der große Mehrwert wird dabei nicht von der Erkennung generiert, sondern durch die durchdachte Integration in TestStand. Der Leistungsumfang einer Softwarelösung ist immer dadurch limitiert, wie viel Zeit der Entwickler in die Entwicklung investieren kann.** Die TEaMVision-Bausteine befähigen einen großen Kreis von Anwendern, ihre Applikationen **selbständig** zu entwickeln. Dadurch werden Entwicklungskapazität und Produktivität massiv erhöht.

Das Thema dieser Arbeit sind Vision-Systeme. Die Problemstellung ist jedoch eine allgemeine. Durch geschicktes Software-Engineering mit modularen Komponenten, offenen Schnittstellen und Anwenderorientierter Bedienung wird eine Kernfunktionalität einem breiten Kreis von Nicht-Experten zugänglich gemacht. Dieses Vorgehen führte hier zum Erfolg und lässt sich auch auf viele andere Themengebiete übertragen.

9 Literaturverzeichnis

- Association for Advanced Automation. (12. Juni 2026). *GigE Vision*. Von automate.org: <https://www.automate.org/vision/vision-standards/vision-standards-gige-vision> abgerufen
- Baumer GmbH. (12. Juni 2026). *Industriekameras*. Von Baumer Website: https://media.baumer.com/mamfile_1845866_720Wx540H_c.png abgerufen
- Festo SE & Co.KG. (01. September 2025). Festo Mediathek (intern).
- Festo SE & Co.KG. (01. September 2025). *Unternehmensgeschichte*. Von Festo Website: https://www.festo.com/de/de/e/ueber-festo/unternehmen/unternehmensgeschichte-id_34983/ abgerufen
- Festo SE & Co.KG. (01. September 2025). *Valve Terminal VTUX*. Von Festo Website: https://ftp.festo.com/Public/PNEUMATIC/SOFTWARE_SERVICE/Documentation/2024/PT/VTUX_EN.PDF abgerufen
- Festo SE & Co.KG. (01. September 2025). *Zahlen, Daten, Fakten*. Von Festo Website: https://www.festo.com/de/de/e/ueber-festo/unternehmen/zahlen-daten-fakten-id_3756/ abgerufen
- He, K. L. (2008). Gaussian Noise Removal of Image on the Local Feature. *2008 Second International Symposium on Intelligent Information Technology Application*, 869.
- International Organization for Standardization. (07. Dezember 2025). *ISO 11898-1:2015*. Von ISO Website: <https://www.iso.org/standard/63648.html> abgerufen
- Intisar, M., Khan, M. M., Islam, M., & Masud, M. (2021). Computer Vision Based Robotic Arm Controlled Using Interactive GUI. *Intelligent Automation & Soft Computing*, 534-548.
- IO-Link Community. (07. Dezember 2025). *IO-Link*. Von IO-Link Website: <https://io-link.com/> abgerufen
- Keyence. (12. Juni 2026). *All-In-One Kamerasystem mit AI*. Von Keyence Website: <https://www.keyence.de/products/vision/vision-sys/vs/> abgerufen
- Kurek, P. (20. Juni 2026). *Multiton - Compendium*. Von Github Pages: https://pietrel.github.io/compendium/design_patterns/creational/multiton.html abgerufen
- Laughton, M. (17. Juni 2026). *What is libdmtx?* Von Libdmtx Website: <https://libdmtx.org/> abgerufen
- Maddula, S. (14. November 2025). *Building Industrial Vision Systems with Open-Source Imaging and Computer Vision Toolkits*. Von EURTH Techtronics Website: <https://www.eurthtech.com/post/building-industrial-vision-systems-with-open-source-imaging-and-computer-vision-toolkits> abgerufen
- Microsoft. (07. Dezember 2025). *Einführung in .Net*. Von Microsoft Learn: <https://learn.microsoft.com/de-de/dotnet/core/introduction> abgerufen
- Microsoft. (13. Mai 2026). *Dynamic Link Library (DLL) - Windows Client*. Von Microsoft Learn: <https://learn.microsoft.com/de-de/troubleshoot/windows-client/setup-upgrade-and-drivers/dynamic-link-library> abgerufen
- Microsoft. (27. Juni 2026). *Low-fragmentation Heap*. Von Microsoft Learn: <https://learn.microsoft.com/en-us/windows/win32/memory/low-fragmentation-heap> abgerufen

Microsoft. (14. Mai 2026). *Marshal.Copy Method*. Von Microsoft Learn: <https://learn.microsoft.com/en-us/dotnet/api/system.runtime.interopservices.marshal.copy?view=netframework-4.8.1&viewFallbackFrom=net-10.0> abgerufen

Microsoft. (14. Mai 2026). *WriteableBitmap Class*. Von Microsoft Learn: <https://learn.microsoft.com/de-de/dotnet/api/system.windows.media.imaging.writeablebitmap?view=netframework-4.8.1&viewFallbackFrom=windowsdesktop-10.0> abgerufen

MVTec. (12. Juni 2026). *Halcon Quick Guide*. Von MVTec Website: https://www.mvtec.com/fileadmin/Redaktion/mvtec.com/products/halcon/documentation/halcon/quick_guide.pdf abgerufen

MVTec. (12. Juni 2026). *Halcon Solution Guide*. Von MVTec Website: https://www.mvtec.com/fileadmin/Redaktion/mvtec.com/products/halcon/documentation/solution_guide/solution_guide_i.pdf abgerufen

MVTec. (12. Juni 2026). *HDevelop Users Guide*. Von MVTec Website: https://www.mvtec.com/fileadmin/Redaktion/mvtec.com/products/halcon/documentation/halcon/hdevelop_users_guide.pdf abgerufen

NI. (07. Dezember 2025). *NI TDMS-Dateiformat*. Von NI Website: <https://www.ni.com/de/support/documentation/supplemental/06/the-ni-tdms-file-format.html> abgerufen

NI. (07. Dezember 2025). *TestStand*. Von NI Website: <https://www.ni.com/de-de/shop/product/teststand.html> abgerufen

NI. (07. Dezember 2025). *Was ist NI LabVIEW?* Von NI Website: <https://www.ni.com/de/shop/labview.html> abgerufen

NI. (13. Mai 2026). *NI-Test- und Messlösungen von Emerson*. Von NI Website: <https://www.ni.com/de.html> abgerufen

Open Source Vision Foundation. (04. Dezember 2025). *opencv.org*. Von OpenCV Website: <https://opencv.org/> abgerufen

OpenCV. (12. Juni 2026). *Eroding and Dilating*. Von OpenCV Dokumentation: https://docs.opencv.org/3.4.20/db/df6/tutorial_erosion_dilatation.html abgerufen

OpenCV. (12. Juni 2026). *More Morphology Transformations*. Von OpenCV Dokumentation: https://docs.opencv.org/3.4.20/d3/db6/tutorial_opening_closing_hats.html abgerufen

OpenCV Team. (13. Juni 2026). *OpenCV Platforms*. Von OpenCV Website: <https://opencv.org/platforms/> abgerufen

Pekat Vision. (12. Juni 2026). *Machine Vision in Quality Inspection: Traditional vs. Deep Learning Approaches*. Von Pekat Vision Website: <https://www.pekatvision.com/machine-vision-in-quality-inspection-traditional-vs-deep-learning-approaches/> abgerufen

PROFIBUS Nutzerorganisation. (07. Dezember 2025). *Profibus - der bewährte Feldbus*. Von Profibus Website: <https://www.profibus.de/profibus> abgerufen

Rudorfer, M. (13. Juni 2026). *service-based architectures for machine vision*. Von Github Pages: <https://mrudorfer.github.io/research/industrial-vision-services> abgerufen

Seppälä, I. (20. 06 2026). *Multiton Pattern in Java: Mastering Advanced Singleton Variants*. Von <https://java-design-patterns.com/patterns/multiton/> abgerufen

Tesseract OCR. (17. Juni 2026). *The world's best free OCR engine*. Von Tesseract Website:
<https://tesseractocr.org/> abgerufen

Zhu, Q., Zhang, Y., Luan, J., & Hu, L. (2022). A Machine Vision Development Framework for Product Appearance Quality Inspection. *Applied Sciences*.